

Babel

Code

Version 26.7.123173
2026/05/15

Javier Bezos
Current maintainer

Johannes L. Braams
Original author

Localization and
internationalization

Unicode

T_EX

LuaT_EX

pdfT_EX

XeT_EX

Contents

1	Identification and loading of required files	3
2	locale directory	3
3	Tools	3
3.1	A few core definitions	8
3.2	LaTeX: babel.sty (start)	8
3.3	base	10
3.4	key=value options and other general option	10
3.5	Post-process some options	12
3.6	Plain: babel.def (start)	13
4	babel.sty and babel.def (common)	13
4.1	Selecting the language	15
4.2	Errors	23
4.3	More on selection	24
4.4	Short tags	25
4.5	Compatibility with language.def	25
4.6	Hooks	26
4.7	Setting up language files	27
4.8	Shorthands	29
4.9	Language attributes	38
4.10	Support for saving and redefining macros	40
4.11	French spacing	41
4.12	Hyphens	42
4.13	Multiencoding strings	44
4.14	Tailor captions	48
4.15	Making glyphs available	49
4.15.1	Quotation marks	49
4.15.2	Letters	51
4.15.3	Shorthands for quotation marks	52
4.15.4	Umlauts and tremas	52
4.16	Layout	54
4.17	Load engine specific macros	54
4.18	Creating and modifying languages	54
4.19	Main loop in ‘provide’	62
4.20	Processing keys in ini	67
4.21	French spacing (again)	72
4.22	Handle language system	73
4.23	Numerals	74
4.24	Casing	75
4.25	Getting info	76
4.26	BCP 47 related commands	77
5	Adjusting the Babel behavior	78
5.1	Cross referencing macros	80
5.2	Layout	83
5.3	Marks	85
5.4	Other packages	85
5.4.1	ifthen	85
5.4.2	varioref	86
5.4.3	hhline	87
5.5	Encoding and fonts	87
5.6	Basic bidi support	89
5.7	Local Language Configuration	92
5.8	Language options	92

6	The kernel of Babel	96
7	Error messages	97
8	Loading hyphenation patterns	100
9	luatex + xetex: common stuff	104
10	Hooks for XeTeX and LuaTeX	108
10.1	XeTeX	108
10.2	Support for interchar	109
10.3	Layout	111
10.4	8-bit TeX	113
10.5	LuaTeX	114
10.6	Southeast Asian scripts	121
10.7	CJK line breaking	122
10.8	Arabic justification	124
10.9	Common stuff	128
10.10	Automatic fonts and ids switching	129
10.11	Bidi	135
10.12	Layout	138
10.13	Lua: transforms	148
10.14	Lua: Auto bidi with basic and basic-r	158
11	Data for CJK	169
12	The ‘nil’ language	169
13	Calendars	170
13.1	Islamic	171
13.2	Hebrew	172
13.3	Persian	176
13.4	Coptic and Ethiopic	177
13.5	Julian	177
13.6	Buddhist	178
14	Support for Plain T_EX (plain.def)	179
14.1	Not renaming hyphen.tex	179
14.2	Emulating some L ^A T _E X features	180
14.3	General tools	180
14.4	Encoding related macros	184
15	Acknowledgements	187

The babel package is being developed incrementally, which means parts of the code are under development and therefore incomplete. Only documented features are considered complete. In other words, use babel in real documents only as documented (except, of course, if you want to explore and test them).

1. Identification and loading of required files

The babel package after unpacking consists of the following files:

babel.sty is the $\LaTeX$ package, which set options and load language styles.

babel.def is loaded by Plain.

switch.def defines macros to set and switch languages (it loads part babel.def).

plain.def is not used, and just loads babel.def, for compatibility.

hyphen.cfg is the file to be used when generating the formats to load hyphenation patterns.

There some additional tex, def and lua files.

The babel installer extends docstrip with a few “pseudo-guards” to set “variables” used at installation time. They are used with `<@name@>` at the appropriate places in the source code and defined with either `<<name=value>>`, or with a series of lines between `<<*name>>` and `<</name>>`. The latter is cumulative (e.g., with *More package options*). That brings a little bit of literate programming. The guards `<-name>` and `<+name>` have been redefined, too. See babel.ins for further details.

2. locale directory

A required component of babel is a set of ini files with basic definitions for about 300 languages. They are distributed as a separate zip file, not packed as dtx. Many of them are essentially finished (except bugs and mistakes, of course). Some of them are still incomplete (but they will be usable), and there are some omissions (e.g., there are no geographic areas in Spanish). Not all include LICR variants.

babel-*.ini files contain the actual data; babel-*.tex files are basically proxies to the corresponding ini files.

See [Keys in ini files](#) in the the babel site.

3. Tools

```
1 <<version=26.7.123173>>
2 <<date=2026/05/15>>
```

Do not use the following macros in ldf files. They may change in the future. This applies mainly to those recently added for replacing, trimming and looping. The older ones, like `\bbl@afterfi`, will not change. We define some basic macros which just make the code cleaner. `\bbl@add` is now used internally instead of `\addto` because of the unpredictable behavior of the latter. Used in babel.def and in babel.sty, which means in $\LaTeX$ is executed twice, but we need them when defining options and babel.def cannot be load until options have been defined. This does not hurt, but should be fixed somehow.

```
3 <<*Basic macros>> ≡
4 \bbl@trace{Basic macros}
5 \def\bbl@stripslash{\expandafter\@gobble\string}
6 \def\bbl@add#1#2{%
7   \bbl@ifunset{\bbl@stripslash#1}%
8     {\def#1{#2}}%
9     {\expandafter\def\expandafter#1\expandafter{#1#2}}
10 \def\bbl@xin@{\@expandtwoargs\in@}
11 \def\bbl@carg#1#2{\expandafter#1\csname#2\endcsname}%
12 \def\bbl@ncarg#1#2#3{\expandafter#1\expandafter#2\csname#3\endcsname}%
13 \def\bbl@ccarg#1#2#3{%
14   \expandafter#1\csname#2\expandafter\endcsname\csname#3\endcsname}%
15 \def\bbl@carg#1#2{\expandafter#1\csname bbl@#2\endcsname}%
16 \def\bbl@cs#1{\csname bbl@#1\endcsname}
17 \def\bbl@c#1{\csname bbl@#1\language\endcsname}
18 \def\bbl@loop#1#2#3{\bbl@loop#1{#3}#2,\@nnil,}
19 \def\bbl@loopx#1#2{\expandafter\bbl@loop\expandafter#1\expandafter{#2}}
```

```

20 \def\bbl@loop#1#2#3,{%
21   \ifx\@nnil#3\relax\else
22     \def#1{#3}#2\bbl@afterfi\bbl@loop#1{#2}%
23   \fi}
24 \def\bbl@for#1#2#3{\bbl@loopx#1{#2}{\ifx#1\@empty\else#3\fi}}

```

\bbl@add@list This internal macro adds its second argument to a comma separated list in its first argument. When the list is not defined yet (or empty), it will be initiated. It presumes expandable character strings.

```

25 \def\bbl@add@list#1#2{%
26   \edef#1{%
27     \bbl@ifunset{\bbl@stripslash#1}%
28     }%
29     {\ifx#1\@empty\else#1,\fi}%
30   #2}}

```

\bbl@afterelse

\bbl@afterfi Because the code that is used in the handling of active characters may need to look ahead, we take extra care to ‘throw’ it over the `\else` and `\fi` parts of an `\if`-statement¹. These macros will break if another `\if... \fi` statement appears in one of the arguments and it is not enclosed in braces.

```

31 \long\def\bbl@afterelse#1\else#2\fi{\fi#1}
32 \long\def\bbl@afterfi#1\fi{\fi#1}

```

\bbl@exp Now, just syntactical sugar, but it makes partial expansion of some code a lot more simple and readable. Here `\` stands for `\noexpand`, `\<.` for `\noexpand` applied to a built macro name (which does not define the macro if undefined to `\relax`, because it is created locally), and `\[. . .]` for one-level expansion (where `. . .` is the macro name without the backslash). The result may be followed by extra arguments, if necessary.

```

33 \def\bbl@exp#1{%
34   \begingroup
35   \let\<\noexpand
36   \let\<\bbl@exp@en
37   \let\[\bbl@exp@ue
38   \edef\bbl@exp@aux{\endgroup#1}%
39   \bbl@exp@aux}
40 \def\bbl@exp@en#1>{\expandafter\noexpand\csname#1\endcsname}%
41 \def\bbl@exp@ue#1]{%
42   \unexpanded\expandafter\expandafter\expandafter{\csname#1\endcsname}}%

```

\bbl@trim The following piece of code is stolen (with some changes) from `keyval`, by David Carlisle. It defines two macros: `\bbl@trim` and `\bbl@trim@def`. The first one strips the leading and trailing spaces from the second argument and then applies the first argument (a macro, `\toks@` and the like). The second one, as its name suggests, defines the first argument as the stripped second argument.

```

43 \def\bbl@tempa#1{%
44   \long\def\bbl@trim##1##2{%
45     \futurelet\bbl@trim@a\bbl@trim@c##2\@nil\@nil#1\@nil\relax{##1}}%
46   \def\bbl@trim@c{%
47     \ifx\bbl@trim@a\sptoken
48       \expandafter\bbl@trim@b
49     \else
50       \expandafter\bbl@trim@b\expandafter#1%
51     \fi}%
52   \long\def\bbl@trim@b#1##1 \@nil{\bbl@trim@i##1}}
53 \bbl@tempa{ }
54 \long\def\bbl@trim@i#1\@nil#2\relax#3{#3{#1}}
55 \long\def\bbl@trim@def#1{\bbl@trim{\def#1}}

```

¹This code is based on code presented in TUGboat vol. 12, no2, June 1991 in “An expansion Power Lemma” by Sonja Maus.

\bbl@ifunset To check if a macro is defined, we create a new macro, which does the same as `\ifundefined`. However, in an ϵ -tex engine, it is based on `\ifcsname`, which is more efficient, and does not waste memory. Defined inside a group, to avoid `\ifcsname` being implicitly set to `\relax` by the `\csname` test.

```

56 \begingroup
57 \gdef\bbl@ifunset#1{%
58   \expandafter\ifx\csname#1\endcsname\relax
59     \expandafter\@firstoftwo
60   \else
61     \expandafter\@secondoftwo
62   \fi}
63 \bbl@ifunset{ifcsname}%
64 {}%
65 {\gdef\bbl@ifunset#1{%
66   \ifcsname#1\endcsname
67     \expandafter\ifx\csname#1\endcsname\relax
68       \bbl@afterelse\expandafter\@firstoftwo
69     \else
70       \bbl@afterfi\expandafter\@secondoftwo
71     \fi
72   \else
73     \expandafter\@firstoftwo
74   \fi}}
75 \endgroup

```

\bbl@ifblank A tool from url, by Donald Arseneau, which tests if a string is empty or space. The companion macros tests if a macro is defined with some ‘real’ value, i.e., not `\relax` and not empty,

```

76 \def\bbl@ifblank#1{%
77   \bbl@ifblank@i#1\@nil\@nil\@secondoftwo\@firstoftwo\@nil}
78 \long\def\bbl@ifblank@i#1#2\@nil#3#4#5\@nil#4#5%
79 \def\bbl@ifset#1#2#3{%
80   \bbl@ifunset{#1}{#3}{\bbl@exp{\bbl@ifblank{\@nameuse{#1}}}{#3}{#2}}}

```

For each element in the comma separated `<key>=<value>` list, execute `<code>` with #1 and #2 as the key and the value of current item (trimmed). In addition, the item is passed verbatim as #3. With the `<key>` alone, it passes `\@empty` as value (i.e., the macro thus named, not an empty argument, which is what you get with `<key>=` and no value).

```

81 \def\bbl@forkv#1#2{%
82   \def\bbl@kvcmd##1##2##3{#2}%
83   \bbl@kvnext#1,\@nil,}
84 \def\bbl@kvnext#1,{%
85   \ifx\@nil#1\relax\else
86     \bbl@ifblank{#1}{\bbl@forkv@eq#1=\@empty=\@nil{#1}}%
87     \expandafter\bbl@kvnext
88   \fi}
89 \def\bbl@forkv@eq#1=#2=#3\@nil#4{%
90   \bbl@trim\def\bbl@forkv@a{#1}%
91   \bbl@trim{\expandafter\bbl@kvcmd\expandafter{\bbl@forkv@a}}{#2}{#4}}

```

A *for* loop. Each item (trimmed) is #1. It cannot be nested (it’s doable, but we don’t need it).

```

92 \def\bbl@vforeach#1#2{%
93   \def\bbl@forcmd##1{#2}%
94   \bbl@fornext#1,\@nil,}
95 \def\bbl@fornext#1,{%
96   \ifx\@nil#1\relax\else
97     \bbl@ifblank{#1}{\bbl@trim\bbl@forcmd{#1}}%
98     \expandafter\bbl@fornext
99   \fi}
100 \def\bbl@foreach#1{\expandafter\bbl@vforeach\expandafter{#1}}

```

Some code should be executed once. The first argument is a flag.

```

101 \global\let\bbl@done\@empty

```

```

102 \def\bbl@once#1#2{%
103   \bbl@xin@{,#1,}{,\bbl@done,}%
104   \ifin@ \else
105     #2%
106   \xdef\bbl@done{\bbl@done,#1,}%
107   \fi}
108 %   \end{macrodef}
109 %
110 % \macro{\bbl@replace}
111 %
112 % Returns implicitly |\toks@| with the modified string.
113 %
114 %   \begin{macrocode}
115 \def\bbl@replace#1#2#3{% in #1 -> repl #2 by #3
116   \toks@{}%
117   \def\bbl@replace@aux##1#2##2#2{%
118     \ifx\bbl@nil##2%
119       \toks@ \expandafter{\the\toks@##1}%
120     \else
121       \toks@ \expandafter{\the\toks@##1#3}%
122       \bbl@afterfi
123       \bbl@replace@aux##2#2%
124     \fi}%
125   \expandafter\bbl@replace@aux#1#2\bbl@nil#2%
126   \edef#1{\the\toks@}}

```

An extension to the previous macro. It takes into account the parameters, and it is string based (i.e., if you replace elax by ho, then \relax becomes \rho). No checking is done at all, because it is not a general purpose macro, and it is used by babel only when it works (an example where it does *not* work is in \bbl@TG@@date, and also fails if there are macros with spaces, because they are retokenized). It may change! (or even merged with \bbl@replace; I'm not sure checking the replacement is really necessary or just paranoia).

```

127 \ifx\detokenize\undefined \else % Unused macros if old Plain TeX
128   \bbl@exp{\def\\bbl@parsedef##1\detokenize{macro:}}#2->#3\relax{%
129     \def\bbl@tempa{#1}%
130     \def\bbl@tempb{#2}%
131     \def\bbl@tempe{#3}}
132 \def\bbl@sreplace#1#2#3{%
133   \begingroup
134     \expandafter\bbl@parsedef\meaning#1\relax
135     \def\bbl@tempc{#2}%
136     \edef\bbl@tempc{\expandafter\strip@prefix\meaning\bbl@tempc}%
137     \def\bbl@tempd{#3}%
138     \edef\bbl@tempd{\expandafter\strip@prefix\meaning\bbl@tempd}%
139     \bbl@xin@{\bbl@tempc}{\bbl@tempe}% If not in macro, do nothing
140     \ifin@
141       \bbl@exp{\\bbl@replace\\bbl@tempe{\bbl@tempc}{\bbl@tempd}}%
142       \def\bbl@tempc{% Expanded an executed below as 'uplevel'
143         \\makeatletter % "internal" macros with @ are assumed
144         \\scantokens{%
145           \bbl@tempa\\@namedef{\bbl@stripslash#1}\bbl@tempb{\bbl@tempe}%
146           \noexpand\noexpand}%
147         \catcode64=\the\catcode64\relax}% Restore @
148     \else
149       \let\bbl@tempc\empty % Not \relax
150     \fi
151     \bbl@exp{% For the 'uplevel' assignments
152   \endgroup
153   \bbl@tempc}} % empty or expand to set #1 with changes
154 \fi

```

Two further tools. \bbl@ifsamestring first expand its arguments and then compare their expansion (sanitized, so that the catcodes do not matter). \bbl@engine takes the following values: 0 is pdf_{La}TeX, 1 is luatex, and 2 is xetex. You may use the latter it in your language style if you want.

```

155 \def\bbl@ifsamestring#1#2{%
156   \begingroup
157   \protected@edef\bbl@tempb{#1}%
158   \edef\bbl@tempb{\expandafter\strip@prefix\meaning\bbl@tempb}%
159   \protected@edef\bbl@tempc{#2}%
160   \edef\bbl@tempc{\expandafter\strip@prefix\meaning\bbl@tempc}%
161   \ifx\bbl@tempb\bbl@tempc
162     \aftergroup\@firstoftwo
163   \else
164     \aftergroup\@secondoftwo
165   \fi
166 \endgroup}
167 \chardef\bbl@engine=%
168 \ifx\directlua\@undefined
169   \ifx\XeTeXinputencoding\@undefined
170     \z@
171   \else
172     \tw@
173   \fi
174 \else
175   \@ne
176 \fi

```

A somewhat hackish tool (hence its name) to avoid spurious spaces in some contexts.

```

177 \def\bbl@bsphack{%
178   \ifhmode
179     \hskip\z@skip
180     \def\bbl@esphack{\loop\ifdim\lastskip>\z@\unskip\repeat\unskip}%
181   \else
182     \let\bbl@esphack\@empty
183   \fi}

```

Another hackish tool, to apply case changes inside a protected macros. It's based on the internal `\let's` made by `\MakeUppercase` and `\MakeLowercase` between things like `\oe` and `\OE`.

```

184 \def\bbl@cased{%
185   \ifx\oe\OE
186     \expandafter\in@\expandafter
187     {\expandafter\OE\expandafter}\expandafter{\oe}%
188   \ifin@
189     \bbl@afterelse\expandafter\MakeUppercase
190   \else
191     \bbl@afterfi\expandafter\MakeLowercase
192   \fi
193 \else
194   \expandafter\@firstofone
195 \fi}

```

The following adds some code to `\extras...` both before and after, while avoiding doing it twice. It's somewhat convoluted, to deal with `#`'s. Used to deal with `alph`, `Alph` and frenchspacing when there are already changes (with `\babel@save`).

```

196 \def\bbl@extras@wrap#1#2#3{% 1:in-test, 2:before, 3:after
197   \toks@\expandafter\expandafter\expandafter{%
198     \csname extras\language\endcsname}%
199   \bbl@exp{\in{#1}{\the\toks@}}%
200   \ifin@\else
201     \@temptokena{#2}%
202     \edef\bbl@tempc{\the\@temptokena\the\toks@}%
203     \toks@\expandafter{\bbl@tempc#3}%
204     \expandafter\edef\csname extras\language\endcsname{\the\toks@}%
205   \fi}
206 <</Basic macros>>

```

Some files identify themselves with a $\TeX$ macro. The following code is placed before them to define (and then undefine) if not in $\TeX$.


```

207 <<*Make sure ProvidesFile is defined>> ≡
208 \ifx\ProvidesFile\@undefined
209   \def\ProvidesFile#1[#2 #3 #4]{%
210     \wlog{File: #1 #4 #3 <#2>}%
211     \let\ProvidesFile\@undefined}
212 \fi
213 <</Make sure ProvidesFile is defined>>

```

3.1. A few core definitions

\language Just for compatibility, for not to touch `hyphen.cfg`.

```

214 <<*Define core switching macros>> ≡
215 \ifx\language\@undefined
216   \csname newcount\endcsname\language
217 \fi
218 <</Define core switching macros>>

```

\last@language Another counter is used to keep track of the allocated languages. $\TeX$ and $\LaTeX$ reserves for this purpose the count 19.

\addlanguage This macro was introduced for $\TeX < 2$. Preserved for compatibility.

```

219 <<*Define core switching macros>> ≡
220 \countdef\last@language=19
221 \def\addlanguage{\csname newlanguage\endcsname}
222 <</Define core switching macros>>

```

Now we make sure all required files are loaded. When the command `\AtBeginDocument` doesn't exist we assume that we are dealing with a plain-based format. In that case the file `plain.def` is needed (which also defines `\AtBeginDocument`, and therefore it is not loaded twice). We need the first part when the format is created, and `\orig@dump` is used as a flag. Otherwise, we need to use the second part, so `\orig@dump` is not defined (`plain.def` undefines it).

Check if the current version of `switch.def` has been previously loaded (mainly, `hyphen.cfg`). If not, load it now. We cannot load `babel.def` here because we first need to declare and process the package options.

3.2. $\LaTeX$: `babel.sty` (start)

Here starts the style file for $\LaTeX$. It also takes care of a number of compatibility issues with other packages.

```

223 <*package>
224 \NeedsTeXFormat{LaTeX2e}
225 \ProvidesPackage{babel}%
226 [<@date@> v<@version@>
227   The multilingual framework for LuaLaTeX, pdfLaTeX and XeLaTeX]

```

Start with some “private” debugging tools, and then define macros for errors. The global lua ‘space’ Babel is declared here, too (inside the test for debug).

```

228 \ifpackagewith{babel}{debug}
229   {\providecommand\bbl@trace[1]{\message{^^J[ #1 ]}}%
230    \let\bbl@debug\@firstofone
231    \ifx\directlua\@undefined\else
232      \directlua{
233        Babel = Babel or {}
234        Babel.debug = true }%
235      \input{babel-debug.tex}%
236    \fi}
237 {\providecommand\bbl@trace[1]{}%
238  \let\bbl@debug\@gobble
239  \ifx\directlua\@undefined\else
240    \directlua{
241      Babel = Babel or {}
242      Babel.debug = false }%

```

```

243 \fi}
244 % Temporary:
245 \newif\ifBabelDebugGerman
246 \ifpackagewith{babel}{debug-german}
247 {\BabelDebugGermantrue}
248 {\BabelDebugGermanfalse}

```

Macros to deal with errors, warnings, etc. Errors are stored in a separate file.

```

249 \def\bbl@error#1{% Implicit #2#3#4
250 \begingroup
251 \catcode`\=0 \catcode`\==12 \catcode`\`=12
252 \input errbabel.def
253 \endgroup
254 \bbl@error{#1}}
255 \def\bbl@warning#1{%
256 \begingroup
257 \def\{\MessageBreak}%
258 \PackageWarning{babel}{#1}%
259 \endgroup}
260 \def\bbl@infowarn#1{%
261 \begingroup
262 \def\{\MessageBreak}%
263 \PackageNote{babel}{#1}%
264 \endgroup}
265 \def\bbl@info#1{%
266 \begingroup
267 \def\{\MessageBreak}%
268 \PackageInfo{babel}{#1}%
269 \endgroup}

```

Many of the following options don't do anything themselves, they are just defined in order to make it possible for babel and language definition files to check if one of them was specified by the user.

But first, include here the *Basic macros* defined above.

```

270 <@Basic macros>
271 \ifpackagewith{babel}{silent}
272 {\let\bbl@info@gobble
273 \let\bbl@infowarn@gobble
274 \let\bbl@warning@gobble}
275 {}
276 %
277 \def\AfterBabelLanguage#1{%
278 \global\expandafter\bbl@add\csname#1.ldf-h@k\endcsname}%

```

If the format created a list of loaded languages (in \bbl@languages), get the name of the 0-th to show the actual language used. Also available with base, because it just shows info.

```

279 \ifx\bbl@languages\undefined\else
280 \begingroup
281 \catcode`\^^I=12
282 \@ifpackagewith{babel}{showlanguages}{%
283 \begingroup
284 \def\bbl@elt#1#2#3#4{\wlog{#2^^I#1^^I#3^^I#4}}%
285 \wlog{<*languages>}%
286 \bbl@languages
287 \wlog{</languages>}%
288 \endgroup{}}
289 \endgroup
290 \def\bbl@elt#1#2#3#4{%
291 \ifnum#2=z@
292 \gdef\bbl@nulllanguage{#1}%
293 \def\bbl@elt##1##2##3##4{%
294 \fi}%
295 \bbl@languages
296 \fi

```

3.3. base

The first ‘real’ option to be processed is base, which set the hyphenation patterns then resets `ver@babel.sty` so that $\TeX$ forgets about the first loading. After a subset of `babel.def` has been loaded (the old `switch.def`) and `\AfterBabelLanguage` defined, it exits.

Now the base option. With it we can define (and load, with `luatex`) hyphenation patterns, even if we are not interested in the rest of `babel`.

```
297 \bbl@trace{Defining option 'base'}
298 \@ifpackagewith{babel}{base}{%
299   \let\bbl@onlyswitch\@empty
300   \let\bbl@provide@locale\relax
301   \input babel.def
302   \let\bbl@onlyswitch\@undefined
303   \ifx\directlua\@undefined
304     \DeclareOption*{\bbl@patterns{\CurrentOption}}%
305   \else
306     \input luababel.def
307     \DeclareOption*{\bbl@patterns@lua{\CurrentOption}}%
308   \fi
309   \DeclareOption{base}{}%
310   \DeclareOption{showlanguages}{}%
311   \ProcessOptions
312   \global\expandafter\let\csname opt@babel.sty\endcsname\relax
313   \global\expandafter\let\csname ver@babel.sty\endcsname\relax
314   \global\let\@ifl@ter@@\@ifl@ter
315   \def\@ifl@ter#1#2#3#4#5{\global\let\@ifl@ter\@ifl@ter@@}%
316   \endinput}{}%
```

3.4. key=value options and other general option

The following macros extract language modifiers, and only real package options are kept in the option list. Modifiers are saved and assigned to `\BabelModifiers` at `\bbl@load@language`; when no modifiers have been given, the former is `\relax`.

```
317 \bbl@trace{key=value and another general options}
318 \bbl@csarg\let{tempa\expandafter}\csname opt@babel.sty\endcsname
319 \def\bbl@tempb#1.#2{% Removes trailing dot
320   #1\ifx\@empty#2\else,\bbl@afterfi\bbl@tempb#2\fi}%
321 \def\bbl@tempe#1=#2\@@{%
322   \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}}
323 \def\bbl@tempd#1.#2\@nnil{%
324   \ifx\@empty#2%
325     \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
326   \else
327     \in@{,provide=}{, #1}%
328     \ifin@
329       \edef\bbl@tempc{%
330         \ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.\bbl@tempb#2}%
331     \else
332       \in@{$modifiers$}{$#1$}%
333       \ifin@
334         \bbl@tempe#2\@@
335       \else
336         \in@{=}{#1}%
337         \ifin@
338           \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.#2}%
339         \else
340           \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
341           \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}%
342         \fi
343       \fi
344     \fi
345   \fi}
346 \let\bbl@tempc\@empty
```

```

347 \bbl@foreach\bbl@tempa{\bbl@tempd#1.\@empty\@nnil}
348 \expandafter\let\csname opt@babel.sty\endcsname\bbl@tempc

```

The next option tells babel to leave shorthand characters active at the end of processing the package. This is *not* the default as it can cause problems with other packages, but for those who want to use the shorthand characters in the preamble of their documents this can help.

```

349 \DeclareOption{KeepShorthandsActive}{}
350 \DeclareOption{activeacute}{}
351 \DeclareOption{activegrave}{}
352 \DeclareOption{debug}{}
353 \DeclareOption{debug-german}{} % Temporary
354 \DeclareOption{noconfigs}{}
355 \DeclareOption{showlanguages}{}
356 \DeclareOption{silent}{}
357 \DeclareOption{shorthands=off}{\bbl@tempa shorthands=\bbl@tempa}
358 \chardef\bbl@iniflag\z@
359 \DeclareOption{provide=*}{\chardef\bbl@iniflag\@ne} % main = 1
360 \DeclareOption{provide+=*}{\chardef\bbl@iniflag\tw@} % second = 2
361 \DeclareOption{provide*=*}{\chardef\bbl@iniflag\thr@@} % second + main
362 \chardef\bbl@ldfflag\z@
363 \DeclareOption{provide=!}{\chardef\bbl@ldfflag\@ne} % main = 1
364 \DeclareOption{provide+=!}{\chardef\bbl@ldfflag\tw@} % second = 2
365 \DeclareOption{provide*=!}{\chardef\bbl@ldfflag\thr@@} % second + main
366 \DeclareOption{licr=extended}{}
367 \DeclareOption{licr=unextended}{}
368 % Don't use. Experimental.
369 \newif\ifbbl@single
370 \DeclareOption{selectors=off}{\bbl@singletrue}
371 <@More package options@>

```

Handling of package options is done in three passes. (I [JBL] am not very happy with the idea, anyway.) The first one processes options which has been declared above or follow the syntax $\langle key \rangle = \langle value \rangle$, the second one loads the requested languages, except the main one if set with the key main, and the third one loads the latter. First, we “flag” valid keys with a nil value.

```

372 \let\bbl@opt@shorthands\@nnil
373 \let\bbl@opt@config\@nnil
374 \let\bbl@opt@main\@nnil
375 \let\bbl@opt@headfoot\@nnil
376 \let\bbl@opt@layout\@nnil
377 \let\bbl@opt@provide\@nnil

```

The following tool is defined temporarily to store the values of options.

```

378 \def\bbl@tempa#1=#2\bbl@tempa{%
379   \bbl@csarg@ifx{opt#1}\@nnil
380     \bbl@csarg\edef{opt#1}{#2}%
381   \else
382     \bbl@error{bad-package-option}{#1}{#2}{}%
383   \fi}

```

Now the option list is processed, taking into account only currently declared options (including those declared with a =), and $\langle key \rangle = \langle value \rangle$ options (the former take precedence). Unrecognized options are saved in \bbl@language@opts, because they are language options.

```

384 \let\bbl@language@opts\@empty
385 \DeclareOption*{%
386   \bbl@xin@{\string=}{\CurrentOption}%
387   \ifin@
388     \expandafter\bbl@tempa\CurrentOption\bbl@tempa
389   \else
390     \bbl@add@list\bbl@language@opts{\CurrentOption}%
391   \fi}

```

Now we finish the first pass (and start over).

```

392 \ProcessOptions*

```

3.5. Post-process some options

```
393 \ifx\bbl@opt@provide\@nnil
394 \let\bbl@opt@provide\@empty %%%% MOVE above
395 \else
396 \chardef\bbl@iniflag\@ne
397 \bbl@exp{\bbl@forkv{\@nameuse{@raw@opt@babel.sty}}}{%
398 \in@{,provide,}{, #1,}%
399 \ifin@
400 \def\bbl@opt@provide{#2}%
401 \fi}
402 \fi
```

If there is no `shorthands=<chars>`, the original babel macros are left untouched, but if there is, these macros are wrapped (in `babel.def`) to define only those given.

A bit of optimization: if there is no `shorthands=`, then `\bbl@ifshorthand` is always true, and it is always false if `shorthands` is empty. Also, some code makes sense only with `shorthands=...`

```
403 \bbl@trace{Conditional loading of shorthands}
404 \def\bbl@sh@string#1{%
405 \ifx#1\@empty\else
406 \ifx#1t\string~%
407 \else\ifx#1c\string,%
408 \else\string#1%
409 \fi\fi
410 \expandafter\bbl@sh@string
411 \fi}
412 \ifx\bbl@opt@shorthands\@nnil
413 \def\bbl@ifshorthand#1#2#3{#2}%
414 \else\ifx\bbl@opt@shorthands\@empty
415 \def\bbl@ifshorthand#1#2#3{#3}%
416 \else
```

The following macro tests if a shorthand is one of the allowed ones.

```
417 \def\bbl@ifshorthand#1{%
418 \bbl@xin@{\string#1}{\bbl@opt@shorthands}%
419 \ifin@
420 \expandafter\@firstoftwo
421 \else
422 \expandafter\@secondoftwo
423 \fi}
```

We make sure all chars in the string are ‘other’, with the help of an auxiliary macro defined above (which also zaps spaces).

```
424 \edef\bbl@opt@shorthands{%
425 \expandafter\bbl@sh@string\bbl@opt@shorthands\@empty}%
```

The following is ignored with `shorthands=off`, since it is intended to take some additional actions for certain chars.

```
426 \bbl@ifshorthand{'}%
427 {\PassOptionsToPackage{activeacute}{babel}}{}
428 \bbl@ifshorthand{`}%
429 {\PassOptionsToPackage{activegrave}{babel}}{}
430 \fi\fi
```

With `headfoot=lang` we can set the language used in heads/feet. For example, in `babel/3796` just add `headfoot=english`. It misuses `\@resetactivechars`, but seems to work.

```
431 \ifx\bbl@opt@headfoot\@nnil\else
432 \g@addto@macro\@resetactivechars{%
433 \set@typeset@protect
434 \expandafter\select@language@x\expandafter{\bbl@opt@headfoot}%
435 \let\protect\noexpand}
436 \fi
```

For the option `safe` we use a different approach – `\bbl@opt@safe` says which macros are redefined (B for bibs and R for refs). By default, both are currently set, but in a future release it will be set to `none`.

```
437 \ifx\bbl@opt@safe\@undefined
```

```

438 \def\bbl@opt@safe{BR}
439 % \let\bbl@opt@safe\@empty % Pending of \cite
440 \fi

For layout an auxiliary macro is provided, available for packages and language styles.
Optimization: if there is no layout, just do nothing.
441 \bbl@trace{Defining IfBabelLayout}
442 \ifx\bbl@opt@layout\@nnil
443 \newcommand\IfBabelLayout[3]{#3}%
444 \else
445 \bbl@exp{\bbl@forkv{\@nameuse{raw@opt@babel.sty}}}{%
446 \in{,layout,}{, #1,}%
447 \ifin@
448 \def\bbl@opt@layout{#2}%
449 \bbl@replace\bbl@opt@layout{ }{.}%
450 \fi}
451 \newcommand\IfBabelLayout[1]{%
452 \@expandtwoargs\in{.#1.}{.\bbl@opt@layout.}%
453 \ifin@
454 \expandafter\@firstoftwo
455 \else
456 \expandafter\@secondoftwo
457 \fi}
458 \fi
459 </package>

```

3.6. Plain: babel.def (start)

Because of the way docstrip works, we need to insert some code for Plain here. However, the tools provided by the babel installer for literate programming makes this section a short interlude, because the actual code is below, tagged as *Emulate LaTeX*.

First, exit immediately if previously loaded.

```

460 < *core >
461 \ifx\ldf@quit\undefined\else
462 \endinput\fi % Same line!
463 <@Make sure ProvidesFile is defined@>
464 \ProvidesFile{babel.def}[<@date@> v<@version@> Babel common definitions]
465 \ifx\AtBeginDocument\undefined
466 <@Emulate LaTeX@>
467 \fi
468 <@Basic macros@>
469 </core>

```

That is all for the moment. Now follows some common stuff, for both Plain and $\LaTeX$. After it, we will resume the $\LaTeX$ -only stuff.

4. babel.sty and babel.def (common)

```

470 < *package | core >
471 \def\bbl@version{<@version@>}
472 \def\bbl@date{<@date@>}
473 <@Define core switching macros@>

```

\adddialect The macro `\adddialect` can be used to add the name of a dialect or variant language, for which an already defined hyphenation table can be used.

```

474 \def\adddialect#1#2{%
475 \global\chardef#1#2\relax
476 \bbl@usehooks{adddialect}{#1}{#2}%
477 \begingroup
478 \count@#1\relax
479 \def\bbl@elt##1##2##3##4{%
480 \ifnum\count@=##2\relax
481 \edef\bbl@tempa{\expandafter\@gobbletwo\string#1}%
482 \bbl@info{Hyphen rules for '\expandafter\@gobble\bbl@tempa'

```

```

483             set to \expandafter\string\csname l@##1\endcsname\\%
484             (\string\language\the\count@). Reported}%
485     \def\bbl@elt###1###2###3###4{}%
486     \fi}%
487     \bbl@cs{languages}%
488 \endgroup}

```

\bbl@iflanguage executes code only if the language l@ exists. Otherwise raises an error.

The argument of \bbl@fixname has to be a macro name, as it may get “fixed” if casing (lc/uc) is wrong. It’s an attempt to fix a long-standing bug when \foreignlanguage and the like appear in a \MakeXXXcase. However, a lowercase form is not imposed to improve backward compatibility (perhaps you defined a language named MYLANG, but unfortunately mixed case names cannot be trapped). Note l@ is encapsulated, so that its case does not change.

```

489 \def\bbl@fixname#1{%
490   \begingroup
491   \def\bbl@tempe{l@}%
492   \edef\bbl@tempd{\noexpand\@ifundefined{\noexpand\bbl@tempe#1}}%
493   \bbl@tempd
494     {\lowercase\expandafter{\bbl@tempd}%
495     {\uppercase\expandafter{\bbl@tempd}%
496     \@empty
497     {\edef\bbl@tempd{\def\noexpand#1{#1}}%
498     {\uppercase\expandafter{\bbl@tempd}}}%
499     {\edef\bbl@tempd{\def\noexpand#1{#1}}%
500     {\lowercase\expandafter{\bbl@tempd}}}%
501     \@empty
502   \edef\bbl@tempd{\endgroup\def\noexpand#1{#1}}%
503   \bbl@tempd
504   \bbl@exp{\bbl@usehooks{language}{\language}{#1}}}
505 \def\bbl@iflanguage#1{%
506   \@ifundefined{l@#1}{\@nolanerr{#1}\@gobble}\@firstofone}

```

After a name has been ‘fixed’, the selectors will try to load the language. If even the fixed name is not defined, will load it on the fly, either based on its name, or if activated, its BCP 47 code.

We first need a couple of macros for a simple BCP 47 look up. It also makes sure, with \bbl@bcpcase, casing is the correct one, so that sr-latn-ba becomes fr-Latn-BA. Note #4 may contain some \@empty’s, but they are eventually removed.

\bbl@bcpllookup either returns the found ini tag or it is \relax.

```

507 \def\bbl@bcpcase#1#2#3#4\@#5{%
508   \ifx\@empty#3%
509     \uppercase{\def#5{#1#2}}%
510   \else
511     \uppercase{\def#5{#1}}%
512     \lowercase{\edef#5{#5#2#3#4}}%
513   \fi}
514 \def\bbl@bcpllookup#1-#2-#3-#4\@{%
515   \let\bbl@bcp\relax
516   \lowercase{\def\bbl@tempa{#1}}%
517   \ifx\@empty#2%
518     \IfFileExists{babel-\bbl@tempa.ini}{\let\bbl@bcp\bbl@tempa}{}%
519   \else\ifx\@empty#3%
520     \bbl@bcpcase#2\@empty\@empty\@{\bbl@tempb}
521     \IfFileExists{babel-\bbl@tempa-\bbl@tempb.ini}%
522       {\edef\bbl@bcp{\bbl@tempa-\bbl@tempb}}%
523       {}%
524   \ifx\bbl@bcp\relax
525     \IfFileExists{babel-\bbl@tempa.ini}{\let\bbl@bcp\bbl@tempa}{}%
526   \fi
527   \else
528     \bbl@bcpcase#2\@empty\@empty\@{\bbl@tempb}
529     \bbl@bcpcase#3\@empty\@empty\@{\bbl@tempc}
530     \IfFileExists{babel-\bbl@tempa-\bbl@tempb-\bbl@tempc.ini}%
531       {\edef\bbl@bcp{\bbl@tempa-\bbl@tempb-\bbl@tempc}}%
532       {}%

```

```

533 \ifx\bb@bcp\relax
534 \IfFileExists{babel-\bb@tempa-\bb@tempc.ini}%
535 {\edef\bb@bcp{\bb@tempa-\bb@tempc}}}%
536 {}%
537 \fi
538 \ifx\bb@bcp\relax
539 \IfFileExists{babel-\bb@tempa-\bb@tempc.ini}%
540 {\edef\bb@bcp{\bb@tempa-\bb@tempc}}}%
541 {}%
542 \fi
543 \ifx\bb@bcp\relax
544 \IfFileExists{babel-\bb@tempa.ini}{\let\bb@bcp\bb@tempa}}}%
545 \fi
546 \fi\fi}
547 \let\bb@initoload\relax

```

\iflanguage Users might want to test (in a private package for instance) which language is currently active. For this we provide a test macro, `\iflanguage`, that has three arguments. It checks whether the first argument is a known language. If so, it compares the first argument with the value of `\language`. Then, depending on the result of the comparison, it executes either the second or the third argument.

```

548 \def\iflanguage#1{%
549 \bb@iflanguage{#1}{%
550 \ifnum\csname l@#1\endcsname=\language
551 \expandafter\@firstoftwo
552 \else
553 \expandafter\@secondoftwo
554 \fi}}

```

4.1. Selecting the language

\selectlanguage It checks whether the language is already defined before it performs its actual task, which is to update `\language` and activate language-specific definitions.

```

555 \let\bb@select@type\z@
556 \edef\selectlanguage{%
557 \noexpand\protect
558 \expandafter\noexpand\csname selectlanguage \endcsname}

```

Because the command `\selectlanguage` could be used in a moving argument it expands to `\protect\selectlanguage`. Therefore, we have to make sure that a macro `\protect` exists. If it doesn't it is `\let` to `\relax`.

```

559 \ifx\@undefined\protect\let\protect\relax\fi

```

The following definition is preserved for backwards compatibility (e.g., arabi, koma). It is related to a trick for 2.09, now discarded.

```

560 \let\xstring\string

```

Since version 3.5 babel writes entries to the auxiliary files in order to typeset table of contents etc. in the correct language environment.

\bb@pop@language But when the language change happens *inside* a group the end of the group doesn't write anything to the auxiliary files. Therefore we need TeX's `aftergroup` mechanism to help us. The command `\aftergroup` stores the token immediately following it to be executed when the current group is closed. So we define a temporary control sequence `\bb@pop@language` to be executed at the end of the group. It calls `\bb@set@language` with the name of the current language as its argument.

\bb@language@stack The previous solution works for one level of nesting groups, but as soon as more levels are used it is no longer adequate. For that case we need to keep track of the nested languages using a stack mechanism. This stack is called `\bb@language@stack` and initially empty.

```

561 \def\bb@language@stack{}

```

When using a stack we need a mechanism to push an element on the stack and to retrieve the information afterwards.

\bbl@push@language

\bbl@pop@language The stack is simply a list of languagenames, separated with a '+' sign; the push function can be simple:

```
562 \def\bbl@push@language{%
563   \ifx\language\undefined\else
564     \ifx\currentgrouplevel\undefined
565       \xdef\bbl@language@stack{\language+\bbl@language@stack}%
566     \else
567       \ifnum\currentgrouplevel=\z@
568         \xdef\bbl@language@stack{\language+}%
569       \else
570         \xdef\bbl@language@stack{\language+\bbl@language@stack}%
571       \fi
572     \fi
573 }
```

Retrieving information from the stack is a little bit less simple, as we need to remove the element from the stack while storing it in the macro \language. For this we first define a helper function.

\bbl@pop@lang This macro stores its first element (which is delimited by the '+'-sign) in \language and stores the rest of the string in \bbl@language@stack.

```
574 \def\bbl@pop@lang#1+#2\@@{%
575   \edef\language{#1}%
576   \xdef\bbl@language@stack{#2}}
```

The reason for the somewhat weird arrangement of arguments to the helper function is the fact it is called in the following way. This means that before \bbl@pop@lang is executed TeX first *expands* the stack, stored in \bbl@language@stack. The result of that is that the argument string of \bbl@pop@lang contains one or more language names, each followed by a '+'-sign (zero language names won't occur as this macro will only be called after something has been pushed on the stack).

```
577 \let\bbl@ifrestoring\@secondoftwo
578 \def\bbl@pop@language{%
579   \expandafter\bbl@pop@lang\bbl@language@stack\@@
580   \let\bbl@ifrestoring\@firstoftwo
581   \expandafter\bbl@set@language\expandafter{\language}%
582   \let\bbl@ifrestoring\@secondoftwo}
```

Once the name of the previous language is retrieved from the stack, it is fed to \bbl@set@language to do the actual work of switching everything that needs switching.

An alternative way to identify languages (in the babel sense) with a numerical value is introduced in 3.30. This is one of the first steps for a new interface based on the concept of locale, which explains the name of \localeid. This means \l@... will be reserved for hyphenation patterns (so that two locales can share the same rules).

```
583 \chardef\localeid\z@
584 \gdef\bbl@id@last{0} % No real need for a new counter
585 \def\bbl@id@assign{%
586   \bbl@ifunset\bbl@id@\language{%
587     {\count@\bbl@id@last\relax
588      \advance\count@\@ne
589      \global\bbl@csarg\chardef{id@\language}\count@
590      \xdef\bbl@id@last{\the\count@}%
591      \ifcase\bbl@engine\or
592        \directlua{
593          Babel.locale_props[\bbl@id@last] = {}
594          Babel.locale_props[\bbl@id@last].name = '\language'
595          Babel.locale_props[\bbl@id@last].vars = {}
596        }%
597      \fi}%
598   }%
599   \chardef\localeid\bbl@c{l{id@}}
```

The unprotected part of \selectlanguage. In case it is used as environment, declare \endselectlanguage, just for safety.

```

600 \let\bbl@select@opts\@empty
601 \expandafter\def\csname selectlanguage \endcsname{%
602   \ifnextchar[\bbl@select@s{\bbl@select@s[]}}
603 \def\bbl@select@s[#1]#2{%
604   \def\bbl@select@opts{#1}%
605   \ifnum\bbl@hymapsel=\@ccclv\let\bbl@hymapsel\tw\fi
606   \bbl@push@language
607   \aftergroup\bbl@pop@language
608   \bbl@set@language{#2}}
609 \let\endselectlanguage\relax

```

\bbl@set@language The macro \bbl@set@language takes care of switching the language environment *and* of writing entries on the auxiliary files. For historical reasons, language names can be either language of \language. To catch either form a trick is used, but unfortunately as a side effect the catcodes of letters in \language are messed up. This is a bug, but preserved for backwards compatibility. The list of auxiliary files can be extended by redefining \BabelContentsFiles, but make sure they are loaded inside a group (as aux, toc, lof, and lot do) or the last language of the document will remain active afterwards.

We also write a command to change the current language in the auxiliary files.

\bbl@savelastskip is used to deal with skips before the write whatsit (as suggested by U Fischer). Adapted from hyperref, but it might fail, so I'll consider it a temporary hack, while I study other options (the ideal, but very likely unfeasible except perhaps in luatex, is to avoid the \write altogether when not needed).

```

610 \def\BabelContentsFiles{toc,lof,lot}
611 \def\bbl@set@language#1{% from selectlanguage, pop@
612   % The old buggy way. Preserved for compatibility, but simplified
613   \edef\language{\expandafter\string#1\@empty}%
614   \select@language{\language}%
615   \bbl@xin@{,main,},{,\bbl@select@opts,}%
616   \ifin@
617     \let\bbl@main@language\localename
618     \let\mainlocalename\localename
619   \fi
620   \let\bbl@select@opts\@empty
621   % write to aux files
622   \expandafter\ifx\csname date\language\endcsname\relax\else
623     \if@filesw
624       \bbl@xin@{,nofiles,},{,\bbl@select@opts,}%
625       \ifin@\else
626         \ifx\babel@aux\@gobbletwo\else % Set if single in the first, redundant
627           \bbl@savelastskip
628           \protected@write\auxout{}\string\babel@aux{\bbl@auxname}{}}%
629           \bbl@restorelastskip
630         \fi
631         \bbl@usehooks{write}}}%
632       \fi
633     \fi
634   \fi}
635 %
636 \let\bbl@restorelastskip\relax
637 \let\bbl@savelastskip\relax
638 %
639 \def\select@language#1{% from set@, babel@aux, babel@toc
640   \ifx\bbl@select@name\@empty
641     \def\bbl@select@name{select}%
642   \fi
643   % set hymap
644   \ifnum\bbl@hymapsel=\@ccclv\chardef\bbl@hymapsel4\relax\fi
645   % set name (when coming from babel@aux)
646   \edef\language{#1}%
647   \bbl@fixname\language
648   % define \localename when coming from set@, with a trick
649   \ifx\scantokens\@undefined

```

```

650 \def\localename{??}%
651 \else
652 \bbl@exp{\scantokens{\def\localename{\language}\noexpand}\relax}%
653 \fi
654 \bbl@provide@locale
655 \bbl@iflanguage\language{\%
656 \let\bbl@select@type\z@
657 \expandafter\bbl@switch\expandafter{\language}}
658 \def\babel@aux#1#2{%
659 \select@language{#1}%
660 \bbl@foreach\BabelContentsFiles{% \relax -> don't assume vertical mode
661 \@writefile{##1}{\babel@toc{#1}{#2}\relax}}}%
662 \def\babel@toc#1#2{%
663 \select@language{#1}}

```

First, check if the user asks for a known language. If so, update the value of `\language` and call `\originalTeX` to bring `TeX` in a certain pre-defined state.

The name of the language is stored in the control sequence `\language`.

Then we have to *redefine* `\originalTeX` to compensate for the things that have been activated. To save memory space for the macro definition of `\originalTeX`, we construct the control sequence name for the `\noextras{language}` command at definition time by expanding the `\csname` primitive.

Now activate the language-specific definitions. This is done by constructing the names of three macros by concatenating three words with the argument of `\selectlanguage`, and calling these macros.

The switching of the values of `\lefthyphenmin` and `\righthyphenmin` is somewhat different. First we save their current values, then we check if `\languagehyphenmins` is defined. If it is not, we set default values (2 and 3), otherwise the values in `\languagehyphenmins` will be used.

No text is supposed to be added with switching captions and date, so we remove any spurious spaces with `\bbl@bsphack` and `\bbl@esphack`.

```

664 \newif\ifbbl@usedategroup
665 \let\bbl@savextras\empty
666 \def\bbl@switch#1{% from select@, foreign@
667 % restore
668 \originalTeX
669 \expandafter\def\expandafter\originalTeX\expandafter{%
670 \csname noextras#1\endcsname
671 \let\originalTeX\empty
672 \babel@beginsave}%
673 \bbl@usehooks{afterreset}}}%
674 \languageshorthands{none}%
675 % set the locale id
676 \bbl@id@assign
677 % switch captions, date
678 \bbl@bsphack
679 \ifcase\bbl@select@type
680 \csname captions#1\endcsname\relax
681 \csname date#1\endcsname\relax
682 \else
683 \bbl@xin@{,captions,},{, \bbl@select@opts,}%
684 \ifin@
685 \csname captions#1\endcsname\relax
686 \fi
687 \bbl@xin@{,date,},{, \bbl@select@opts,}%
688 \ifin@ % if \foreign... within \language date
689 \csname date#1\endcsname\relax
690 \fi
691 \fi
692 \bbl@esphack
693 % switch extras
694 \csname bbl@preextras@#1\endcsname
695 \bbl@usehooks{beforeextras}}}%
696 \csname extras#1\endcsname\relax
697 \bbl@usehooks{afterextras}}}%

```

```

698 % > babel-ensure
699 % > babel-sh-<short>
700 % > babel-bidi
701 % > babel-fontspec
702 \let\bbbl@savedextras\@empty
703 % hyphenation - case mapping
704 \ifcase\bbbl@opt@hyphenmap\or
705   \def\BabelLower##1##2{\lccode##1=##2\relax}%
706   \ifnum\bbbl@hymapsel>4\else
707     \csname\language\name @bbbl@hyphenmap\endcsname
708   \fi
709   \chardef\bbbl@opt@hyphenmap\z@
710 \else
711   \ifnum\bbbl@hymapsel>\bbbl@opt@hyphenmap\else
712     \csname\language\name @bbbl@hyphenmap\endcsname
713   \fi
714 \fi
715 \let\bbbl@hymapsel\@cclv
716 % hyphenation - select rules
717 \ifnum\csname l@\language\endcsname=\l@unhyphenated
718   \edef\bbbl@tempa{u}%
719 \else
720   \edef\bbbl@tempa{\bbbl@cl{\lnbrk}}%
721 \fi
722 % linebreaking - handle u, e, k (v in the future)
723 \bbbl@xin@{/u}{/\bbbl@tempa}%
724 \ifin@ \else \bbbl@xin@{/e}{/\bbbl@tempa} \fi % elongated forms
725 \ifin@ \else \bbbl@xin@{/k}{/\bbbl@tempa} \fi % only kashida
726 \ifin@ \else \bbbl@xin@{/p}{/\bbbl@tempa} \fi % padding (e.g., Tibetan)
727 \ifin@ \else \bbbl@xin@{/v}{/\bbbl@tempa} \fi % variable font
728 % hyphenation - save mins
729 \babel@savevariable\lefthyphenmin
730 \babel@savevariable\righthyphenmin
731 \ifnum\bbbl@engine=\@ne
732   \babel@savevariable\hyphenationmin
733 \fi
734 \ifin@
735   % unhyphenated/kashida/elongated/padding = allow stretching
736   \language\l@unhyphenated
737   \babel@savevariable\emergencystretch
738   \emergencystretch\maxdimen
739   \babel@savevariable\hbadness
740   \hbadness\@M
741 \else
742   % other = select patterns
743   \bbbl@patterns{#1}%
744 \fi
745 % hyphenation - set mins
746 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
747   \set@hyphenmins\tw@\thr@@\relax
748   \@nameuse{bbbl@hyphenmins@}%
749 \else
750   \expandafter\expandafter\expandafter\set@hyphenmins
751     \csname #1hyphenmins\endcsname\relax
752 \fi
753 \@nameuse{bbbl@hyphenmins@}%
754 \@nameuse{bbbl@hyphenmins@\language\name}%
755 \@nameuse{bbbl@hyphenatmin@}%
756 \@nameuse{bbbl@hyphenatmin@\language\name}%
757 \let\bbbl@selectortname\@empty}

```

otherlanguage It can be used as an alternative to using the `\selectlanguage` declarative command. The `\ignorespaces` command is necessary to hide the environment when it is entered in horizontal

mode.

```
758 \edef\otherlanguage{%
759   \noexpand\protect
760   \expandafter\noexpand\csname otherlanguage \endcsname}
761 \expandafter\def\csname otherlanguage \endcsname{%
762   \@ifstar{\@nameuse{otherlanguage*}}\bbl@otherlanguage}
763 \def\bbl@otherlanguage#1{%
764   \def\bbl@selectorname{other}%
765   \ifnum\bbl@hymapsel=\@cclv\let\bbl@hymapsel\thr@@\fi
766   \csname selectlanguage \endcsname{#1}%
767   \ignorespaces}
```

The `\endotherlanguage` part of the environment tries to hide itself when it is called in horizontal mode.

```
768 \long\def\endotherlanguage{\@ignoretrue\ignorespaces}
```

otherlanguage* It is meant to be used when a large part of text from a different language needs to be typeset, but without changing the translation of words such as ‘figure’. It makes use of `\foreign@language`.

```
769 \expandafter\def\csname otherlanguage*\endcsname{%
770   \@ifnextchar[\bbl@otherlanguage@s{\bbl@otherlanguage@s{}}]
771 \def\bbl@otherlanguage@s[#1]#2{%
772   \def\bbl@selectorname{other*}%
773   \ifnum\bbl@hymapsel=\@cclv\chardef\bbl@hymapsel4\relax\fi
774   \def\bbl@select@opts{#1}%
775   \foreign@language{#2}}
```

At the end of the environment we need to switch off the extra definitions. The grouping mechanism of the environment will take care of resetting the correct hyphenation rules and “extras”.

```
776 \expandafter\let\csname endotherlanguage*\endcsname\relax
```

\foreignlanguage This command takes two arguments, the first argument is the name of the language to use for typesetting the text specified in the second argument.

Unlike `\selectlanguage` this command doesn’t switch *everything*, it only switches the hyphenation rules and the extra definitions for the language specified. It does this within a group and assumes the `\extras<language>` command doesn’t make any \global changes. The coding is very similar to part of `\selectlanguage`.

`\bbl@beforeforeign` is a trick to fix a bug in bidi texts. `\foreignlanguage` is supposed to be a ‘text’ command, and therefore it must emit a `\leavevmode`, but it does not, and therefore the indent is placed on the opposite margin. For backward compatibility, however, it is done only if a right-to-left script is requested; otherwise, it is no-op.

(3.11) `\foreignlanguage*` is a temporary, experimental macro for a few lines with a different script direction, while preserving the paragraph format (thank the braces around `\par`, things like `\hangindent` are not reset). Do not use it in production, because its semantics and its syntax may change (and very likely will, or even it could be removed altogether). Currently it enters in vmode and then selects the language (which in turn sets the paragraph direction).

(3.11) Also experimental are the hook `foreign` and `foreign*`. With them you can redefine `\BabelText` which by default does nothing. Its behavior is not well defined yet. So, use it in horizontal mode only if you do not want surprises.

In other words, at the beginning of a paragraph `\foreignlanguage` enters into hmode with the surrounding lang, and with `\foreignlanguage*` with the new lang.

```
777 \providecommand\bbl@beforeforeign{}
778 \edef\foreignlanguage{%
779   \noexpand\protect
780   \expandafter\noexpand\csname foreignlanguage \endcsname}
781 \expandafter\def\csname foreignlanguage \endcsname{%
782   \@ifstar\bbl@foreign@s\bbl@foreign@x}
783 \providecommand\bbl@foreign@x[3][{}]{%
784   \begingroup
785     \def\bbl@selectorname{foreign}%
786     \def\bbl@select@opts{#1}%
787     \let\BabelText\@firstofone
```

```

788 \bbl@beforeforeign
789 \foreign@language{#2}%
790 \bbl@usehooks{foreign}{}%
791 \BabelText{#3}% Now in horizontal mode!
792 \endgroup}
793 \def\bbl@foreign@s#1#2{%
794 \begingroup
795 {\par}%
796 \def\bbl@select@name{foreign*}%
797 \let\bbl@select@opts\@empty
798 \let\BabelText\@firstofone
799 \foreign@language{#1}%
800 \bbl@usehooks{foreign*}{}%
801 \bbl@dirparastext
802 \BabelText{#2}% Still in vertical mode!
803 {\par}%
804 \endgroup}
805 \providecommand\BabelWrapText[1]{%
806 \def\bbl@tempa{\def\BabelText###1}%
807 \expandafter\bbl@tempa\expandafter{\BabelText{#1}}}

```

\foreign@language This macro does the work for \foreignlanguage and the otherlanguage* environment. First we need to store the name of the language and check that it is a known language. Then it just calls bbl@switch.

```

808 \def\foreign@language#1{%
809 % set name
810 \edef\language#1}%
811 \ifbbl@usedategroup
812 \bbl@add\bbl@select@opts{,date,}%
813 \bbl@usedategroupfalse
814 \fi
815 \bbl@fixname\language
816 \let\localname\language
817 \bbl@provide@locale
818 \bbl@iflanguage\language{%
819 \let\bbl@select@type\@ne
820 \expandafter\bbl@switch\expandafter{\language}}

```

The following macro executes conditionally some code based on the selector being used.

```

821 \def\IfBabelSelectorTF#1{%
822 \bbl@xin@{\bbl@select@name,}{,\zap@space#1 \@empty,}%
823 \ifin@
824 \expandafter\@firstoftwo
825 \else
826 \expandafter\@secondoftwo
827 \fi}

```

\bbl@patterns This macro selects the hyphenation patterns by changing the \language register. If special hyphenation patterns are available specifically for the current font encoding, use them instead of the default.

It also sets hyphenation exceptions, but only once, because they are global (here language \lccode's has been set, too). \bbl@hyphenation@ is set to relax until the very first \babelhyphenation, so do nothing with this value. If the exceptions for a language (by its number, not its name, so that :ENC is taken into account) has been set, then use \hyphenation with both global and language exceptions and empty the latter to mark they must not be set again.

```

828 \let\bbl@hyphlist\@empty
829 \let\bbl@hyphenation@relax
830 \let\bbl@pttnlist\@empty
831 \let\bbl@patterns@relax
832 \let\bbl@hymapsel=\ccclv
833 \def\bbl@patterns#1{%
834 \language=\expandafter\ifx\csname l@#1:\f@encoding\endcsname\relax

```

```

835     \csname l@#1\endcsname
836     \edef\bbl@tempa{#1}%
837     \else
838     \csname l@#1:\f@encoding\endcsname
839     \edef\bbl@tempa{#1:\f@encoding}%
840     \fi
841     \@expandtwoargs\bbl@usehooks{patterns}{{#1}{\bbl@tempa}}%
842     % > luatex
843     \ifundefined{bbl@hyphenation@}{\relax!
844     \begingroup
845     \bbl@xin@{,\number\language,}{,\bbl@hyphlist}%
846     \ifin@ \else
847     \@expandtwoargs\bbl@usehooks{hyphenation}{{#1}{\bbl@tempa}}%
848     \hyphenation%
849     \bbl@hyphenation@
850     \ifundefined{bbl@hyphenation@#1}%
851     \empty
852     {\space\csname bbl@hyphenation@#1\endcsname}}%
853     \xdef\bbl@hyphlist{\bbl@hyphlist\number\language,}%
854     \fi
855     \endgroup}}

```

hyphenrules It can be used to select *just* the hyphenation rules. It does *not* change `\language` and when the hyphenation rules specified were not loaded it has no effect. Note however, `\lccode`'s and font encodings are not set at all, so in most cases you should use `otherlanguage*`.

```

856 \def\hyphenrules#1{%
857   \edef\bbl@tempf{#1}%
858   \bbl@fixname\bbl@tempf
859   \bbl@iflanguage\bbl@tempf{%
860     \expandafter\bbl@patterns\expandafter{\bbl@tempf}%
861     \ifx\languageshorthands\undefined\else
862       \languageshorthands{none}%
863     \fi
864     \expandafter\ifx\csname\bbl@tempf hyphenmins\endcsname\relax
865       \set@hyphenmins\tw@\thr@@\relax
866     \else
867       \expandafter\expandafter\expandafter\set@hyphenmins
868       \csname\bbl@tempf hyphenmins\endcsname\relax
869     \fi}}
870 \let\endhyphenrules\empty

```

\providehyphenmins The macro `\providehyphenmins` should be used in the language definition files to provide a *default* setting for the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`. If the macro `\(language)hyphenmins` is already defined this command has no effect.

```

871 \def\providehyphenmins#1#2{%
872   \expandafter\ifx\csname #1hyphenmins\endcsname\relax
873     \@namedef{#1hyphenmins}{#2}%
874   \fi}

```

\set@hyphenmins This macro sets the values of `\lefthyphenmin` and `\righthyphenmin`. It expects two values as its argument.

```

875 \def\set@hyphenmins#1#2{%
876   \lefthyphenmin#1\relax
877   \righthyphenmin#2\relax}

```

\ProvidesLanguage The identification code for each file is something that was introduced in $\text{\LaTeX 2\epsilon}$. When the command `\ProvidesFile` does not exist, a dummy definition is provided temporarily. For use in the language definition file the command `\ProvidesLanguage` is defined by `babel`.

Depending on the format, i.e., or if the former is defined, we use a similar definition or not.

```

878 \ifx\ProvidesFile\undefined

```

```

879 \def\ProvidesLanguage#1[#2 #3 #4]{%
880   \wlog{Language: #1 #4 #3 <#2>}%
881 }
882 \else
883 \def\ProvidesLanguage#1{%
884   \begingroup
885     \catcode\ 10 %
886     \@makeother\/%
887     \@ifnextchar[%]
888       {\@provideslanguage{#1}}{\@provideslanguage{#1}[]}
889 \def\@provideslanguage#1[#2]{%
890   \wlog{Language: #1 #2}%
891   \expandafter\xdef\csname ver@#1.ldf\endcsname{#2}%
892   \endgroup}
893 \fi

```

originalTeX The macro `\originalTeX` should be known to $\TeX$ at this moment. As it has to be expandable we `\let` it to `\@empty` instead of `\relax`.

```
894 \ifx\originalTeX\@undefined\let\originalTeX\@empty\fi
```

Because this part of the code can be included in a format, we make sure that the macro which initializes the save mechanism, `\babel@beginsave`, is not considered to be undefined.

```
895 \ifx\babel@beginsave\@undefined\let\babel@beginsave\relax\fi
```

A few macro names are reserved for future releases of babel, which will use the concept of ‘locale’:

```

896 \providecommand\setlocale{\bbl@error{not-yet-available}}{}{}
897 \let\uselocale\setlocale
898 \let\locale\setlocale
899 \let\selectlocale\setlocale
900 \let\textlocale\setlocale
901 \let\textlanguage\setlocale
902 \let\languagegettext\setlocale

```

4.2. Errors

\@nolanerr

\@nopatterns The babel package will signal an error when a documents tries to select a language that hasn’t been defined earlier. When a user selects a language for which no hyphenation patterns were loaded into the format he will be given a warning about that fact. We revert to the patterns for `\language=0` in that case. In most formats that will be (US)english, but it might also be empty.

\@noopterr When the package was loaded without options not everything will work as expected. An error message is issued in that case.

When the format knows about `\PackageError` it must be $\LaTeX 2\epsilon$, so we can safely use its error handling interface. Otherwise we’ll have to ‘keep it simple’.

Infos are not written to the console, but on the other hand many people think warnings are errors, so a further message type is defined: an important info which is sent to the console.

```

903 \edef\bbl@nulllanguage{\string\language=0}
904 \def\bbl@nocaption{\protect\bbl@nocaption@i}
905 \def\bbl@nocaption@i#1#2{% 1: text to be printed 2: caption macro \langXname
906   \global\@namedef{#2}{\textbf{?#1?}}%
907   \@nameuse{#2}%
908   \edef\bbl@tempa{#1}%
909   \bbl@sreplace\bbl@tempa{name}}}%
910   \bbl@sreplace\bbl@tempa{NAME}}}%
911   \bbl@warning{%
912     \@backslashchar#1 not set for '\language'. Please,\\%
913     define it after the language has been loaded\\%
914     (typically in the preamble) with:\\%
915     \string\setlocalecaption{\language}{\bbl@tempa}{..}\\%
916     Feel free to contribute on github.com/latex3/babel.\\%
917     Reported}}

```



```

918 \def\bbl@tentative{\protect\bbl@tentative@i}
919 \def\bbl@tentative@i#1{%
920   \bbl@warning{%
921     Some functions for '#1' are tentative.\\%
922     They might not work as expected and their behavior\\%
923     could change in the future.\\%
924     Reported}}
925 \def\@nolanerr#1{\bbl@error{undefined-language}{#1}{}}
926 \def\@nopatterns#1{%
927   \bbl@warning
928     {No hyphenation patterns were preloaded for\\%
929     the language '#1' into the format.\\%
930     Please, configure your TeX system to add them and\\%
931     rebuild the format. Now I will use the patterns\\%
932     preloaded for \bbl@nulllanguage\space instead}}
933 \let\bbl@usehooks\@gobbletwo

Here ended the now discarded switch.def.
Here also (currently) ends the base option.

934 \ifx\bbl@onlyswitch\@empty\endinput\fi

```

4.3. More on selection

\babelensure The user command just parses the optional argument and creates a new macro named `\bbl@e@{language}`. We register a hook at the `afterextras` event which just executes this macro in a “complete” selection (which, if undefined, is `\relax` and does nothing). This part is somewhat involved because we have to make sure things are expanded the correct number of times.

The macro `\bbl@e@{language}` contains `\bbl@ensure{<include>}{<exclude>}{<fontenc>}`, which in turn loops over the macros names in `\bbl@captionslist`, excluding (with the help of `\in@`) those in the `exclude` list. If the `fontenc` is given (and not `\relax`), the `\fontencoding` is also added. Then we loop over the `include` list, but if the macro already contains `\foreignlanguage`, nothing is done. Note this macro (1) is not restricted to the preamble, and (2) changes are local.

```

935 \bbl@trace{Defining babelensure}
936 \newcommand\babelensure[2][]{%
937   \AddBabelHook{babel-ensure}{afterextras}{%
938     \ifcase\bbl@select@type
939       \bbl@cl{e}%
940     \fi}%
941   \begingroup
942     \let\bbl@ens@include\@empty
943     \let\bbl@ens@exclude\@empty
944     \def\bbl@ens@fontenc{\relax}%
945     \def\bbl@tempb##1{%
946       \ifx\@empty##1\else\noexpand##1\expandafter\bbl@tempb\fi}%
947     \edef\bbl@tempa{\bbl@tempb#1\@empty}%
948     \def\bbl@tempb##1=##2\@{\@namedef{\bbl@ens@##1}{##2}}%
949     \bbl@foreach\bbl@tempa{\bbl@tempb##1\@}%
950     \def\bbl@tempc{\bbl@ensure}%
951     \expandafter\bbl@add\expandafter\bbl@tempc\expandafter{%
952       \expandafter{\bbl@ens@include}}%
953     \expandafter\bbl@add\expandafter\bbl@tempc\expandafter{%
954       \expandafter{\bbl@ens@exclude}}%
955     \toks@\expandafter{\bbl@tempc}%
956     \bbl@exp{%
957   \endgroup
958   \def<\bbl@e@#2>{\the\toks@{\bbl@ens@fontenc}}}%
959 \def\bbl@ensure#1#2#3{% 1: include 2: exclude 3: fontenc
960   \def\bbl@tempb##1{% elt for (excluding) \bbl@captionslist list
961     \ifx##1\undefined % 3.32 - Don't assume the macro exists
962       \edef##1{\noexpand\bbl@nocaption
963         {\bbl@stripslash##1}{\language\name\bbl@stripslash##1}}%
964     \fi
965     \ifx##1\@empty\else

```

```

966 \in@{##1}{#2}%
967 \ifin@else
968 \let\bbl@tempc\language
969 \bbl@replace\bbl@tempc{-}{@}%
970 \bbl@ifunset{\bbl@ensure@\bbl@tempc}%r
971 {\bbl@exp{%
972 \\\DeclareRobustCommand\<bbl@ensure@\bbl@tempc>[1]{%
973 \\\foreignlanguage{\language}\language}%
974 {\ifx\relax#3\else
975 \\\fontencoding{#3}\selectfont
976 \fi
977 #####1}}}%
978 }%
979 \toks@{\expandafter{##1}%
980 \edef##1{%
981 \bbl@csarg\noexpand{ensure@\bbl@tempc}%
982 {\the\toks@}}}%
983 \fi
984 \expandafter\bbl@tempb
985 \fi}%
986 \expandafter\bbl@tempb\bbl@captionslist\today\@empty
987 \def\bbl@tempa##1{% elt for include list
988 \ifx##1\@empty\else
989 \let\bbl@tempc\language
990 \bbl@replace\bbl@tempc{-}{@}%
991 \bbl@csarg\in@{ensure@\bbl@tempc\expandafter}\expandafter{##1}%
992 \ifin@else
993 \bbl@tempb##1\@empty
994 \fi
995 \expandafter\bbl@tempa
996 \fi}%
997 \bbl@tempa#1\@empty}
998 \def\bbl@captionslist{%
999 \prefacename\refname\abstractname\bibname\chaptername\appendixname
1000 \contentsname\listfigurename\listtablename\indexname\figurename
1001 \tablename\partname\enclname\ccname\headtoname\pagename\seename
1002 \alsoname\proofname\glossaryname}

```

4.4. Short tags

\babeltags This macro is straightforward. After zapping spaces, we loop over the list and define the macros `\text{<tag>}` and `\<tag>`. Definitions are first expanded so that they don't contain `\csname` but the actual macro.

```

1003 \bbl@trace{Short tags}
1004 \newcommand\babeltags[1]{%
1005 \edef\bbl@tempa{\zap@space#1 \@empty}%
1006 \def\bbl@tempb##1=##2\@{#%
1007 \edef\bbl@tempc{%
1008 \noexpand\newcommand
1009 \expandafter\noexpand\csname ##1\endcsname{%
1010 \noexpand\protect
1011 \expandafter\noexpand\csname otherlanguage*\endcsname{##2}}
1012 \noexpand\newcommand
1013 \expandafter\noexpand\csname text##1\endcsname{%
1014 \noexpand\foreignlanguage{##2}}}%
1015 \bbl@tempc}%
1016 \bbl@for\bbl@tempa\bbl@tempa{%
1017 \expandafter\bbl@tempb\bbl@tempa\@{}}

```

4.5. Compatibility with language.def

Plain e-TeX doesn't rely on language.dat, but babel can be made compatible with this format easily.

```

1018 \bbl@trace{Compatibility with language.def}
1019 \ifx\directlua\@undefined\else
1020   \ifx\bbl@luapatterns\@undefined
1021     \input luabel.def
1022   \fi
1023 \fi
1024 \ifx\bbl@languages\@undefined
1025   \ifx\directlua\@undefined
1026     \openin1 = language.def
1027     \ifeof1
1028       \closein1
1029       \message{I couldn't find the file language.def}
1030     \else
1031       \closein1
1032       \begingroup
1033         \def\addlanguage#1#2#3#4#5{%
1034           \expandafter\ifx\csname lang@#1\endcsname\relax\else
1035             \global\expandafter\let\csname l@#1\endcsname
1036               \csname lang@#1\endcsname
1037           \fi}%
1038         \def\uselanguage#1{%
1039           \input language.def
1040         \endgroup
1041       \fi
1042     \fi
1043   \chardef\l@english\z@
1044 \fi

```

\addto It takes two arguments, a *<control sequence>* and $\TeX$ -code to be added to the *<control sequence>*.

If the *<control sequence>* has not been defined before it is defined now. The control sequence could also expand to `\relax`, in which case a circular definition results. The net result is a stack overflow. Note there is an inconsistency, because the assignment in the last branch is global.

```

1045 \def\addto#1#2{%
1046   \ifx#1\@undefined
1047     \def#1{#2}%
1048   \else
1049     \ifx#1\relax
1050       \def#1{#2}%
1051     \else
1052       {\toks@\expandafter{#1#2}%
1053        \xdef#1{\the\toks@}}%
1054     \fi
1055   \fi}

```

4.6. Hooks

Admittedly, the current implementation is a somewhat simplistic and does very little to catch errors, but it is meant for developers, after all. `\bbl@usehooks` is the commands used by babel to execute hooks defined for an event.

```

1056 \bbl@trace{Hooks}
1057 \newcommand\AddBabelHook[3][[]]{%
1058   \bbl@iifunset\bbl@hk@#2}{\EnableBabelHook{#2}}}%
1059 \def\bbl@tempa##1,#3=##2,##3\@empty{\def\bbl@tempb{##2}}%
1060 \expandafter\bbl@tempa\bbl@evargs,#3=,\@empty
1061 \bbl@iifunset\bbl@ev@#2@#3@#1{%
1062   {\bbl@csarg\bbl@add{ev@#3@#1}{\bbl@elth{#2}}}%
1063   {\bbl@csarg\let{ev@#2@#3@#1}\relax}%
1064   \bbl@csarg\newcommand{ev@#2@#3@#1}{\bbl@tempb}}
1065 \newcommand\EnableBabelHook[1]{\bbl@csarg\let{hk@#1}\@firstofone}
1066 \newcommand\DisableBabelHook[1]{\bbl@csarg\let{hk@#1}\@gobble}
1067 \def\bbl@usehooks{\bbl@usehooks@lang\language}

```

```

1068 \def\bbl@usehooks@lang#1#2#3{% Test for Plain
1069 \ifx\UseHook\@undefined\else\UseHook{babel/*/#2}\fi
1070 \def\bbl@elth##1{%
1071 \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#3}}%
1072 \bbl@cs{ev@#2@}%
1073 \ifx\language\@undefined\else % Test required for Plain (?)
1074 \ifx\UseHook\@undefined\else\UseHook{babel/#1/#2}\fi
1075 \def\bbl@elth##1{%
1076 \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#1#3}}%
1077 \bbl@cs{ev@#2@#1}%
1078 \fi}

```

To ensure forward compatibility, arguments in hooks are set implicitly. So, if a further argument is added in the future, there is no need to change the existing code. Note events intended for hyphen.cfg are also loaded (just in case you need them for some reason).

```

1079 \def\bbl@evargs{,% <- don't delete this comma
1080 everylanguage=1,loadkernel=1,loadpatterns=1,loadexceptions=1,%
1081 adddialect=2,patterns=2,defaultcommands=0,encodedcommands=2,write=0,%
1082 beforeextras=0,afterextras=0,stopcommands=0,stringprocess=0,%
1083 hyphenation=2,initiateactive=3,afterreset=0,foreign=0,foreign*=0,%
1084 beforestart=0,language=2,begindocument=1}
1085 \ifx\NewHook\@undefined\else % Test for Plain (?)
1086 \def\bbl@tempa#1=#2@@{\NewHook{babel/#1}}
1087 \bbl@foreach\bbl@evargs{\bbl@tempa#1\@@}
1088 \fi

```

Since the following command is meant for a hook (although a $\LaTeX$ one), it's placed here.

```

1089 \providecommand\PassOptionsToLocale[2]{%
1090 \bbl@csarg\bbl@add@list{passto@#2}{#1}}

```

4.7. Setting up language files

\LdfInit $\LdfInit$ macro takes two arguments. The first argument is the name of the language that will be defined in the language definition file; the second argument is either a control sequence or a string from which a control sequence should be constructed. The existence of the control sequence indicates that the file has been processed before.

At the start of processing a language definition file we always check the category code of the at-sign. We make sure that it is a 'letter' during the processing of the file. We also save its name as the last called option, even if not loaded.

Another character that needs to have the correct category code during processing of language definition files is the equals sign, '=', because it is sometimes used in constructions with the $\let$ primitive. Therefore we store its current catcode and restore it later on.

Now we check whether we should perhaps stop the processing of this file. To do this we first need to check whether the second argument that is passed to $\LdfInit$ is a control sequence. We do that by looking at the first token after passing #2 through string. When it is equal to $\@backslashchar$ we are dealing with a control sequence which we can compare with $\@undefined$.

If so, we call $\ldf@quit$ to set the main language, restore the category code of the @-sign and call $\endinput$

When #2 was *not* a control sequence we construct one and compare it with $\relax$.

Finally we check $\originalTeX$.

```

1091 \bbl@trace{Macros for setting language files up}
1092 \def\bbl@ldfinit{%
1093 \let\bbl@screset\@empty
1094 \let\BabelStrings\bbl@opt@string
1095 \let\BabelOptions\@empty
1096 \let\BabelLanguages\relax
1097 \ifx\originalTeX\@undefined
1098 \let\originalTeX\@empty
1099 \else
1100 \originalTeX
1101 \fi}
1102 \def\LdfInit#1#2{%
1103 \chardef\atcatcode=\catcode`\@

```

```

1104 \catcode`\@=11\relax
1105 \chardef\eqcatcode=\catcode`\=
1106 \catcode`\==12\relax
1107 \@ifpackagewith{babel}{ensureinfo=off}{}%
1108 {\ifx\InputIfFileExists\@undefined\else
1109   \bbl@ifunset{bbl@lname@#1}%
1110   {\let\bbl@ensuring\@empty % Flag used in babel-serbianc.tex
1111     \def\language{#1}%
1112     \bbl@id@assign
1113     \bbl@load@info{#1}}}%
1114   {}%
1115   \fi}%
1116 \expandafter\if\expandafter\@backslashchar
1117   \expandafter\@car\string#2\@nil
1118   \ifx#2\@undefined\else
1119     \ldf@quit{#1}%
1120     \fi
1121   \else
1122     \expandafter\ifx\csname#2\endcsname\relax\else
1123       \ldf@quit{#1}%
1124       \fi
1125   \fi
1126   \bbl@ldfinit}

```

\ldf@quit This macro interrupts the processing of a language definition file. Remember `\endinput` is not executed immediately, but delayed to the end of the current line in the input file.

```

1127 \def\ldf@quit#1{%
1128   \expandafter\main@language\expandafter{#1}%
1129   \catcode`\@=\atcatcode \let\atcatcode\relax
1130   \catcode`\==\eqcatcode \let\eqcatcode\relax
1131   \endinput}

```

\ldf@finish This macro takes one argument. It is the name of the language that was defined in the language definition file.

We load the local configuration file if one is present, we set the main language (taking into account that the argument might be a control sequence that needs to be expanded) and reset the category code of the `@`-sign.

```

1132 \def\bbl@afterldf{%
1133   \bbl@afterlang
1134   \let\bbl@afterlang\relax
1135   \let\BabelModifiers\relax
1136   \let\bbl@screset\relax}%
1137 \def\ldf@finish#1{%
1138   \loadlocalcfg{#1}%
1139   \bbl@afterldf
1140   \expandafter\main@language\expandafter{#1}%
1141   \catcode`\@=\atcatcode \let\atcatcode\relax
1142   \catcode`\==\eqcatcode \let\eqcatcode\relax}

```

After the preamble of the document the commands `\LdfInit`, `\ldf@quit` and `\ldf@finish` are no longer needed. Therefore they are turned into warning messages in *TeX*.

```

1143 \@onlypreamble\LdfInit
1144 \@onlypreamble\ldf@quit
1145 \@onlypreamble\ldf@finish

```

\main@language

\bbl@main@language This command should be used in the various language definition files. It stores its argument in `\bbl@main@language`; to be used to switch to the correct language at the beginning of the document.

```

1146 \def\main@language#1{%
1147   \def\bbl@main@language{#1}%

```

```

1148 \let\language\bbbl@main@language
1149 \let\locale\bbbl@main@language
1150 \let\mainlocale\bbbl@main@language
1151 \bbbl@id@assign
1152 \ifcase\bbbl@engine\or
1153   \ifx\setattribute\@undefined\else
1154     \setattribute\bbbl@attr@locale\localeid
1155   \fi
1156 \fi
1157 \bbbl@patterns{\language}

```

We also have to make sure that some code gets executed at the beginning of the document, either when the aux file is read or, if it does not exist, when the `\AtBeginDocument` is executed. Languages do not set `\pagedir`, so we set here for the whole document to the main `\bodydir`.

The code written to the aux file attempts to avoid errors if babel is removed from the document.

```

1158 \def\bbbl@beforestart{%
1159   \def\@nolanerr##1{%
1160     \bbbl@carg\chardef{l@##1}\z@
1161     \bbbl@warning{Undefined language '##1' in aux.\\Reported}}%
1162   \bbbl@usehooks{beforestart}{}%
1163   \global\let\bbbl@beforestart\relax
1164 \AtBeginDocument{%
1165   {\@nameuse{bbbl@beforestart}}% Group!
1166   \if@filesw
1167     \providecommand\babel@aux[2]{}%
1168     \immediate\write\@mainaux{unexpanded{%
1169       \providecommand\babel@aux[2]{\global\let\babel@toc\@gobbletwo}}}%
1170     \immediate\write\@mainaux{string\@nameuse{bbbl@beforestart}}}%
1171   \fi
1172   \expandafter\selectlanguage\expandafter{\bbbl@main@language}%
1173   \ifbbbl@single % must go after the line above.
1174     \renewcommand\selectlanguage[1]{}%
1175     \renewcommand\foreignlanguage[2]{#2}%
1176     \global\let\babel@aux\@gobbletwo % Also as flag
1177   \fi}
1178 %
1179 \ifcase\bbbl@engine\or
1180   \AtBeginDocument{\pagedir\bodydir}
1181 \fi

```

A bit of optimization. Select in heads/feet the language only if necessary.

```

1182 \def\select@language@x#1{%
1183   \ifcase\bbbl@select@type
1184     \bbbl@ifsamestring\language\@nameuse{#1}{\select@language{#1}}%
1185   \else
1186     \select@language{#1}%
1187   \fi}

```

4.8. Shorthands

The macro `\initiate@active@char` below takes all the necessary actions to make its argument a shorthand character. The real work is performed once for each character. But first we define a little tool.

```

1188 \bbbl@trace{Shorhands}
1189 \def\bbbl@withactive#1#2{%
1190   \begingroup
1191     \lccode`~=#2\relax
1192     \lowercase{\endgroup#1~}}

```

`\bbbl@add@special` The macro `\bbbl@add@special` is used to add a new character (or single character control sequence) to the macro `\dospecials` (and `\@sanitize` if $\TeX$ is used). It is used only at one place, namely when `\initiate@active@char` is called (which is ignored if the char has been made active before). Because `\@sanitize` can be undefined, we put the definition inside a conditional.

Items are added to the lists without checking its existence or the original catcode. It does not hurt, but should be fixed. It's already done with `\nfss@catcodes`, added in 3.10.

```

1193 \def\bbl@add@special#1{% 1:a macro like \", \?, etc.
1194   \bbl@add\dospecials{\do#1}% test @sanitize = \relax, for back. compat.
1195   \bbl@ifunset{@sanitize}{\bbl@add@sanitize{\@makeother#1}}%
1196   \ifx\nfss@catcodes\undefined\else
1197     \begingroup
1198       \catcode`#1\active
1199       \nfss@catcodes
1200       \ifnum\catcode`#1=\active
1201         \endgroup
1202         \bbl@add\nfss@catcodes{\@makeother#1}%
1203       \else
1204         \endgroup
1205       \fi
1206   \fi}

```

\initiate@active@char A language definition file can call this macro to make a character active. This macro takes one argument, the character that is to be made active. When the character was already active this macro does nothing. Otherwise, this macro defines the control sequence `\normal@char<char>` to expand to the character in its ‘normal state’ and it defines the active character to expand to `\normal@char<char>` by default (`<char>` being the character to be made active). Later its definition can be changed to expand to `\active@char<char>` by calling `\bbl@activate{<char>}`.

For example, to make the double quote character active one could have `\initiate@active@char{"}` in a language definition file. This defines `"` as `\active@prefix "\active@char` (where the first `"` is the character with its original catcode, when the shorthand is created, and `\active@char` is a single token). In protected contexts, it expands to `\protect "` or `\noexpand "` (i.e., with the original `"`); otherwise `\active@char` is executed. This macro in turn expands to `\normal@char` in “safe” contexts (e.g., `\label`), but `\user@active` in normal “unsafe” ones. The latter search a definition in the user, language and system levels, in this order, but if none is found, `\normal@char` is used. However, a deactivated shorthand (with `\bbl@deactivate` defined as `\active@prefix "\normal@char`).

The following macro is used to define shorthands in the three levels. It takes 4 arguments: the (string’ed) character, `\<level>@group`, `\<level>@active` and `\<next-level>@active` (except in system).

```

1207 \def\bbl@active@def#1#2#3#4{%
1208   \@namedef{#3#1}{%
1209     \expandafter\ifx\csname#2@sh@#1\endcsname\relax
1210       \bbl@afterelse\bbl@sh@select#2#1{#3@arg#1}{#4#1}%
1211     \else
1212       \bbl@afterfi\csname#2@sh@#1\endcsname
1213     \fi}%

```

When there is also no current-level shorthand with an argument we will check whether there is a next-level defined shorthand for this active character.

```

1214   \long\@namedef{#3@arg#1}##1{%
1215     \expandafter\ifx\csname#2@sh@#1\string##1\endcsname\relax
1216       \bbl@afterelse\csname#4#1\endcsname##1%
1217     \else
1218       \bbl@afterfi\csname#2@sh@#1\string##1\endcsname
1219     \fi}}%

```

`\initiate@active@char` calls `\@initiate@active@char` with 3 arguments. All of them are the same character with different catcodes: active, other (`\string’ed`) and the original one. This trick simplifies the code a lot.

```

1220 \def\initiate@active@char#1{%
1221   \bbl@ifunset{active@char\string#1}%
1222   {\bbl@withactive
1223     {\expandafter\@initiate@active@char\expandafter}#1\string#1#1}%
1224   {}}

```

The very first thing to do is saving the original catcode and the original definition, even if not active, which is possible (undefined characters require a special treatment to avoid making them `\relax` and preserving some degree of protection).

```

1225 \def\@initiate@active@char#1#2#3{%
1226   \bbl@csarg\edef{oricat@#2}{\catcode`#2=\the\catcode`#2\relax}%
1227   \ifx#1\@undefined
1228     \bbl@csarg\def{oridef@#2}{\def#1{\active@prefix#1\@undefined}}%
1229   \else
1230     \bbl@csarg\let{oridef@#2}#1%
1231     \bbl@csarg\edef{oridef@#2}{%
1232       \let\noexpand#1%
1233       \expandafter\noexpand\csname bbl@oridef@#2\endcsname}%
1234   \fi

```

If the character is already active we provide the default expansion under this shorthand mechanism. Otherwise we write a message in the transcript file, and define `\normal@char⟨char⟩` to expand to the character in its default state. If the character is mathematically active when babel is loaded (for example `'`) the normal expansion is somewhat different to avoid an infinite loop (but it does not prevent the loop if the mathcode is set to `"8000 a posteriori`).

```

1235   \ifx#1#3\relax
1236     \expandafter\let\csname normal@char#2\endcsname#3%
1237   \else
1238     \bbl@info{Making #2 an active character}%
1239     \ifnum\mathcode`#2=\ifodd\bbl@engine"1000000 \else"8000 \fi
1240     \@namedef{normal@char#2}{%
1241       \textormath{#3}{\csname bbl@oridef@#2\endcsname}}%
1242   \else
1243     \@namedef{normal@char#2}{#3}%
1244   \fi

```

To prevent problems with the loading of other packages after babel we reset the catcode of the character to the original one at the end of the package and of each language file (except with `KeepShorthandsActive`). It is re-activate again at `\begin{document}`. We also need to make sure that the shorthands are active during the processing of the aux file. Otherwise some citations may give unexpected results in the printout when a shorthand was used in the optional argument of `\bibitem` for example. Then we make it active (not strictly necessary, but done for backward compatibility).

```

1245   \bbl@restoreactive{#2}%
1246   \AtBeginDocument{%
1247     \catcode`#2\active
1248     \if@filesw
1249       \immediate\write\@mainaux{\catcode`\string#2\active}%
1250     \fi}%
1251   \expandafter\bbl@add@special\csname#2\endcsname
1252   \catcode`#2\active
1253   \fi

```

Now we have set `\normal@char⟨char⟩`, we must define `\active@char⟨char⟩`, to be executed when the character is activated. We define the first level expansion of `\active@char⟨char⟩` to check the status of the `@safe@actives` flag. If it is set to true we expand to the ‘normal’ version of this character; otherwise we call `\user@active⟨char⟩` to start the search of a definition in the user, language and system levels (or eventually `normal@char⟨char⟩`).

```

1254   \let\bbl@tempa\@firstoftwo
1255   \if\string^#2%
1256     \def\bbl@tempa{\noexpand\textormath}%
1257   \else
1258     \ifx\bbl@mathnormal\@undefined\else
1259       \let\bbl@tempa\bbl@mathnormal
1260     \fi
1261   \fi
1262   \expandafter\edef\csname active@char#2\endcsname{%
1263     \bbl@tempa
1264     {\noexpand\if@safe@actives
1265       \noexpand\expandafter
1266       \expandafter\noexpand\csname normal@char#2\endcsname
1267     \noexpand\else
1268       \noexpand\expandafter
1269       \expandafter\noexpand\csname bbl@doactive#2\endcsname

```



```

1270 \noexpand\fi}%
1271 {\expandafter\noexpand\csname normal@char#2\endcsname}}}%
1272 \bbl@csarg\edef{doactive#2}{%
1273 \expandafter\noexpand\csname user@active#2\endcsname}%

```

We now define the default values which the shorthand is set to when activated or deactivated. It is set to the deactivated form (globally), so that the character expands to

`\active@prefix <char> \normal@char<char>`

(where `\active@char<char>` is *one* control sequence!).

```

1274 \bbl@csarg\edef{active@#2}{%
1275 \noexpand\active@prefix\noexpand#1%
1276 \expandafter\noexpand\csname active@char#2\endcsname}%
1277 \bbl@csarg\edef{normal@#2}{%
1278 \noexpand\active@prefix\noexpand#1%
1279 \expandafter\noexpand\csname normal@char#2\endcsname}%
1280 \bbl@ncarg\let#1\bbl@normal@#2}%

```

The next level of the code checks whether a user has defined a shorthand for himself with this character. First we check for a single character shorthand. If that doesn't exist we check for a shorthand with an argument.

```

1281 \bbl@active@def#2\user@group{user@active}{language@active}%
1282 \bbl@active@def#2\language@group{language@active}{system@active}%
1283 \bbl@active@def#2\system@group{system@active}{normal@char}%

```

In order to do the right thing when a shorthand with an argument is used by itself at the end of the line we provide a definition for the case of an empty argument. For that case we let the shorthand character expand to its non-active self. Also, When a shorthand combination such as `' '` ends up in a heading $\TeX$ would see `\protect'\protect'`. To prevent this from happening a couple of shorthand needs to be defined at user level.

```

1284 \expandafter\edef\csname\user@group @sh@#2@\endcsname
1285 {\expandafter\noexpand\csname normal@char#2\endcsname}%
1286 \expandafter\edef\csname\user@group @sh@#2@\string\protect\endcsname
1287 {\expandafter\noexpand\csname user@active#2\endcsname}%

```

Finally, a couple of special cases are taken care of. (1) If we are making the right quote (`'`) active we need to change `\prim@s` as well. Also, make sure that a single `'` in math mode ‘does the right thing’. (2) If we are using the caret (`^`) as a shorthand character special care should be taken to make sure math still works. Therefore an extra level of expansion is introduced with a check for math mode on the upper level.

```

1288 \if\string'#2%
1289 \let\prim@s\bbl@prim@s
1290 \let\active@math@prime#1%
1291 \fi
1292 \bbl@usehooks{initiateactive}{{#1}{#2}{#3}}

```

The following package options control the behavior of shorthands in math mode.

```

1293 << *More package options >> ≡
1294 \DeclareOption{math=active}{}
1295 \DeclareOption{math=normal}{\def\bbl@mathnormal{\noexpand\textormath}}
1296 << /More package options >>

```

Initiating a shorthand makes active the char. That is not strictly necessary but it is still done for backward compatibility. So we need to restore the original catcode at the end of package *and* the end of the *ldf*.

```

1297 \ifpackagewith{babel}{KeepShorthandsActive}%
1298 {\let\bbl@restoreactive@gobble}%
1299 {\def\bbl@restoreactive#1{%
1300 \bbl@exp{%
1301 \\\AfterBabelLanguage\\CurrentOption
1302 {\catcode`#1=\the\catcode`#1\relax}%
1303 \\\AtEndOfPackage
1304 {\catcode`#1=\the\catcode`#1\relax}}}%
1305 \AtEndOfPackage{\let\bbl@restoreactive@gobble}}

```

\bbl@sh@select This command helps the shorthand supporting macros to select how to proceed.

Note that this macro needs to be expandable as do all the shorthand macros in order for them to work in expansion-only environments such as the argument of \hyphenation.

This macro expects the name of a group of shorthands in its first argument and a shorthand character in its second argument. It will expand to either \bbl@firstcs or \bbl@scndcs. Hence two more arguments need to follow it.

```
1306 \def\bbl@sh@select#1#2{%
1307   \expandafter\ifx\csname#1@sh@#2@sel\endcsname\relax
1308     \bbl@afterelse\bbl@scndcs
1309   \else
1310     \bbl@afterfi\csname#1@sh@#2@sel\endcsname
1311   \fi}
```

\active@prefix Used in the expansion of active characters has a function similar to \OT1-cmd in that it \protects the active character whenever \protect is *not* \@typeset@protect. The \@gobble is needed to remove a token such as \activechar: (when the double colon was the active character to be dealt with). There are two definitions, depending of \ifincsname is available. If there is, the expansion will be more robust.

```
1312 \begingroup
1313 \bbl@ifunset{ifincsname}
1314 {\gdef\active@prefix#1{%
1315   \ifx\protect\@typeset@protect
1316   \else
1317     \ifx\protect\@unexpandable@protect
1318       \noexpand#1%
1319     \else
1320       \protect#1%
1321     \fi
1322   \expandafter\@gobble
1323   \fi}}
1324 {\gdef\active@prefix#1{%
1325   \ifincsname
1326     \string#1%
1327     \expandafter\@gobble
1328   \else
1329     \ifx\protect\@typeset@protect
1330     \else
1331       \ifx\protect\@unexpandable@protect
1332         \noexpand#1%
1333       \else
1334         \protect#1%
1335       \fi
1336     \expandafter\expandafter\expandafter\@gobble
1337     \fi
1338   \fi}}
1339 \endgroup
```

\if@safe@actives In some circumstances it is necessary to be able to reset the shorthand to its ‘normal’ value (usually the character with catcode ‘other’) on the fly. For this purpose the switch @safe@actives is available. The setting of this switch should be checked in the first level expansion of \active@char<char>. When this expansion mode is active (with \@safe@activetrue), something like "13"13 becomes "12"12 in an \edef (in other words, shorthands are \string'ed). This contrasts with \protected@edef, where catcodes are always left unchanged. Once converted, they can be used safely even after this expansion mode is deactivated (with \@safe@activefalse).

```
1340 \newif\if@safe@actives
1341 \@safe@activesfalse
```

\bbl@restore@actives When the output routine kicks in while the active characters were made “safe” this must be undone in the headers to prevent unexpected typeset results. For this situation we define a command to make them “unsafe” again.

```
1342 \def\bbl@restore@actives{\if@safe@actives\@safe@activefalse\fi}
```

\bbl@activate

\bbl@deactivate Both macros take one argument, like `\initiate@active@char`. The macro is used to change the definition of an active character to expand to `\active@char<char>` in the case of `\bbl@activate`, or `\normal@char<char>` in the case of `\bbl@deactivate`.

```
1343 \chardef\bbl@activated\z@
1344 \def\bbl@activate#1{%
1345   \chardef\bbl@activated\@ne
1346   \bbl@withactive{\expandafter\let\expandafter}#1%
1347   \csname bbl@active@string#1\endcsname}
1348 \def\bbl@deactivate#1{%
1349   \chardef\bbl@activated\tw@
1350   \bbl@withactive{\expandafter\let\expandafter}#1%
1351   \csname bbl@normal@string#1\endcsname}
```

\bbl@firstcs

\bbl@scndcs These macros are used only as a trick when declaring shorthands.

```
1352 \def\bbl@firstcs#1#2{\csname#1\endcsname}
1353 \def\bbl@scndcs#1#2{\csname#2\endcsname}
```

\declare@shorthand Used to declare a shorthand on a certain level. It takes three arguments:

1. a name for the collection of shorthands, i.e., ‘system’, or ‘dutch’;
2. the character (sequence) that makes up the shorthand, i.e., ~ or "a;
3. the code to be executed when the shorthand is encountered.

The auxiliary macro `\babel@texpdf` improves the interoperativity with `hyperref` and takes 4 arguments: (1) The $\TeX$ code in text mode, (2) the string for `hyperref`, (3) the $\TeX$ code in math mode, and (4), which is currently ignored, but it's meant for a string in math mode, like a minus sign instead of an hyphen (currently `hyperref` doesn't discriminate the mode). This macro may be used in `ldf` files.

```
1354 \def\babel@texpdf#1#2#3#4{%
1355   \ifx\texorpdfstring\undefined
1356     \textormath{#1}{#3}%
1357   \else
1358     \texorpdfstring{\textormath{#1}{#3}}{#2}%
1359     % \texorpdfstring{\textormath{#1}{#3}}{\textormath{#2}{#4}}%
1360   \fi}
1361 %
1362 \def\declare@shorthand#1#2{\@decl@short{#1}#2\@nil}
1363 \def\@decl@short#1#2#3\@nil#4{%
1364   \def\bbl@tempa{#3}%
1365   \ifx\bbl@tempa\@empty
1366     \expandafter\let\csname #1@sh@string#2@sel\endcsname\bbl@scndcs
1367     \bbl@ifunset{#1@sh@string#2@}{}%
1368     {\def\bbl@tempa{#4}%
1369      \expandafter\ifx\csname#1@sh@string#2@endcsname\bbl@tempa
1370       \else
1371         \bbl@info
1372           {Redefining #1 shorthand \string#2\}%
1373           in language \CurrentOption}%
1374     \fi}%
1375   \@namedef{#1@sh@string#2@}{#4}%
1376 \else
1377   \expandafter\let\csname #1@sh@string#2@sel\endcsname\bbl@firstcs
1378   \bbl@ifunset{#1@sh@string#2@string#3@}{}%
1379   {\def\bbl@tempa{#4}%
1380    \expandafter\ifx\csname#1@sh@string#2@string#3@endcsname\bbl@tempa
1381     \else
1382       \bbl@info
1383         {Redefining #1 shorthand \string#2\string#3\}%
1384         in language \CurrentOption}%
1385     \fi}%
1386   \@namedef{#1@sh@string#2@string#3@}{#4}%
1387 \fi}
```

\textormath Some of the shorthands that will be declared by the language definition files have to be usable in both text and mathmode. To achieve this the helper macro `\textormath` is provided.

```
1388 \def\textormath{%
1389   \ifmmode
1390     \expandafter\@secondoftwo
1391   \else
1392     \expandafter\@firstoftwo
1393   \fi}
```

\user@group

\language@group

\system@group The current concept of ‘shorthands’ supports three levels or groups of shorthands. For each level the name of the level or group is stored in a macro. The default is to have a user group; use language group ‘english’ and have a system group called ‘system’.

```
1394 \def\user@group{user}
1395 \def\language@group{english}
1396 \def\system@group{system}
```

\usesshorthands This is the user level macro. It initializes and activates the character for use as a shorthand character (i.e., it’s active in the preamble). Languages can deactivate shorthands, so a starred version is also provided which activates them always after the language has been switched.

```
1397 \def\usesshorthands{%
1398   \ifstar\bbl@usesesh@s{\bbl@usesesh@x{}}
1399 \def\bbl@usesesh@s#1{%
1400   \bbl@usesesh@x
1401   {\AddBabelHook{babel-sh-\string#1}{afterextras}{\bbl@activate{#1}}}%
1402   {#1}}
1403 \def\bbl@usesesh@x#1#2{%
1404   \bbl@ifshorthand{#2}%
1405   {\def\user@group{user}%
1406    \initiate@active@char{#2}%
1407    #1%
1408    \bbl@activate{#2}}%
1409   {\bbl@error{shorthand-is-off}{#2}}}
```

\defineshorthand Currently we only support two groups of user level shorthands, named internally user and user@(*language*) (language-dependent user shorthands). By default, only the first one is taken into account, but if the former is also used (in the optional argument of `\defineshorthand`) a new level is inserted for it (user@generic, done by `\bbl@set@user@generic`); we make also sure {} and \protect are taken into account in this new top level.

```
1410 \def\user@language@group{user@\language@group}
1411 \def\bbl@set@user@generic#1#2{%
1412   \bbl@ifunset{user@generic@active#1}%
1413   {\bbl@active@def#1\user@language@group{user@active}{user@generic@active}%
1414    \bbl@active@def#1\user@group{user@generic@active}{\language@active}%
1415    \expandafter\edef\csname#2@sh@#1@\endcsname{%
1416      \expandafter\noexpand\csname normal@char#1\endcsname}%
1417    \expandafter\edef\csname#2@sh@#1@\string\protect@\endcsname{%
1418      \expandafter\noexpand\csname user@active#1\endcsname}}%
1419   \@empty}
1420 \newcommand\defineshorthand[3][user]{%
1421   \edef\bbl@tempa{\zap@space#1 \@empty}%
1422   \bbl@for\bbl@tempb\bbl@tempa{%
1423     \if*\expandafter\@car\bbl@tempb\@nil
1424     \edef\bbl@tempb{user@\expandafter\@gobble\bbl@tempb}%
1425     \@expandtwoargs
1426     \bbl@set@user@generic{\expandafter\string\@car#2\@nil}\bbl@tempb
1427   \fi
1428   \declare@shorthand{\bbl@tempb}{#2}{#3}}
```

\languageshorthands A user level command to change the language from which shorthands are used. Unfortunately, babel currently does not keep track of defined groups, and therefore there is no way to catch a possible change in casing to fix it in the same way languages names are fixed.

```

1429 \def\languageshorthands#1{%
1430   \bbl@ifsamestring{none}{#1}{}%
1431   \bbl@once{short-\localename-#1}{%
1432     \bbl@info{'\localename' activates '#1' shorthands.\Reported}}}%
1433   \def\language@group{#1}}

```

\aliasshorthand *Deprecated.* First the new shorthand needs to be initialized. Then, we define the new shorthand in terms of the original one, but note with `\aliasshorthands{"}{/}` is `\active@prefix /\active@char/`, so we still need to let the latter to `\active@char`.

```

1434 \def\aliasshorthand#1#2{%
1435   \bbl@ifshorthand{#2}%
1436   {\expandafter\ifx\csname active@char\string#2\endcsname\relax
1437     \ifx\document\@notprerr
1438       \@notshorthand{#2}%
1439     \else
1440       \initiate@active@char{#2}%
1441       \bbl@ccarg\let{active@char\string#2}{active@char\string#1}%
1442       \bbl@ccarg\let{normal@char\string#2}{normal@char\string#1}%
1443       \bbl@activate{#2}%
1444     \fi
1445   \fi}%
1446   {\bbl@error{shorthand-is-off}{#2}{}}}

```

\@notshorthand

```

1447 \def\@notshorthand#1{\bbl@error{not-a-shorthand}{#1}{}}

```

\shorthandon

\shorthandoff The first level definition of these macros just passes the argument on to `\bbl@switch@sh`, adding `\@nil` at the end to denote the end of the list of characters.

```

1448 \newcommand*\shorthandon[1]{\bbl@switch@sh\@one#1\@nnil}
1449 \DeclareRobustCommand*\shorthandoff{%
1450   \@ifstar{\bbl@shorthandoff\tw@}{\bbl@shorthandoff\z@}}
1451 \def\bbl@shorthandoff#1#2{\bbl@switch@sh#1#2\@nnil}

```

\bbl@switch@sh The macro `\bbl@switch@sh` takes the list of characters apart one by one and subsequently switches the category code of the shorthand character according to the first argument of `\bbl@switch@sh`.

But before any of this switching takes place we make sure that the character we are dealing with is known as a shorthand character. If it is, a macro such as `\active@char` should exist.

Switching off and on is easy – we just set the category code to ‘other’ (12) and `\active`. With the starred version, the original catcode and the original definition, saved in `@initiate@active@char`, are restored.

```

1452 \def\bbl@switch@sh#1#2{%
1453   \ifx#2\@nnil\else
1454     \bbl@ifunset{\bbl@active@\string#2}%
1455     {\bbl@error{not-a-shorthand-b}{#2}{}}%
1456     {\ifcase#1%   off, on, off*
1457       \catcode`#2\relax
1458     \or
1459       \catcode`#2\active
1460       \bbl@ifunset{\bbl@shdef@\string#2}%
1461       {}%
1462       {\bbl@withactive{\expandafter\let\expandafter}#2%
1463         \csname bbl@shdef@\string#2\endcsname
1464         \bbl@csarg\let{shdef@\string#2}\relax}%
1465       \ifcase\bbl@activated\or
1466         \bbl@activate{#2}%

```

```

1467         \else
1468         \bbl@deactivate{#2}%
1469     \fi
1470 \or
1471     \bbl@ifunset{bbl@shdef@\string#2}%
1472     {\bbl@withactive{\bbl@csarg\let{shdef@\string#2}}{#2}%
1473     }%
1474     \csname bbl@oricat@\string#2\endcsname
1475     \csname bbl@oridef@\string#2\endcsname
1476 \fi}%
1477 \bbl@afterfi\bbl@switch@sh#1%
1478 \fi}

```

Note the value is that at the expansion time; e.g., in the preamble shorthands are usually deactivated.

```

1479 \def\babelshorthand{\active@prefix\babelshorthand\bbl@putsh}
1480 \def\bbl@putsh#1{%
1481     \bbl@ifunset{bbl@active@\string#1}%
1482     {\bbl@putsh@i#1\@empty\@nnil}%
1483     {\csname bbl@active@\string#1\endcsname}}
1484 \def\bbl@putsh@i#1#2\@nnil{%
1485     \csname\language@group @sh@\string#1@%
1486     \ifx\@empty#2\else\string#2\fi\endcsname}
1487 %
1488 \ifx\bbl@opt@shorthands\@nnil\else
1489     \let\bbl@s@initiate@active@char\initiate@active@char
1490     \def\initiate@active@char#1{%
1491         \bbl@ifshorthand{#1}{\bbl@s@initiate@active@char{#1}}{}}
1492     \let\bbl@s@switch@sh\bbl@switch@sh
1493     \def\bbl@switch@sh#1#2{%
1494         \ifx#2\@nnil\else
1495             \bbl@afterfi
1496             \bbl@ifshorthand{#2}{\bbl@s@switch@sh#1{#2}}{\bbl@switch@sh#1}%
1497         \fi}
1498     \let\bbl@s@activate\bbl@activate
1499     \def\bbl@activate#1{%
1500         \bbl@ifshorthand{#1}{\bbl@s@activate{#1}}{}}
1501     \let\bbl@s@deactivate\bbl@deactivate
1502     \def\bbl@deactivate#1{%
1503         \bbl@ifshorthand{#1}{\bbl@s@deactivate{#1}}{}}
1504 \fi

```

You may want to test if a character is a shorthand. Note it does not test whether the shorthand is on or off.

```

1505 \newcommand\ifbabelshorthand[3]{\bbl@ifunset{bbl@active@\string#1}{#3}{#2}}

```

\bbl@prim@s

\bbl@pr@m@s One of the internal macros that are involved in substituting \prime for each right quote in mathmode is \prim@s. This checks if the next character is a right quote. When the right quote is active, the definition of this macro needs to be adapted to look also for an active right quote; the hat could be active, too.

```

1506 \def\bbl@prim@s{%
1507     \prime\futurelet\@let@token\bbl@pr@m@s}
1508 \def\bbl@if@primes#1#2{%
1509     \ifx#1\@let@token
1510         \expandafter\@firstoftwo
1511     \else\ifx#2\@let@token
1512         \bbl@afterelse\expandafter\@firstoftwo
1513     \else
1514         \bbl@afterfi\expandafter\@secondoftwo
1515     \fi\fi}
1516 \begingroup
1517 \catcode`\^=7 \catcode`\*=\active \lccode`\*='\^

```

```

1518 \catcode`\'=12 \catcode`\="=\active \lccode`\="=\`
1519 \lowercase{%
1520 \gdef\bbl@pr@ms{%
1521 \bbl@if@primes"%
1522 \pr@@s
1523 {\bbl@if@primes*^{\pr@@t\egroup}}
1524 \endgroup

```

Usually the ~ is active and expands to `\penalty\M\`. When it is written to the aux file it is written expanded. To prevent that and to be able to use the character ~ as a start character for a shorthand, it is redefined here as a one character shorthand on system level. The system declaration is in most cases redundant (when ~ is still a non-break space), and in some cases is inconvenient (if ~ has been redefined); however, for backward compatibility it is maintained (some existing documents may rely on the babel value).

```

1525 \initiate@active@char{~}
1526 \declare@shorthand{system}{~}{\leavevmode\nobreak\ }
1527 \bbl@activate{~}

```

\OT1dqpos

\T1dqpos The position of the double quote character is different for the OT1 and T1 encodings. It will later be selected using the `\f@encoding` macro. Therefore we define two macros here to store the position of the character in these encodings.

```

1528 \expandafter\def\csname OT1dqpos\endcsname{127}
1529 \expandafter\def\csname T1dqpos\endcsname{4}

```

When the macro `\f@encoding` is undefined (as it is in plain $\TeX$) we define it here to expand to OT1

```

1530 \ifx\f@encoding\undefined
1531 \def\f@encoding{OT1}
1532 \fi

```

4.9. Language attributes

Language attributes provide a means to give the user control over which features of the language definition files he wants to enable.

\languageattribute The macro `\languageattribute` checks whether its arguments are valid and then activates the selected language attribute. First check whether the language is known, and then process each attribute in the list.

```

1533 \bbl@trace{Language attributes}
1534 \newcommand\languageattribute[2]{%
1535 \def\bbl@tempc{#1}%
1536 \bbl@fixname\bbl@tempc
1537 \bbl@iflanguage\bbl@tempc{%
1538 \bbl@vforeach{#2}{%

```

To make sure each attribute is selected only once, we store the already selected attributes in `\bbl@known@attrs`. When that control sequence is not yet defined this attribute is certainly not selected before.

```

1539 \ifx\bbl@known@attrs\undefined
1540 \in@false
1541 \else
1542 \bbl@xin@{\bbl@tempc-##1,}{,\bbl@known@attrs,}%
1543 \fi
1544 \ifin@
1545 \bbl@warning{%
1546 You have more than once selected the attribute '##1'\%
1547 for language #1. Reported}%
1548 \else

```

When we end up here the attribute is not selected before. So, we add it to the list of selected attributes and execute the associated $\TeX$ -code.

```

1549 \bbl@info{Activated '##1' attribute for\\%

```

```

1550      '\bbl@tempc'. Reported}%
1551      \bbl@exp{%
1552      \\bbl@add@list\\bbl@known@attribs{\bbl@tempc-##1}}%
1553      \edef\bbl@tempa{\bbl@tempc-##1}%
1554      \expandafter\bbl@ifknown@ttrib\expandafter{\bbl@tempa}\bbl@attributes%
1555      {\csname\bbl@tempc @attr##1\endcsname}%
1556      {\@attrerr{\bbl@tempc}{##1}}%
1557      \fi}}
1558 \onlypreamble\languageattribute

The error text to be issued when an unknown attribute is selected.

1559 \newcommand*{\@attrerr}[2]{%
1560   \bbl@error{unknown-attribute}{#1}{#2}{}}

```

\bbl@declare@ttribute This command adds the new language/attribute combination to the list of known attributes.

Then it defines a control sequence to be executed when the attribute is used in a document. The result of this should be that the macro `\extras...` for the current language is extended, otherwise the attribute will not work as its code is removed from memory at `\begin{document}`.

```

1561 \def\bbl@declare@ttribute#1#2#3{%
1562   \bbl@xin@{,#2,}{,\BabelModifiers,}%
1563   \ifin@
1564     \AfterBabelLanguage{#1}{\languageattribute{#1}{#2}}%
1565   \fi
1566   \bbl@add@list\bbl@attributes{#1-#2}%
1567   \expandafter\def\csname#1@attr#2\endcsname{#3}}

```

\bbl@ifattributeset This internal macro has 4 arguments. It can be used to interpret $\TeX$ code based on whether a certain attribute was set. This command should appear inside the argument to `\AtBeginDocument` because the attributes are set in the document preamble, *after* babel is loaded.

The first argument is the language, the second argument the attribute being checked, and the third and fourth arguments are the true and false clauses.

```

1568 \def\bbl@ifattributeset#1#2#3#4{%
1569   \ifx\bbl@known@attribs\@undefined
1570     \in@false
1571   \else
1572     \bbl@xin@{,#1-#2,}{,\bbl@known@attribs,}%
1573   \fi
1574   \ifin@
1575     \bbl@afterelse#3%
1576   \else
1577     \bbl@afterfi#4%
1578   \fi}

```

\bbl@ifknown@ttrib An internal macro to check whether a given language/attribute is known. The macro takes 4 arguments, the language/attribute, the attribute list, the $\TeX$ -code to be executed when the attribute is known and the $\TeX$ -code to be executed otherwise.

We first assume the attribute is unknown. Then we loop over the list of known attributes, trying to find a match.

```

1579 \def\bbl@ifknown@ttrib#1#2{%
1580   \let\bbl@tempa\@secondoftwo
1581   \bbl@loopx\bbl@tempb{#2}{%
1582     \expandafter\in@\expandafter{\expandafter,\bbl@tempb,}{,#1,}%
1583     \ifin@
1584       \let\bbl@tempa\@firstoftwo
1585     \else
1586       \fi}%
1587   \bbl@tempa}

```


\bbl@clear@ttribs This macro removes all the attribute code from $\TeX$'s memory at `\begin{document}` time (if any is present).

```

1588 \def\bbl@clear@ttribs{%
1589   \ifx\bbl@attributes\undefined\else
1590     \bbl@loopx\bbl@tempa{\bbl@attributes}{%
1591       \expandafter\bbl@clear@ttrib\bbl@tempa.}%
1592     \let\bbl@attributes\undefined
1593   \fi}
1594 \def\bbl@clear@ttrib#1-#2.{%
1595   \expandafter\let\csname#1@attr@#2\endcsname\undefined}
1596 \AtBeginDocument{\bbl@clear@ttribs}

```

4.10. Support for saving and redefining macros

To save the meaning of control sequences using `\babel@save`, we use temporary control sequences. To save hash table entries for these control sequences, we don't use the name of the control sequence to be saved to construct the temporary name. Instead we simply use the value of a counter, which is reset to zero each time we begin to save new values. This works well because we release the saved meanings before we begin to save a new set of control sequence meanings (see `\selectlanguage` and `\originalTeX`). Note undefined macros are not undefined any more when saved – they are `\relax`'ed.

\babel@savecnt

\babel@beginsave The initialization of a new save cycle: reset the counter to zero.

```

1597 \bbl@trace{Macros for saving definitions}
1598 \def\babel@beginsave{\babel@savecnt\z@}

```

Before it's forgotten, allocate the counter and initialize all.

```

1599 \newcount\babel@savecnt
1600 \babel@beginsave

```

\babel@save

\babel@savevariable The macro `\babel@save<csname>` saves the current meaning of the control sequence `<csname>` to `\originalTeX` (which has to be expandable, i.e., you shouldn't let it to `\relax`). To do this, we let the current meaning to a temporary control sequence, the restore commands are appended to `\originalTeX` and the counter is incremented. The macro `\babel@savevariable<variable>` saves the value of the variable. `<variable>` can be anything allowed after the `\the` primitive. To avoid messing saved definitions up, they are saved only the very first time.

```

1601 \def\babel@save#1{%
1602   \def\bbl@tempa{,{#1,}}% Clumsy, for Plain
1603   \expandafter\bbl@add\expandafter\bbl@tempa\expandafter{%
1604     \expandafter{\expandafter,\bbl@savextrs,}}%
1605   \expandafter\in@\bbl@tempa
1606   \ifin@else
1607     \bbl@add\bbl@savextrs{,{#1,}}%
1608     \bbl@carg\let\babel@number\babel@savecnt\#1\relax
1609     \toks@{\expandafter{\originalTeX\let#1=}}%
1610     \bbl@exp{%
1611       \def\\originalTeX{\the\toks@<\babel@number\babel@savecnt>\relax}}%
1612     \advance\babel@savecnt\@ne
1613   \fi}
1614 \def\babel@savevariable#1{%
1615   \toks@{\expandafter{\originalTeX #1=}}%
1616   \bbl@exp{\def\\originalTeX{\the\toks@\\the#1\relax}}}

```

\bbl@redefine To redefine a command, we save the old meaning of the macro. Then we redefine it to call the original macro with the 'sanitized' argument. The reason why we do it this way is that we don't want to redefine the $\TeX$ macros completely in case their definitions change (they have changed in the past). A macro named `\macro` will be saved new control sequences named `\org@macro`.

```

1617 \def\bb@l@redefine#1{%
1618   \edef\bb@tempa{\bb@stripslash#1}%
1619   \expandafter\let\csname org@bb@tempa\endcsname#1%
1620   \expandafter\def\csname\bb@tempa\endcsname}
1621 \@onlypreamble\bb@l@redefine

```

\bb@l@redefine@long This version of \babel@redefine can be used to redefine \long commands such as \ifthenelse.

```

1622 \def\bb@l@redefine@long#1{%
1623   \edef\bb@tempa{\bb@stripslash#1}%
1624   \expandafter\let\csname org@bb@tempa\endcsname#1%
1625   \long\expandafter\def\csname\bb@tempa\endcsname}
1626 \@onlypreamble\bb@l@redefine@long

```

\bb@l@redefineroobust For commands that are redefined, but which *might* be robust we need a slightly more intelligent macro. A robust command foo is defined to expand to \protect\foo_. So it is necessary to check whether \foo_ exists. The result is that the command that is being redefined is always robust afterwards. Therefore all we need to do now is define \foo_.

```

1627 \def\bb@l@redefineroobust#1{%
1628   \edef\bb@tempa{\bb@stripslash#1}%
1629   \bb@ifunset{\bb@tempa\space}%
1630   {\expandafter\let\csname org@bb@tempa\endcsname#1%
1631     \bb@exp{\def\#1{\protect\<\bb@tempa\space>}}}%
1632   {\bb@exp{\let\<org@bb@tempa>\<\bb@tempa\space>}}}%
1633   \@namedef{\bb@tempa\space}}
1634 \@onlypreamble\bb@l@redefineroobust

```

4.11. French spacing

\bb@l@frenchspacing

\bb@l@nonfrenchspacing Some languages need to have \frenchspacing in effect. Others don't want that. The command \bb@l@frenchspacing switches it on when it isn't already in effect and \bb@l@nonfrenchspacing switches it off if necessary.

```

1635 \def\bb@l@frenchspacing{%
1636   \ifnum\the\sfcodes\<.\<=\@m
1637     \let\bb@l@nonfrenchspacing\relax
1638   \else
1639     \frenchspacing
1640     \let\bb@l@nonfrenchspacing\nonfrenchspacing
1641   \fi}
1642 \let\bb@l@nonfrenchspacing\nonfrenchspacing

```

A more refined way to switch the catcodes is done with ini files. Here an auxiliary macro is defined, but the main part is in \babelprovide. This new method should be ideally the default one.

```

1643 \let\bb@l@elt\relax
1644 \edef\bb@l@fs@chars{%
1645   \bb@elt{\string.}\@m{3000}\bb@elt{\string?}\@m{3000}%
1646   \bb@elt{\string!}\@m{3000}\bb@elt{\string:}\@m{2000}%
1647   \bb@elt{\string;}\@m{1500}\bb@elt{\string,}\@m{1250}}
1648 \def\bb@l@pre@fs{%
1649   \def\bb@l@elt##1##2##3{\sfcode`##1=\the\sfcodes`##1\relax}%
1650   \edef\bb@l@save@sfcodes{\bb@l@fs@chars}%
1651 \def\bb@l@post@fs{%
1652   \bb@l@save@sfcodes
1653   \edef\bb@l@tempa{\bb@cl{frspc}}%
1654   \edef\bb@l@tempa{\expandafter\@car\bb@l@tempa\@nil}%
1655   \if u\bb@l@tempa % do nothing
1656   \else\if n\bb@l@tempa % non french
1657     \def\bb@l@elt##1##2##3{%
1658       \ifnum\sfcodes`##1=##2\relax
1659       \babel@savevariable{\sfcode`##1}%

```

```

1660 \sfcode`##1=##3\relax
1661 \fi}%
1662 \bbl@fs@chars
1663 \else\if y\bbl@tempa % french
1664 \def\bbl@elt##1##2##3{%
1665 \ifnum\sffcode`##1=##3\relax
1666 \babel@savevariable{\sfcode`##1}%
1667 \sfcode`##1=##2\relax
1668 \fi}%
1669 \bbl@fs@chars
1670 \fi\fi\fi}

```

\labelhyphenation This macro saves hyphenation exceptions. Two macros are used to store them: `\bbl@hyphenation@` for the global ones and `\bbl@hyphenation@(language)` for language ones. See `\bbl@patterns` above for further details. We make sure there is a space between words when multiple commands are used.

```

1671 \bbl@trace{Hyphens}
1672 \@onlypreamble\babelhyphenation
1673 \AtEndOfPackage{%
1674   \newcommand\babelhyphenation[2][\@empty]{%
1675     \ifx\bbl@hyphenation@\relax
1676       \let\bbl@hyphenation@\@empty
1677     \fi
1678     \ifx\bbl@hyphlist\@empty\else
1679       \bbl@warning{%
1680         You must not intermingle \string\selectlanguage\space and\\%
1681         \string\babelhyphenation\space or some exceptions will not\\%
1682         be taken into account. Reported}%
1683     \fi
1684     \ifx\@empty#1%
1685       \protected@edef\bbl@hyphenation@{\bbl@hyphenation@\space#2}%
1686     \else
1687       \bbl@vforeach{#1}{%
1688         \def\bbl@tempa{##1}%
1689         \bbl@fixname\bbl@tempa
1690         \bbl@iflanguage\bbl@tempa{%
1691           \bbl@csarg\protected@edef{hyphenation@\bbl@tempa}{%
1692             \bbl@ifunset{\bbl@hyphenation@\bbl@tempa}%
1693             {}%
1694             {\csname bbl@hyphenation@\bbl@tempa\endcsname\space}%
1695             #2}}}%
1696       \fi}}

```

\labelhyphenmins Only L^AT_EX (basically because it's defined with a L^AT_EX tool).

```

1697 \ifx\NewDocumentCommand\undefined\else
1698   \NewDocumentCommand\babelhyphenmins{sommo}{%
1699     \IfNoValueTF{#2}%
1700       {\protected@edef\bbbl@hyphenmins@{\set@hyphenmins{#3}{#4}}}%
1701       \IfValueT{#5}{%
1702         \protected@edef\bbbl@hyphenatmin@{\hyphenationmin=#5\relax}}%
1703       \IfBooleanT{#1}{%
1704         \lefthyphenmin=#3\relax
1705         \righthyphenmin=#4\relax
1706         \IfValueT{#5}{\hyphenationmin=#5\relax}}}%
1707   {\edef\bbbl@tempb{\zap@space#2 \@empty}%
1708     \bbbl@for\bbbl@tempa\bbbl@tempb{%
1709       \@namedef{bbbl@hyphenmins@\bbbl@tempa}{\set@hyphenmins{#3}{#4}}}%
1710     \IfValueT{#5}{%
1711       \@namedef{bbbl@hyphenatmin@\bbbl@tempa}{\hyphenationmin=#5\relax}}}%
1712   \IfBooleanT{#1}{\bbbl@error{hyphenmins-args}{}}{}{}{}{}{}

```

1713 \fi

\bbl@allowhyphens This macro makes hyphenation possible. Basically its definition is nothing more than `\nobreak\hskip 0pt plus 0pt`. TeX begins and ends a word for hyphenation at a glue node. The penalty prevents a linebreak at this glue node.

```
1714 \def\bbl@allowhyphens{\ifvmode\else\nobreak\hskip\z@skip\fi}
1715 \def\bbl@t@one{T1}
1716 \def\allowhyphens{\ifx\cf@encoding\bbl@t@one\else\bbl@allowhyphens\fi}
```

\babelhyphen Macros to insert common hyphens. Note the space before @ in `\babelhyphen`. Instead of protecting it with `\DeclareRobustCommand`, which could insert a `\relax`, we use the same procedure as shorthands, with `\active@prefix`.

```
1717 \newcommand\babelnullhyphen{\char\hyphenchar\font}
1718 \def\babelhyphen{\active@prefix\babelhyphen\bbl@hyphen}
1719 \def\bbl@hyphen{%
1720   \ifstar{\bbl@hyphen@i @}{\bbl@hyphen@i \@empty}}
1721 \def\bbl@hyphen@i#1#2{%
1722   \lowercase{\bbl@ifunset{\bbl@hy@#1#2\@empty}}%
1723   {\csname bbl@#1usehyphen\endcsname\discretionary{#2}{}{#2}}}%
1724   {\lowercase{\csname bbl@hy@#1#2\@empty\endcsname}}}
```

The following two commands are used to wrap the “hyphen” and set the behavior of the rest of the word – the version with a single @ is used when further hyphenation is allowed, while that with @@ if no more hyphens are allowed. In both cases, if the hyphen is preceded by a positive space, breaking after the hyphen is disallowed.

There should not be a discretionary after a hyphen at the beginning of a word, so it is prevented if preceded by a skip. Unfortunately, this does handle cases like “(-suffix)”. `\nobreak` is always preceded by `\leavevmode`, in case the shorthand starts a paragraph.

```
1725 \def\bbl@usehyphen#1{%
1726   \leavevmode
1727   \ifdim\lastskip>\z@\mbox{#1}\else\nobreak#1\fi
1728   \nobreak\hskip\z@skip}
1729 \def\bbl@@usehyphen#1{%
1730   \leavevmode\ifdim\lastskip>\z@\mbox{#1}\else#1\fi}
```

The following macro inserts the hyphen char.

```
1731 \def\bbl@hyphenchar{%
1732   \ifnum\hyphenchar\font=\m@ne
1733     \babelnullhyphen
1734   \else
1735     \char\hyphenchar\font
1736   \fi}
```

Finally, we define the hyphen “types”. Their names will not change, so you may use them in `\ldf`’s. After a space, the `\mbox` in `\bbl@hy@nobreak` is redundant.

```
1737 \def\bbl@hy@soft{\bbl@usehyphen\discretionary{\bbl@hyphenchar}{}}{}
1738 \def\bbl@hy@@soft{\bbl@usehyphen\discretionary{\bbl@hyphenchar}{}}{}
1739 \def\bbl@hy@hard{\bbl@usehyphen\bbl@hyphenchar}
1740 \def\bbl@hy@@hard{\bbl@usehyphen\bbl@hyphenchar}
1741 \def\bbl@hy@nobreak{\bbl@usehyphen\mbox{\bbl@hyphenchar}}
1742 \def\bbl@hy@@nobreak{\mbox{\bbl@hyphenchar}}
1743 \def\bbl@hy@repeat{%
1744   \bbl@usehyphen%
1745   \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}
1746 \def\bbl@hy@@repeat{%
1747   \bbl@usehyphen%
1748   \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}
1749 \def\bbl@hy@empty{\hskip\z@skip}
1750 \def\bbl@hy@@empty{\discretionary{}{}{}}
```

\bbl@disc For some languages the macro `\bbl@disc` is used to ease the insertion of discretionaries for letters that behave ‘abnormally’ at a breakpoint.

```
1751 \def\bbl@disc#1#2{\nobreak\discretionary{#2-}{}{#1}\bbl@allowhyphens}
```

4.13. Multiencoding strings

The aim following commands is to provide a common interface for strings in several encodings. They also contains several hooks which can be used by luatex and xetex. The code is organized here with pseudo-guards, so we start with the basic commands.

Tools But first, a tool. It makes global a local variable. This is not the best solution, but it works.

```
1752 \bbl@trace{Multiencoding strings}
1753 \def\bbl@tglobal#1{\global\let#1#1}
```

The following option is currently no-op. It was meant for the deprecated `\SetCase`.

```
1754 <<{*More package options}>> ≡
1755 \DeclareOption{nocase}{}
1756 <</More package options>>
```

The following package options control the behavior of `\SetString`.

```
1757 <<{*More package options}>> ≡
1758 \let\bbl@opt@strings\@nnil % accept strings=value
1759 \DeclareOption{strings}{\def\bbl@opt@strings{\BabelStringsDefault}}
1760 \DeclareOption{strings=encoded}{\let\bbl@opt@strings\relax}
1761 \def\BabelStringsDefault{generic}
1762 <</More package options>>
```

Main command This is the main command. With the first use it is redefined to omit the basic setup in subsequent blocks. We make sure strings contain actual letters in the range 128-255, not active characters.

```
1763 \@onlypreamble\StartBabelCommands
1764 \def\StartBabelCommands{%
1765   \begingroup
1766   \@tempcnta="7F
1767   \def\bbl@tempa{%
1768     \ifnum\@tempcnta>"FF\else
1769       \catcode\@tempcnta=11
1770       \advance\@tempcnta\@ne
1771       \expandafter\bbl@tempa
1772     \fi}%
1773   \bbl@tempa
1774   <@Macros local to BabelCommands@>
1775   \def\bbl@provstring##1##2{%
1776     \providecommand##1{##2}%
1777     \bbl@tglobal##1}%
1778   \global\let\bbl@scafter\@empty
1779   \let\StartBabelCommands\bbl@startcmds
1780   \ifx\BabelLanguages\relax
1781     \let\BabelLanguages\CurrentOption
1782   \fi
1783   \begingroup
1784   \let\bbl@screset\@nnil % local flag - disable 1st stopcommands
1785   \StartBabelCommands}
1786 \def\bbl@startcmds{%
1787   \ifx\bbl@screset\@nnil\else
1788     \bbl@usehooks{stopcommands}{}%
1789   \fi
1790   \endgroup
1791   \begingroup
1792   \@ifstar
1793     {\ifx\bbl@opt@strings\@nnil
1794       \let\bbl@opt@strings\BabelStringsDefault
1795     \fi
1796     \bbl@startcmds@i}%
1797   \bbl@startcmds@i}
1798 \def\bbl@startcmds@i#1#2{%
1799   \edef\bbl@L{\zap@space#1 \@empty}%
```

```

1800 \edef\bbl@G{\zap@space#2 \@empty}%
1801 \bbl@startcmds@ii}
1802 \let\bbl@startcommands\StartBabelCommands

```

Parse the encoding info to get the label, input, and font parts.

Select the behavior of \SetString. There are two main cases, depending of if there is an optional argument: without it and strings=encoded, strings are defined always; otherwise, they are set only if they are still undefined (i.e., fallback values). With labelled blocks and strings=encoded, define the strings, but with another value, define strings only if the current label or font encoding is the value of strings; otherwise (i.e., no strings or a block whose label is not in strings=) do nothing.

We presume the current block is not loaded, and therefore set (above) a couple of default values to gobble the arguments. Then, these macros are redefined if necessary according to several parameters.

```

1803 \newcommand\bbl@startcmds@ii[1][\@empty]{%
1804 \let\SetString\@gobbletwo
1805 \let\bbl@stringdef\@gobbletwo
1806 \let\AfterBabelCommands\@gobble
1807 \ifx\@empty#1%
1808 \def\bbl@sc@label{generic}%
1809 \def\bbl@encstring##1##2{%
1810 \ProvideTextCommandDefault##1{##2}%
1811 \bbl@tglobal##1%
1812 \expandafter\bbl@tglobal\csname\string?\string##1\endcsname}%
1813 \let\bbl@sctest\in@true
1814 \else
1815 \let\bbl@sc@charset\space % <- zapped below
1816 \let\bbl@sc@fontenc\space % <- " "
1817 \def\bbl@tempa##1=##2\@nil{%
1818 \bbl@csarg\edef{sc\zap@space##1 \@empty}{##2 }}%
1819 \bbl@vforeach{label=#1}{\bbl@tempa##1\@nil}%
1820 \def\bbl@tempa##1 ##2{% space -> comma
1821 ##1%
1822 \ifx\@empty##2\else\ifx,##1,\else,\fi\bbl@afterfi\bbl@tempa##2\fi}%
1823 \edef\bbl@sc@fontenc{\expandafter\bbl@tempa\bbl@sc@fontenc\@empty}%
1824 \edef\bbl@sc@label{\expandafter\zap@space\bbl@sc@label\@empty}%
1825 \edef\bbl@sc@charset{\expandafter\zap@space\bbl@sc@charset\@empty}%
1826 \def\bbl@encstring##1##2{%
1827 \bbl@foreach\bbl@sc@fontenc{%
1828 \bbl@ifunset{T@###1}%
1829 }%
1830 {\ProvideTextCommand##1{####1}{##2}%
1831 \bbl@tglobal##1%
1832 \expandafter
1833 \bbl@tglobal\csname###1\string##1\endcsname}}}%
1834 \def\bbl@sctest{%
1835 \bbl@xin@{\bbl@opt@strings,}{,\bbl@sc@label,\bbl@sc@fontenc,}%
1836 \fi
1837 \ifx\bbl@opt@strings\@nnil % i.e., no strings key -> defaults
1838 \else\ifx\bbl@opt@strings\relax % i.e., strings=encoded
1839 \let\AfterBabelCommands\bbl@aftercmds
1840 \let\SetString\bbl@setstring
1841 \let\bbl@stringdef\bbl@encstring
1842 \else % i.e., strings=value
1843 \bbl@sctest
1844 \ifin@
1845 \let\AfterBabelCommands\bbl@aftercmds
1846 \let\SetString\bbl@setstring
1847 \let\bbl@stringdef\bbl@provstring
1848 \fi\fi\fi
1849 \bbl@scswitch
1850 \ifx\bbl@G\@empty
1851 \def\SetString##1##2{%
1852 \bbl@error{missing-group}{##1}{}}}%

```

```

1853 \fi
1854 \ifx\@empty#1%
1855   \bbl@usehooks{defaultcommands}{}%
1856 \else
1857   \@expandtwoargs
1858   \bbl@usehooks{encodedcommands}{\bbl@sc@charset}{\bbl@sc@fontenc}}%
1859 \fi}

```

There are two versions of `\bbl@scswitch`. The first version is used when `ldfs` are read, and it makes sure `\<group>\<language>` is reset, but only once (`\bbl@screset` is used to keep track of this). The second version is used in the preamble and packages loaded after `babel` and does nothing.

The macro `\bbl@forlang` loops `\bbl@L` but its body is executed only if the value is in `\BabelLanguages` (inside `babel`) or `\date<language>` is defined (after `babel` has been loaded). There are also two version of `\bbl@forlang`. The first one skips the current iteration if the language is not in `\BabelLanguages` (used in `ldfs`), and the second one skips undefined languages (after `babel` has been loaded).

```

1860 \def\bbl@forlang#1#2{%
1861   \bbl@for#1\bbl@L{%
1862     \bbl@xin@{,#1,}{,\BabelLanguages,}%
1863     \ifin@#2\relax\fi}}
1864 \def\bbl@scswitch{%
1865   \bbl@forlang\bbl@tempa{%
1866     \ifx\bbl@G\@empty\else
1867       \ifx\SetString@gobbletwo\else
1868         \edef\bbl@GL{\bbl@G\bbl@tempa}%
1869         \bbl@xin@{\bbl@GL,}{,\bbl@screset,}%
1870       \ifin@\else
1871         \global\expandafter\let\csname\bbl@GL\endcsname\@undefined
1872         \xdef\bbl@screset{\bbl@screset,\bbl@GL}%
1873       \fi
1874     \fi
1875   \fi}}
1876 \AtEndOfPackage{%
1877   \def\bbl@forlang#1#2{\bbl@for#1\bbl@L{\bbl@ifunset{date#1}{\#2}}}%
1878   \let\bbl@scswitch\relax}
1879 \@onlypreamble\EndBabelCommands
1880 \def\EndBabelCommands{%
1881   \bbl@usehooks{stopcommands}{}%
1882   \endgroup
1883   \endgroup
1884   \bbl@scafter}
1885 \let\bbl@endcommands\EndBabelCommands

```

Now we define commands to be used inside `\StartBabelCommands`.

Strings The following macro is the actual definition of `\SetString` when it is “active”

First save the “switcher”. Create it if undefined. Strings are defined only if undefined (i.e., like `\providescommand`). With the event `stringprocess` you can preprocess the string by manipulating the value of `\BabelString`. If there are several hooks assigned to this event, preprocessing is done in the same order as defined. Finally, the string is set.

```

1886 \def\bbl@setstring#1#2{ e.g., \prefacename{<string>}
1887   \bbl@forlang\bbl@tempa{%
1888     \edef\bbl@LC{\bbl@tempa\bbl@stripslash#1}%
1889     \bbl@ifunset{\bbl@LC} e.g., \germanchaptername
1890     {\bbl@exp%
1891       \global\bbbl@add\<\bbl@G\bbl@tempa{\bbbl@scset\#1\<\bbl@LC>}}}%
1892     {}%
1893   \def\BabelString{#2}%
1894   \bbl@usehooks{stringprocess}{}%
1895   \expandafter\bbl@stringdef
1896   \csname\bbl@LC\expandafter\endcsname\expandafter{\BabelString}}

```

A little auxiliary command sets the string. Formerly used with casing. Very likely no longer necessary, although it's used in `\setlocalecaption`.

```
1897 \def\bbl@scset#1#2{\def#1{#2}}
```

Define `\SetStringLoop`, which is actually set inside `\StartBabelCommands`. The current definition is somewhat complicated because we need a count, but `\count@` is not under our control (remember `\SetString` may call hooks). Instead of defining a dedicated count, we just “pre-expand” its value.

```
1898 << *Macros local to BabelCommands >> ≡
1899 \def\SetStringLoop##1##2{%
1900   \def\bbl@templ####1{\expandafter\noexpand\csname##1\endcsname}%
1901   \count@\z@
1902   \bbl@loop\bbl@tempa{##2}{% empty items and spaces are ok
1903     \advance\count@\@ne
1904     \toks@\expandafter{\bbl@tempa}%
1905     \bbl@exp{%
1906       \\SetString\bbl@templ{\romannumeral\count@}{\the\toks@}%
1907       \count@=\the\count@\relax}}}%
1908 <</Macros local to BabelCommands >>
```

Delaying code Now the definition of `\AfterBabelCommands` when it is activated.

```
1909 \def\bbl@aftercmds#1{%
1910   \toks@\expandafter{\bbl@scafter#1}%
1911   \xdef\bbl@scafter{\the\toks@}}
```

Case mapping The command `\SetCase` is deprecated. Currently it consists in a definition with a hack just for backward compatibility in the macro mapping.

```
1912 << *Macros local to BabelCommands >> ≡
1913 \newcommand\SetCase[3][]{%
1914   \def\bbl@tempa####1####2{%
1915     \ifx####1\@empty\else
1916       \bbl@carg\bbl@add{extras\CurrentOption}{%
1917         \bbl@carg\babel@save{c__text_uppercase\_string####1_tl}%
1918         \bbl@carg\def{c__text_uppercase\_string####1_tl}{####2}%
1919         \bbl@carg\babel@save{c__text_lowercase\_string####2_tl}%
1920         \bbl@carg\def{c__text_lowercase\_string####2_tl}{####1}%
1921         \expandafter\bbl@tempa
1922       \fi}%
1923   \bbl@tempa##1\@empty\@empty
1924   \bbl@carg\bbl@toglobal{extras\CurrentOption}}%
1925 <</Macros local to BabelCommands >>
```

Macros to deal with case mapping for hyphenation. To decide if the document is monolingual or multilingual, we make a rough guess – just see if there is a comma in the languages list, built in the first pass of the package options.

```
1926 << *Macros local to BabelCommands >> ≡
1927 \newcommand\SetHyphenMap[1]{%
1928   \bbl@forlang\bbl@tempa{%
1929     \expandafter\bbl@stringdef
1930     \csname\bbl@tempa @bbl@hyphenmap\endcsname{##1}}}%
1931 <</Macros local to BabelCommands >>
```

There are 3 helper macros which do most of the work for you.

```
1932 \newcommand\BabelLower[2]{% one to one.
1933   \ifnum\lccode#1=#2\else
1934     \babel@savevariable{\lccode#1}%
1935     \lccode#1=#2\relax
1936   \fi}
1937 \newcommand\BabelLowerMM[4]{% many-to-many
1938   \@tempcnta=#1\relax
1939   \@tempcntb=#4\relax
1940   \def\bbl@tempa{%
1941     \ifnum\@tempcnta>#2\else
1942       \@expandtwoargs\BabelLower{\the\@tempcnta}{\the\@tempcntb}%
1943       \advance\@tempcnta#3\relax
```



```

1944 \advance\@tempcntb#3\relax
1945 \expandafter\bbbl@tempa
1946 \fi}%
1947 \bbbl@tempa}
1948 \newcommand\BabelLowerM0[4]{% many-to-one
1949 \@tempcnta=#1\relax
1950 \def\bbbl@tempa{%
1951 \ifnum\@tempcnta>#2\else
1952 \expandafter\BabelLower{\the\@tempcnta}{#4}%
1953 \advance\@tempcnta#3
1954 \expandafter\bbbl@tempa
1955 \fi}%
1956 \bbbl@tempa}

```

The following package options control the behavior of hyphenation mapping.

```

1957 <<{*More package options}>> ≡
1958 \DeclareOption{hyphenmap=off}{\chardef\bbbl@opt@hyphenmap\z@}
1959 \DeclareOption{hyphenmap=first}{\chardef\bbbl@opt@hyphenmap\@ne}
1960 \DeclareOption{hyphenmap=select}{\chardef\bbbl@opt@hyphenmap\tw@}
1961 \DeclareOption{hyphenmap=other}{\chardef\bbbl@opt@hyphenmap\thr@@}
1962 \DeclareOption{hyphenmap=other*}{\chardef\bbbl@opt@hyphenmap4\relax}
1963 <</More package options>>

```

Initial setup to provide a default behavior if hyphenmap is not set.

```

1964 \AtEndOfPackage{%
1965 \ifx\bbbl@opt@hyphenmap\undefined
1966 \bbbl@xin@{,}{\bbbl@language@opts}%
1967 \chardef\bbbl@opt@hyphenmap\ifin@4\else\@ne\fi
1968 \fi}

```

4.14. Tailor captions

A general tool for resetting the caption names with a unique interface. With the old way, which mixes the switcher and the string, we convert it to the new one, which separates these two steps.

```

1969 \newcommand\setlocalecaption{%
1970 \ifstar\bbbl@setcaption@s\bbbl@setcaption@x}
1971 \def\bbbl@setcaption@x#1#2#3{% language caption-name string
1972 \bbbl@trim@def\bbbl@tempa{#2}%
1973 \bbbl@xin@{.template}{\bbbl@tempa}%
1974 \ifin@
1975 \bbbl@ini@captions@template{#3}{#1}%
1976 \else
1977 \edef\bbbl@tempd{%
1978 \expandafter\expandafter\expandafter
1979 \strip@prefix\expandafter\meaning\csname captions#1\endcsname}%
1980 \bbbl@xin@
1981 {\expandafter\string\csname #2name\endcsname}%
1982 {\bbbl@tempd}%
1983 \ifin@ % Renew caption
1984 \bbbl@xin@{\string\bbbl@scset}{\bbbl@tempd}%
1985 \ifin@
1986 \bbbl@exp{%
1987 \\bbbl@ifsamestring{\bbbl@tempa}{\language name}%
1988 {\bbbl@scset\<#2name>\<#1#2name>}}%
1989 {}}%
1990 \else % Old way converts to new way
1991 \bbbl@ifunset{#1#2name}%
1992 {\bbbl@exp{%
1993 \\bbbl@add\<captions#1>{\def\<#2name>{\<#1#2name>}}%
1994 \\bbbl@ifsamestring{\bbbl@tempa}{\language name}%
1995 {\def\<#2name>{\<#1#2name>}}%
1996 {}}}%
1997 {}%

```

```

1998     \fi
1999 \else
2000     \bbl@xin@{\string\bbl@scset}{\bbl@tempd}% New
2001     \ifin@ % New way
2002     \bbl@exp{%
2003         \\bbl@add<captions#1>{\bbl@scset<#2name>\<#1#2name>}%
2004         \\bbl@ifsamestring{\bbl@tempa}{\language}%
2005         {\bbl@scset<#2name>\<#1#2name>}%
2006         }%
2007     \else % Old way, but defined in the new way
2008     \bbl@exp{%
2009         \\bbl@add<captions#1>{\def<#2name>{\<#1#2name>}}%
2010         \\bbl@ifsamestring{\bbl@tempa}{\language}%
2011         {\def<#2name>{\<#1#2name>}}%
2012         }%
2013     \fi%
2014 \fi
2015 \@namedef{#1#2name}{#3}%
2016 \toks@{\expandafter\bbl@captionslist}%
2017 \bbl@exp{\in{\<#2name>}{\the\toks@}}%
2018 \ifin\else
2019     \bbl@exp{\bbl@add\bbl@captionslist{\<#2name>}}%
2020     \bbl@tglobal\bbl@captionslist
2021 \fi
2022 \fi}

```

4.15. Making glyphs available

This section makes a number of glyphs available that either do not exist in the OT1 encoding and have to be ‘faked’, or that are not accessible through Tlenc.def.

\set@low@box The following macro is used to lower quotes to the same level as the comma. It prepares its argument in box register 0.

```

2023 \bbl@trace{Macros related to glyphs}
2024 \def\set@low@box#1{\setbox\tw@{#1}\setbox\z@{\hbox{#1}}%
2025     \dimen\z@{\ht\z@ \advance\dimen\z@ -\ht\tw@}%
2026     \setbox\z@{\hbox{\lower\dimen\z@ \box\z@}\ht\z@{\ht\tw@ \dp\z@{\dp\tw@}}

```

\save@sf@q The macro \save@sf@q is used to save and reset the current space factor.

```

2027 \def\save@sf@q#1{\leavevmode
2028     \begingroup
2029     \edef\@SF{\spacefactor\the\spacefactor}\@SF
2030     \endgroup}

```

4.15.1. Quotation marks

\quotedblbase In the T1 encoding the opening double quote at the baseline is available as a separate character; accessible via \quotedblbase. In the OT1 encoding it is not available, therefore we make it available by lowering the normal open quote character to the baseline.

```

2031 \ProvideTextCommand{\quotedblbase}{OT1}{%
2032     \save@sf@q{\set@low@box{\textquotedblright}/}%
2033     \box\z@{\kern-.04em\bbl@allowhyphens}}

```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```

2034 \ProvideTextCommandDefault{\quotedblbase}{%
2035     \UseTextSymbol{OT1}{\quotedblbase}}

```

\quotesinglbase We also need the single quote character at the baseline.

```

2036 \ProvideTextCommand{\quotesinglbase}{OT1}{%
2037     \save@sf@q{\set@low@box{\textquoteright}/}%
2038     \box\z@{\kern-.04em\bbl@allowhyphens}}

```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```
2039 \ProvideTextCommandDefault{\quotesinglbase}{%
2040   \UseTextSymbol{OT1}{\quotesinglbase}}
```

\guillemetleft

\guillemetright The guillemet characters are not available in OT1 encoding. They are faked. (Wrong names with o preserved for compatibility.)

```
2041 \ProvideTextCommand{\guillemetleft}{OT1}{%
2042   \ifmmode
2043     \ll
2044   \else
2045     \save@sf@q{\nobreak
2046       \raise.2ex\hbox{$\scriptscriptstyle\ll$}\bbl@allowhyphens}%
2047   \fi}
2048 \ProvideTextCommand{\guillemetright}{OT1}{%
2049   \ifmmode
2050     \gg
2051   \else
2052     \save@sf@q{\nobreak
2053       \raise.2ex\hbox{$\scriptscriptstyle\gg$}\bbl@allowhyphens}%
2054   \fi}
2055 \ProvideTextCommand{\guillemotleft}{OT1}{%
2056   \ifmmode
2057     \ll
2058   \else
2059     \save@sf@q{\nobreak
2060       \raise.2ex\hbox{$\scriptscriptstyle\ll$}\bbl@allowhyphens}%
2061   \fi}
2062 \ProvideTextCommand{\guillemotright}{OT1}{%
2063   \ifmmode
2064     \gg
2065   \else
2066     \save@sf@q{\nobreak
2067       \raise.2ex\hbox{$\scriptscriptstyle\gg$}\bbl@allowhyphens}%
2068   \fi}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2069 \ProvideTextCommandDefault{\guillemetleft}{%
2070   \UseTextSymbol{OT1}{\guillemetleft}}
2071 \ProvideTextCommandDefault{\guillemetright}{%
2072   \UseTextSymbol{OT1}{\guillemetright}}
2073 \ProvideTextCommandDefault{\guillemotleft}{%
2074   \UseTextSymbol{OT1}{\guillemotleft}}
2075 \ProvideTextCommandDefault{\guillemotright}{%
2076   \UseTextSymbol{OT1}{\guillemotright}}
```

\guilsinglleft

\guilsinglright The single guillemets are not available in OT1 encoding. They are faked.

```
2077 \ProvideTextCommand{\guilsinglleft}{OT1}{%
2078   \ifmmode
2079     <%
2080   \else
2081     \save@sf@q{\nobreak
2082       \raise.2ex\hbox{$\scriptscriptstyle<$}\bbl@allowhyphens}%
2083   \fi}
2084 \ProvideTextCommand{\guilsinglright}{OT1}{%
2085   \ifmmode
2086     >%
2087   \else
2088     \save@sf@q{\nobreak
2089       \raise.2ex\hbox{$\scriptscriptstyle>$}\bbl@allowhyphens}%
2090   \fi}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2091 \ProvideTextCommandDefault{\guilsinglleft}{%
2092 \UseTextSymbol{OT1}{\guilsinglleft}}
2093 \ProvideTextCommandDefault{\guilsinglright}{%
2094 \UseTextSymbol{OT1}{\guilsinglright}}
```

4.15.2. Letters

\ij

\IJ The dutch language uses the letter ‘ij’. It is available in T1 encoded fonts, but not in the OT1 encoded fonts. Therefore we fake it for the OT1 encoding.

```
2095 \DeclareTextCommand{\ij}{OT1}{%
2096 i\kern-0.02em\bbl@allowhyphens j}
2097 \DeclareTextCommand{\IJ}{OT1}{%
2098 I\kern-0.02em\bbl@allowhyphens J}
2099 \DeclareTextCommand{\ij}{T1}{\char188}
2100 \DeclareTextCommand{\IJ}{T1}{\char156}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2101 \ProvideTextCommandDefault{\ij}{%
2102 \UseTextSymbol{OT1}{\ij}}
2103 \ProvideTextCommandDefault{\IJ}{%
2104 \UseTextSymbol{OT1}{\IJ}}
```

\dj

\DJ The croatian language needs the letters \dj and \DJ; they are available in the T1 encoding, but not in the OT1 encoding by default.

Some code to construct these glyphs for the OT1 encoding was made available to me by Stipčević Mario, (stipcevic@olimp.irb.hr).

```
2105 \def\crrtic@{\hrule height0.1ex width0.3em}
2106 \def\crrtic@{\hrule height0.1ex width0.33em}
2107 \def\ddj@{%
2108 \setbox0\hbox{d}\dimen@=\ht0
2109 \advance\dimen@lex
2110 \dimen@.45\dimen@
2111 \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2112 \advance\dimen@ii.5ex
2113 \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crrtic@}}}}
2114 \def\DDJ@{%
2115 \setbox0\hbox{D}\dimen@=.55\ht0
2116 \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2117 \advance\dimen@ii.15ex % correction for the dash position
2118 \advance\dimen@ii-.15\fontdimen7\font % correction for cmtt font
2119 \dimen\thr@\expandafter\rem@pt\the\fontdimen7\font\dimen@
2120 \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crrtic@}}}}
2121 %
2122 \DeclareTextCommand{\dj}{OT1}{\ddj@ d}
2123 \DeclareTextCommand{\DJ}{OT1}{\DDJ@ D}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2124 \ProvideTextCommandDefault{\dj}{%
2125 \UseTextSymbol{OT1}{\dj}}
2126 \ProvideTextCommandDefault{\DJ}{%
2127 \UseTextSymbol{OT1}{\DJ}}
```

\SS For the T1 encoding \SS is defined and selects a specific glyph from the font, but for other encodings it is not available. Therefore we make it available here.

```
2128 \DeclareTextCommand{\SS}{OT1}{SS}
2129 \ProvideTextCommandDefault{\SS}{\UseTextSymbol{OT1}{\SS}}
```

4.15.3. Shorthands for quotation marks

Shorthands are provided for a number of different quotation marks, which make them usable both outside and inside mathmode. They are defined with `\ProvideTextCommandDefault`, but this is very likely not required because their definitions are based on encoding-dependent macros.

`\glq`

`\grq` The ‘german’ single quotes.

```
2130 \ProvideTextCommandDefault{\glq}{%
2131 \textormath{\quotesinglbase}{\mbox{\quotesinglbase}}}
```

The definition of `\grq` depends on the fontencoding. With T1 encoding no extra kerning is needed.

```
2132 \ProvideTextCommand{\grq}{T1}{%
2133 \textormath{\kern\z@\textquoteleft}{\mbox{\textquoteleft}}}
2134 \ProvideTextCommand{\grq}{TU}{%
2135 \textormath{\textquoteleft}{\mbox{\textquoteleft}}}
2136 \ProvideTextCommand{\grq}{OT1}{%
2137 \save@sf@q{\kern-.0125em
2138 \textormath{\textquoteleft}{\mbox{\textquoteleft}}}%
2139 \kern.07em\relax}}
2140 \ProvideTextCommandDefault{\grq}{\UseTextSymbol{OT1}\grq}
```

`\glqq`

`\grqq` The ‘german’ double quotes.

```
2141 \ProvideTextCommandDefault{\glqq}{%
2142 \textormath{\quotedblbase}{\mbox{\quotedblbase}}}
```

The definition of `\grqq` depends on the fontencoding. With T1 encoding no extra kerning is needed.

```
2143 \ProvideTextCommand{\grqq}{T1}{%
2144 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}
2145 \ProvideTextCommand{\grqq}{TU}{%
2146 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}
2147 \ProvideTextCommand{\grqq}{OT1}{%
2148 \save@sf@q{\kern-.07em
2149 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}%
2150 \kern.07em\relax}}
2151 \ProvideTextCommandDefault{\grqq}{\UseTextSymbol{OT1}\grqq}
```

`\flq`

`\frq` The ‘french’ single guillemets.

```
2152 \ProvideTextCommandDefault{\flq}{%
2153 \textormath{\guilsinglleft}{\mbox{\guilsinglleft}}}
2154 \ProvideTextCommandDefault{\frq}{%
2155 \textormath{\guilsinglright}{\mbox{\guilsinglright}}}
```

`\flqq`

`\frqq` The ‘french’ double guillemets.

```
2156 \ProvideTextCommandDefault{\flqq}{%
2157 \textormath{\guillemetleft}{\mbox{\guillemetleft}}}
2158 \ProvideTextCommandDefault{\frqq}{%
2159 \textormath{\guillemetright}{\mbox{\guillemetright}}}
```

4.15.4. Umlauts and tremas

The command `\` needs to have a different effect for different languages. For German for instance, the ‘umlaut’ should be positioned lower than the default position for placing it over the letters a, o, u, A, O and U. When placed over an e, i, E or I it can retain its normal position. For Dutch the same glyph is always placed in the lower position.

`\umlauthigh`

\umlautlow To be able to provide both positions of `\` we provide two commands to switch the positioning, the default will be `\umlauthigh` (the normal positioning).

```

2160 \def\umlauthigh{%
2161   \def\bbbl@umlauta##1{\leavevmode\bgroup%
2162     \accent\csname\f@encoding dqpos\endcsname
2163     ##1\bbbl@allowhyphens\egroup}%
2164   \let\bbbl@umlaute\bbbl@umlauta}
2165 \def\umlautlow{%
2166   \def\bbbl@umlauta{\protect\lower@umlaut}}
2167 \def\umlautelowlow{%
2168   \def\bbbl@umlaute{\protect\lower@umlaut}}
2169 \umlauthigh

```

\lower@umlaut Used to position the `\` closer to the letter. We want the umlaut character lowered, nearer to the letter. To do this we need an extra *(dimen)* register.

```

2170 \expandafter\ifx\csname U@D\endcsname\relax
2171   \csname newdimen\endcsname U@D
2172 \fi

```

The following code fools $\TeX$'s `make\_accent` procedure about the current x-height of the font to force another placement of the umlaut character. First we have to save the current x-height of the font, because we'll change this font dimension and this is always done globally.

Then we compute the new x-height in such a way that the umlaut character is lowered to the base character. The value of `.45ex` depends on the METAFONT parameters with which the fonts were built. (Just try out, which value will look best.) If the new x-height is too low, it is not changed. Finally we call the `\accent` primitive, reset the old x-height and insert the base character in the argument.

```

2173 \def\lower@umlaut#1{%
2174   \leavevmode\bgroup
2175     \U@D lex%
2176     {\setbox\z@\hbox{%
2177       \char\csname\f@encoding dqpos\endcsname}%
2178       \dimen@ -.45ex\advance\dimen@\ht\z@
2179       \ifdim lex<\dimen@ \fontdimen5\font\dimen@ \fi}%
2180     \accent\csname\f@encoding dqpos\endcsname
2181     \fontdimen5\font\U@D #1%
2182   \egroup}

```

For all vowels we declare `\` to be a composite command which uses `\bbbl@umlauta` or `\bbbl@umlaute` to position the umlaut character. We need to be sure that these definitions override the ones that are provided when the package `fontenc` with option `OT1` is used. Therefore these declarations are postponed until the beginning of the document. Note these definitions only apply to some languages, but `babel` sets them for *all* languages – you may want to redefine `\bbbl@umlauta` and/or `\bbbl@umlaute` for a language in the corresponding `ldf` (using the `babel` switching mechanism, of course).

```

2183 \AtBeginDocument{%
2184   \DeclareTextCompositeCommand{\}{OT1}{a}{\bbbl@umlauta{a}}%
2185   \DeclareTextCompositeCommand{\}{OT1}{e}{\bbbl@umlaute{e}}%
2186   \DeclareTextCompositeCommand{\}{OT1}{i}{\bbbl@umlaute{i}}%
2187   \DeclareTextCompositeCommand{\}{OT1}{\i}{\bbbl@umlaute{i}}%
2188   \DeclareTextCompositeCommand{\}{OT1}{o}{\bbbl@umlauta{o}}%
2189   \DeclareTextCompositeCommand{\}{OT1}{u}{\bbbl@umlauta{u}}%
2190   \DeclareTextCompositeCommand{\}{OT1}{A}{\bbbl@umlauta{A}}%
2191   \DeclareTextCompositeCommand{\}{OT1}{E}{\bbbl@umlaute{E}}%
2192   \DeclareTextCompositeCommand{\}{OT1}{I}{\bbbl@umlaute{I}}%
2193   \DeclareTextCompositeCommand{\}{OT1}{O}{\bbbl@umlauta{O}}%
2194   \DeclareTextCompositeCommand{\}{OT1}{U}{\bbbl@umlauta{U}}%

```

Finally, make sure the default hyphenrules are defined (even if empty). For internal use, another empty `\language` is defined. Currently used in `Amharic`.

```

2195 \ifx\l@english\undefined
2196   \chardef\l@english\z@
2197 \fi

```

```

2198 % The following is used to cancel rules in ini files (see Amharic).
2199 \ifx\l@unhyphenated\@undefined
2200   \newlanguage\l@unhyphenated
2201 \fi

```

4.16. Layout

Layout is mainly intended to set bidi documents, but there is at least a tool useful in general.

```

2202 \bbl@trace{Bidi layout}
2203 \providecommand\IfBabelLayout[3]{#3}%

```

4.17. Load engine specific macros

Some macros are not defined in all engines, so, after loading the files define them if necessary to raise an error.

```

2204 \bbl@trace{Input engine specific macros}
2205 \ifcase\bbl@engine
2206   \input txtbabel.def
2207 \or
2208   \input luababel.def
2209 \or
2210   \input xebabel.def
2211 \fi
2212 \providecommand\babelfont{\bbl@error{only-lua-xe}{}}{}{}
2213 \providecommand\babelprehyphenation{\bbl@error{only-lua}{}}{}{}
2214 \ifx\babelposthyphenation\@undefined
2215   \let\babelposthyphenation\babelprehyphenation
2216   \let\babelpatterns\babelprehyphenation
2217   \let\babelcharproperty\babelprehyphenation
2218 \fi
2219 </package | core>

```

4.18. Creating and modifying languages

Continue with $\LaTeX$ only.

`\babelprovide` is a general purpose tool for creating and modifying languages. It creates the language infrastructure, and loads, if requested, an ini file. It may be used in conjunction to previously loaded `ldf` files.

```

2220 < *package>
2221 \bbl@trace{Creating languages and reading ini files}
2222 \let\bbl@extend@ini\@gobble
2223 \newcommand\babelprovide[2][]{%
2224   \let\bbl@savelangname\languagename
2225   \edef\bbl@savelocaleid{\the\localeid}%
2226   \global\let\bbl@afterload\@empty
2227   % Set name and locale id
2228   \edef\languagename{#2}%
2229   \bbl@id@assign
2230   % Initialize keys
2231   \bbl@vforeach{captions,date,import,main,script,language,%
2232     hyphenrules,linebreaking,justification,mapfont,maparabic,%
2233     mapdigits,intraspaces,intrapenalty,onchar,transforms,alph,%
2234     Alph,labels,labels*,mapdot,calendar,date,casing,interchar,%
2235     @import}%
2236     {\bbl@csarg\let{KVP@##1}\@nnil}%
2237   \global\let\bbl@release@transforms\@empty
2238   \global\let\bbl@release@casing\@empty
2239   \let\bbl@calendars\@empty
2240   \global\let\bbl@inidata\@empty
2241   \global\let\bbl@extend@ini\@gobble
2242   \global\let\bbl@included@inis\@empty
2243   \gdef\bbl@key@list{;}%

```

```

2244 \bbl@ifunset{bbl@passto@#2}%
2245   {\def\bbl@tempa{#1}}%
2246   {\bbl@exp{\def\\bbl@tempa{[bbl@passto@#2],\unexpanded{#1}}}%
2247 \expandafter\bbl@forkv\expandafter{\bbl@tempa}{%
2248   \in@{/}{##1}% With /, (re)sets a value in the ini
2249   \ifin@
2250     \bbl@renewinikey##1\@{##2}%
2251   \else
2252     \bbl@csarg\ifx{KVP@##1}\@nnil\else
2253       \bbl@error{unknown-provide-key}{##1}{}}%
2254     \fi
2255     \bbl@csarg\def{KVP@##1}{##2}%
2256   \fi}%
2257 \chardef\bbl@howloaded=0:none;1:ldf without ini;2:ini
2258 \bbl@ifunset{date#2}\z@{\bbl@ifunset{bbl@llevel@#2}\@ne\tw@}%
2259 % == init ==
2260 \ifx\bbl@screset\undefined
2261   \bbl@ldfinit
2262 \fi
2263 % ==
2264 % If there is no import (last wins), use @import (internal, there
2265 % must be just one). To consider any order (because
2266 % \PassOptionsToLocale).
2267 \ifx\bbl@KVP@import\@nnil
2268   \let\bbl@KVP@import\bbl@KVP@@import
2269 \fi
2270 % == date (as option) ==
2271 % \ifx\bbl@KVP@date\@nnil\else
2272 % \fi
2273 % ==
2274 \let\bbl@lbkflag\relax % \@empty = do setup linebreak, only in 3 cases:
2275 \ifcase\bbl@howloaded
2276   \let\bbl@lbkflag\@empty % new
2277 \else
2278   \ifx\bbl@KVP@hyphenrules\@nnil\else
2279     \let\bbl@lbkflag\@empty
2280   \fi
2281   \ifx\bbl@KVP@import\@nnil\else
2282     \let\bbl@lbkflag\@empty
2283   \fi
2284 \fi
2285 % == import, captions ==
2286 \ifx\bbl@KVP@import\@nnil\else
2287   \bbl@exp{\\bbl@ifblank{\bbl@KVP@import}}%
2288   {\ifx\bbl@initoload\relax
2289     \begingroup
2290       \def\BabelBeforeIni##1##2{\gdef\bbl@KVP@import{##1}\endinput}%
2291       \bbl@input@texini{##2}%
2292     \endgroup
2293   \else
2294     \xdef\bbl@KVP@import{\bbl@initoload}%
2295   \fi}%
2296   {}%
2297   \let\bbl@KVP@date\@empty
2298 \fi
2299 \let\bbl@KVP@captions@@\bbl@KVP@captions
2300 \ifx\bbl@KVP@captions\@nnil
2301   \let\bbl@KVP@captions\bbl@KVP@import
2302 \fi
2303 % ==
2304 \ifx\bbl@KVP@transforms\@nnil\else
2305   \bbl@replace\bbl@KVP@transforms{ }{,}%
2306 \fi

```



```

2307 % ==
2308 \ifx\bbk@KVP@mapdot\@nnil\else
2309   \def\bbk@tempa{\@empty}%
2310   \ifx\bbk@KVP@mapdot\bbk@tempa\else
2311     \bbk@exp{\gdef\<bbk@map@. @\language>{\bbk@KVP@mapdot}}}%
2312   \fi
2313 \fi
2314 % Load ini
2315 % -----
2316 \ifcase\bbk@howloaded
2317   \bbk@provide@new{#2}%
2318 \else
2319   \bbk@ifblank{#1}%
2320     {}% With \bbk@load@basic below
2321     {\bbk@provide@renew{#2}}}%
2322 \fi
2323 % Post tasks
2324 % -----
2325 % == subsequent calls after the first provide for a locale ==
2326 \ifx\bbk@inidata\@empty\else
2327   \bbk@extend@ini{#2}%
2328 \fi
2329 % == ensure captions ==
2330 \ifx\bbk@KVP@captions\@nnil\else
2331   \bbk@ifunset{\bbk@extracaps@#2}%
2332     {\bbk@exp{\bbk@babel@ensure[exclude=\\today]{#2}}}%
2333     {\bbk@exp{\bbk@babel@ensure[exclude=\\today,
2334       include=\bbk@extracaps@#2]{#2}}}%
2335   \let\bbk@tempc\language
2336   \bbk@replace\bbk@tempc{-}{@}%
2337   \bbk@ifunset{\bbk@ensure@\bbk@tempc}%
2338     {\bbk@exp{%
2339       \\DeclareRobustCommand\<bbk@ensure@\bbk@tempc>[1]{%
2340         \\foreignlanguage{\language}%
2341         {###1}}}%
2342     }%
2343   \bbk@exp{%
2344     \\bbk@tglobal\<bbk@ensure@\bbk@tempc>%
2345     \\bbk@tglobal\<bbk@ensure@\bbk@tempc\space>%
2346 \fi

```

At this point all parameters are defined if 'import'. Now we execute some code depending on them. But what about if nothing was imported? We just set the basic parameters, but still loading the whole ini file.

```

2347 \bbk@load@basic{#2}%
2348 % == script, language ==
2349 % Override the values from ini or defines them
2350 \ifx\bbk@KVP@script\@nnil\else
2351   \bbk@csarg\edef{sname@#2}{\bbk@KVP@script}%
2352 \fi
2353 \ifx\bbk@KVP@language\@nnil\else
2354   \bbk@csarg\edef{lname@#2}{\bbk@KVP@language}%
2355 \fi
2356 \ifcase\bbk@engine\or
2357   \bbk@ifunset{\bbk@chrng@\language}%
2358     {\directlua{
2359       Babel.set_chranges_b('\bbk@cl{sbc}', '\bbk@cl{chrng}') }}%
2360 \fi
2361 % == Line breaking: intraspace, intrapenalty ==
2362 % For CJK, East Asian, Southeast Asian, if interspace in ini
2363 \ifx\bbk@KVP@intraspace\@nnil\else % We can override the ini or set
2364   \bbk@csarg\edef{intsp@#2}{\bbk@KVP@intraspace}%
2365 \fi

```

```

2366 \bbl@provide@intraspace
2367 % == Line breaking: justification ==
2368 \ifx\bbl@KVP@justification\@nnil\else
2369 \let\bbl@KVP@linebreaking\bbl@KVP@justification
2370 \fi
2371 \ifx\bbl@KVP@linebreaking\@nnil\else
2372 \bbl@xin@{\,\bbl@KVP@linebreaking,}%
2373 {,elongated,kashida,cjk,padding,unhyphenated,}%
2374 \ifin@
2375 \bbl@csarg\xdef
2376 {\lnbrk@{\language\name}{\expandafter\@car\bbl@KVP@linebreaking\@nil}%
2377 \fi
2378 \fi
2379 \bbl@xin@{/e}{/\bbl@cl{\lnbrk}}%
2380 \ifin@ \else \bbl@xin@{/k}{/\bbl@cl{\lnbrk}} \fi
2381 \ifin@ \bbl@arabicjust \fi
2382 \bbl@xin@{/p}{/\bbl@cl{\lnbrk}}%
2383 \ifin@ \AtBeginDocument{\@nameuse{\bbl@tibetanjust}} \fi
2384 % == Line breaking: hyphenate.other.(locale|script) ==
2385 \ifx\bbl@lbfkflag\@empty
2386 \bbl@ifunset{\bbl@hyotl@{\language\name}}{ }%
2387 {\bbl@csarg\bbl@replace{\hyotl@{\language\name}}{ }{,}%
2388 \bbl@startcommands*{\language\name}}{ }%
2389 \bbl@csarg\bbl@foreach{\hyotl@{\language\name}}{ }%
2390 \ifcase\bbl@engine
2391 \ifnum##1<257
2392 \SetHyphenMap{\BabelLower{##1}{##1}}%
2393 \fi
2394 \else
2395 \SetHyphenMap{\BabelLower{##1}{##1}}%
2396 \fi}%
2397 \bbl@endcommands}%
2398 \bbl@ifunset{\bbl@hyots@{\language\name}}{ }%
2399 {\bbl@csarg\bbl@replace{\hyots@{\language\name}}{ }{,}%
2400 \bbl@csarg\bbl@foreach{\hyots@{\language\name}}{ }%
2401 \ifcase\bbl@engine
2402 \ifnum##1<257
2403 \global\lccode##1=##1\relax
2404 \fi
2405 \else
2406 \global\lccode##1=##1\relax
2407 \fi}}%
2408 \fi
2409 % == Counters: maparabic ==
2410 % Native digits, if provided in ini (TeX level, xe and lua)
2411 \ifcase\bbl@engine\else
2412 \bbl@ifunset{\bbl@dgnat@{\language\name}}{ }%
2413 {\expandafter\ifx\csname\bbl@dgnat@{\language\name}\endcsname\@empty\else
2414 \expandafter\expandafter\expandafter
2415 \bbl@setdigits\csname\bbl@dgnat@{\language\name}\endcsname
2416 \ifx\bbl@KVP@maparabic\@nnil\else
2417 \ifx\bbl@latinarabic\@undefined
2418 \expandafter\let\expandafter\@arabic
2419 \csname\bbl@counter@{\language\name}\endcsname
2420 \else % i.e., if layout=counters, which redefines \@arabic
2421 \expandafter\let\expandafter\bbl@latinarabic
2422 \csname\bbl@counter@{\language\name}\endcsname
2423 \fi
2424 \fi
2425 \fi}%
2426 \fi
2427 % == Counters: mapdigits ==
2428 % > luababel.def

```

```

2429 % == Counters: alph, Alph ==
2430 \ifx\bbbl@KVP@alph\@nnil\else
2431   \bbbl@exp{%
2432     \\\bbbl@add\<bbbl@preextras@\language\name>{%
2433       \\\babel@save\\\@alph
2434       \let\\\@alph\<bbbl@cntr@\bbbl@KVP@alph @\language\name>}}%
2435 \fi
2436 \ifx\bbbl@KVP@Alph\@nnil\else
2437   \bbbl@exp{%
2438     \\\bbbl@add\<bbbl@preextras@\language\name>{%
2439       \\\babel@save\\\@Alph
2440       \let\\\@Alph\<bbbl@cntr@\bbbl@KVP@Alph @\language\name>}}%
2441 \fi
2442 % == Counters: mapdot ==
2443 \ifx\bbbl@KVP@mapdot\@nnil\else
2444   \bbbl@foreach\bbbl@list@the{%
2445     \bbbl@ifunset{the##1}{}%
2446     {{\bbbl@ncarg\let\bbbl@tempd{the##1}%
2447       \bbbl@carg\bbbl@sreplace{the##1}{.}{\bbbl@map@lbl{.}}%
2448       \expandafter\ifx\csname the##1\endcsname\bbbl@tempd\else
2449         \bbbl@exp{\gdef\<the##1>{{\the##1}}}%
2450       \fi}}}%
2451 \edef\bbbl@tempb{enumi,enumii,enumiii,enumiv}%
2452 \bbbl@foreach\bbbl@tempb{%
2453   \bbbl@ifunset{label##1}{}%
2454   {{\bbbl@ncarg\let\bbbl@tempd{label##1}%
2455     \bbbl@carg\bbbl@sreplace{label##1}{.}{\bbbl@map@lbl{.}}%
2456     \expandafter\ifx\csname label##1\endcsname\bbbl@tempd\else
2457       \bbbl@exp{\gdef\<label##1>{{\label##1}}}%
2458     \fi}}}%
2459 \fi
2460 % == Casing ==
2461 \bbbl@release@casing
2462 \ifx\bbbl@KVP@casing\@nnil\else
2463   \bbbl@csarg\xdef{casing@\language\name}%
2464   {\@nameuse{bbbl@casing@\language\name}\bbbl@maybextx\bbbl@KVP@casing}%
2465 \fi
2466 % == Calendars ==
2467 \ifx\bbbl@KVP@calendar\@nnil
2468   \edef\bbbl@KVP@calendar{\bbbl@cl{calpr}}%
2469 \fi
2470 \def\bbbl@tempe##1 ##2\@{% Get first calendar
2471   \def\bbbl@tempa{##1}}%
2472   \bbbl@exp{\\\bbbl@tempe\bbbl@KVP@calendar\space\\\@}%
2473 \def\bbbl@tempe##1.##2.##3\@{%
2474   \def\bbbl@tempc{##1}%
2475   \def\bbbl@tempb{##2}}%
2476 \expandafter\bbbl@tempe\bbbl@tempa..\@
2477 \bbbl@csarg\edef{calpr@\language\name}{%
2478   \ifx\bbbl@tempc\@empty\else
2479     calendar=\bbbl@tempc
2480   \fi
2481   \ifx\bbbl@tempb\@empty\else
2482     ,variant=\bbbl@tempb
2483   \fi}%
2484 % == engine specific extensions ==
2485 % Defined in XXXbabel.def
2486 \bbbl@provide@extra{#2}%
2487 % == require.babel in ini ==
2488 % To load or reload the babel-*.tex, if require.babel in ini
2489 \ifx\bbbl@beforestart\relax\else % But not in doc aux or body
2490   \bbbl@ifunset{bbbl@rqtex@\language\name}{}%
2491   {\expandafter\ifx\csname bbl@rqtex@\language\name\endcsname\@empty\else

```

```

2492     \let\BabelBeforeIni\@gobbletwo
2493     \chardef\atcatcode=\catcode`\@
2494     \catcode`\@=11\relax
2495     \def\CurrentOption{#2}%
2496     \bbl@input@texini{\bbl@cs{rqtex@\language}}}%
2497     \catcode`\@=\atcatcode
2498     \let\atcatcode\relax
2499     \global\bbl@csarg\let{rqtex@\language}\relax
2500     \fi}%
2501     \bbl@foreach\bbl@calendars{%
2502     \bbl@ifunset{\bbl@ca##1}{%
2503     \chardef\atcatcode=\catcode`\@
2504     \catcode`\@=11\relax
2505     \InputIfFileExists{babel-ca-##1.tex}{}{}%
2506     \catcode`\@=\atcatcode
2507     \let\atcatcode\relax}%
2508     {}}%
2509 \fi
2510 % == frenchspacing ==
2511 \ifcase\bbl@howloaded\in@true\else\in@false\fi
2512 \ifin@else\bbl@xin@{typography/frenchspacing}{\bbl@key@list}\fi
2513 \ifin@
2514     \bbl@extras@wrap{\bbl@pre@fs}%
2515     {\bbl@pre@fs}%
2516     {\bbl@post@fs}%
2517 \fi
2518 % == transforms ==
2519 % > luababel.def
2520 \def\CurrentOption{#2}%
2521 \@nameuse{\bbl@icsave#2}%
2522 % == main ==
2523 \ifx\bbl@KVP@main\@nnil % Restore only if not 'main'
2524     \let\language\bbl@savelangname
2525     \chardef\localeid\bbl@savelocaleid\relax
2526 \fi
2527 % == ==
2528 \ifx\ldf@finish\@onlypreamble\else
2529     \bbl@afterload
2530 \fi
2531 % == hyphenrules (apply if current) ==
2532 \ifx\bbl@KVP@hyphenrules\@nnil\else
2533     \ifnum\bbl@savelocaleid=\localeid
2534         \language\@nameuse{l@\language}%
2535     \fi
2536 \fi}

```

Depending on whether or not the language exists (based on `\date<language>`), we define two macros. Remember `\bbl@startcommands` opens a group.

```

2537 \def\bbl@provide@new#1{%
2538     \@namedef{date#1}{}% marks lang exists - required by \StartBabelCommands
2539     \@namedef{extras#1}{}%
2540     \@namedef{noextras#1}{}%
2541     \bbl@startcommands*{#1}{captions}%
2542     \ifx\bbl@KVP@captions\@nnil % and also if import, implicit
2543         \def\bbl@tempb##1% elt for \bbl@captionslist
2544         \ifx##1\@nnil\else
2545             \bbl@exp{%
2546                 \\SetString\\##1%
2547                 \\bbl@nocaption{\bbl@stripslash##1}{#1\bbl@stripslash##1}}%
2548             \expandafter\bbl@tempb
2549         \fi}%
2550     \expandafter\bbl@tempb\bbl@captionslist\@nnil
2551 \else

```

```

2552     \ifx\bbbl@initoload\relax
2553     \bbbl@read@ini{\bbbl@KVP@captions}2% % Here letters cat = 11
2554     \else
2555     \bbbl@read@ini{\bbbl@initoload}2% % Same
2556     \fi
2557 \fi
2558 \StartBabelCommands*{#1}{date}%
2559 \ifx\bbbl@KVP@date\@nnil
2560     \bbbl@exp{%
2561         \\SetString\\today{\\bbbl@nocaption{today}{#1today}}}%
2562     \else
2563     \bbbl@savetoday
2564     \bbbl@savedate
2565     \fi
2566 \bbbl@endcommands
2567 \bbbl@load@basic{#1}%
2568 % == hyphenmins == (only if new)
2569 \bbbl@exp{%
2570     \gdef\<#1hyphenmins>{%
2571         {\bbbl@ifunset{\bbbl@lfthm@#1}{2}{\bbbl@cs{lfthm@#1}}}%
2572         {\bbbl@ifunset{\bbbl@rgthm@#1}{3}{\bbbl@cs{rgthm@#1}}}%
2573     % == hyphenrules (also in renew) ==
2574     \bbbl@provide@hyphens{#1}%
2575     % == main ==
2576     \ifx\bbbl@KVP@main\@nnil\else
2577     \expandafter\main@language\expandafter{#1}%
2578     \fi}
2579 %
2580 \def\bbbl@provide@renew#1{%
2581     \ifx\bbbl@KVP@captions\@nnil\else
2582     \StartBabelCommands*{#1}{captions}%
2583     \bbbl@read@ini{\bbbl@KVP@captions}2% % Here all letters cat = 11
2584     \EndBabelCommands
2585     \fi
2586     \ifx\bbbl@KVP@date\@nnil\else
2587     \StartBabelCommands*{#1}{date}%
2588     \bbbl@savetoday
2589     \bbbl@savedate
2590     \EndBabelCommands
2591     \fi
2592     % == hyphenrules (also in new) ==
2593     \ifx\bbbl@lbfkflag\@empty
2594     \bbbl@provide@hyphens{#1}%
2595     \fi
2596     % == main ==
2597     \ifx\bbbl@KVP@main\@nnil\else
2598     \expandafter\main@language\expandafter{#1}%
2599     \fi}

```

Load the basic parameters (ids, typography, counters, and a few more), while captions and dates are left out. But it may happen some data has been loaded before automatically, so we first discard the saved values.

```

2600 \def\bbbl@load@basic#1{%
2601     \ifcase\bbbl@howloaded\or\or
2602     \ifcase\csname bbl@llevel@\language\endcsname
2603     \bbbl@csarg\let\lname@\language\relax
2604     \fi
2605     \fi
2606     \bbbl@ifunset{\bbbl@lname@#1}%
2607     {\def\BabelBeforeIni##1##2{%
2608         \begingroup
2609         \let\bbbl@ini@captions@aux\@gobbletwo
2610         \def\bbbl@inidate ####1.####2.####3.####4\relax ####5####6}%

```

```

2611         \bbl@read@ini{##1}l%
2612         \ifx\bbl@initoload\relax\endinput\fi
2613     \endgroup}%
2614     \begingroup      % boxed, to avoid extra spaces:
2615     \ifx\bbl@initoload\relax
2616         \bbl@input@texini{#1}%
2617     \else
2618         \setbox\z@\hbox{\BabelBeforeIni{\bbl@initoload}{}}%
2619     \fi
2620 \endgroup}%
2621 {}

```

The following ini reader ignores everything but the identification section. It is called when a font is defined (i.e., when the language is first selected) to know which script/language must be enabled. This means we must make sure a few characters are not active. The ini is not read directly, but with a proxy tex file named as the language (which means any code in it must be skipped, too).

```

2622 \def\bbl@load@info#1{%
2623   \def\BabelBeforeIni##1##2{%
2624     \begingroup
2625       \bbl@read@ini{##1}0%
2626       \endinput      % babel- .tex may contain onlypreamble's
2627     \endgroup}%      boxed, to avoid extra spaces:
2628   {\bbl@input@texini{#1}}}

```

The hyphenrules option is handled with an auxiliary macro. This macro is called in three cases: when a language is first declared with \babelprovide, with hyphenrules and with import.

```

2629 \def\bbl@provide@hyphens#1{%
2630   \@tempcnta\m@ne % a flag
2631   \ifx\bbl@KVP@hyphenrules\@nnil\else
2632     \bbl@replace\bbl@KVP@hyphenrules{ },}%
2633     \bbl@foreach\bbl@KVP@hyphenrules{%
2634       \ifnum\@tempcnta=\m@ne % if not yet found
2635         \bbl@ifsamestring{##1}{+}%
2636         {\bbl@carg\addlanguage{l@##1}}%
2637         {}%
2638         \bbl@ifunset{l@##1}% After a possible +
2639         {}%
2640         {\@tempcnta\@nameuse{l@##1}}%
2641       \fi}%
2642   \ifnum\@tempcnta=\m@ne
2643     \bbl@warning{%
2644       Requested 'hyphenrules' for '\language' not found:\\%
2645       \bbl@KVP@hyphenrules.\\%
2646       Using the default value. Reported}%
2647   \fi
2648 \fi
2649 \ifnum\@tempcnta=\m@ne % if no opt or no language in opt found
2650   \ifx\bbl@KVP@captions@\@nnil
2651     \bbl@ifunset\bbl@hyphr@#1{}% use value in ini, if exists
2652     {\bbl@exp{\bbl@ifblank{\bbl@cs{hyphr@#1}}}%
2653      {}%
2654      {\bbl@ifunset{l@\bbl@cl{hyphr}}%
2655       {}%
2656       {\@tempcnta\@nameuse{l@\bbl@cl{hyphr}}}}}%
2657   \fi
2658 \fi
2659 \bbl@ifunset{l@#1}%
2660 {\ifnum\@tempcnta=\m@ne
2661   \bbl@carg\adddialect{l@#1}\language
2662 \else
2663   \bbl@carg\adddialect{l@#1}\@tempcnta
2664 \fi}%
2665 {\ifnum\@tempcnta=\m@ne\else
2666   \global\bbl@carg\chardef{l@#1}\@tempcnta

```

```
2667 \fi}}
```

The reader of babel-...tex files. We reset temporarily some catcodes (and make sure no space is accidentally inserted).

```
2668 \def\bbl@input@texini#1{%
2669 \bbl@sphack
2670 \bbl@exp{%
2671 \catcode\%%=14 \catcode\==0
2672 \catcode\{=1 \catcode\}=2
2673 \lowercase{\InputIfFileExists{babel-#1.tex}{}}%
2674 \catcode\==\the\catcode\%\relax
2675 \catcode\==\the\catcode\%\relax
2676 \catcode\{=\the\catcode\%\relax
2677 \catcode\}= \the\catcode\%\relax}%
2678 \bbl@esphack}
```

The following macros read and store ini files (but don't process them). For each line, there are 3 possible actions: ignore if starts with ;, switch section if starts with [, and store otherwise. There are used in the first step of \bbl@read@ini.

```
2679 \def\bbl@iniline#1\bbl@iniline{%
2680 \ifnextchar[\bbl@inisect{\ifnextchar;\bbl@iniskip\bbl@inistore}#1\@@}% ]
2681 \def\bbl@inisect[#1]#2\@@{\def\bbl@section{#1}}
2682 \def\bbl@iniskip#1\@@{% if starts with ;
2683 \def\bbl@inistore#1=#2\@@{% full (default)
2684 \bbl@trim@def\bbl@tempa{#1}%
2685 \bbl@trim\toks@{#2}%
2686 \bbl@ifsamestring{\bbl@tempa}{@include}%
2687 {\bbl@read@subini{\the\toks@}}%
2688 {\bbl@xin@;\bbl@section/\bbl@tempa;}{\bbl@key@list}%
2689 \ifin@
2690 \bbl@xin@{,identification/include.}%
2691 {,\bbl@section/\bbl@tempa}%
2692 \ifin@\xdef\bbl@included@inis{\the\toks@}\fi
2693 \bbl@exp{%
2694 \\\g@addto@macro\\bbl@inidata{%
2695 \\\bbl@elt{\bbl@section}{\bbl@tempa}{\the\toks@}}}%
2696 \fi}}
2697 \def\bbl@inistore@min#1=#2\@@{% minimal (maybe set in \bbl@read@ini)
2698 \bbl@trim@def\bbl@tempa{#1}%
2699 \bbl@trim\toks@{#2}%
2700 \bbl@xin@{.identification.}{.\bbl@section.}%
2701 \ifin@
2702 \bbl@exp{\\g@addto@macro\\bbl@inidata{%
2703 \\\bbl@elt{identification}{\bbl@tempa}{\the\toks@}}}%
2704 \fi}
```

4.19. Main loop in 'provide'

Now, the 'main loop', \bbl@read@ini, which ****must be executed inside a group****. At this point, \bbl@inidata may contain data declared in \babelprovide, with 'slashed' keys. There are 3 steps: first read the ini file and store it; then traverse the stored values, and process some groups if required (date, captions, labels, counters); finally, 'export' some values by defining global macros (identification, typography, characters, numbers). The second argument is 0 when called to read the minimal data for fonts; with \babelprovide it's either 1 (without import) or 2 (which import). The value -1 is used with \DocumentMetadata.

\bbl@loop@ini is the reader, line by line (1: stream), and calls \bbl@iniline to save the key/value pairs. If \bbl@inistore finds the @include directive, the input stream is switched temporarily and \bbl@read@subini is called.

When the language is being set based on the document metadata (#2 in \bbl@read@ini is -1), there is an interlude to get the name, after the data have been collected, and before it's processed.

```
2705 \def\bbl@loop@ini#1{%
2706 \loop
2707 \if T\ifeof#1 F\fi T\relax % Trick, because inside \loop
```

```

2708     \endlinechar\m@ne
2709     \read#1 to \bbl@line
2710     \endlinechar`\^^M
2711     \ifx\bbl@line\@empty\else
2712         \expandafter\bbl@iniline\bbl@line\bbl@iniline
2713     \fi
2714 \repeat}
2715 %
2716 \def\bbl@read@subini#1{%
2717     \ifx\bbl@readsubstream\@undefined
2718         \csname newread\endcsname\bbl@readsubstream
2719     \fi
2720     \openin\bbl@readsubstream=babel-#1.ini
2721     \ifeof\bbl@readsubstream
2722         \bbl@error{no-ini-file}{#1}{}}%
2723     \else
2724         {\bbl@loop@ini\bbl@readsubstream}%
2725     \fi
2726     \closein\bbl@readsubstream}
2727 %
2728 \ifx\bbl@readstream\@undefined
2729     \csname newread\endcsname\bbl@readstream
2730 \fi
2731 \def\bbl@read@ini#1#2{%
2732     \global\let\bbl@extend@ini\@gobble
2733     \openin\bbl@readstream=babel-#1.ini
2734     \ifeof\bbl@readstream
2735         \bbl@error{no-ini-file}{#1}{}}%
2736     \else
2737         % == Store ini data in \bbl@inidata ==
2738         \catcode\ =10 \catcode\ "=12
2739         \catcode\[ =12 \catcode\ ] =12 \catcode\ ==12 \catcode\ & =12
2740         \catcode\ ; =12 \catcode\ | =12 \catcode\ % =14 \catcode\ - =12
2741         \ifnum#2=\m@ne % Just for the info
2742             \edef\language\tag \bbl@metalang}%
2743         \fi
2744         \bbl@info{\ifnum#2=\m@ne Fetching locale name for tag \bbl@metalang
2745             \else Importing
2746                 \ifcase#2font and identification \or basic \fi
2747                 data for \language
2748             \fi}%
2749         from babel-#1.ini. Reported}%
2750     \ifnum#2<\@ne
2751         \global\let\bbl@inidata\@empty
2752         \let\bbl@inistore\bbl@inistore@min % Remember it's local
2753     \fi
2754     \def\bbl@section{identification}%
2755     \bbl@exp{%
2756         \\bbl@inistore tag.ini=#1\\@@
2757         \\bbl@inistore load.level=\ifnum#2<\@ne 0\else #2\fi\\@@}%
2758     \bbl@loop@ini\bbl@readstream
2759     % == Process stored data ==
2760     \ifnum#2=\m@ne
2761         \def\bbl@tempa##1 ##2\@{##1}% Get first name
2762         \def\bbl@elt###1##2##3{%
2763             \bbl@ifsamestring{identification/name.babel}{##1/##2}%
2764             {\edef\language{\bbl@tempa##3 \@}%
2765             \let\localname\language
2766             \bbl@id@assign
2767             \def\bbl@elt####1####2####3{}}%
2768             {}}%
2769         \bbl@inidata
2770     \fi

```



```

2771 \bbl@csarg\xdef\luni@\language\name}{#1}%
2772 \bbl@read@ini@aux
2773 % == 'Export' data ==
2774 \bbl@ini@exports{#2}%
2775 \global\bbl@csarg\let\inidata@\language\name\bbl@inidata
2776 \global\let\bbl@inidata\empty
2777 \bbl@exp{\bbl@add@list\bbl@ini@loaded{\language\name}}%
2778 \bbl@tglobal\bbl@ini@loaded
2779 \@ifpackagewith{babel}{licr=unextended}{}%
2780 {\ifcase\bbl@engine\else % Find a better place
2781 \bbl@xin@{\bbl@cl{sbcpr},}{Cyril}%
2782 \ifin@
2783 \bbl@once{licr-cryl}{\g@addto@macro\bbl@afterload{\input{cyril2uni.def}}}%
2784 \fi
2785 \fi}%
2786 \fi
2787 \closein\bbl@readstream}
2788 \def\bbl@read@ini@aux{%
2789 \let\bbl@savestrings\empty
2790 \let\bbl@savetoday\empty
2791 \let\bbl@savestate\empty
2792 \def\bbl@elt##1##2##3{%
2793 \def\bbl@section{##1}%
2794 \in@{=date.}{=##1}% Find a better place
2795 \ifin@
2796 \bbl@ifunset{\bbl@inikv@##1}%
2797 {\bbl@ini@calendar{##1}}%
2798 {}%
2799 \fi
2800 \bbl@ifunset{\bbl@inikv@##1}{}%
2801 {\csname bbl@inikv@##1\endcsname{##2}{##3}}%
2802 \bbl@inidata}

```

A variant to be used when the ini file has been already loaded, because it's not the first \babelprovide for this language.

```

2803 \def\bbl@extend@ini@aux#1{%
2804 \bbl@startcommands*{#1}{captions}%
2805 % Activate captions/... and modify exports
2806 \bbl@csarg\def\inikv@captions.licr{##1##2}%
2807 \setlocalecaption{#1}{##1}{##2}%
2808 \def\bbl@inikv@captions##1##2{%
2809 \bbl@ini@captions@aux{##1}{##2}%
2810 \def\bbl@stringdef##1##2{\gdef##1{##2}}%
2811 \def\bbl@exportkey##1##2##3{%
2812 \bbl@ifunset{\bbl@kv@##2}{%
2813 {\expandafter\ifx\csname bbl@kv@##2\endcsname\empty\else
2814 \bbl@exp{\global\let<bbl@##1@\language\name>\<bbl@kv@##2>}}%
2815 \fi}}%
2816 % As with \bbl@read@ini, but with some changes
2817 \bbl@read@ini@aux
2818 \bbl@ini@exports\tw@
2819 % Update inidata@lang by pretending the ini is read.
2820 \def\bbl@elt##1##2##3{%
2821 \def\bbl@section{##1}%
2822 \bbl@iniline##2=##3\bbl@iniline}%
2823 \csname bbl@inidata@#1\endcsname
2824 \global\bbl@csarg\let\inidata@#1\bbl@inidata
2825 \StartBabelCommands*{#1}{date}% And from the import stuff
2826 \def\bbl@stringdef##1##2{\gdef##1{##2}}%
2827 \bbl@savetoday
2828 \bbl@savestate
2829 \bbl@endcommands}

```

A somewhat hackish tool to handle calendar sections.

```

2830 \def\bb@ini@calendar#1{%
2831 \lowercase{\def\bb@tempa{=#1=}}%
2832 \bb@replace\bb@tempa{=date.gregorian}{}%
2833 \bb@replace\bb@tempa{=date.}{}%
2834 \in@{.licr=}{#1=}%
2835 \ifin@
2836 \ifcase\bb@engine
2837 \bb@replace\bb@tempa{.licr=}{}%
2838 \else
2839 \let\bb@tempa\relax
2840 \fi
2841 \fi
2842 \ifx\bb@tempa\relax\else
2843 \bb@replace\bb@tempa{=}{}%
2844 \ifx\bb@tempa@empty\else
2845 \xdef\bb@calendars{\bb@calendars,\bb@tempa}%
2846 \fi
2847 \bb@exp{%
2848 \def\<bb@inikv@#1>###1###2{%
2849 \\\bb@inidate###1...\relax{###2}{\bb@tempa}}}%
2850 \fi}

```

A key with a slash in \babelprovide replaces the value in the ini file (which is ignored altogether). The mechanism is simple (but suboptimal): add the data to the ini one (at this point the ini file has not yet been read), and define a dummy macro. When the ini file is read, just skip the corresponding key and reset the macro (in \bb@inistore above).

```

2851 \def\bb@renewinikey#1/#2\@#3{%
2852 \global\let\bb@extend@ini\bb@extend@ini@aux
2853 \edef\bb@tempa{\zap@space #1 \@empty}% section
2854 \edef\bb@tempb{\zap@space #2 \@empty}% key
2855 \bb@trim\toks@{#3}% value
2856 \bb@exp{%
2857 \edef\\bb@key@list{\bb@key@list \bb@tempa/\bb@tempb;}%
2858 \\g@addto@macro\\bb@inidata{%
2859 \\\bb@elt{\bb@tempa}{\bb@tempb}{\the\toks@}}}%

```

The previous assignments are local, so we need to export them. If the value is empty, we can provide a default value.

```

2860 \def\bb@exportkey#1#2#3{%
2861 \bb@iifunset{\bb@kv@#2}%
2862 {\bb@csarg\gdef{#1@\language}\{#3}}%
2863 {\expandafter\ifx\csname \bb@kv@#2\endcsname\@empty
2864 \bb@csarg\gdef{#1@\language}\{#3}%
2865 \else
2866 \bb@exp{\global\let\<bb@#1@\language>\<bb@kv@#2>}%
2867 \fi}}

```

Key-value pairs are treated differently depending on the section in the ini file. The following macros are the readers for identification and typography. Note \bb@ini@exports is called always (via \bb@inise), while \bb@after@ini must be called explicitly after \bb@read@ini if necessary.

Although BCP 47 doesn't treat '-x-' as an extension, the CLDR and many other sources do (as a *private use extension*). For consistency with other single-letter subtags or 'singletons', here is considered an extension, too.

The identification section is used internally by babel in the following places [to be completed]: BCP 47 script tag in the Unicode ranges, which is in turn used by onchar; the language system is set with the names, and then fontspec maps them to the opentype tags, but if the latter package doesn't define them, then babel does it; encodings are used in pdftex to select a font encoding valid (and preloaded) for a language loaded on the fly.

```

2868 \def\bb@iniwarning#1{%
2869 \bb@iifunset{\bb@kv@identification.warning#1}{}%
2870 {\bb@warning%
2871 From babel-\bb@cs{lini@\language}.ini:\\%
2872 \bb@cs{@kv@identification.warning#1}\\%

```

```

2873         Reported}}
2874 %
2875 \let\bbl@release@transforms\@empty
2876 \let\bbl@release@casing\@empty

Relevant keys are ‘exported’, i.e., global macros with short names are created with values taken
from the corresponding keys. The number of exported keys depends on the loading level (#1): –1
and 0 only info (the identification section), 1 also basic (like linebreaking or character ranges), 2 also
(re)new (with date and captions).

2877 \def\bbl@ini@exports#1{%
2878   % Identification always exported
2879   \bbl@iniwarning{}}%
2880   \ifcase\bbl@engine
2881     \bbl@iniwarning{.pdf\latex}%
2882   \or
2883     \bbl@iniwarning{.lua\latex}%
2884   \or
2885     \bbl@iniwarning{.xel\latex}%
2886   \fi%
2887   \bbl@exportkey{lllevel}{identification.load.level}{}%
2888   \bbl@exportkey{elname}{identification.name.english}{}%
2889   \bbl@exp{\bbl@exportkey{lname}{identification.name.opentype}%
2890     {\csname bbl@elname\language\endcsname}}%
2891   \bbl@exportkey{tbc}{identification.tag.bcp47}{}%
2892   \bbl@exportkey{casing}{identification.tag.bcp47}{}%
2893   \bbl@exportkey{lbc}{identification.language.tag.bcp47}{}%
2894   \bbl@exportkey{lotf}{identification.tag.opentype}{dflt}%
2895   \bbl@exportkey{esname}{identification.script.name}{}%
2896   \bbl@exp{\bbl@exportkey{sname}{identification.script.name.opentype}%
2897     {\csname bbl@esname\language\endcsname}}%
2898   \bbl@exportkey{sbc}{identification.script.tag.bcp47}{}%
2899   \bbl@exportkey{sotf}{identification.script.tag.opentype}{DFLT}%
2900   \bbl@exportkey{rbcp}{identification.region.tag.bcp47}{}%
2901   \bbl@exportkey{vbc}{identification.variant.tag.bcp47}{}%
2902   \bbl@exportkey{extt}{identification.extension.t.tag.bcp47}{}%
2903   \bbl@exportkey{extu}{identification.extension.u.tag.bcp47}{}%
2904   \bbl@exportkey{extx}{identification.extension.x.tag.bcp47}{}%
2905   % Also maps bcp47 -> language
2906   \bbl@csarg\edef{bcp@map@bbl@cl{tbc}}{\language}%
2907   \ifcase\bbl@engine\or
2908     \directlua{%
2909       Babel.locale_props[\the\bbl@cs{id@\language}].script
2910       = '\bbl@cl{sbc}}}%
2911   \fi
2912   % Conditional
2913   \ifnum#1>\z@ % -1 or 0 = only info, 1 = basic, 2 = (re)new
2914     \bbl@exportkey{calpr}{date.calendar.preferred}{}%
2915     \bbl@exportkey{lbrk}{typography.linebreaking}{h}%
2916     \bbl@exportkey{hyphr}{typography.hyphenrules}{}%
2917     \bbl@exportkey{lftm}{typography.lefthyphenmin}{2}%
2918     \bbl@exportkey{rgthm}{typography.righthyphenmin}{3}%
2919     \bbl@exportkey{prehc}{typography.prehyphenchar}{}%
2920     \bbl@exportkey{hyotl}{typography.hyphenate.other.locale}{}%
2921     \bbl@exportkey{hyots}{typography.hyphenate.other.script}{}%
2922     \bbl@exportkey{intsp}{typography.intraspaces}{}%
2923     \bbl@exportkey{frspc}{typography.frenchspacing}{u}%
2924     \bbl@exportkey{chrng}{characters.ranges}{}%
2925     \bbl@exportkey{quote}{characters.delimiters.quotes}{}%
2926     \bbl@exportkey{dgnat}{numbers.digits.native}{}%
2927     \ifnum#1=\tw@ % only (re)new
2928       \bbl@exportkey{rqtex}{identification.require.babel}{}%
2929       \bbl@tglobal\bbl@savetoday
2930       \bbl@tglobal\bbl@savestate

```

```

2931      \bbl@savestrings
2932      \fi
2933      \fi}

```

4.20. Processing keys in ini

A shared handler for key=val lines to be stored in \bbl@kv@<section>.<key>.

```

2934 \def\bbl@inikv#1#2{%      key=value
2935   \toks@{#2}%              This hides #'s from ini values
2936   \bbl@csarg\edef{@kv@\bbl@section.#1}{\the\toks@}}

```

By default, the following sections are just read. Actions are taken later.

```

2937 \let\bbl@inikv@identification\bbl@inikv
2938 \let\bbl@inikv@date\bbl@inikv
2939 \let\bbl@inikv@typography\bbl@inikv
2940 \let\bbl@inikv@numbers\bbl@inikv

```

The characters section also stores the values, but casing is treated in a different fashion. Much like transforms, a set of commands calling the parser are stored in \bbl@release@casing, which is executed in \babelprovide.

```

2941 \def\bbl@maybextx{-\bbl@csarg\ifx{extx@\language}\empty x-\fi}
2942 \def\bbl@inikv@characters#1#2{%
2943   \bbl@ifsamestring{#1}{casing}% e.g., casing = uV
2944   {\bbl@exp{%
2945     \\g@addto@macro\\bbl@release@casing{%
2946       \\bbl@casemapping}{\language}{\unexpanded{#2}}}%
2947   {\in{${casing}.}{$#1}% e.g., casing.Uv = uV
2948   \ifin@
2949     \lowercase{\def\bbl@tempb{#1}%
2950     \bbl@replace\bbl@tempb{casing.}{}%
2951     \bbl@exp{\\g@addto@macro\\bbl@release@casing{%
2952       \\bbl@casemapping
2953       {\bbl@maybextx\bbl@tempb}{\language}{\unexpanded{#2}}}%
2954     \else
2955       \bbl@inikv{#1}{#2}%
2956     \fi}}

```

Additive numerals require an additional definition. When .1 is found, two macros are defined – the basic one, without .1 called by \localenumeral, and another one preserving the trailing .1 for the ‘units’.

```

2957 \def\bbl@inikv@counters#1#2{%
2958   \bbl@ifsamestring{#1}{digits}%
2959   {\bbl@error{digits-is-reserved}{}}}%
2960   {}%
2961   \def\bbl@tempc{#1}%
2962   \bbl@trim@def{\bbl@tempb*}{#2}%
2963   \in{.1$}{#1$}%
2964   \ifin@
2965     \bbl@replace\bbl@tempc{.1}{}%
2966     \bbl@csarg\protected@xdef{cntr@\bbl@tempc @\language}{%
2967       \noexpand\bbl@alphanumeric{\bbl@tempc}}%
2968   \fi
2969   \in{.F.}{#1}%
2970   \ifin@else\in{.S.}{#1}\fi
2971   \ifin@
2972     \bbl@csarg\protected@xdef{cntr@#1@\language}{\bbl@tempb*}%
2973   \else
2974     \toks@{}% Required by \bbl@buildifcase, which returns \bbl@tempa
2975     \expandafter\bbl@buildifcase\bbl@tempb* \ \ % Space after \
2976     \bbl@csarg{\global\expandafter\let}{cntr@#1@\language}\bbl@tempa
2977   \fi}

```

Now captions and captions.licr, depending on the engine. And below also for dates. They rely on a few auxiliary macros. It is expected the ini file provides the complete set in Unicode and LICR, in that order.

```

2978 \ifcase\bbl@engine
2979   \bbl@csarg\def{inikv@captions.licr}#1#2{%
2980     \bbl@ini@captions@aux{#1}{#2}}
2981 \else
2982   \def\bbl@inikv@captions#1#2{%
2983     \bbl@ini@captions@aux{#1}{#2}}
2984 \fi

```

The auxiliary macro for captions define \<caption>name.

```

2985 \def\bbl@ini@captions@template#1#2{% string language tempa=capt-name
2986   \bbl@replace\bbl@tempa{.template}{}%
2987   \def\bbl@toreplace{#1}{}%
2988   \bbl@replace\bbl@toreplace[ ]{\nobreakspace{}}%
2989   \bbl@replace\bbl@toreplace[{}]{\csname}%
2990   \bbl@replace\bbl@toreplace[{}]{\csname the}%
2991   \bbl@replace\bbl@toreplace[{}]{name\endcsname{}}%
2992   \bbl@replace\bbl@toreplace[{}]{\endcsname{}}%
2993   \bbl@xin@{,\bbl@tempa,}{,chapter,appendix,part,}%
2994   \ifin@
2995     \@nameuse{\bbl@patch\bbl@tempa}%
2996     \global\bbl@csarg\let{\bbl@tempa fmt@#2}\bbl@toreplace
2997   \fi
2998   \bbl@xin@{,\bbl@tempa,}{,figure,table,}%
2999   \ifin@
3000     \global\bbl@csarg\let{\bbl@tempa fmt@#2}\bbl@toreplace
3001     \bbl@exp{\gdef\<fnum@\bbl@tempa>{%
3002       \\bbl@ifunset{\bbl@bbl@tempa fmt@\\language}%
3003       {[fnum@\bbl@tempa]}%
3004       {\\@nameuse{\bbl@bbl@tempa fmt@\\language}}}%
3005     \fi}
3006 %
3007 \def\bbl@ini@captions@aux#1#2{%
3008   \bbl@trim\def\bbl@tempa{#1}%
3009   \bbl@xin@{.template}{\bbl@tempa}%
3010   \ifin@
3011     \bbl@ini@captions@template{#2}\language
3012   \else
3013     \bbl@ifblank{#2}%
3014     {\bbl@exp{%
3015       \toks@{\\bbl@nocaption{\bbl@tempa name}{\language\bbl@tempa name}}}%
3016     {\bbl@trim\toks@{#2}}}%
3017     \bbl@exp{%
3018       \\bbl@add\\bbl@savestrings{%
3019         \\SetString\<\bbl@tempa name>{\the\toks@}}%
3020       \toks@\expandafter{\bbl@captionslist}%
3021       \bbl@exp{\\in@{\<\bbl@tempa name>}{\the\toks@}}%
3022     \ifin@else
3023       \bbl@exp{%
3024         \\bbl@add\<\bbl@extracaps@language>{\<\bbl@tempa name>}%
3025         \\bbl@tglobal\<\bbl@extracaps@language>}%
3026       \fi
3027     \fi}

```

Labels. Captions must contain just strings, no format at all, so there is new group in ini files.

```

3028 \def\bbl@list@the{%
3029   part,chapter,section,subsection,subsubsection,paragraph,%
3030   subparagraph,enumi,enumii,enumiii,enumiv,equation,figure,%
3031   table,page,footnote,mpfootnote,mpfn}
3032 %
3033 \def\bbl@map@cnt#1{% #1:roman,etc, // #2:enumi,etc

```

```

3034 \bbl@ifunset{bbl@map@#1@\language}%
3035 {\@nameuse{#1}}%
3036 {\@nameuse{bbl@map@#1@\language}}%
3037 %
3038 \def\bbl@map@lbl#1{% #1:a sign, eg, .
3039 \ifincsname#1\else
3040 \bbl@ifunset{bbl@map@@#1@\language}%
3041 {#1}%
3042 {\@nameuse{bbl@map@@#1@\language}}%
3043 \fi}
3044 %
3045 \def\bbl@inikv@labels#1#2{%
3046 \in@{.map}{#1}%
3047 \ifin@
3048 \in@{,dot.map,}{, #1,}%
3049 \ifin@
3050 \global\@namedef{bbl@map@@@\language}{#2}%
3051 \fi
3052 \ifx\bbl@KVP@labels\@nnil\else
3053 \bbl@xin@{ map }{ \bbl@KVP@labels\space}%
3054 \ifin@
3055 \def\bbl@tempc{#1}%
3056 \bbl@replace\bbl@tempc{.map}{}%
3057 \in@{, #2,}{,arabic,roman,Roman,alph,Alph,fnsymbol,}%
3058 \bbl@exp{%
3059 \gdef\<bbl@map@\bbl@tempc @\language>%
3060 {\ifin@<#2>\else\\localecounter{#2}\fi}}%
3061 \bbl@foreach\bbl@list@the{%
3062 \bbl@ifunset{the##1}{}%
3063 {\bbl@ncarg\let\bbl@tempd{the##1}%
3064 \bbl@exp{%
3065 \\bbl@sreplace\<the##1>%
3066 {\<\bbl@tempc>{##1}}%
3067 {\bbl@map@cnt{\bbl@tempc}{##1}}%
3068 \\bbl@sreplace\<the##1>%
3069 {\<\@empty @\bbl@tempc>\<c@##1>%
3070 {\bbl@map@cnt{\bbl@tempc}{##1}}%
3071 \\bbl@sreplace\<the##1>%
3072 {\bbl@csname @\bbl@tempc\\endcsname\<c@##1>%
3073 {\bbl@map@cnt{\bbl@tempc}{##1}}}%
3074 \expandafter\ifx\csname the##1\endcsname\bbl@tempd\else
3075 \bbl@exp{\gdef\<the##1>{\[the##1]}}%
3076 \fi}}%
3077 \fi
3078 \fi
3079 %
3080 \else
3081 % The following code is still under study. You can test it and make
3082 % suggestions. E.g., enumerate.2 = ([enumi]).([enumii]). It's
3083 % language dependent.
3084 \in@{enumerate.}{#1}%
3085 \ifin@
3086 \def\bbl@tempa{#1}%
3087 \bbl@replace\bbl@tempa{enumerate.}{}%
3088 \def\bbl@toreplace{#2}%
3089 \bbl@replace\bbl@toreplace{[ ]}{\nobreakspace}%
3090 \bbl@replace\bbl@toreplace{[ ]}{\csname the}%
3091 \bbl@replace\bbl@toreplace{[ ]}{\endcsname}}%
3092 \toks@{\expandafter\bbl@toreplace}%
3093 \bbl@exp{%
3094 \\bbl@add\<extras\language>{%
3095 \\babel@save\<labelenum\romannumeral\bbl@tempa>%
3096 \def\<labelenum\romannumeral\bbl@tempa>\the\toks@}%

```

```

3097      \\bbl@tglobal<extras\languagename>}%
3098      \fi
3099      \fi}

```

To show correctly some captions in a few languages, we need to patch some internal macros, because the order is hardcoded. For example, in Japanese the chapter number is surrounded by two string, while in Hungarian is placed after. These replacement works in many classes, but not all. Actually, the following lines are somewhat tentative.

```

3100 \def\bbl@chapttype{chapter}
3101 \ifx\@makechapterhead\undefined
3102   \let\bbl@patchchapter\relax
3103 \else\ifx\thechapter\undefined
3104   \let\bbl@patchchapter\relax
3105 \else\ifx\ps@headings\undefined
3106   \let\bbl@patchchapter\relax
3107 \else
3108   \def\bbl@patchchapter{%
3109     \global\let\bbl@patchchapter\relax
3110     \gdef\bbl@chfmt{%
3111       \bbl@ifunset{bbl@\bbl@chapttype fmt@\languagename}%
3112       {\@chapapp\space\thechapter}%
3113       {\@nameuse{bbl@\bbl@chapttype fmt@\languagename}}}%
3114     \bbl@add\appendix{\def\bbl@chapttype{appendix}}% Not harmful, I hope
3115     \bbl@sreplace\ps@headings{\@chapapp\ \thechapter}{\bbl@chfmt}%
3116     \bbl@sreplace\chaptermark{\@chapapp\ \thechapter}{\bbl@chfmt}%
3117     \bbl@sreplace\@makechapterhead{\@chapapp\space\thechapter}{\bbl@chfmt}%
3118     \bbl@tglobal\appendix
3119     \bbl@tglobal\ps@headings
3120     \bbl@tglobal\chaptermark
3121     \bbl@tglobal\@makechapterhead}
3122   \let\bbl@patchappendix\bbl@patchchapter
3123 \fi\fi\fi
3124 \ifx\@part\undefined
3125   \let\bbl@patchpart\relax
3126 \else
3127   \def\bbl@patchpart{%
3128     \global\let\bbl@patchpart\relax
3129     \gdef\bbl@partformat{%
3130       \bbl@ifunset{bbl@partfmt@\languagename}%
3131       {\@partname\nobreakspace\thepart}%
3132       {\@nameuse{bbl@partfmt@\languagename}}}%
3133     \bbl@sreplace\@part{\@partname\nobreakspace\thepart}{\bbl@partformat}%
3134     \bbl@tglobal\@part}
3135 \fi

```

Date. Arguments (year, month, day) are *not* protected, on purpose. In \today, arguments are always gregorian, and therefore always converted with other calendars.

```

3136 \let\bbl@calendar\@empty
3137 \DeclareRobustCommand\localedate[1][\bbl@localedate{#1}]
3138 \def\bbl@localedate#1#2#3#4{%
3139   \begingroup
3140     \edef\bbl@they{#2}%
3141     \edef\bbl@them{#3}%
3142     \edef\bbl@thed{#4}%
3143     \edef\bbl@tempe{%
3144       \bbl@ifunset{bbl@calpr@\languagename}{\bbl@cl{calpr}},%
3145       #1}%
3146     \bbl@exp{\lowercase{\edef\\bbl@tempe{\bbl@tempe}}}%
3147     \bbl@replace\bbl@tempe{ }{}%
3148     \bbl@replace\bbl@tempe{convert}{convert=}%
3149     \let\bbl@ld@calendar\@empty
3150     \let\bbl@ld@variant\@empty
3151     \let\bbl@ld@convert\relax
3152     \def\bbl@tempb##1=##2\@@{\@namedef{bbl@ld@##1}{##2}}%

```

```

3153 \bbl@foreach\bbl@tempe{\bbl@tempb##1\@@}%
3154 \bbl@replace\bbl@ld@calendar{gregorian}{}%
3155 \ifx\bbl@ld@calendar\@empty\else
3156 \ifx\bbl@ld@convert\relax\else
3157 \babelcalendar[\bbl@they-\bbl@them-\bbl@thed]%
3158 {\bbl@ld@calendar}\bbl@they\bbl@them\bbl@thed
3159 \fi
3160 \fi
3161 \@nameuse{\bbl@precalendar}% Remove, e.g., +, -civil (-ca-islamic)
3162 \edef\bbl@calendar{% Used in \month..., too
3163 \bbl@ld@calendar
3164 \ifx\bbl@ld@variant\@empty\else
3165 .\bbl@ld@variant
3166 \fi}%
3167 \bbl@cased
3168 {\@nameuse{\bbl@date@\language name @\bbl@calendar}%
3169 \bbl@they\bbl@them\bbl@thed}%
3170 \endgroup}
3171 %
3172 \def\bbl@printdate#1{%
3173 \ifnextchar[{\bbl@printdatei{#1}}{\bbl@printdatei{#1}[]}}
3174 \def\bbl@printdatei#1[#2]#3#4#5{%
3175 \bbl@usedategroupt true
3176 \def\bbl@tempc{#1}%
3177 \bbl@replace\bbl@tempc{-}{@}%
3178 \@nameuse{\bbl@ensure@\bbl@tempc}{\localedate[#2]{#3}{#4}{#5}}}
3179 %
3180 e.g.: 1=months, 2=wide, 3=1, 4=dummy, 5=value, 6=calendar
3181 \def\bbl@inidate#1.#2.#3.#4\relax#5#6{%
3182 \bbl@trim@def\bbl@tempa{#1.#2}%
3183 \bbl@ifsamestring{\bbl@tempa}{months.wide}% to savedate
3184 {\bbl@trim@def\bbl@tempa{#3}%
3185 \bbl@trim\toks@{#5}%
3186 \@temptokena\expandafter{\bbl@savedate}%
3187 \bbl@exp{% Reverse order - in ini last wins
3188 \def\\bbl@savedate{%
3189 \\SetString\<month\romannumeral\bbl@tempa#6name>{\the\toks@}%
3190 \the\@temptokena}}}%
3191 {\bbl@ifsamestring{\bbl@tempa}{date.long}% defined now
3192 {\lowercase{\def\bbl@tempb{#6}}}%
3193 \bbl@trim@def\bbl@toreplace{#5}%
3194 \bbl@TG@@date
3195 \global\bbl@csarg\let{date@\language name @\bbl@tempb}\bbl@toreplace
3196 \ifx\bbl@savetoday\@empty
3197 \bbl@exp{%
3198 \\AfterBabelCommands{%
3199 \gdef\<\language name date>{\protect\<\language name date >}%
3200 \gdef\<\language name date >{\bbl@printdate{\language name}}}%
3201 \def\\bbl@savetoday{%
3202 \\SetString\\today{%
3203 \<\language name date>[convert]%
3204 {\the\year}{\the\month}{\the\day}}}%
3205 \fi}%
3206 {}}}

```

Dates will require some macros for the basic formatting. They may be redefined by language, so “semi-public” names (camel case) are used. Oddly enough, the CLDR places particles like “de” inconsistently in either in the date or in the month name. Note after \bbl@replace \toks@ contains the resulting string, which is used by \bbl@replace@finish@iii (this implicit behavior doesn’t seem a good idea, but it’s efficient).

```

3207 \let\bbl@calendar\@empty
3208 \newcommand\babelcalendar[2][\the\year-\the\month-\the\day]{%
3209 \@nameuse{\bbl@ca@#2}#1\@@}

```



```

3210 \newcommand\BabelDateSpace{\nobreakspace}
3211 \newcommand\BabelDateDot{.\@}
3212 \newcommand\BabelDated[1]{\{\number#1\}}
3213 \newcommand\BabelDatedd[1]{\{\ifnum#1<10 0\fi\number#1\}}
3214 \newcommand\BabelDateM[1]{\{\number#1\}}
3215 \newcommand\BabelDateMM[1]{\{\ifnum#1<10 0\fi\number#1\}}
3216 \newcommand\BabelDateMMMM[1]{\{%
3217   \csname month\romannumeral#1\bbbl@calendar name\endcsname\}}%
3218 \newcommand\BabelDatey[1]{\{\number#1\}}%
3219 \newcommand\BabelDateyy[1]{\{%
3220   \ifnum#1<10 0\number#1 %
3221   \else\ifnum#1<100 \number#1 %
3222   \else\ifnum#1<1000 \expandafter\@gobble\number#1 %
3223   \else\ifnum#1<10000 \expandafter\@gobbletwo\number#1 %
3224   \else
3225     \bbbl@error{limit-two-digits}{\}\{\}\}%
3226   \fi\fi\fi\fi}
3227 \newcommand\BabelDateyyyy[1]{\{\number#1\}}
3228 \newcommand\BabelDateU[1]{\{\number#1\}}%
3229 \def\bbbl@replace@finish@iii#1{%
3230   \bbbl@exp{\def\#1####1####2####3{\the\toks@}}
3231 \def\bbbl@TG@@date{%
3232   \bbbl@replace\bbbl@toreplace{[ ]}{\BabelDateSpace}}%
3233   \bbbl@replace\bbbl@toreplace{[. ]}{\BabelDateDot}}%
3234   \bbbl@replace\bbbl@toreplace{[y]}{\BabelDatey{####1}}%
3235   \bbbl@replace\bbbl@toreplace{[y]}{\bbbl@datecctr[####1]}%
3236   \bbbl@replace\bbbl@toreplace{[yy]}{\BabelDateyy{####1}}%
3237   \bbbl@replace\bbbl@toreplace{[yyyy]}{\BabelDateyyyy{####1}}%
3238   \bbbl@replace\bbbl@toreplace{[M]}{\BabelDateM{####2}}%
3239   \bbbl@replace\bbbl@toreplace{[M]}{\bbbl@datecctr[####2]}%
3240   \bbbl@replace\bbbl@toreplace{[MM]}{\BabelDateMM{####2}}%
3241   \bbbl@replace\bbbl@toreplace{[MMMM]}{\BabelDateMMMM{####2}}%
3242   \bbbl@replace\bbbl@toreplace{[d]}{\BabelDated{####3}}%
3243   \bbbl@replace\bbbl@toreplace{[d]}{\bbbl@datecctr[####3]}%
3244   \bbbl@replace\bbbl@toreplace{[dd]}{\BabelDatedd{####3}}%
3245   \bbbl@replace\bbbl@toreplace{[U]}{\BabelDateU{####1}}%
3246   \bbbl@replace\bbbl@toreplace{[U]}{\bbbl@datecctr[####1]}%
3247   \bbbl@replace@finish@iii\bbbl@toreplace}
3248 \def\bbbl@datecctr{\expandafter\bbbl@xdatecctr\expandafter}
3249 \def\bbbl@xdatecctr[#1|#2]{\localenumeral{#2}{#1}}

```

4.21. French spacing (again)

For the following declarations, see issue #240. \nonfrenchspacing is set by document too early, so it's a hack.

```

3250 \AddToHook{begindocument/before}{%
3251   \let\bbbl@normalsf\normalsfcodes
3252   \let\normalsfcodes\relax}
3253 \AtBeginDocument{%
3254   \ifx\bbbl@normalsf\@empty
3255     \ifnum\sfcodes\@m
3256       \let\normalsfcodes\frenchspacing
3257     \else
3258       \let\normalsfcodes\nonfrenchspacing
3259     \fi
3260   \else
3261     \let\normalsfcodes\bbbl@normalsf
3262   \fi}

```

Transforms.

Process the transforms read from ini files, converts them to a form close to the user interface (with \babelprehyphenation and \babelprehyphenation), wrapped with \bbbl@transforms@aux ... \relax, and stores them in \bbbl@release@transforms. However, since building a list enclosed in

braces isn't trivial, the replacements are added after a comma, and then `\bbl@transforms@aux` adds the braces.

```

3263 \bbl@csarg\let{inikv@transforms.prehyphenation}\bbl@inikv
3264 \bbl@csarg\let{inikv@transforms.posthyphenation}\bbl@inikv
3265 \def\bbl@transforms@aux#1#2#3#4,#5\relax{%
3266   #1[#2]{#3}{#4}{#5}}
3267 \begingroup
3268   \catcode`\%=12
3269   \catcode`\&=14
3270   \gdef\bbl@transforms#1#2#3{%&
3271     \directlua{
3272       local str = [==[#2]==]
3273       str = str:gsub('%.%d+%.%d+$', '')
3274       token.set_macro('babeltempa', str)
3275     }&
3276     \def\babeltempc{}&
3277     \bbl@xin@{,\babeltempa,}{,\bbl@KVP@transforms,}&
3278     \ifin@else
3279       \bbl@xin@{:,\babeltempa,}{,\bbl@KVP@transforms,}&
3280     \fi
3281     \ifin@
3282       \bbl@foreach\bbl@KVP@transforms{%&
3283         \bbl@xin@{:,\babeltempa,}{,##1,}&
3284         \ifin@ & font:font:transform syntax
3285         \directlua{
3286           local t = {}
3287           for m in string.gmatch('##1'..' ':'(.)') do
3288             table.insert(t, m)
3289           end
3290           table.remove(t)
3291           token.set_macro('babeltempc', ',fonts=' .. table.concat(t, ' '))
3292         }&
3293       \fi}&
3294     \in@{.0$}{#2$}&
3295     \ifin@
3296       \directlua{%& (\attribute) syntax
3297         local str = string.match([[ \bbl@KVP@transforms]],
3298           '%([^(%[%])[%^%)]-\babeltempa')
3299         if str == nil then
3300           token.set_macro('babeltempb', '')
3301         else
3302           token.set_macro('babeltempb', ',attribute=' .. str)
3303         end
3304       }&
3305     \toks@{#3}&
3306     \bbl@exp{%&
3307       \\g@addto@macro\\bbl@release@transforms{%&
3308         \relax & Closes previous \bbl@transforms@aux
3309         \\bbl@transforms@aux
3310         \\#1{label=\babeltempa\babeltempb\babeltempc}&
3311         {\language\the\toks@}}&
3312     \else
3313       \g@addto@macro\bbl@release@transforms{, {#3}}&
3314     \fi
3315   \fi}
3316 \endgroup

```

4.22. Handle language system

The language system (i.e., Language and Script) to be used when defining a font or setting the direction are set with the following macros. It also deals with unhyphenated line breaking in xetex (e.g., Thai and traditional Sanskrit), which is done with a hack at the font level because this engine

doesn't support it.

4.23. Numerals

```

3342 \def\bbl@setdigits#1#2#3#4#5{%
3343   \bbl@exp{%
3344     \def\<\language name digits>####1{%          i.e., \langdigits
3345       \<bbl@digits@\language name>#####1\\@nil}%
3346       \let\<bbl@cuntr@digits@\language name>\<\language name digits>%
3347       \def\<\language name counter>####1{%        i.e., \langcounter
3348         \\expandafter\<bbl@counter@\language name>%
3349         \\c sname c#####1\endcsname}%
3350       \def\<bbl@counter@\language name>####1% i.e., \bbl@counter@lang
3351         \\expandafter\<bbl@digits@\language name>%
3352         \\number####1\\@nil}}}%
3353 \def\bbl@tempa##1##2###3###4###5{%
3354   \bbl@exp{%      Wow, quite a lot of hashes! :-(
3355     \def\<bbl@digits@\language name>#####1{%
3356       \\ifx#####1\\@nil                      % i.e., \bbl@digits@lang
3357       \\else
3358         \\ifx0#####1#1%
3359         \\else\\ifx1#####1#2%
3360         \\else\\ifx2#####1#3%
3361         \\else\\ifx3#####1#4%
3362         \\else\\ifx4#####1#5%
3363         \\else\\ifx5#####1##1%
3364         \\else\\ifx6#####1##2%
3365         \\else\\ifx7#####1##3%
3366         \\else\\ifx8#####1##4%
3367         \\else\\ifx9#####1##5%
3368         \\else#####1%
3369         \\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi
3370         \\expandafter\<bbl@digits@\language name>%
3371         \\fi}}} }%
```

```
3372 \bbl@tempa}
```

Alphabetic counters must be converted from a space separated list to an \ifcase structure.

```
3373 \def\bbl@buildifcase#1 {% Returns \bbl@tempa, requires \toks@={}%
3374 \ifx\\#1% % \\ before, in case #1 is multiletter
3375 \bbl@exp{%
3376 \def\\bbl@tempa###1{%
3377 \<ifcase>###1\space\the\toks@<else>\\@ctrerr<fi>}}%
3378 \else
3379 \toks@<expandafter{\the\toks@<or #1>%
3380 \expandafter\bbl@buildifcase
3381 \fi}
```

The code for additive counters is somewhat tricky and it's based on the fact the arguments just before @@ collects digits which have been left 'unused' in previous arguments, the first of them being the number of digits in the number to be converted. This explains the reverse set 76543210. Digits above 10000 are not handled yet. When the key contains the subkey .F., the number after is treated as an special case, for a fixed form (see babel-he.ini, for example).

```
3382 \newcommand\localenumeral[2]{%
3383 \bbl@ifunset{bbl@cntr@#1@\language}%
3384 {#2}%
3385 {\bbl@cs{cntr@#1@\language}{#2}}}
3386 \def\bbl@localecntr#1#2{\localenumeral{#2}{#1}}
3387 \newcommand\localecounter[2]{%
3388 \expandafter\bbl@localecntr
3389 \expandafter{\number\csname c@#2\endcsname}{#1}}
3390 \def\bbl@alphnumeral#1#2{%
3391 \expandafter\bbl@alphnumeral@i\number#2 76543210\@@{#1}}
3392 \def\bbl@alphnumeral@i#1#2#3#4#5#6#7#8\@@#9{%
3393 \ifcase\@car#8\@nil\or % Currently <10000, but prepared for bigger
3394 \bbl@alphnumeral@ii{#9}00000#1\or
3395 \bbl@alphnumeral@ii{#9}00000#1#2\or
3396 \bbl@alphnumeral@ii{#9}0000#1#2#3\or
3397 \bbl@alphnumeral@ii{#9}000#1#2#3#4\else
3398 \bbl@alphnum@invalid{>9999}%
3399 \fi}
3400 \def\bbl@alphnumeral@ii#1#2#3#4#5#6#7#8{%
3401 \bbl@ifunset{bbl@cntr@#1.F.\number#5#6#7#8@\language}%
3402 {\bbl@cs{cntr@#1.4@\language}{#5}%
3403 \bbl@cs{cntr@#1.3@\language}{#6}%
3404 \bbl@cs{cntr@#1.2@\language}{#7}%
3405 \bbl@cs{cntr@#1.1@\language}{#8}%
3406 \ifnum#6#7#8>\z@
3407 \bbl@ifunset{bbl@cntr@#1.S.321@\language}{}%
3408 {\bbl@cs{cntr@#1.S.321@\language}{}%
3409 \fi}%
3410 {\bbl@cs{cntr@#1.F.\number#5#6#7#8@\language}}}
3411 \def\bbl@alphnum@invalid#1{%
3412 \bbl@error{alphabetic-too-large}{#1}{}}
```

4.24. Casing

```
3413 \newcommand\BabelUppercaseMapping[3]{%
3414 \DeclareUppercaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}
3415 \newcommand\BabelTitlecaseMapping[3]{%
3416 \DeclareTitlecaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}
3417 \newcommand\BabelLowercaseMapping[3]{%
3418 \DeclareLowercaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}
```

The parser for casing and casing. (variant).

```
3419 \ifcase\bbl@engine % Converts utf8 to its code (expandable)
3420 \def\bbl@utftocode#1{\the\numexpr\decode@UTFviii#1\relax}
3421 \else
3422 \def\bbl@utftocode#1{\expandafter\string#1}
```

```

3423 \fi
3424 \def\bbl@casemapping#1#2#3{% 1:variant
3425   \def\bbl@tempa##1 ##2{% Loop
3426     \bbl@casemapping@i{##1}%
3427     \ifx\@empty##2\else\bbl@afterfi\bbl@tempa##2\fi}%
3428   \edef\bbl@templ{\@nameuse{bbl@casing@#2}#1}% Language code
3429   \def\bbl@tempe{0}% Mode (upper/lower...)
3430   \def\bbl@tempc{#3}% Casing list
3431   \expandafter\bbl@tempa\bbl@tempc\@empty}
3432 \def\bbl@casemapping@i#1{%
3433   \def\bbl@tempb{#1}%
3434   \ifcase\bbl@engine % Handle utf8 in pdftex, by surrounding chars with {}
3435     \@nameuse{regex_replace_all:nnN}%
3436     {[\\x{c0}-\\x{ff}][\\x{80}-\\x{bf}]*}{\\0}}\bbl@tempb
3437   \else
3438     \@nameuse{regex_replace_all:nnN}{.}{\\0}}\bbl@tempb
3439   \fi
3440   \expandafter\bbl@casemapping@ii\bbl@tempb\@{
3441 \def\bbl@casemapping@ii#1#2#3\@{
3442   \in@{#1#3}{<>}% i.e., if <u>, <l>, <t>
3443   \ifin@
3444     \edef\bbl@tempe{%
3445       \if#2u1 \else\if#2l2 \else\if#2t3 \fi\fi\fi}%
3446   \else
3447     \ifcase\bbl@tempe\relax
3448       \DeclareUppercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3449       \DeclareLowercaseMapping[\bbl@templ]{\bbl@uftocode{#2}}{#1}%
3450     \or
3451       \DeclareUppercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3452     \or
3453       \DeclareLowercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3454     \or
3455       \DeclareTitlecaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3456   \fi
3457   \fi}

```

4.25. Getting info

The information in the identification section can be useful, so the following macro just exposes it with a user command.

```

3458 \def\bbl@localeinfo#1#2{%
3459   \bbl@ifunset{bbl@info@#2}{#1}%
3460   {\bbl@ifunset{bbl@csname bbl@info@#2\endcsname @\languagename}{#1}%
3461     {\bbl@cs{csname bbl@info@#2\endcsname @\languagename}}}%
3462 \newcommand\localeinfo[1]{%
3463   \ifx*#1\@empty
3464     \bbl@afterelse\bbl@localeinfo{%
3465   \else
3466     \bbl@localeinfo
3467     {\bbl@error{no-ini-info}}{}}}%
3468   {#1}%
3469   \fi}
3470 % \@namedef{bbl@info@name.locale}{lcname}
3471 \@namedef{bbl@info@tag.ini}{lini}
3472 \@namedef{bbl@info@name.english}{elname}
3473 \@namedef{bbl@info@name.opentype}{lname}
3474 \@namedef{bbl@info@tag.bcp47}{tbc}
3475 \@namedef{bbl@info@language.tag.bcp47}{lbc}
3476 \@namedef{bbl@info@tag.opentype}{lotf}
3477 \@namedef{bbl@info@script.name}{esname}
3478 \@namedef{bbl@info@script.name.opentype}{sname}
3479 \@namedef{bbl@info@script.tag.bcp47}{sbcp}
3480 \@namedef{bbl@info@script.tag.opentype}{sotf}
3481 \@namedef{bbl@info@region.tag.bcp47}{rbcp}

```

```

3482 \@namedef{bbl@info@variant.tag.bcp47}{vbcp}
3483 \@namedef{bbl@info@extension.t.tag.bcp47}{extt}
3484 \@namedef{bbl@info@extension.u.tag.bcp47}{extu}
3485 \@namedef{bbl@info@extension.x.tag.bcp47}{extx}

```

With version 3.75 \BabelEnsureInfo is executed always, but there is an option to disable it. Since the info in ini files are always loaded, it has been made no-op in version 25.8.

```

3486 <<*<More package options>> >> \
3487 \DeclareOption{ensureinfo=off}{}
3488 <</More package options>>
3489 \let\BabelEnsureInfo\relax

```

More general, but non-expandable, is \getlocaleproperty.

```

3490 \newcommand\getlocaleproperty{%
3491   \@ifstar\bbl@getproperty@s\bbl@getproperty@x}
3492 \def\bbl@getproperty@s#1#2#3{%
3493   \let#1\relax
3494   \def\bbl@elt##1##2##3{%
3495     \bbl@ifsamestring{##1/##2}{#3}%
3496     {\providecommand#1{##3}%
3497     \def\bbl@elt####1####2####3{}}}%
3498   {}}%
3499   \bbl@cs{inidata@#2}}%
3500 \def\bbl@getproperty@x#1#2#3{%
3501   \bbl@getproperty@s{#1}{#2}{#3}%
3502   \ifx#1\relax
3503     \bbl@error{unknown-locale-key}{#1}{#2}{#3}%
3504   \fi}

```

To inspect every possible loaded ini, we define \LocaleForEach, where \bbl@ini@loaded is a comma-separated list of locales, built by \bbl@read@ini.

```

3505 \let\bbl@ini@loaded\@empty
3506 \newcommand\LocaleForEach{\bbl@foreach\bbl@ini@loaded}
3507 \def\ShowLocaleProperties#1{%
3508   \typeout{}}%
3509   \typeout{*** Properties for language '#1' ***}%
3510   \def\bbl@elt##1##2##3{\typeout{##1/##2 = \unexpanded{##3}}}%
3511   \@nameuse{bbl@inidata@#1}%
3512   \typeout{*****}}

```

4.26. BCP 47 related commands

This macro is called by language selectors when the language isn't recognized. So, it's the core for (1) mapping from a BCP 27 tag to the actual language, if bcp47.toname is enabled (i.e., if bbl@bcptoname is true), and (2) lazy loading. With autoload.bcp47 enabled *and* lazy loading, we must first build a name for the language, with the help of autoload.bcp47.prefix. Then we use \provideprovide passing the options set with autoload.bcp47.options (by default import). Finally, and if the locale has not been loaded before, we use \provideprovide with the language name as passed to the selector.

```

3513 \newif\ifbbl@bcppallowed
3514 \bbl@bcppallowedfalse
3515 \def\bbl@autoload@options{@import}
3516 \def\bbl@provide@locale{%
3517   \ifx\babelprovide\@undefined
3518     \bbl@error{base-on-the-fly}{}{}{}%
3519   \fi
3520   \let\bbl@auxname\language
3521   \ifbbl@bcptoname
3522     \bbl@ifunset{bbl@bcp@map@\language}{}% Move uplevel??
3523     {\edef\language{\@nameuse{bbl@bcp@map@\language}}}%
3524     \let\locale\language}%
3525   \fi
3526   \ifbbl@bcppallowed

```

```

3527 \expandafter\ifx\csname date\language\endcsname\relax
3528 \expandafter
3529 \bbl@bcplookup\language-\@empty-\@empty-\@empty\@@
3530 \ifx\bbl@bcp\relax\else % Returned by \bbl@bcplookup
3531 \edef\language{\bbl@bcp@prefix\bbl@bcp}%
3532 \let\localname\language
3533 \expandafter\ifx\csname date\language\endcsname\relax
3534 \let\bbl@initoload\bbl@bcp
3535 \bbl@exp{\babelprovide[\bbl@autoload@bcptoptions]{\language}}%
3536 \let\bbl@initoload\relax
3537 \fi
3538 \bbl@csarg\xdef{bcp@map@\bbl@bcp}{\localname}%
3539 \fi
3540 \fi
3541 \fi
3542 \expandafter\ifx\csname date\language\endcsname\relax
3543 \IfFileExists{babel-\language.tex}%
3544 {\bbl@exp{\babelprovide[\bbl@autoload@options]{\language}}}%
3545 {}%
3546 \fi}

```

$\TeX$ needs to know the BCP 47 codes for some features. For that, it expects `\BCPdata` to be defined. While language, region, script, and variant are recognized, extension.`(s)` for singletons may change.

Still somewhat hackish. Note `\str_if_eq:nnTF` is fully expandable (`\bbl@ifsamestring` isn't). The argument is the prefix to `tag.bcp47`.

```

3547 \providecommand\BCPdata{}
3548 \ifx\renewcommand\@undefined\else
3549 \renewcommand\BCPdata[1]{\bbl@bcpdata@i#1\@empty\@empty\@empty}
3550 \def\bbl@bcpdata@i#1#2#3#4#5#6\@empty{%
3551 \@nameuse{str_if_eq:nnTF}{#1#2#3#4#5}{main.}%
3552 {\bbl@bcpdata@ii{#6}\bbl@main@language}%
3553 {\bbl@bcpdata@ii{#1#2#3#4#5#6}\language}}%
3554 \def\bbl@bcpdata@ii#1#2{%
3555 \bbl@ifunset{bbl@info@#1.tag.bcp47}%
3556 {\bbl@error{unknown-ini-field}{#1}{}}}%
3557 {\bbl@ifunset{bbl\csname bbl@info@#1.tag.bcp47\endcsname @#2}{}}%
3558 {\bbl@cs\csname bbl@info@#1.tag.bcp47\endcsname @#2}}}%
3559 \fi
3560 \@namedef{bbl@info@casing.tag.bcp47}{casing}
3561 \@namedef{bbl@info@tag.tag.bcp47}{tbc} % For \BCPdata

```

5. Adjusting the Babel behavior

A generic high level interface is provided to adjust some global and general settings.

```

3562 \newcommand\babeladjust[1]{%
3563 \bbl@forkv{#1}{%
3564 \bbl@ifunset{bbl@ADJ@##1@##2}%
3565 {\bbl@cs{ADJ@##1}{##2}}%
3566 {\bbl@cs{ADJ@##1@##2}}}%
3567 %
3568 \def\bbl@adjust@lua#1#2{%
3569 \ifvmode
3570 \ifnum\currentgrouplevel=\z@
3571 \directlua{ Babel.#2 }%
3572 \expandafter\expandafter\expandafter\@gobble
3573 \fi
3574 \fi
3575 {\bbl@error{adjust-only-vertical}{#1}{}}}% Gobbled if everything went ok.
3576 \@namedef{bbl@ADJ@bidi.mirroring@on}{%
3577 \bbl@adjust@lua{bidi}{mirroring_enabled=true}}
3578 \@namedef{bbl@ADJ@bidi.mirroring@off}{%

```

```

3579 \bbl@adjust@lua{bidi}{mirroring_enabled=false}}
3580 \@namedef{bbl@ADJ@bidi.text@on}{%
3581 \bbl@adjust@lua{bidi}{bidi_enabled=true}}
3582 \@namedef{bbl@ADJ@bidi.text@off}{%
3583 \bbl@adjust@lua{bidi}{bidi_enabled=false}}
3584 \@namedef{bbl@ADJ@bidi.math@on}{%
3585 \let\bbl@noamsmath\@empty}
3586 \@namedef{bbl@ADJ@bidi.math@off}{%
3587 \let\bbl@noamsmath\relax}
3588 %
3589 \@namedef{bbl@ADJ@bidi.mapdigits@on}{%
3590 \bbl@adjust@lua{bidi}{digits_mapped=true}}
3591 \@namedef{bbl@ADJ@bidi.mapdigits@off}{%
3592 \bbl@adjust@lua{bidi}{digits_mapped=false}}
3593 %
3594 \@namedef{bbl@ADJ@linebreak.sea@on}{%
3595 \bbl@adjust@lua{linebreak}{sea_enabled=true}}
3596 \@namedef{bbl@ADJ@linebreak.sea@off}{%
3597 \bbl@adjust@lua{linebreak}{sea_enabled=false}}
3598 \@namedef{bbl@ADJ@linebreak.cjk@on}{%
3599 \bbl@adjust@lua{linebreak}{cjk_enabled=true}}
3600 \@namedef{bbl@ADJ@linebreak.cjk@off}{%
3601 \bbl@adjust@lua{linebreak}{cjk_enabled=false}}
3602 \@namedef{bbl@ADJ@justify.arabic@on}{%
3603 \bbl@adjust@lua{linebreak}{arabic.justify_enabled=true}}
3604 \@namedef{bbl@ADJ@justify.arabic@off}{%
3605 \bbl@adjust@lua{linebreak}{arabic.justify_enabled=false}}
3606 %
3607 \def\bbl@adjust@layout#1{%
3608 \ifvmode
3609 #1%
3610 \expandafter\@gobble
3611 \fi
3612 {\bbl@error{layout-only-vertical}}{}}}% Gobbled if everything went ok.
3613 \@namedef{bbl@ADJ@layout.tabular@on}{%
3614 \ifnum\bbl@tabular@mode=\tw@
3615 \bbl@adjust@layout{\let\@tabular\bbl@NL@@@tabular}%
3616 \else
3617 \chardef\bbl@tabular@mode\@ne
3618 \fi}
3619 \@namedef{bbl@ADJ@layout.tabular@off}{%
3620 \ifnum\bbl@tabular@mode=\tw@
3621 \bbl@adjust@layout{\let\@tabular\bbl@OL@@@tabular}%
3622 \else
3623 \chardef\bbl@tabular@mode\@z@
3624 \fi}
3625 \@namedef{bbl@ADJ@layout.lists@on}{%
3626 \bbl@adjust@layout{\let\list\bbl@NL@list}}
3627 \@namedef{bbl@ADJ@layout.lists@off}{%
3628 \bbl@adjust@layout{\let\list\bbl@OL@list}}
3629 %
3630 \@namedef{bbl@ADJ@autoload.bcp47@on}{%
3631 \bbl@bcpallowedtrue}
3632 \@namedef{bbl@ADJ@autoload.bcp47@off}{%
3633 \bbl@bcpallowedfalse}
3634 \@namedef{bbl@ADJ@autoload.bcp47.prefix}#1{%
3635 \def\bbl@bcp@prefix{#1}}
3636 \def\bbl@bcp@prefix{bcp47-}
3637 \@namedef{bbl@ADJ@autoload.options}#1{%
3638 \def\bbl@autoload@options{#1}}
3639 \def\bbl@autoload@bcptoptions{import}
3640 \@namedef{bbl@ADJ@autoload.bcp47.options}#1{%
3641 \def\bbl@autoload@bcptoptions{#1}}

```



```

3642 \newif\ifbbl@bcptoname
3643 %
3644 \@namedef{bbl@ADJ@bcp47.toname@on}{%
3645   \bbl@bcptonametrue}
3646 \@namedef{bbl@ADJ@bcp47.toname@off}{%
3647   \bbl@bcptonamefalse}
3648 %
3649 \@namedef{bbl@ADJ@prehyphenation.disable@nohyphenation}{%
3650   \directlua{ Babel.ignore_pre_char = function(node)
3651     return (node.lang == \the\csname l@nohyphenation\endcsname)
3652   end }}
3653 \@namedef{bbl@ADJ@prehyphenation.disable@off}{%
3654   \directlua{ Babel.ignore_pre_char = function(node)
3655     return false
3656   end }}
3657 %
3658 \@namedef{bbl@ADJ@interchar.disable@nohyphenation}{%
3659   \def\bbl@ignoreinterchar{%
3660     \ifnum\language=\l@nohyphenation
3661       \expandafter\@gobble
3662     \else
3663       \expandafter\@firstofone
3664     \fi}}
3665 \@namedef{bbl@ADJ@interchar.disable@off}{%
3666   \let\bbl@ignoreinterchar\@firstofone}
3667 %
3668 \@namedef{bbl@ADJ@select.write@shift}{%
3669   \let\bbl@restorelastskip\relax
3670   \def\bbl@savelastskip{%
3671     \let\bbl@restorelastskip\relax
3672     \ifvmode
3673       \ifdim\lastskip=\z@
3674         \let\bbl@restorelastskip\nobreak
3675       \else
3676         \bbl@exp{%
3677           \def\\bbl@restorelastskip{%
3678             \skip@=\the\lastskip
3679             \\nobreak \vskip-\skip@ \vskip\skip@}}%
3680         \fi
3681       \fi}}
3682 \@namedef{bbl@ADJ@select.write@keep}{%
3683   \let\bbl@restorelastskip\relax
3684   \let\bbl@savelastskip\relax}
3685 \@namedef{bbl@ADJ@select.write@omit}{%
3686   \AddBabelHook{babel-select}{beforestart}{%
3687     \expandafter\babel@aux\expandafter{\bbl@main@language}{}}%
3688   \let\bbl@restorelastskip\relax
3689   \def\bbl@savelastskip##1\bbl@restorelastskip{}}
3690 \@namedef{bbl@ADJ@select.encoding@off}{%
3691   \let\bbl@encoding@select@off\@empty}

```

5.1. Cross referencing macros

The $\LaTeX$ book states:

The *key* argument is any sequence of letters, digits, and punctuation symbols; upper- and lowercase letters are regarded as different.

When the above quote should still be true when a document is typeset in a language that has active characters, special care has to be taken of the category codes of these characters when they appear in an argument of the cross referencing macros.

When a cross referencing command processes its argument, all tokens in this argument should be character tokens with category ‘letter’ or ‘other’.

The following package options control which macros are to be redefined.

```

3692 << *More package options >> ≡
3693 \DeclareOption{safe=none}{\let\bbl@opt@safe@empty}
3694 \DeclareOption{safe=bib}{\def\bbl@opt@safe{B}}
3695 \DeclareOption{safe=ref}{\def\bbl@opt@safe{R}}
3696 \DeclareOption{safe=refbib}{\def\bbl@opt@safe{BR}}
3697 \DeclareOption{safe=bibref}{\def\bbl@opt@safe{BR}}
3698 << /More package options >>

```

\@newl@bel First we open a new group to keep the changed setting of `\protect` local and then we set the `@safe@actives` switch to true to make sure that any shorthand that appears in any of the arguments immediately expands to its non-active self.

```

3699 \bbl@trace{Cross referencing macros}
3700 \ifx\bbl@opt@safe@empty\else % i.e., if 'ref' and/or 'bib'
3701   \def\@newl@bel#1#2#3{%
3702     {\@safe@activetrue
3703       \bbl@ifunset{#1@#2}%
3704         \relax
3705         {\gdef\@multiplelabels{%
3706           \@latex@warning@no@line{There were multiply-defined labels}}%
3707           \@latex@warning@no@line{Label `#2' multiply defined}}%
3708       \global\@namedef{#1@#2}{#3}}

```

\@testdef An internal $\TeX$ macro used to test if the labels that have been written on the aux file have changed. It is called by the `\enddocument` macro.

```

3709 \CheckCommand*\@testdef[3]{%
3710   \def\reserved@a{#3}%
3711   \expandafter\ifx\csname#1@#2\endcsname\reserved@a
3712     \else
3713       \@tempswatrue
3714     \fi}

```

Now that we made sure that `\@testdef` still has the same definition we can rewrite it. First we make the shorthands ‘safe’. Then we use `\bbl@tempa` as an ‘alias’ for the macro that contains the label which is being checked. Then we define `\bbl@tempb` just as `\@newl@bel` does it. When the label is defined we replace the definition of `\bbl@tempa` by its meaning. If the label didn’t change, `\bbl@tempa` and `\bbl@tempb` should be identical macros.

```

3715 \def\@testdef#1#2#3{%
3716   \@safe@activetrue
3717   \expandafter\let\expandafter\bbl@tempa\csname #1@#2\endcsname
3718   \def\bbl@tempb{#3}%
3719   \@safe@activesfalse
3720   \ifx\bbl@tempa\relax
3721     \else
3722       \edef\bbl@tempa{\expandafter\strip@prefix\meaning\bbl@tempa}%
3723     \fi
3724     \edef\bbl@tempb{\expandafter\strip@prefix\meaning\bbl@tempb}%
3725     \ifx\bbl@tempa\bbl@tempb
3726       \else
3727         \@tempswatrue
3728       \fi}
3729 \fi

```

\ref

\pageref The same holds for the macro `\ref` that references a label and `\pageref` to reference a page. We make them robust as well (if they weren’t already) to prevent problems if they should become expanded at the wrong moment.

```

3730 \bbl@xin@{R}\bbl@opt@safe
3731 \ifin@
3732   \edef\bbl@tempc{\expandafter\string\csname ref code\endcsname}%
3733   \bbl@xin@{\expandafter\strip@prefix\meaning\bbl@tempc}%
3734   {\expandafter\strip@prefix\meaning\ref}%

```

```

3735 \ifin@
3736 \bbl@redefine\@kernel@ref#1{%
3737 \@safe@activetrue\org@@kernel@ref{#1}\@safe@activesfalse}
3738 \bbl@redefine\@kernel@pageref#1{%
3739 \@safe@activetrue\org@@kernel@pageref{#1}\@safe@activesfalse}
3740 \bbl@redefine\@kernel@sref#1{%
3741 \@safe@activetrue\org@@kernel@sref{#1}\@safe@activesfalse}
3742 \bbl@redefine\@kernel@spageref#1{%
3743 \@safe@activetrue\org@@kernel@spageref{#1}\@safe@activesfalse}
3744 \else
3745 \bbl@redefineroquest\ref#1{%
3746 \@safe@activetrue\org@ref{#1}\@safe@activesfalse}
3747 \bbl@redefineroquest\pageref#1{%
3748 \@safe@activetrue\org@pageref{#1}\@safe@activesfalse}
3749 \fi
3750 \else
3751 \let\org@ref\ref
3752 \let\org@pageref\pageref
3753 \fi

```

\@citex The macro used to cite from a bibliography, \cite, uses an internal macro, \@citex. It is this internal macro that picks up the argument(s), so we redefine this internal macro and leave \cite alone. The first argument is used for typesetting, so the shorthands need only be deactivated in the second argument.

```

3754 \bbl@xin@{B}\bbl@opt@safe
3755 \ifin@
3756 \bbl@redefine\@citex[#1]#2{%
3757 \@safe@activetrue\edef\bbl@tempa{#2}\@safe@activesfalse
3758 \org@@citex[#1]{\bbl@tempa}}

```

Unfortunately, the packages natbib and cite need a different definition of \@citex... To begin with, natbib has a definition for \@citex with *three* arguments... We only know that a package is loaded when \begin{document} is executed, so we need to postpone the different redefinition.

Notice that we use \def here instead of \bbl@redefine because \org@@citex is already defined and we don't want to overwrite that definition (it would result in parameter stack overflow because of a circular definition).

(Recent versions of natbib change dynamically \@citex, so PR4087 doesn't seem fixable in a simple way. Just load natbib before.)

```

3759 \AtBeginDocument{%
3760 \@ifpackageloaded{natbib}{%
3761 \def\@citex[#1][#2]#3{%
3762 \@safe@activetrue\edef\bbl@tempa{#3}\@safe@activesfalse
3763 \org@@citex[#1][#2]{\bbl@tempa}}%
3764 }}

```

The package cite has a definition of \@citex where the shorthands need to be turned off in both arguments.

```

3765 \AtBeginDocument{%
3766 \@ifpackageloaded{cite}{%
3767 \def\@citex[#1]#2{%
3768 \@safe@activetrue\org@@citex[#1][#2]\@safe@activesfalse}%
3769 }}

```

\nocite The macro \nocite which is used to instruct BiBTeX to extract uncited references from the database.

```

3770 \bbl@redefine\nocite#1{%
3771 \@safe@activetrue\org@nocite{#1}\@safe@activesfalse}

```

\bibcite The macro that is used in the aux file to define citation labels. When packages such as natbib or cite are not loaded its second argument is used to typeset the citation label. In that case, this second argument can contain active characters but is used in an environment where \@safe@activetrue is in effect. This switch needs to be reset inside the \hbox which contains the

citation label. In order to determine during aux file processing which definition of `\biblecite` is needed we define `\biblecite` in such a way that it redefines itself with the proper definition. We call `\bbl@cite@choice` to select the proper definition for `\biblecite`. This new definition is then activated.

```
3772 \bbl@redefine\biblecite{%
3773   \bbl@cite@choice
3774   \biblecite}
```

\bbl@biblecite The macro `\bbl@biblecite` holds the definition of `\biblecite` needed when neither `natbib` nor `cite` is loaded.

```
3775 \def\bbl@biblecite#1#2{%
3776   \org@biblecite{#1}{\@safe@activesfalse#2}}
```

\bbl@cite@choice The macro `\bbl@cite@choice` determines which definition of `\biblecite` is needed. First we give `\biblecite` its default definition.

```
3777 \def\bbl@cite@choice{%
3778   \global\let\biblecite\bbl@biblecite
3779   \@ifpackageloaded{natbib}{\global\let\biblecite\org@biblecite}{}%
3780   \@ifpackageloaded{cite}{\global\let\biblecite\org@biblecite}{}%
3781   \global\let\bbl@cite@choice\relax}
```

When a document is run for the first time, no aux file is available, and `\biblecite` will not yet be properly defined. In this case, this has to happen before the document starts.

```
3782 \AtBeginDocument{\bbl@cite@choice}
```

\@bibitem One of the two internal $\TeX$ macros called by `\bibitem` that write the citation label on the aux file.

```
3783 \bbl@redefine\@bibitem#1{%
3784   \@safe@activestrue\org@@bibitem{#1}\@safe@activesfalse}
3785 \else
3786   \let\org@nocite\nocite
3787   \let\org@@citex\citex
3788   \let\org@biblecite\biblecite
3789   \let\org@@bibitem\@bibitem
3790 \fi
```

5.2. Layout

```
3791 \newcommand\BabelPatchSection[1]{%
3792   \@ifundefined{#1}{}{%
3793     \bbl@exp{\let<bbl@ss@#1>\<#1>}%
3794     \@namedef{#1}{%
3795       \@ifstar{\bbl@presec@s{#1}}%
3796       {\@dblarg{\bbl@presec@x{#1}}}}}%
3797 \def\bbl@presec@x#1[#2]#3{%
3798   \bbl@exp{%
3799     \\\select@language@x{\bbl@main@language}%
3800     \\\bbl@cs{sspre@#1}%
3801     \\\bbl@cs{ss@#1}%
3802     [\\foreignlanguage{\language@name}{\unexpanded{#2}}}%
3803     {\\foreignlanguage{\language@name}{\unexpanded{#3}}}%
3804     \\\select@language@x{\language@name}}}%
3805 \def\bbl@presec@s#1#2{%
3806   \bbl@exp{%
3807     \\\select@language@x{\bbl@main@language}%
3808     \\\bbl@cs{sspre@#1}%
3809     \\\bbl@cs{ss@#1}*%
3810     {\\foreignlanguage{\language@name}{\unexpanded{#2}}}%
3811     \\\select@language@x{\language@name}}}%
3812 %
3813 \IfBabelLayout{sectioning}%
3814   {\BabelPatchSection{part}}%
```

```

3815 \BabelPatchSection{chapter}%
3816 \BabelPatchSection{section}%
3817 \BabelPatchSection{subsection}%
3818 \BabelPatchSection{subsubsection}%
3819 \BabelPatchSection{paragraph}%
3820 \BabelPatchSection{subparagraph}%
3821 \def\babel@toc#1{%
3822   \select@language@x{\bbl@main@language}}{}
3823 \IfBabelLayout{captions}%
3824 {\BabelPatchSection{caption}}{}

```

\BabelFootnote Footnotes.

```

3825 \bbl@trace{Footnotes}
3826 \def\bbl@footnote#1#2#3{%
3827   \@ifnextchar[%
3828     {\bbl@footnote@o{#1}{#2}{#3}}%
3829     {\bbl@footnote@x{#1}{#2}{#3}}}
3830 \long\def\bbl@footnote@x#1#2#3#4{%
3831   \bgroup
3832   \select@language@x{\bbl@main@language}%
3833   \bbl@fn@footnote{#2#1{\ignorespaces#4}#3}%
3834   \egroup}
3835 \long\def\bbl@footnote@o#1#2#3[#4]#5{%
3836   \bgroup
3837   \select@language@x{\bbl@main@language}%
3838   \bbl@fn@footnote[#4]{#2#1{\ignorespaces#5}#3}%
3839   \egroup}
3840 \def\bbl@footnotetext#1#2#3{%
3841   \@ifnextchar[%
3842     {\bbl@footnotetext@o{#1}{#2}{#3}}%
3843     {\bbl@footnotetext@x{#1}{#2}{#3}}}
3844 \long\def\bbl@footnotetext@x#1#2#3#4{%
3845   \bgroup
3846   \select@language@x{\bbl@main@language}%
3847   \bbl@fn@footnotetext{#2#1{\ignorespaces#4}#3}%
3848   \egroup}
3849 \long\def\bbl@footnotetext@o#1#2#3[#4]#5{%
3850   \bgroup
3851   \select@language@x{\bbl@main@language}%
3852   \bbl@fn@footnotetext[#4]{#2#1{\ignorespaces#5}#3}%
3853   \egroup}
3854 \def\BabelFootnote#1#2#3#4{%
3855   \ifx\bbl@fn@footnote\undefined
3856     \let\bbl@fn@footnote\footnote
3857   \fi
3858   \ifx\bbl@fn@footnotetext\undefined
3859     \let\bbl@fn@footnotetext\footnotetext
3860   \fi
3861   \bbl@ifblank{#2}%
3862     {\def#1{\bbl@footnote{\@firstofone}{#3}{#4}}
3863      \@namedef{\bbl@stripslash#1text}%
3864        {\bbl@footnotetext{\@firstofone}{#3}{#4}}}%
3865     {\def#1{\bbl@exp{\bbl@footnote{\foreignlanguage{#2}}}{#3}{#4}}%
3866      \@namedef{\bbl@stripslash#1text}%
3867        {\bbl@exp{\bbl@footnotetext{\foreignlanguage{#2}}}{#3}{#4}}}%
3868 \IfBabelLayout{footnotes}%
3869 {\let\bbl@0L@footnote\footnote
3870  \BabelFootnote\footnote\languagename{}}{}%
3871 \BabelFootnote\localfootnote\languagename{}}{}%
3872 \BabelFootnote\mainfootnote{}}{}%
3873 {}

```

5.3. Marks

\markright Because the output routine is asynchronous, we must pass the current language attribute to the head lines. To achieve this we need to adapt the definition of `\markright` and `\markboth` somewhat. However, headlines and footlines can contain text outside marks; for that we must take some actions in the output routine if the 'headfoot' options is used.

We need to make some redefinitions to the output routine to avoid an endless loop and to correctly handle the page number in bidi documents.

```
3874 \bbl@trace{Marks}
3875 \IfBabelLayout{sectioning}
3876 {\ifx\bbl@opt@headfoot\@nnil
3877   \g@addto@macro\@resetactivechars{%
3878     \set@typeset@protect
3879     \expandafter\select@language\x\expandafter{\bbl@main@language}%
3880     \let\protect\noexpand
3881     \ifcase\bbl@bidimode\else % Only with bidi. See also above
3882       \edef\thepage{%
3883         \noexpand\babelsublr{\unexpanded\expandafter{\thepage}}}%
3884     \fi}%
3885 \fi}
3886 {\ifbbl@single\else
3887   \bbl@ifunset{markright } \bbl@redefine\bbl@redefineroobust
3888   \markright#1{%
3889     \bbl@ifblank{#1}%
3890     {\org@markright{}}}%
3891     {\toks@{#1}%
3892     \bbl@exp{%
3893       \\org@markright{\\protect\\foreignlanguage{\language}%
3894         {\protect\\bbl@restore@actives\the\toks@}}}%}
```

\markboth

\@mkboth The definition of `\markboth` is equivalent to that of `\markright`, except that we need two token registers. The documentclasses report and book define and set the headings for the page. While doing so they also store a copy of `\markboth` in `\@mkboth`. Therefore we need to check whether `\@mkboth` has already been set. If so we need to do that again with the new definition of `\markboth`. (As of Oct 2019, $\text{\LaTeX}$ stores the definition in an intermediate macro, so it's not necessary anymore, but it's preserved for older versions.)

```
3895   \ifx\@mkboth\markboth
3896     \def\bbl@tempc{\let\@mkboth\markboth}%
3897   \else
3898     \def\bbl@tempc{}%
3899   \fi
3900   \bbl@ifunset{markboth } \bbl@redefine\bbl@redefineroobust
3901   \markboth#1#2{%
3902     \protected@edef\bbl@tempb##1{%
3903       \protect\foreignlanguage
3904       {\language}{\protect\bbl@restore@actives##1}}%
3905     \bbl@ifblank{#1}%
3906     {\toks@{}}%
3907     {\toks@\expandafter{\bbl@tempb{#1}}}%
3908     \bbl@ifblank{#2}%
3909     {\@temptokena{}}%
3910     {\@temptokena\expandafter{\bbl@tempb{#2}}}%
3911     \bbl@exp{\\org@markboth{\the\toks@}{\the\@temptokena}}}%
3912     \bbl@tempc
3913   \fi} % end ifbbl@single, end \IfBabelLayout
```

5.4. Other packages

5.4.1. ifthen

\ifthenelse Sometimes a document writer wants to create a special effect depending on the page a certain fragment of text appears on. This can be achieved by the following piece of code:

```
% \ifthenelse{\isodd{\pageref{some-label}}}{
%      {code for odd pages}
%      {code for even pages}
%}
```

In order for this to work the argument of `\isodd` needs to be fully expandable. With the above redefinition of `\pageref` it is not in the case of this example. To overcome that, we add some code to the definition of `\ifthenelse` to make things work.

We want to revert the definition of `\pageref` and `\ref` to their original definition for the first argument of `\ifthenelse`, so we first need to store their current meanings.

Then we can set the `\@safe@actives` switch and call the original `\ifthenelse`. In order to be able to use shorthands in the second and third arguments of `\ifthenelse` the resetting of the switch *and* the definition of `\pageref` happens inside those arguments.

```
3914 \bbl@trace{Preventing clashes with other packages}
3915 \ifx\org@ref\@undefined\else
3916   \bbl@xin@{R}\bbl@opt@safe
3917   \ifin@
3918     \AtBeginDocument{%
3919       \@ifpackageloaded{ifthen}{%
3920         \bbl@redefine@long\ifthenelse#1#2#3{%
3921           \let\bbl@temp@pref\pageref
3922           \let\pageref\org@pageref
3923           \let\bbl@temp@ref\ref
3924           \let\ref\org@ref
3925           \@safe@activestrue
3926           \org@ifthenelse{#1}%
3927             {\let\pageref\bbl@temp@pref
3928              \let\ref\bbl@temp@ref
3929              \@safe@activesfalse
3930              #2}%
3931             {\let\pageref\bbl@temp@pref
3932              \let\ref\bbl@temp@ref
3933              \@safe@activesfalse
3934              #3}%
3935           }%
3936         }{}%
3937       }
3938 \fi
```

5.4.2. varioref

`\@@vpageref`
`\vrefpagemum`

`\Ref` When the package `varioref` is in use we need to modify its internal command `\@@vpageref` in order to prevent problems when an active character ends up in the argument of `\vref`. The same needs to happen for `\vrefpagemum`.

```
3939 \AtBeginDocument{%
3940   \@ifpackageloaded{varioref}{%
3941     \bbl@redefine\@@vpageref#1[#2]#3{%
3942       \@safe@activestrue
3943       \org@@@vpageref{#1}[#2]{#3}%
3944       \@safe@activesfalse}%
3945     \bbl@redefine\vrefpagemum#1#2{%
3946       \@safe@activestrue
3947       \org@vrefpagemum{#1}[#2]%
3948       \@safe@activesfalse}%
3949   }
```

The package `varioref` defines `\Ref` to be a robust command which uppercases the first character of the reference text. In order to be able to do that it needs to access the expandable form of `\ref`. So we employ a little trick here. We redefine the (internal) command `\Ref_` to call `\org@ref` instead of `\ref`. The disadvantage of this solution is that whenever the definition of `\Ref` changes, this definition needs to be updated as well.

```

3949 \expandafter\def\csname Ref \endcsname#1{%
3950 \protected@edef\@tempa{\org@ref{#1}}\expandafter\MakeUppercase\@tempa}
3951 }{}%
3952 }
3953 \fi

```

5.4.3. hline

\hline Delaying the activation of the shorthand characters has introduced a problem with the hline package. The reason is that it uses the ‘:’ character which is made active by the french support in babel. Therefore we need to *reload* the package when the ‘:’ is an active character. Note that this happens *after* the category code of the @-sign has been changed to other, so we need to temporarily change it to letter again.

```

3954 \AtEndOfPackage{%
3955 \AtBeginDocument{%
3956 \@ifpackageloaded{hline}%
3957 {\expandafter\ifx\csname normal@char\string\endcsname\relax
3958 \else
3959 \makeatletter
3960 \def\@currname{hline}\input{hline.sty}\makeatother
3961 \fi}%
3962 {}}}

```

\substitutefontfamily *Deprecated.* It creates an fd file on the fly. The first argument is an encoding mnemonic, the second and third arguments are font family names. Use the tools provided by L^AT_EX (\DeclareFontFamilySubstitution).

```

3963 \def\substitutefontfamily#1#2#3{%
3964 \lowercase{\immediate\openout15=#1#2.fd\relax}%
3965 \immediate\write15{%
3966 \string\ProvidesFile{#1#2.fd}%
3967 [\the\year/\two@digits{\the\month}/\two@digits{\the\day}
3968 \space generated font description file]^J
3969 \string\DeclareFontFamily{#1}{#2}{}}^J
3970 \string\DeclareFontShape{#1}{#2}{m}{n}{<->ssub * #3/m/n}{}}^J
3971 \string\DeclareFontShape{#1}{#2}{m}{it}{<->ssub * #3/m/it}{}}^J
3972 \string\DeclareFontShape{#1}{#2}{m}{sl}{<->ssub * #3/m/sl}{}}^J
3973 \string\DeclareFontShape{#1}{#2}{m}{sc}{<->ssub * #3/m/sc}{}}^J
3974 \string\DeclareFontShape{#1}{#2}{b}{n}{<->ssub * #3/bx/n}{}}^J
3975 \string\DeclareFontShape{#1}{#2}{b}{it}{<->ssub * #3/bx/it}{}}^J
3976 \string\DeclareFontShape{#1}{#2}{b}{sl}{<->ssub * #3/bx/sl}{}}^J
3977 \string\DeclareFontShape{#1}{#2}{b}{sc}{<->ssub * #3/bx/sc}{}}^J
3978 }%
3979 \closeout15
3980 }
3981 \@onlypreamble\substitutefontfamily

```

5.5. Encoding and fonts

Because documents may use non-ASCII font encodings, we make sure that the logos of T_EX and L^AT_EX always come out in the right encoding. There is a list of non-ASCII encodings. Requested encodings are currently stored in \fontenc@load@list. If a non-ASCII has been loaded, we define versions of \TeX and \LaTeX for them using \ensureascii. The default ASCII encoding is set, too (in reverse order): the “main” encoding (when the document begins), the last loaded, or OT1.

\ensureascii

```

3982 \bbl@trace{Encoding and fonts}
3983 \newcommand\BabelNonASCII{LGR,LGI,X2,OT2,OT3,OT6,LHE,LWN,LMA,LMC,LMS,LMU}
3984 \newcommand\BabelNonText{TS1,T3,TS3}
3985 \let\org@TeX\TeX
3986 \let\org@LaTeX\LaTeX
3987 \let\ensureascii\@firstofone
3988 \let\asciientcoding\@empty

```



```

3989 \AtBeginDocument{%
3990   \def\elt#1{, #1,}%
3991   \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%
3992   \let\elt\relax
3993   \let\bbl@tempb\@empty
3994   \def\bbl@tempc{OT1}%
3995   \bbl@foreach\BabelNonASCII{% LGR loaded in a non-standard way
3996     \bbl@ifunset{T@#1}{\def\bbl@tempb{#1}}}%
3997   \bbl@foreach\bbl@tempa{%
3998     \bbl@xin@{, #1,}{, \BabelNonASCII,}%
3999     \ifin@
4000       \def\bbl@tempb{#1}% Store last non-ascii
4001     \else\bbl@xin@{, #1,}{, \BabelNonText,}% Pass
4002       \ifin@else
4003         \def\bbl@tempc{#1}% Store last ascii
4004       \fi
4005     \fi}%
4006   \ifx\bbl@tempb\@empty\else
4007     \bbl@xin@{\cf@encoding,}{, \BabelNonASCII, \BabelNonText,}%
4008     \ifin@else
4009       \edef\bbl@tempc{\cf@encoding}% The default if ascii wins
4010     \fi
4011     \let\asciencoding\bbl@tempc
4012     \renewcommand\ensureascii[1]{%
4013       {\fontencoding{\asciencoding}\selectfont#1}}%
4014     \DeclareTextCommandDefault{\TeX}{\ensureascii{\org@TeX}}%
4015     \DeclareTextCommandDefault{\LaTeX}{\ensureascii{\org@LaTeX}}%
4016   \fi}

```

Now comes the old deprecated stuff (with a little change in 3.9l, for fontspec). The first thing we need to do is to determine, at `\begin{document}`, which latin fontencoding to use.

Latinencoding When text is being typeset in an encoding other than ‘latin’ (OT1 or T1), it would be nice to still have Roman numerals come out in the Latin encoding. So we first assume that the current encoding at the end of processing the package is the Latin encoding.

```

4017 \AtEndOfPackage{\edef\latinencoding{\cf@encoding}}

```

But this might be overruled with a later loading of the package fontenc. Therefore we check at the execution of `\begin{document}` whether it was loaded with the T1 option. The normal way to do this (using `\ifpackageloaded`) is disabled for this package. Now we have to revert to parsing the internal macro `\@filelist` which contains all the filenames loaded.

```

4018 \AtBeginDocument{%
4019   \ifpackageloaded{fontspec}%
4020     {\xdef\latinencoding{%
4021       \ifx\UTFencname\@undefined
4022         EU\ifcase\bbl@engine\or2\or1\fi
4023       \else
4024         \UTFencname
4025       \fi}}%
4026   {\gdef\latinencoding{OT1}%
4027     \ifx\cf@encoding\bbl@t@one
4028       \xdef\latinencoding{\bbl@t@one}%
4029     \else
4030       \def\elt#1{, #1,}%
4031       \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%
4032       \let\elt\relax
4033       \bbl@xin@{, T1,}\bbl@tempa
4034       \ifin@
4035         \xdef\latinencoding{\bbl@t@one}%
4036       \fi
4037     \fi}}

```

\latintext Then we can define the command `\latintext` which is a declarative switch to a latin font-encoding. Usage of this macro is deprecated.

```
4038 \DeclareRobustCommand{\latintext}{%
4039   \fontencoding{\latinencoding}\selectfont
4040   \def\encodingdefault{\latinencoding}}
```

\textlatin This command takes an argument which is then typeset using the requested font encoding. In order to avoid many encoding switches it operates in a local scope.

```
4041 \ifx\@undefined\DeclareTextFontCommand
4042   \DeclareRobustCommand{\textlatin}[1]{\leavevmode{\latintext #1}}
4043 \else
4044   \DeclareTextFontCommand{\textlatin}{\latintext}
4045 \fi
```

For several functions, we need to execute some code with `\selectfont`. With $\TeX$ 2021-06-01, there is a hook for this purpose.

```
4046 \def\bbbl@patchfont#1{\AddToHook{selectfont}{#1}}
```

5.6. Basic bidi support

This code is currently placed here for practical reasons. It will be moved to the correct place soon, I hope.

It is loosely based on `rlbabel.def`, but most of it has been developed from scratch. This babel module (by Johannes Braams and Boris Lavva) has served the purpose of typesetting R documents for two decades, and despite its flaws I think it is still a good starting point (some parts have been copied here almost verbatim), partly thanks to its simplicity. I've also looked at `ARABI` (by Youssef Jabri), which is compatible with babel.

There are two ways of modifying macros to make them “bidi”, namely, by patching the internal low-level macros (which is what I have done with lists, columns, counters, tocs, much like `rlbabel` did), and by introducing a “middle layer” just below the user interface (sectioning, footnotes).

- `pdftex` provides a minimal support for bidi text, and it must be done by hand. Vertical typesetting is not possible.
- `xetex` is somewhat better, thanks to its font engine (even if not always reliable) and a few additional tools. However, very little is done at the paragraph level. Another challenging problem is text direction does not honour $\TeX$ grouping.
- `luatex` can provide the most complete solution, as we can manipulate almost freely the node list, the generated lines, and so on, but bidi text does not work out of the box and some development is necessary. It also provides tools to properly set left-to-right and right-to-left page layouts. As `Lua $\TeX$ -ja` shows, vertical typesetting is possible, too.

```
4047 \bbl@trace{Loading basic (internal) bidi support}
4048 \ifodd\bbbl@engine
4049 \else % Any xe+lua bidi
4050   \ifnum\bbbl@bidimode>100 \ifnum\bbbl@bidimode<200
4051     \bbl@error{bidi-only-lua}{}}{}{}%
4052     \let\bbbl@beforeforeign\leavevmode
4053     \AtEndOfPackage{%
4054       \EnableBabelHook{babel-bidi}%
4055       \bbl@xebidipar}
4056 \fi\fi
4057 \def\bbbl@loadxebidi#1{%
4058   \ifx\RTLfootnotetext\@undefined
4059     \AtEndOfPackage{%
4060       \EnableBabelHook{babel-bidi}%
4061       \ifx\fontspec\@undefined
4062         \usepackage{fontspec}% bidi needs fontspec
4063       \fi
4064       \usepackage#1{bidi}%
4065       \let\bbbl@digitsdotdash\DigitsDotDashInterCharToks
4066       \def\DigitsDotDashInterCharToks{% See the 'bidi' package
4067         \ifnum\@nameuse{bbl@wdir@language}=\tw@ % 'AL' bidi
```

```

4068         \bbl@digitsdotdash % So ignore in 'R' bidi
4069     \fi}}%
4070 \fi}
4071 \ifnum\bbl@bidimode>200 % Any xe bidi=
4072     \ifcase\expandafter\gobbletwo\the\bbl@bidimode\or
4073         \bbl@tentative{bidi=bidi}
4074         \bbl@loadxebidi{}
4075     \or
4076         \bbl@loadxebidi{[rldocument]}
4077     \or
4078         \bbl@loadxebidi{}
4079     \fi
4080 \fi
4081 \fi
4082 \ifnum\bbl@bidimode=\@ne % bidi=default
4083     \let\bbl@beforeforeign\leavevmode
4084     \ifodd\bbl@engine % lua
4085         \newattribute\bbl@attr@dir
4086         \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
4087         \bbl@exp{\output{\bodydir\pagedir\the\output}}
4088     \fi
4089     \AtEndOfPackage{%
4090         \EnableBabelHook{babel-bidi}% pdf/lua/xe
4091         \ifodd\bbl@engine\else % pdf/xe
4092             \bbl@xebidipar
4093         \fi}
4094 \fi

```

Now come the macros used to set the direction when a language is switched. Testing are based on script names, because it's the user interface (including language and script in \babelprovide. First the (mostly) common macros.

```

4095 \bbl@trace{Macros to switch the text direction}
4096 \def\bbl@alscripts{%
4097     ,Arabic,Syriac,Thaana,Hanifi Rohingya,Hanifi,Sogdian,}
4098 \def\bbl@rscripts{%
4099     Adlam,Avestan,Chorasmian,Cypriot,Elymaic,Garay,%
4100     Hatran,Hebrew,Imperial Aramaic,Inscriptional Pahlavi,%
4101     Inscriptional Parthian,Kharoshthi,Lydian,Mandaic,Manichaean,%
4102     Mende Kikakui,Meroitic Cursive,Meroitic Hieroglyphs,Nabataean,%
4103     Nko,Old Hungarian,Old North Arabian,Old Sogdian,%
4104     Old South Arabian,Old Turkic,Old Uyghur,Palmyrene,Phoenician,%
4105     Psalter Pahlavi,Samaritan,Yezidi,Mandaean,%
4106     Meroitic,N'Ko,Orkhon,Todhri}
4107 %
4108 \def\bbl@provide@dirs#1{%
4109     \bbl@xin@{\csname bbl@sname@#1\endcsname}{\bbl@alscripts\bbl@rscripts}%
4110     \ifin@
4111         \global\bbl@csarg\chardef{wdir@#1}\@ne
4112         \bbl@xin@{\csname bbl@sname@#1\endcsname}{\bbl@alscripts}%
4113         \ifin@
4114             \global\bbl@csarg\chardef{wdir@#1}\tw@
4115         \fi
4116     \else
4117         \global\bbl@csarg\chardef{wdir@#1}\z@
4118     \fi
4119     \ifodd\bbl@engine
4120         \bbl@csarg\ifcase{wdir@#1}%
4121             \directlua{ Babel.locale_props[\the\localeid].texdir = 'l' }%
4122         \or
4123             \directlua{ Babel.locale_props[\the\localeid].texdir = 'r' }%
4124         \or
4125             \directlua{ Babel.locale_props[\the\localeid].texdir = 'al' }%
4126         \fi

```

```

4127 \fi}
4128 %
4129 \def\bbl@switchdir{%
4130 \bbl@ifunset\bbl@sys{\language}\bbl@provide@sys{\language}}{}%
4131 \bbl@ifunset\bbl@wdir{\language}\bbl@provide@dirs{\language}}{}%
4132 \bbl@exp{\bbl@setdirs\bbl@cl{wdir}}
4133 \def\bbl@setdirs#1{%
4134 \ifcase\bbl@select@type
4135 \bbl@bodydir{#1}%
4136 \bbl@pardir{#1}% <- Must precede \bbl@textdir
4137 \fi
4138 \bbl@textdir{#1}}
4139 \ifnum\bbl@bidimode>\z@
4140 \AddBabelHook{babel-bidi}{afterextras}{\bbl@switchdir}
4141 \DisableBabelHook{babel-bidi}
4142 \fi

```

Now the engine-dependent macros.

```

4143 \ifodd\bbl@engine % luatex=1
4144 \else % pdftex=0, xetex=2
4145 \newcount\bbl@dirlevel
4146 \chardef\bbl@thetextdir\z@
4147 \chardef\bbl@thepardir\z@
4148 \def\bbl@textdir#1{%
4149 \ifcase#1\relax
4150 \chardef\bbl@thetextdir\z@
4151 \@nameuse{setlatin}%
4152 \bbl@textdir@i\beginL\endL
4153 \else
4154 \chardef\bbl@thetextdir@ne
4155 \@nameuse{setnonlatin}%
4156 \bbl@textdir@i\beginR\endR
4157 \fi}
4158 \def\bbl@textdir@i#1#2{%
4159 \ifhmode
4160 \ifnum\currentgrouplevel>\z@
4161 \ifnum\currentgrouplevel=\bbl@dirlevel
4162 \bbl@error{multiple-bidi}{}{}%
4163 \bgroup\aftergroup#2\aftergroup\egroup
4164 \else
4165 \ifcase\currentgrouptype\or % 0 bottom
4166 \aftergroup#2% 1 simple {}
4167 \or
4168 \bgroup\aftergroup#2\aftergroup\egroup % 2 hbox
4169 \or
4170 \bgroup\aftergroup#2\aftergroup\egroup % 3 adj hbox
4171 \or\or\or % vbox vtop align
4172 \or
4173 \bgroup\aftergroup#2\aftergroup\egroup % 7 noalign
4174 \or\or\or\or\or % output math disc insert vcent mathchoice
4175 \or
4176 \aftergroup#2% 14 \begingroup
4177 \else
4178 \bgroup\aftergroup#2\aftergroup\egroup % 15 adj
4179 \fi
4180 \fi
4181 \bbl@dirlevel\currentgrouplevel
4182 \fi
4183 #1%
4184 \fi}
4185 \def\bbl@pardir#1{\chardef\bbl@thepardir#1\relax}
4186 \let\bbl@bodydir@gobble
4187 \def\bbl@dirparastext{\chardef\bbl@thepardir\bbl@thetextdir}

```

The following command is executed only if there is a right-to-left script (once). It activates the `\everypar` hack for xetex, to properly handle the par direction. Note text and par dirs are decoupled to some extent (although not completely).

```

4188 \def\bbl@xebidipar{%
4189   \let\bbl@xebidipar\relax
4190   \TeXeTstate\@ne
4191   \def\bbl@xeeverypar{%
4192     \ifcase\bbl@thepardir
4193       \ifcase\bbl@thetextdir\else\beginR\fi
4194       \else
4195         {\setbox\z@\lastbox\beginR\box\z@}%
4196         \fi}%
4197   \AddToHook{para/begin}{\bbl@xeeverypar}}
4198 \ifnum\bbl@bidimode>200 % Any xe bidi=
4199   \let\bbl@textdir@i\@gobbletwo
4200   \let\bbl@xebidipar\@empty
4201   \AddBabelHook{bidi}{foreign}{%
4202     \ifcase\bbl@thetextdir
4203       \BabelWrapText{\LR{##1}}}%
4204     \else
4205       \BabelWrapText{\RL{##1}}}%
4206     \fi}
4207   \def\bbl@pardir#1{\ifcase#1\relax\setLR\else\setRL\fi}
4208 \fi
4209 \fi

A tool for weak L (mainly digits). We also disable warnings with hyperref.

4210 \DeclareRobustCommand\babelsublr[1]{\leavevmode\bbl@textdir\z@#1}}
4211 \AtBeginDocument{%
4212   \ifx\pdfstringdefDisableCommands\@undefined\else
4213     \ifx\pdfstringdefDisableCommands\relax\else
4214       \pdfstringdefDisableCommands{\let\babelsublr\@firstofone}%
4215     \fi
4216   \fi}

```

5.7. Local Language Configuration

\loadlocalcfg At some sites it may be necessary to add site-specific actions to a language definition file. This can be done by creating a file with the same name as the language definition file, but with the extension `.cfg`. For instance the file `norsk.cfg` will be loaded when the language definition file `norsk.ldf` is loaded.

For plain-based formats we don't want to override the definition of `\loadlocalcfg` from `plain.def`.

```

4217 \bbl@trace{Local Language Configuration}
4218 \ifx\loadlocalcfg\@undefined
4219   \ifpackagewith{babel}{noconfigs}%
4220     {\let\loadlocalcfg\@gobble}%
4221     {\def\loadlocalcfg#1{%
4222       \InputIfFileExists{#1.cfg}%
4223       {\typeout{*****^J%
4224                 * Local config file #1.cfg used^^J%
4225                 *}}}%
4226       \@empty}}
4227 \fi

```

5.8. Language options

Languages are loaded when processing the corresponding option *except* if a main language has been set. In such a case, it is not loaded until all options has been processed. The following macro inputs the ldf file and does some additional checks (`\input` works, too, but possible errors are not caught).

```

4228 \bbl@trace{Language options}
4229 \def\BabelDefinitionFile#1#2#3{

```

```

4230 \let\bbl@afterlang\relax
4231 \let\BabelModifiers\relax
4232 \let\bbl@loaded@empty
4233 \def\bbl@load@language#1{%
4234   \InputIfFileExists{#1.ldf}%
4235   {\edef\bbl@loaded{\CurrentOption
4236     \ifx\bbl@loaded@empty\else,\bbl@loaded\fi}%
4237     \expandafter\let\expandafter\bbl@afterlang
4238     \csname\CurrentOption.ldr-h@k\endcsname
4239     \expandafter\let\expandafter\BabelModifiers
4240     \csname bbl@mod@\CurrentOption\endcsname
4241     \bbl@exp{\AtBeginDocument{%
4242       \bbl@usehooks@lang{\CurrentOption}{\begin{document}}{\CurrentOption}}}%
4243     {\bbl@error{unknown-package-option}}{}}}%

```

Another way to extend the list of ‘known’ options for babel was to create the file `bblopts.cfg` in which one can add option declarations. However, this mechanism is deprecated – if you want an alternative name for a language, just create a new `ldf` file loading the actual one. You can also set the name of the file with the package option `config=<name>`, which will load `<name>.cfg` instead.

If the language has been set as metadata, read the info from the corresponding `ini` file and extract the babel name. Then added it as a package option at the end, so that it becomes the main language. The behavior of a metatag with a global language option is not well defined, so if there is not a main option we set here explicitly.

Tagging PDF Span elements requires horizontal mode. With `DocumentMetadata` we also force it with `\foreignlanguage` (this is also done in `bidi texts`).

```

4244 \ifx\GetDocumentProperties@undefined\else
4245   \let\bbl@beforeforeign\leavevmode
4246   \edef\bbl@metalang{\GetDocumentProperties{document/lang}}%
4247   \ifx\bbl@metalang@empty\else
4248     \begin{group}
4249       \expandafter
4250       \bbl@bcplookup\bbl@metalang-\@empty-\@empty-\@empty@@
4251       \ifx\bbl@bcp\relax
4252         \ifx\bbl@opt@main@nnil
4253           \bbl@error{no-locale-for-meta}{\bbl@metalang}{}}%
4254         \fi
4255       \else
4256         \bbl@read@ini{\bbl@bcp}\m@ne
4257         \xdef\bbl@language@opts{\bbl@language@opts,\language}%
4258         \ifx\bbl@opt@main@nnil
4259           \global\let\bbl@opt@main\language
4260         \fi
4261         \bbl@info{Passing \language\space to babel.\\%
4262           This will be the main language except if\\%
4263           explicitly overridden with 'main='.\\%
4264           Reported}%
4265       \fi
4266     \end{group}
4267   \fi
4268 \fi
4269 \ifx\bbl@opt@config@nnil
4270   \@ifpackagewith{babel}{noconfigs}{}%
4271   {\InputIfFileExists{bblopts.cfg}%
4272     {\bbl@info{Configuration files are deprecated, as\\%
4273       they can break document portability.\\%
4274       Reported}%
4275     \typeout{*****^J%
4276       * Local config file bblopts.cfg used^^J%
4277       *}%
4278     {}}%
4279 \else
4280   \InputIfFileExists{\bbl@opt@config.cfg}%
4281   {\bbl@info{Configuration files are deprecated, as\\%

```

```

4282             they can break document portability.\\%
4283             Reported}%
4284     \typeout{*****^J%
4285             * Local config file \bbl@opt@config.cfg used^^J%
4286             *}}%
4287     {\bbl@error{config-not-found}}{}{}{}%
4288 \fi

```

Recognizing global options in packages not having a closed set of them is not trivial, as for them to be processed they must be defined explicitly. So, package options not yet taken into account and stored in `\bbl@language@opts` are assumed to be languages. If not declared above, the names of the option and the file are the same. We first pre-process the class and package options to determine the available locales, and which version (ldf or ini) will be loaded. This is done by first loading the corresponding `babel-⟨name⟩.tex` file.

The second argument of `\BabelBeforeIni` may contain a `\BabelDefinitionFile` which defines `\bbl@tempa` and `\bbl@tempb` and saves the third argument for the moment of the actual loading. If there is no `\BabelDefinitionFile` the last element is usually empty, and the ini file is loaded. The values are used to build a list in the form ‘main-or-not’ / ‘ldf-or-ldfini-flag’ // ‘option-name’ // ‘bcp-tag’ / ‘ldf-name-or-none’. The ‘main-or-not’ element is 0 by default and set to 10 later if necessary (by prepending 1). The ‘bcp-tag’ is stored here so that the corresponding ini file can be loaded directly (with `@import`).

```

4289 \def\BabelBeforeIni#1#2{%
4290   \def\bbl@tempa{\@m}% <- Default if no \BDefFile
4291   \let\bbl@tempb\@empty
4292   #2%
4293   \edef\bbl@toload{%
4294     \ifx\bbl@toload\@empty\else\bbl@toload,\fi
4295     \bbl@toload@last}%
4296   \edef\bbl@toload@last{0/\bbl@tempa//\CurrentOption//\#1/\bbl@tempb}}
4297 \def\BabelDefinitionFile#1#2#3{%
4298   \def\bbl@tempa{#1}\def\bbl@tempb{#2}%
4299   \@namedef{\bbl@preldf@CurrentOption}{#3}%
4300   \endinput}%

```

For efficiency, first preprocess the class options to remove those with =, which are becoming increasingly frequent (no language should contain this character). Here we use the more robust macro to traverse a clist from the $\TeX$ layer.

```

4301 \def\bbl@tempf{,}
4302 \@nameuse{clist_map_inline:Nn}\@raw@classoptionslist{%
4303   \in@{=}{#1}%
4304   \ifin@
4305     \edef\bbl@tempf{\bbl@tempf\zap@space#1 \@empty,}%
4306   \fi}

```

Store the class/package options in a list. If there is an explicit main, it’s placed as the last option. Then loop it to read the tex files, which can have a `\BabelDefinitionFile`. If there is no tex file, we attempt loading the ldf for the option name; if it fails, an error is raised. Note the option name is surrounded by `//...//`. Class and package options are separated with `@`, because errors and info are dealt with in different ways. Consecutive identical languages count as one.

```

4307 \let\bbl@toload\@empty
4308 \let\bbl@toload@last\@empty
4309 \let\bbl@unkopt\@gobble %% <- Ugly
4310 \edef\bbl@tempc{%
4311   \bbl@tempf,@,\bbl@language@opts
4312   \ifx\bbl@opt@main\@nnil\else,\bbl@opt@main\fi}
4313 \let\BabelLocalesTentative\bbl@tempc
4314 %
4315 \bbl@foreach\bbl@tempc{%
4316   \in@{@@}{#1}% <- Ugly
4317   \ifin@
4318     \def\bbl@unkopt##1{%
4319       \DeclareOption{##1}{\bbl@error{unknown-package-option}}{}{}{}}%
4320   \else
4321     \def\CurrentOption{#1}%

```

```

4322 \bbl@xin@{/#1//}{\bbl@toload@last}% Collapse consecutive
4323 \ifin@else
4324 \lowercase{\InputIfFileExists{babel-#1.tex}}{}{}%
4325 \IfFileExists{#1.ldf}%
4326 {\edef\bbl@toload{%
4327 \ifx\bbl@toload\@empty\else\bbl@toload,\fi
4328 \bbl@toload@last}%
4329 \edef\bbl@toload@last{0/0//\CurrentOption//und/#1}}%
4330 {\bbl@unkopt{#1}}}%
4331 \fi
4332 \fi}

```

We have to determine (1) if no language has been loaded (in which case we fallback to ‘nil’, with a special tag), and (2) the main language. With an explicit ‘main’ language, remove repeated elements. The number 1 flags it as the main language (relevant in *ini* locales), because with 0 becomes 10.

```

4333 \ifx\bbl@opt@main\@nnil
4334 \ifx\bbl@toload@last\@empty
4335 \def\bbl@toload@last{0/0//nil//und-x-nil/nil}
4336 \bbl@info{%
4337 You haven't specified a language as a class or package\\%
4338 option. I'll load 'nil'. Reported}
4339 \fi
4340 \else
4341 \let\bbl@tempc\@empty
4342 \bbl@foreach\bbl@toload{%
4343 \bbl@xin@{/#1//\bbl@opt@main//}{#1}%
4344 \ifin@else
4345 \bbl@add@list\bbl@tempc{#1}%
4346 \fi}
4347 \let\bbl@toload\bbl@tempc
4348 \fi
4349 \edef\bbl@toload{\bbl@toload,1\bbl@toload@last}

```

Finally, load the ‘ini’ file or the pair ‘ini’/‘ldf’ file. Babel resorts to its own mechanism, not the default one based on \ProcessOptions (which is still present to make some internal clean-up). First, handle provide= and friends (with a recursive call if they are present), and then provide= and friend. \count@ is used as flag: 0 if ‘ini’, 1 if ‘ldf’.

```

4350 \def\AfterBabelLanguage#1{%
4351 \bbl@ifsamestring\CurrentOption{#1}{\global\bbl@add\bbl@afterlang{}}}
4352 \NewHook{babel/presets}
4353 \UseHook{babel/presets}
4354 %
4355 \let\bbl@tempb\@empty
4356 \def\bbl@tempc#1/#2//#3//#4/#5\@@{%
4357 \count@z@
4358 \ifnum#2=\@m % if no \BabelDefinitionFile
4359 \ifnum#1=z@ % not main. -- % if provide+=!, provide*=!
4360 \ifnum\bbl@ldfflag>\@ne\bbl@tempc 0/0//#3//#4/#3\@@
4361 \else\bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4362 \fi
4363 \else % 10 = main -- % if provide=!, provide*=!
4364 \ifodd\bbl@ldfflag\bbl@tempc 10/0//#3//#4/#3\@@
4365 \else\bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4366 \fi
4367 \fi
4368 \else
4369 \ifnum#1=z@ % not main
4370 \ifnum\bbl@iniflag>\@ne\else % if ∅, provide
4371 \ifcase#2\count@\@ne\else\ifcase\bbl@engine\count@\@ne\fi\fi
4372 \fi
4373 \else % 10 = main
4374 \ifodd\bbl@iniflag\else % if provide+, provide*
4375 \ifcase#2\count@\@ne\else\ifcase\bbl@engine\count@\@ne\fi\fi
4376 \fi

```



```

4377 \fi
4378 \bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4379 \fi}

Based on the value of \count@, do the actual loading. If 'ldf', we load the basic info from the 'ini' file
before.

4380 \def\bbl@tempd#1#2#3#4#5{%
4381 \DeclareOption{#3}{}%
4382 \ifcase\count@
4383 \bbl@exp{\\bbl@add\\bbl@tempb{%
4384 \global\let\\bbl@afterload\\@empty
4385 \\@nameuse{bbl@preini@#3}%
4386 \\bbl@ldfinit
4387 \def\\CurrentOption{#3}%
4388 \\bbl@provide[@import=#4,\ifnum#1=z@else\bbl@opt@provide,main\fi]{#3}%
4389 \\bbl@afterldf
4390 \\bbl@afterload}}%
4391 \else
4392 \bbl@add\bbl@tempb{%
4393 \global\let\bbl@afterload\@empty
4394 \def\CurrentOption{#3}%
4395 \let\localename\CurrentOption
4396 \let\language\localename
4397 \def\BabelIniTag{#4}%
4398 \@nameuse{bbl@preldf@#3}%
4399 \begingroup
4400 \bbl@id@assign
4401 \bbl@read@ini{\BabelIniTag}0%
4402 \endgroup
4403 \bbl@load@language{#5}%
4404 \bbl@afterload}%
4405 \fi}
4406 %
4407 \bbl@foreach\bbl@toload{\bbl@tempc#1\@@}
4408 \bbl@tempb
4409 \DeclareOption*{}
4410 \ProcessOptions
4411 %
4412 \bbl@exp{%
4413 \\AtBeginDocument{\\bbl@usehooks@lang{/{}}{begindocument}{}}}%
4414 \def\AfterBabelLanguage{\bbl@error{late-after-babel}{}}{}
4415 </package>

```

6. The kernel of Babel

The kernel of the babel system is currently stored in `babel.def`. The file `babel.def` contains most of the code. The file `hyphen.cfg` is a file that can be loaded into the format, which is necessary when you want to be able to switch hyphenation patterns.

Because plain $\text{T}_{\text{E}}\text{X}$ users might want to use some of the features of the babel system too, care has to be taken that plain $\text{T}_{\text{E}}\text{X}$ can process the files. For this reason the current format will have to be checked in a number of places. Some of the code below is common to plain $\text{T}_{\text{E}}\text{X}$ and $\text{\LaTeX}$, some of it is for the $\text{\LaTeX}$ case only.

Plain formats based on `etex` (`etex`, `xetex`, `luatex`) don't load `hyphen.cfg` but `etex.src`, which follows a different naming convention, so we need to define the babel names. It presumes `language.def` exists and it is the same file used when formats were created.

A proxy file for `switch.def`

```

4416 <{*kernel}
4417 \let\bbl@onlyswitch\@empty
4418 \input babel.def
4419 \let\bbl@onlyswitch\@undefined
4420 </kernel>

```

7. Error messages

They are loaded when `\bbl@error` is first called. To save space, the main code just identifies them with a tag, and messages are stored in a separate file. Since it can be loaded anywhere, you make sure some catcodes have the right value, although those for `\`, ```, `^`, `M`, `%` and `=` are reset before loading the file.

```
4421 (*errors)
4422 \catcode`\{=1 \catcode`\}=2 \catcode`\#=6
4423 \catcode`\:=12 \catcode`\,=12 \catcode`\.=12 \catcode`\-=12
4424 \catcode`\'=12 \catcode`\(=12 \catcode`\)=12
4425 \catcode`\@=11 \catcode`\^=7
4426 %
4427 \ifx\MessageBreak\undefined
4428 \gdef\bbl@error@i#1#2{%
4429 \beginingroup
4430 \newlinechar=`^^J
4431 \def\{^^J(babel) }%
4432 \errhelp{#2}\errmessage{\{#1}%
4433 \endgroup}
4434 \else
4435 \gdef\bbl@error@i#1#2{%
4436 \beginingroup
4437 \def\{\MessageBreak}%
4438 \PackageError{babel}{#1}{#2}%
4439 \endgroup}
4440 \fi
4441 \def\bbl@errmessage#1#2#3{%
4442 \expandafter\gdef\csname bbl@err@#1\endcsname##1##2##3{%
4443 \bbl@error@i{#2}{#3}}
4444 % Implicit #2#3#4:
4445 \gdef\bbl@error#1{\csname bbl@err@#1\endcsname}
4446 %
4447 \bbl@errmessage{not-yet-available}
4448 {Not yet available}%
4449 {Find an armchair, sit down and wait}
4450 \bbl@errmessage{bad-package-option}%
4451 {Bad option '#1=#2'. Either you have misspelled the\\%
4452 key or there is a previous setting of '#1'. Valid\\%
4453 keys are, among others, 'shorthands', 'main', 'bidi',\\%
4454 'strings', 'config', 'headfoot', 'safe', 'math'.}%
4455 {See the manual for further details.}
4456 \bbl@errmessage{base-on-the-fly}
4457 {For a language to be defined on the fly 'base'\\%
4458 is not enough, and the whole package must be\\%
4459 loaded. Either delete the 'base' option or\\%
4460 request the languages explicitly}%
4461 {See the manual for further details.}
4462 \bbl@errmessage{undefined-language}
4463 {You haven't defined the language '#1' yet.\\%
4464 Perhaps you misspelled it or your installation\\%
4465 is not complete}%
4466 {Your command will be ignored, type <return> to proceed}
4467 \bbl@errmessage{invalid-ini-name}
4468 {'#1' not valid with the 'ini' mechanism.\\%
4469 I think you want '#2' instead. You may continue,\\%
4470 but you should fix the name. See the babel manual\\%
4471 for the available locales with 'provide'}%
4472 {See the manual for further details.}
4473 \bbl@errmessage{shorthand-is-off}
4474 {I can't declare a shorthand turned off (\string#2)}
4475 {Sorry, but you can't use shorthands which have been\\%
4476 turned off in the package options}
4477 \bbl@errmessage{not-a-shorthand}
```

```

4478 {The character '\string #1' should be made a shorthand character;\%
4479 add the command \string\usesshorthands\string{#1\string} to
4480 the preamble.\%
4481 I will ignore your instruction}%
4482 {You may proceed, but expect unexpected results}
4483 \bbl@errmessage{not-a-shorthand-b}
4484 {I can't switch '\string#2' on or off--not a shorthand\%
4485 This character is not a shorthand. Maybe you made\%
4486 a typing mistake?}%
4487 {I will ignore your instruction.}
4488 \bbl@errmessage{unknown-attribute}
4489 {The attribute #2 is unknown for language #1.}%
4490 {Your command will be ignored, type <return> to proceed}
4491 \bbl@errmessage{missing-group}
4492 {Missing group for string \string#1}%
4493 {You must assign strings to some category, typically\%
4494 captions or extras, but you set none}
4495 \bbl@errmessage{only-lua-xe}
4496 {This macro is available only in LuaLaTeX and XeLaTeX.}%
4497 {Consider switching to these engines.}
4498 \bbl@errmessage{only-lua}
4499 {This macro is available only in LuaLaTeX}%
4500 {Consider switching to that engine.}
4501 \bbl@errmessage{unknown-provide-key}
4502 {Unknown key '#1' in \string\babelprovide}%
4503 {See the manual for valid keys}%
4504 \bbl@errmessage{unknown-mapfont}
4505 {Option '\bbl@KVP@mapfont' unknown for\%
4506 mapfont. Use 'direction'}%
4507 {See the manual for details.}
4508 \bbl@errmessage{no-ini-file}
4509 {There is no ini file for the requested language\%
4510 (#1: \language). Perhaps you misspelled it or your\%
4511 installation is not complete}%
4512 {Fix the name or reinstall babel.}
4513 \bbl@errmessage{digits-is-reserved}
4514 {The counter name 'digits' is reserved for mapping\%
4515 decimal digits}%
4516 {Use another name.}
4517 \bbl@errmessage{limit-two-digits}
4518 {Currently two-digit years are restricted to the\%
4519 range 0-9999}%
4520 {There is little you can do. Sorry.}
4521 \bbl@errmessage{alphabetic-too-large}
4522 {Alphabetic numeral too large (#1)}%
4523 {Currently this is the limit.}
4524 \bbl@errmessage{no-ini-info}
4525 {I've found no info for the current locale.\%
4526 The corresponding ini file has not been loaded\%
4527 Perhaps it doesn't exist}%
4528 {See the manual for details.}
4529 \bbl@errmessage{unknown-ini-field}
4530 {Unknown field '#1' in \string\BCPdata.\%
4531 Perhaps you misspelled it}%
4532 {See the manual for details.}
4533 \bbl@errmessage{unknown-locale-key}
4534 {Unknown key for locale '#2':\%
4535 #3\%
4536 \string#1 will be set to \string\relax}%
4537 {Perhaps you misspelled it.}%
4538 \bbl@errmessage{adjust-only-vertical}
4539 {Currently, #1 related features can be adjusted only\%
4540 in the main vertical list}%

```

```

4541 {Maybe things change in the future, but this is what it is.}
4542 \bbl@errmessage{layout-only-vertical}
4543 {Currently, layout related features can be adjusted only\\%
4544 in vertical mode}%
4545 {Maybe things change in the future, but this is what it is.}
4546 \bbl@errmessage{bidi-only-lua}
4547 {The bidi method 'basic' is available only in\\%
4548 luatex. I'll continue with 'bidi=default', so\\%
4549 expect wrong results.\\%
4550 Suggested actions:\\%
4551 * If possible, switch to luatex, as xetex is not\\%
4552 recommend anymore.\\
4553 * If you can't, try 'bidi=bidi' with xetex.\\%
4554 * With pdftex, only 'bidi=default' is available.}%
4555 {See the manual for further details.}
4556 \bbl@errmessage{multiple-bidi}
4557 {Multiple bidi settings inside a group\\%
4558 I'll insert a new group, but expect wrong results.\\%
4559 Suggested action:\\%
4560 * Add a new group where appropriate.}
4561 {See the manual for further details.}
4562 \bbl@errmessage{unknown-package-option}
4563 {Unknown option '\CurrentOption'.\\%
4564 Suggested actions:\\%
4565 * Make sure you haven't misspelled it\\%
4566 * Check in the babel manual that it's supported\\%
4567 * If supported and it's a language, you may\\%
4568 \space\space need in some distributions a separate\\%
4569 \space\space installation\\%
4570 * If installed, check there isn't an old\\%
4571 \space\space version of the required files in your system\\%
4572 * If it's an unsupported language, create it with\\%
4573 \string\babelprovide (see the manual)}
4574 {Valid options are, among others: shorthands=, KeepShorthandsActive,\\%
4575 activeacute, activegrave, noconfigs, safe=, main=, math=\\%
4576 headfoot=, strings=, config=, hyphenmap=, or a language name.}
4577 \bbl@errmessage{config-not-found}
4578 {Local config file '\bbl@opt@config.cfg' not found.\\%
4579 Suggested actions:\\%
4580 * Make sure you haven't misspelled it in config=\\%
4581 * Check it exists and it's in the correct path}%
4582 {Perhaps you misspelled it.}
4583 \bbl@errmessage{late-after-babel}
4584 {Too late for \string\AfterBabelLanguage}%
4585 {Languages have been loaded, so I can do nothing}
4586 \bbl@errmessage{double-hyphens-class}
4587 {Double hyphens aren't allowed in \string\babelcharclass\\%
4588 because it's potentially ambiguous}%
4589 {See the manual for further info}
4590 \bbl@errmessage{unknown-interchar}
4591 {'#1' for '\language' cannot be enabled.\\%
4592 Maybe there is a typo}%
4593 {See the manual for further details.}
4594 \bbl@errmessage{unknown-interchar-b}
4595 {'#1' for '\language' cannot be disabled.\\%
4596 Maybe there is a typo}%
4597 {See the manual for further details.}
4598 \bbl@errmessage{charproperty-only-vertical}
4599 {\string\babelcharproperty\space can be used only in\\%
4600 vertical mode (preamble or between paragraphs)}%
4601 {See the manual for further info}
4602 \bbl@errmessage{unknown-char-property}
4603 {No property named '#2'. Allowed values are\\%

```

```

4604 direction (bc), mirror (bmg), and linebreak (lb))%
4605 {See the manual for further info}
4606 \bbl@errmessage{bad-transform-option}
4607 {Bad option '#1' in a transform.\\%
4608 I'll ignore it but expect more errors}%
4609 {See the manual for further info.}
4610 \bbl@errmessage{font-conflict-transforms}
4611 {Transforms cannot be re-assigned to different\\%
4612 fonts. The conflict is in '\bbl@kv@label'.\\%
4613 Apply the same fonts or use a different label}%
4614 {See the manual for further details.}
4615 \bbl@errmessage{transform-not-available}
4616 {'#1' for '\language' cannot be enabled.\\%
4617 Maybe there is a typo or it's a font-dependent transform}%
4618 {See the manual for further details.}
4619 \bbl@errmessage{transform-not-available-b}
4620 {'#1' for '\language' cannot be disabled.\\%
4621 Maybe there is a typo or it's a font-dependent transform}%
4622 {See the manual for further details.}
4623 \bbl@errmessage{year-out-range}
4624 {Year out of range.\\%
4625 The allowed range is #1}%
4626 {See the manual for further details.}
4627 \bbl@errmessage{only-pdfTeX-lang}
4628 {The '#1' ldf style doesn't work with #2,\\%
4629 but you can use the ini locale instead.\\%
4630 Try adding 'provide=' to the option list. You may\\%
4631 also want to set 'bidi=' to some value}%
4632 {See the manual for further details.}
4633 \bbl@errmessage{hyphenmins-args}
4634 {\string\babelhyphenmins\ accepts either the optional\\%
4635 argument or the star, but not both at the same time}%
4636 {See the manual for further details.}
4637 \bbl@errmessage{no-locale-for-meta}
4638 {There isn't currently a locale for the 'lang' requested\\%
4639 in the PDF metadata ('#1'). To fix it, you can\\%
4640 set explicitly a similar language (using the same\\%
4641 script) with the key main= when loading babel. If you\\%
4642 continue, I'll fallback to the 'nil' language, with\\%
4643 tag 'und' and script 'Latn', but expect a bad font\\%
4644 rendering with other scripts. You may also need set\\%
4645 explicitly captions and date, too}%
4646 {See the manual for further details.}
4647 </errors>
4648 <*patterns>

```

8. Loading hyphenation patterns

The following code is meant to be read by $\text{\texttt{iniTeX}}$ because it should instruct $\text{\texttt{TeX}}$ to read hyphenation patterns. To this end the `docstrip` option `patterns` is used to include this code in the file `hyphen.cfg`. Code is written with lower level macros.

```

4649 <@Make sure ProvidesFile is defined@>
4650 \ProvidesFile{hyphen.cfg}[<@date@> v<@version@> Babel hyphens]
4651 \xdef\bbl@format{\jobname}
4652 \def\bbl@version{<@version@>}
4653 \def\bbl@date{<@date@>}
4654 \ifx\AtBeginDocument\undefined
4655 \def\@empty{}
4656 \fi
4657 <@Define core switching macros@>

```

\process@line Each line in the file `language.dat` is processed by `\process@line` after it is read. The first thing this macro does is to check whether the line starts with `=`. When the first token of a line is an `=`, the macro `\process@synonym` is called; otherwise the macro `\process@language` will continue.

```
4658 \def\process@line#1#2 #3 #4 {%
4659   \ifx=#1%
4660     \process@synonym{#2}%
4661   \else
4662     \process@language{#1#2}{#3}{#4}%
4663   \fi
4664   \ignorespaces}
```

\process@synonym This macro takes care of the lines which start with an `=`. It needs an empty token register to begin with. `\bbl@languages` is also set to empty.

```
4665 \toks@{}
4666 \def\bbl@languages{}
```

When no languages have been loaded yet, the name following the `=` will be a synonym for hyphenation register 0. So, it is stored in a token register and executed when the first pattern file has been processed. (The `\relax` just helps to the `\if` below catching synonyms without a language.)

Otherwise the name will be a synonym for the language loaded last.

We also need to copy the `hyphenmin` parameters for the synonym.

```
4667 \def\process@synonym#1{%
4668   \ifnum\last@language=\m@ne
4669     \toks@\expandafter{\the\toks@\relax\process@synonym{#1}}%
4670   \else
4671     \expandafter\chardef\csname l@#1\endcsname\last@language
4672     \wlog{\string\l@#1=\string\language\the\last@language}%
4673     \expandafter\let\csname #1hyphenmins\expandafter\endcsname
4674       \csname\language\hyphenmins\endcsname
4675     \let\bbl@elt\relax
4676     \edef\bbl@languages{\bbl@languages\bbl@elt{#1}{\the\last@language}}}%
4677   \fi}
```

\process@language The macro `\process@language` is used to process a non-empty line from the ‘configuration file’. It has three arguments, each delimited by white space. The first argument is the ‘name’ of a language; the second is the name of the file that contains the patterns. The optional third argument is the name of a file containing hyphenation exceptions.

The first thing to do is call `\addlanguage` to allocate a pattern register and to make that register ‘active’. Then the pattern file is read.

For some hyphenation patterns it is needed to load them with a specific font encoding selected. This can be specified in the file `language.dat` by adding for instance ‘:T1’ to the name of the language. The macro `\bbl@get@enc` extracts the font encoding from the language name and stores it in `\bbl@hyph@enc`. The latter can be used in hyphenation files if you need to set a behavior depending on the given encoding (it is set to empty if no encoding is given).

Pattern files may contain assignments to `\lefthyphenmin` and `\righthyphenmin`. $\TeX$ does not keep track of these assignments. Therefore we try to detect such assignments and store them in the `\<language>hyphenmins` macro. When no assignments were made we provide a default setting.

Some pattern files contain changes to the `\lccode` and `\uccode` arrays. Such changes should remain local to the language; therefore we process the pattern file in a group; the `\patterns` command acts globally so its effect will be remembered.

Then we globally store the settings of `\lefthyphenmin` and `\righthyphenmin` and close the group.

When the hyphenation patterns have been processed we need to see if a file with hyphenation exceptions needs to be read. This is the case when the third argument is not empty and when it does not contain a space token. (Note however there is no need to save hyphenation exceptions into the format.)

`\bbl@languages` saves a snapshot of the loaded languages in the form `\bbl@elt{<language-name>}{<number>}{<patterns-file>}{<exceptions-file>}`. Note the last 2 arguments are empty in ‘dialects’ defined in `language.dat` with `=`. Note also the language name can have encoding info.

Finally, if the counter `\language` is equal to zero we execute the synonyms stored.

```
4678 \def\process@language#1#2#3{%
```

```

4679 \expandafter\addlanguage\csname l@#1\endcsname
4680 \expandafter\language\csname l@#1\endcsname
4681 \edef\languagename{#1}%
4682 \bbl@hook@everylanguage{#1}%
4683 % > luatex
4684 \bbl@get@enc#1::\@@@
4685 \begingroup
4686 \lefthyphenmin\m@ne
4687 \bbl@hook@loadpatterns{#2}%
4688 % > luatex
4689 \ifnum\lefthyphenmin=\m@ne
4690 \else
4691 \expandafter\xdef\csname #1hyphenmins\endcsname{%
4692 \the\lefthyphenmin\the\rightthyphenmin}%
4693 \fi
4694 \endgroup
4695 \def\bbl@tempa{#3}%
4696 \ifx\bbl@tempa\@empty\else
4697 \bbl@hook@loadexceptions{#3}%
4698 % > luatex
4699 \fi
4700 \let\bbl@elt\relax
4701 \edef\bbl@languages{%
4702 \bbl@languages\bbl@elt{#1}\the\language}{#2}{\bbl@tempa}}%
4703 \ifnum\the\language=\z@
4704 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
4705 \set@hyphenmins\tw@\thr@@\relax
4706 \else
4707 \expandafter\expandafter\expandafter\set@hyphenmins
4708 \csname #1hyphenmins\endcsname
4709 \fi
4710 \the\toks@
4711 \toks@{}%
4712 \fi}

```

\bbl@get@enc

\bbl@hyph@enc The macro \bbl@get@enc extracts the font encoding from the language name and stores it in \bbl@hyph@enc. It uses delimited arguments to achieve this.

```
4713 \def\bbl@get@enc#1:#2:#3\@@@{\def\bbl@hyph@enc{#2}}
```

Now, hooks are defined. For efficiency reasons, they are dealt here in a special way. Besides luatex, format-specific configuration files are taken into account. loadkernel currently loads nothing, but define some basic macros instead.

```

4714 \def\bbl@hook@everylanguage#1{}
4715 \def\bbl@hook@loadpatterns#1{\input #1\relax}
4716 \let\bbl@hook@loadexceptions\bbl@hook@loadpatterns
4717 \def\bbl@hook@loadkernel#1{%
4718 \def\addlanguage{\csname newlanguage\endcsname}%
4719 \def\adddialect##1##2{%
4720 \global\chardef##1##2\relax
4721 \wlog{\string##1 = a dialect from \string\language##2}}%
4722 \def\iflanguage##1{%
4723 \expandafter\ifx\csname l@##1\endcsname\relax
4724 \nolanner{##1}%
4725 \else
4726 \ifnum\csname l@##1\endcsname=\language
4727 \expandafter\expandafter\expandafter\@firstoftwo
4728 \else
4729 \expandafter\expandafter\expandafter\@secondoftwo
4730 \fi
4731 \fi}%
4732 \def\providehyphenmins##1##2{%
4733 \expandafter\ifx\csname ##1hyphenmins\endcsname\relax

```

```

4734 \namedef{##1hyphenmins}{##2}%
4735 \fi}%
4736 \def\set@hyphenmins##1##2{%
4737 \lefthyphenmin##1\relax
4738 \righthyphenmin##2\relax}%
4739 \def\selectlanguage{%
4740 \errhelp{Selecting a language requires a package supporting it}%
4741 \errmessage{No multilingual package has been loaded}}%
4742 \let\foreignlanguage\selectlanguage
4743 \let\otherlanguage\selectlanguage
4744 \expandafter\let\csname otherlanguage*\endcsname\selectlanguage
4745 \def\bbl@usehooks##1##2{%
4746 \def\setlocale{%
4747 \errhelp{Find an armchair, sit down and wait}%
4748 \errmessage{(babel) Not yet available}}%
4749 \let\uselocale\setlocale
4750 \let\locale\setlocale
4751 \let\selectlocale\setlocale
4752 \let\localename\setlocale
4753 \let\textlocale\setlocale
4754 \let\textlanguage\setlocale
4755 \let\languagetext\setlocale}
4756 \begingroup
4757 \def\AddBabelHook#1#2{%
4758 \expandafter\ifx\csname bbl@hook@#2\endcsname\relax
4759 \def\next{\toks1}%
4760 \else
4761 \def\next{\expandafter\gdef\csname bbl@hook@#2\endcsname###1}%
4762 \fi
4763 \next}
4764 \ifx\directlua\@undefined
4765 \ifx\XeTeXinputencoding\@undefined\else
4766 \input xebabel.def
4767 \fi
4768 \else
4769 \input luababel.def
4770 \fi
4771 \openin1 = babel-\bbl@format.cfg
4772 \ifeof1
4773 \else
4774 \input babel-\bbl@format.cfg\relax
4775 \fi
4776 \closein1
4777 \endgroup
4778 \bbl@hook@loadkernel{switch.def}

```

\readconfigfile The configuration file can now be opened for reading.

```

4779 \openin1 = language.dat

```

See if the file exists, if not, use the default hyphenation file `hyphen.tex`. The user will be informed about this.

```

4780 \def\language{english}%
4781 \ifeof1
4782 \message{I couldn't find the file language.dat,\space
4783 I will try the file hyphen.tex}
4784 \input hyphen.tex\relax
4785 \chardef\l@english\z@
4786 \else

```

Pattern registers are allocated using count register `\last@language`. Its initial value is 0. The definition of the macro `\newlanguage` is such that it first increments the count register and then defines the language. In order to have the first patterns loaded in pattern register number 0 we initialize `\last@language` with the value `-1`.


```
4787 \last@language\m@ne
```

We now read lines from the file until the end is found. While reading from the input, it is useful to switch off recognition of the end-of-line character. This saves us stripping off spaces from the contents of the control sequence.

```
4788 \loop
4789 \endlinechar\m@ne
4790 \readl to \bbl@line
4791 \endlinechar\^^M
```

If the file has reached its end, exit from the loop here. If not, empty lines are skipped. Add 3 space characters to the end of \bbl@line. This is needed to be able to recognize the arguments of \process@line later on. The default language should be the very first one.

```
4792 \if T\ifeoflF\fi T\relax
4793 \ifx\bbl@line\@empty\else
4794 \edef\bbl@line{\bbl@line\space\space\space}%
4795 \expandafter\process@line\bbl@line\relax
4796 \fi
4797 \repeat
```

Check for the end of the file. We must reverse the test for \ifeof without \else. Then reactivate the default patterns, and close the configuration file.

```
4798 \begingroup
4799 \def\bbl@elt#1#2#3#4{%
4800 \global\language=#2\relax
4801 \gdef\language#1}%
4802 \def\bbl@elt##1##2##3##4{}}%
4803 \bbl@languages
4804 \endgroup
4805 \fi
4806 \closeinl
```

We add a message about the fact that babel is loaded in the format and with which language patterns to the \everyjob register.

```
4807 \if/\the\toks@\else
4808 \errhelp{language.dat loads no language, only synonyms}
4809 \errmessage{Orphan language synonym}
4810 \fi
```

Also remove some macros from memory and raise an error if \toks@ is not empty. Finally load switch.def, but the latter is not required and the line inputting it may be commented out.

```
4811 \let\bbl@line\@undefined
4812 \let\process@line\@undefined
4813 \let\process@synonym\@undefined
4814 \let\process@language\@undefined
4815 \let\bbl@get@enc\@undefined
4816 \let\bbl@hyph@enc\@undefined
4817 \let\bbl@tempa\@undefined
4818 \let\bbl@hook@loadkernel\@undefined
4819 \let\bbl@hook@everylanguage\@undefined
4820 \let\bbl@hook@loadpatterns\@undefined
4821 \let\bbl@hook@loadexceptions\@undefined
4822 </patterns>
```

Here the code for `initTeX` ends.

9. luatex + xetex: common stuff

Add the bidi handler just before luaotfload, which is loaded by default by LaTeX. Just in case, consider the possibility it has not been loaded. First, a couple of definitions related to bidi (although default also applies to pdftex).

```
4823 <<*<More package options>> ≡
4824 \chardef\bbl@bidimode\z@
```

```

4825 \DeclareOption{bidi=default}{\chardef\bbbl@bidimode=\@ne}
4826 \DeclareOption{bidi=basic}{\chardef\bbbl@bidimode=101 }
4827 \DeclareOption{bidi=basic-r}{\chardef\bbbl@bidimode=102 }
4828 \DeclareOption{bidi=bidi}{\chardef\bbbl@bidimode=201 }
4829 \DeclareOption{bidi=bidi-r}{\chardef\bbbl@bidimode=202 }
4830 \DeclareOption{bidi=bidi-l}{\chardef\bbbl@bidimode=203 }
4831 <</More package options>>

```

\babelfont With explicit languages, we could define the font at once, but we don't. Just wait and see if the language is actually activated. `bbbl@font` replaces hardcoded font names inside `\. . family` by the corresponding macro `\. . default`.

```

4832 <<*Font selection>> ≡
4833 \bbbl@trace{Font handling with fontspec}
4834 \AddBabelHook{babel-fontspec}{afterextras}{\bbbl@switchfont}
4835 \AddBabelHook{babel-fontspec}{beforestart}{\bbbl@cckcheckstdfonts}
4836 \DisableBabelHook{babel-fontspec}
4837 \@onlypreamble\babelfont
4838 \ifx\NewDocumentCommand\undefined\else % Not plain
4839   \NewDocumentCommand\babelfont{0}{m0}{m0}}{%
4840     \bbbl@bbblfont@o[#1]{#2}{#3,#5}{#4}}
4841 \fi
4842 \newcommand\bbbl@bbblfont@o[2][]{% 1=langs/scripts 2=fam
4843   \ifx\fontspec\undefined
4844     \usepackage{fontspec}%
4845   \fi
4846   \EnableBabelHook{babel-fontspec}%
4847   \edef\bbbl@tempa{#1}%
4848   \def\bbbl@tempb{#2}% Used by \bbbl@bbblfont
4849   \bbbl@bbblfont}
4850 \newcommand\bbbl@bbblfont[2][]{% 1=features 2=fontname, @font=rm|sf|tt
4851   \bbbl@ifunset{\bbbl@tempb family}%
4852     {\bbbl@providfam{\bbbl@tempb}}%
4853   }%
4854   % For the default font, just in case:
4855   \bbbl@ifunset{\bbbl@lsys{\language name}}{\bbbl@provide@lsys{\language name}}{%
4856     \expandafter\bbbl@ifblank\expandafter{\bbbl@tempa}%
4857     {\bbbl@csarg\edef{\bbbl@tempb dflt@}{<>{#1}{#2}}% save \bbbl@rmdflt@
4858     \bbbl@exp{%
4859       \let<\bbbl@bbbl@tempb dflt@\language name>\<\bbbl@bbbl@tempb dflt@>%
4860       \\\bbbl@font@set<\bbbl@bbbl@tempb dflt@\language name>%
4861       \<\bbbl@tempb default>\<\bbbl@tempb family>}}%
4862     {\bbbl@foreach\bbbl@tempa{% i.e., \bbbl@rmdflt@lang / *s crt
4863       \bbbl@csarg\def{\bbbl@tempb dflt@##1}{<>{#1}{#2}}}}}%

```

If the family in the previous command does not exist, it must be defined. Here is how:

```

4864 \def\bbbl@providfam#1{%
4865   \bbbl@exp{%
4866     \\\newcommand\<#1default>{}% Just define it
4867     \\\bbbl@add@list\bbbl@font@fams{#1}%
4868     \\\NewHook{#1family}%
4869     \\\DeclareRobustCommand\<#1family>{%
4870       \\\not@math@alphabet\<#1family>\relax
4871       % \\\prepare@family@series@update{#1}\<#1default>% TODO. Fails
4872       \\\fontfamily\<#1default>%
4873       \\\UseHook{#1family}%
4874       \\\selectfont}%
4875     \\\DeclareTextFontCommand{\<text#1>}{\<#1family>}}}

```

The following macro is activated when the hook `babel-fontspec` is enabled. But before, we define a macro for a warning, which sets a flag to avoid duplicate them.

```

4876 \def\bbbl@nostdfont#1{%
4877   \bbbl@once{nostdfam-\f@family}%
4878   {\bbbl@infowarn{The current font is not a babel standard family:\f@family}}

```

```

4879      #1%
4880      \fontname\font\\%
4881      There is nothing intrinsically wrong, and you can\\%,
4882      ignore this message altogether if you do not need\\%
4883      this font. If they are used in the document, be aware\\%
4884      'babel' will not set Script and Language for it, so\\%
4885      you may consider defining a new family with \string\babelfont.\\%
4886      See the manual for further details about \string\babelfont.
4887      Reported}}%
4888      {}}%
4889 \gdef\bbl@switchfont{%
4890   \bbl@ifunset\bbl@lsys@\language\name\{\bbl@provide@lsys\{\language\name\}\}%
4891   \bbl@exp{% e.g., Arabic -> arabic
4892     \lowercase\edef\\bbl@tempa{\bbl@cl{sname}}}%
4893   \bbl@foreach\bbl@font@fams{%
4894     \bbl@ifunset\bbl@##1dflt@\language\name\%      (1) language?
4895     {\bbl@ifunset\bbl@##1dflt@*\bbl@tempa}%      (2) from script?
4896     {\bbl@ifunset\bbl@##1dflt@}%      2=F - (3) from generic?
4897     {}%      123=F - nothing!
4898     {\bbl@exp{%      3=T - from generic
4899       \global\let<\bbl@##1dflt@\language\name>%
4900       \<\bbl@##1dflt@>}}}%
4901     {\bbl@exp{%      2=T - from script
4902       \global\let<\bbl@##1dflt@\language\name>%
4903       \<\bbl@##1dflt@*\bbl@tempa>}}}%
4904     {}}%      1=T - language, already defined
4905   \def\bbl@tempa{\bbl@nostdfont{}}%
4906   \bbl@foreach\bbl@font@fams{%      don't gather with prev for
4907     \bbl@ifunset\bbl@##1dflt@\language\name\%
4908     {\bbl@cs{famrst@##1}%
4909     \global\bbl@csarg\let{famrst@##1}\relax}%
4910     {\bbl@exp{% order is relevant.
4911       \\bbl@add\\originalTeX{%
4912         \\bbl@font@rst{\bbl@cl{##1dflt}}%
4913         \<##1default>\<##1family>{##1}}}%
4914       \\bbl@font@set<\bbl@##1dflt@\language\name>% the main part!
4915       \<##1default>\<##1family>}}}%
4916   \bbl@ifrestoring{\bbl@tempa}}%

```

The following is executed at the beginning of the aux file or the document to warn about fonts not defined with \babelfont.

```

4917 \ifx\f@family\undefined\else      % if latex
4918 \ifcase\bbl@engine                  % if pdftex
4919   \let\bbl@cckstdfont\relax
4920 \else
4921   \def\bbl@cckstdfont{%
4922     \begingroup
4923     \global\let\bbl@cckstdfont\relax
4924     \let\bbl@tempa\empty
4925     \bbl@foreach\bbl@font@fams{%
4926       \bbl@ifunset\bbl@##1dflt@{%
4927         {\nameuse{##1family}}%
4928         \bbl@csarg\gdef{WFF@f@family}{}}% Flag
4929         \bbl@exp{\\bbl@add\\bbl@tempa{* \<##1family>= \f@family\\}%
4930         \space\space\fontname\font\\}%
4931         \bbl@csarg\xdef{##1dflt@}{\f@family}%
4932         \expandafter\xdef\csname ##1default\endcsname{\f@family}}%
4933       {}}%
4934   \ifx\bbl@tempa\empty\else
4935     \bbl@infowarn{The following font families will use the default\\%
4936     settings for all or some languages:\\%
4937     \bbl@tempa
4938     There is nothing intrinsically wrong with it, but\\%

```

```

4939         'babel' will no set Script and Language, which could\\%
4940         be relevant in some languages. If your document uses\\%
4941         these families, consider redefining them with \string\babelfont.\\%
4942         Reported}%
4943     \fi
4944 \endgroup}
4945 \fi
4946 \fi

```

Now the macros defining the font with fontspec.

When there are repeated keys in fontspec, the last value wins. So, we just place the ini settings at the beginning, and user settings will take precedence. We must deactivate temporarily `\bbl@mapselect` because `\selectfont` is called internally when a font is defined.

For historical reasons, $\TeX$ can select two different series (bx and b), for what is conceptually a single one. This can lead to problems when a single family requires several fonts, depending on the language, mainly because ‘substitutions’ with some combinations are not done consistently – sometimes bx/sc is the correct font, but sometimes points to b/n, even if b/sc exists. So, some substitutions are redefined (in a somewhat hackish way, by inspecting if the variant declaration contains `>ssub*`).

```

4947 \def\bbl@font@set#1#2#3{% e.g., \bbl@rmdflt@lang \rmdefault \rmfamily
4948 \bbl@xin@{<>}{#1}%
4949 \ifin@
4950 \bbl@exp{\\bbl@fontspec@set\\#1\expandafter\@gobbletwo#1\\#3}%
4951 \fi
4952 \bbl@exp{%
4953     \def\\#2{#1}% e.g., \rmdefault{\bbl@rmdflt@lang}
4954     \\bbl@ifsamestring{#2}{\f@family}%
4955     {\\#3%
4956     \\bbl@ifsamestring{\f@series}{\bfdefault}{\\bfseries}}}%
4957     \let\\bbl@tempa\relax}%
4958     {}%

```

Loaded locally, which does its job, but very must be global. The problem is how. This actually defines a font predeclared with `\babelfont`, making sure Script and Language names are defined. If they are not, the corresponding data in the ini file is used. The font is actually set temporarily to get the family name (`\f@family`). There is also a hack because by default some replacements related to the bold series are sometimes assigned to the wrong font (see issue #92).

```

4959 \def\bbl@fontspec@set#1#2#3#4{% eg \bbl@rmdflt@lang fnt-opt fnt-nme \xxfamily
4960 \let\bbl@tempa\bbl@mapselect
4961 \edef\bbl@tempb{\bbl@stripslash#4/}% Catcodes hack (better pass it).
4962 \bbl@exp{\\bbl@replace\\bbl@tempb{\bbl@stripslash\family/}{}}%
4963 \let\bbl@mapselect\relax
4964 \let\bbl@tempa@fam#4% e.g., '\rmfamily', to be restored below
4965 \let#4\@empty % Make sure \renewfontfamily is valid
4966 \bbl@set@renderer
4967 \bbl@exp{%
4968     \let\\bbl@tempa@pfam<\bbl@stripslash#4\space>% e.g., '\rmfamily '
4969     \<keys_if_exist:nnF>{fontspec-opentype}{Script/\bbl@cl{sname}}%
4970     {\\newfontscript{\bbl@cl{sname}}{\bbl@cl{sotf}}}%
4971     \<keys_if_exist:nnF>{fontspec-opentype}{Language/\bbl@cl{lname}}%
4972     {\\newfontlanguage{\bbl@cl{lname}}{\bbl@cl{lotf}}}%
4973     \\renewfontfamily\\#4%
4974     [\bbl@cl{lsys},% xetex removes unknown features :-(
4975     \ifcase\bbl@engine\or RawFeature={family=\bbl@tempb},\fi
4976     #2]}{#3}% i.e., \bbl@exp{..}{#3}
4977 \bbl@unset@renderer
4978 \begingroup
4979     #4%
4980     \xdef#1{\f@family}% e.g., \bbl@rmdflt@lang{FreeSerif(0)}
4981 \endgroup
4982 \bbl@xin@{\string>\string s\string s\string u\string b\string*}%
4983     {\expandafter\meaning\csname TU/#1/bx/sc\endcsname}%
4984 \ifin@
4985     \global\bbl@ccarg\let{TU/#1/bx/sc}{TU/#1/b/sc}%

```

```

4986 \fi
4987 \bbl@xin@{\string>\string s\string s\string u\string b\string*}%
4988 {\expandafter\meaning\csname TU/#1/bx/scit\endcsname}%
4989 \ifin@
4990 \global\bbl@ccarg\let{TU/#1/bx/scit}{TU/#1/b/scit}%
4991 \fi
4992 \let#4\bbl@temp@fam
4993 \bbl@exp{\let<\bbl@stripslash#4\space>}\bbl@temp@pfam
4994 \let\bbl@mapselect\bbl@tempe}%

```

font@rst and famrst are only used when there is no global settings, to save and restore de previous families. Not really necessary, but done for optimization.

```

4995 \def\bbl@font@rst#1#2#3#4{%
4996 \bbl@ccarg\def{famrst@#4}{\bbl@font@set{#1}#2#3}}

```

The default font families. They are eurocentric, but the list can be expanded easily with \babelfont.

```

4997 \def\bbl@font@fams{rm,sf,tt}
4998 <</Font selection>>

```

10. Hooks for XeTeX and LuaTeX

10.1. XeTeX

Unfortunately, the current encoding cannot be retrieved and therefore it is reset always to utf8, which seems a sensible default.

Now, the code.

```

4999 <*<xetex>
5000 \def\BabelStringsDefault{unicode}
5001 \let\xebbl@stop\relax
5002 \AddBabelHook{xetex}{encodedcommands}{%
5003 \def\bbl@tempa{#1}%
5004 \ifx\bbl@tempa\@empty
5005 \XeTeXinputencoding"bytes"%
5006 \else
5007 \XeTeXinputencoding"#1"%
5008 \fi
5009 \def\xebbl@stop{\XeTeXinputencoding"utf8"}}
5010 \AddBabelHook{xetex}{stopcommands}{%
5011 \xebbl@stop
5012 \let\xebbl@stop\relax}
5013 \def\bbl@input@classes{% Used in CJK intraspaces
5014 \input{load-unicode-xetex-classes.tex}%
5015 \let\bbl@input@classes\relax}
5016 \def\bbl@intraspace#1 #2 #3\@@{%
5017 \bbl@ccarg\gdef{xeisp@\languagename}%
5018 {\XeTeXlinebreakskip #1em plus #2em minus #3em\relax}}
5019 \def\bbl@intrapenalty#1\@@{%
5020 \bbl@ccarg\gdef{xeipn@\languagename}%
5021 {\XeTeXlinebreakpenalty #1\relax}}
5022 \def\bbl@provide@intraspace{%
5023 \bbl@xin@{/s}{/\bbl@cl{lnbrk}}}%
5024 \ifin@ \else \bbl@xin@{/c}{/\bbl@cl{lnbrk}} \fi
5025 \ifin@
5026 \bbl@ifunset{bbl@intsp@\languagename}{%
5027 {\expandafter\ifx\csname bbl@intsp@\languagename\endcsname\@empty\else
5028 \ifx\bbl@KVP@intraspace\@nnil
5029 \bbl@exp{%
5030 \\\bbl@intraspace\bbl@cl{intsp}\\\@@}%
5031 \fi
5032 \ifx\bbl@KVP@intrapenalty\@nnil
5033 \bbl@intrapenalty0\@@

```

```

5034 \fi
5035 \fi
5036 \ifx\bbbl@KVP@intraspace\@nnil\else % We may override the ini
5037 \expandafter\bbbl@intraspace\bbbl@KVP@intraspace\@@
5038 \fi
5039 \ifx\bbbl@KVP@intrapenalty\@nnil\else
5040 \expandafter\bbbl@intrapenalty\bbbl@KVP@intrapenalty\@@
5041 \fi
5042 \bbbl@exp{%
5043 \\\bbbl@add<extras\languagename>{%
5044 \XeTeXlinebreaklocale "\bbbl@cl{tbc}"%
5045 \<bbbl@xeisp@\languagename>%
5046 \<bbbl@xeipn@\languagename>%
5047 \\\bbbl@tglobal\<extras\languagename>%
5048 \\\bbbl@add<noextras\languagename>{%
5049 \XeTeXlinebreaklocale ""}%
5050 \\\bbbl@tglobal\<noextras\languagename>%
5051 \ifx\bbbl@ispace\@undefined
5052 \gdef\bbbl@ispace{\bbbl@cl{xeisp}}%
5053 \ifx\AtBeginDocument\@notprerr
5054 \expandafter\@secondoftwo % to execute right now
5055 \fi
5056 \AtBeginDocument{\bbbl@patchfont{\bbbl@ispace}}%
5057 \fi}%
5058 \fi}
5059 \ifx\DisableBabelHook\@undefined\endinput\fi
5060 \let\bbbl@set@renderer\relax
5061 \let\bbbl@unset@renderer\relax
5062 <@Font selection>
5063 \def\bbbl@provide@extra#1{}

Hack for unhyphenated line breaking. See \bbbl@provide@lsys in the common code.

5064 \def\bbbl@xenohyph@d{%
5065 \bbbl@ifset{bbbl@prehc@\languagename}%
5066 {\ifnum\hyphenchar\font=\defaultthyphenchar
5067 \iffontchar\font\bbbl@cl{prehc}\relax
5068 \hyphenchar\font\bbbl@cl{prehc}\relax
5069 \else\iffontchar\font"200B
5070 \hyphenchar\font"200B
5071 \else
5072 \bbbl@warning
5073 {Neither 0 nor ZERO WIDTH SPACE are available\\%
5074 in the current font, and therefore the hyphen\\%
5075 will be printed. Try changing the fontspec's\\%
5076 'HyphenChar' to another value, but be aware\\%
5077 this setting is not safe (see the manual).\\%
5078 Reported}%
5079 \hyphenchar\font\defaultthyphenchar
5080 \fi\fi
5081 \fi}%
5082 {\hyphenchar\font\defaultthyphenchar}}

```

10.2. Support for interchar

xetex reserves some values for CJK (although they are not set in XELATEX), so we make sure they are skipped. Define some user names for the global classes, too.

```

5083 \ifnum\Xe@alloc@intercharclass<\thr@@
5084 \Xe@alloc@intercharclass\thr@@
5085 \fi
5086 \chardef\bbbl@xe@class@default=\z@
5087 \chardef\bbbl@xe@class@cjkideogram=\@ne
5088 \chardef\bbbl@xe@class@cjkleftpunctuation=\tw@
5089 \chardef\bbbl@xe@class@cjkrightpunctuation=\thr@@

```

```

5090 \chardef\bbl@xeclasse@boundary@=4095
5091 \chardef\bbl@xeclasse@ignore@=4096

```

The machinery is activated with a hook (enabled only if actually used). Here \bbl@tempc is pre-set with \bbl@usingxeclasse, defined below. The standard mechanism based on \originalTeX to save, set and restore values is used. \count@ stores the previous char to be set, except at the beginning (0) and after \bbl@upto, which is the previous char negated, as a flag to mark a range.

```

5092 \AddBabelHook{babel-interchar}{beforeextras}{%
5093   \nameuse{bbl@xechars@\language@}}
5094 \DisableBabelHook{babel-interchar}
5095 \protected\def\bbl@charclass#1{%
5096   \ifnum\count@<\z@
5097     \count@-\count@
5098     \loop
5099       \bbl@exp{%
5100         \\babel@savevariable{\XeTeXcharclass`\Uchar\count@}}%
5101         \XeTeXcharclass\count@ \bbl@tempc
5102         \ifnum\count@<`#1\relax
5103           \advance\count@\@ne
5104         \repeat
5105   \else
5106     \babel@savevariable{\XeTeXcharclass`#1}%
5107     \XeTeXcharclass`#1 \bbl@tempc
5108   \fi
5109   \count@`#1\relax}

```

Now the two user macros. Char classes are declared implicitly, and then the macro to be executed at the babel-interchar hook is created. The list of chars to be handled by the hook defined above has internally the form \bbl@usingxeclasse\bbl@xeclasse@punct@english\bbl@charclass{.} \bbl@charclass{,} (etc.), where \bbl@usingxeclasse stores the class to be applied to the subsequent characters. The \ifcat part deals with the alternative way to enter characters as macros (e.g., \). As a special case, hyphens are stored as \bbl@upto, to deal with ranges.

```

5110 \newcommand\bbl@ifinterchar[1]{%
5111   \let\bbl@tempa\@gobble           % Assume to ignore
5112   \edef\bbl@tempb{\zap@space#1 \@empty}%
5113   \ifx\bbl@KVP@interchar\@nnil\else
5114     \bbl@replace\bbl@KVP@interchar{ }{,}%
5115     \bbl@foreach\bbl@tempb{%
5116       \bbl@xin@{,##1,}{, \bbl@KVP@interchar,}%
5117       \ifin@
5118         \let\bbl@tempa\@firstofone
5119       \fi}%
5120   \fi
5121   \bbl@tempa}
5122 \newcommand\IfBabelIntercharT[2]{%
5123   \bbl@carg\bbl@add{\bbl@icsave@CurrentOption}{\bbl@ifinterchar{#1}{#2}}}%
5124 \newcommand\babelcharclass[3]{%
5125   \EnableBabelHook{babel-interchar}%
5126   \bbl@csarg\newXeTeXintercharclass{xeclasse@#2@#1}%
5127   \def\bbl@tempb##1{%
5128     \ifx##1\@empty\else
5129       \ifx##1-%
5130         \bbl@upto
5131       \else
5132         \bbl@charclass{%
5133           \ifcat\noexpand##1\relax\bbl@stripslash##1\else\string##1\fi}%
5134         \fi
5135       \expandafter\bbl@tempb
5136     \fi}%
5137   \bbl@ifunset{bbl@xechars@#1}%
5138   {\toks@{%
5139     \babel@savevariable\XeTeXinterchartokenstate
5140     \XeTeXinterchartokenstate\@ne
5141   }}%

```

```

5142   {\toks@ \expandafter \expandafter \expandafter {%
5143     \csname bbl@xechars@#1\endcsname}}}%
5144   \bbl@csarg \edef {xechars@#1}{%
5145     \the \toks@
5146     \bbl@usingxe@class \csname bbl@xe@class@#2@#1\endcsname
5147     \bbl@tempb#3 \@empty}}
5148   \protected \def \bbl@usingxe@class#1 {\count@ \z@ \let \bbl@tempc#1}
5149   \protected \def \bbl@upto {%
5150     \ifnum \count@ > \z@
5151       \advance \count@ \@ne
5152       \count@ - \count@
5153     \else \ifnum \count@ = \z@
5154       \bbl@charclass {-}%
5155     \else
5156       \bbl@error {double-hyphens-class} {} {} {}%
5157     \fi \fi}

```

And finally, the command with the code to be inserted. If the language doesn't define a class, then use the global one, as defined above. For the definition there is a intermediate macro, which can be 'disabled' with `\bbl@ic@<label>@<language>`.

```

5158 \def \bbl@ignoreinterchar {%
5159   \ifnum \language = \l@nohyphenation
5160     \expandafter \@gobble
5161   \else
5162     \expandafter \@firstofone
5163   \fi}
5164 \newcommand \babelinterchar [5] {} {%
5165   \let \bbl@kv@label \@empty
5166   \bbl@forkv {#1} {\bbl@csarg \edef {kv@##1} {##2}}}%
5167   \namedef {\zap@space bbl@xeinter@ \bbl@kv@label @#3@#4@#2 \@empty}%
5168   {\bbl@ignoreinterchar {#5}}}%
5169   \bbl@csarg \let {ic@ \bbl@kv@label @#2} \@firstofone
5170   \bbl@exp {\bbl@for {\bbl@tempa {\zap@space#3 \@empty}}}%
5171   \bbl@exp {\bbl@for {\bbl@tempb {\zap@space#4 \@empty}}}%
5172   \XeTeXinterchartoks
5173     \@nameuse {bbl@xe@class@ \bbl@tempa @%
5174       \bbl@ifunset {bbl@xe@class@ \bbl@tempa @#2} {} {#2}} %
5175     \@nameuse {bbl@xe@class@ \bbl@tempb @%
5176       \bbl@ifunset {bbl@xe@class@ \bbl@tempb @#2} {} {#2}} %
5177     = \expandafter {%
5178       \csname bbl@ic@ \bbl@kv@label @#2 \expandafter \endcsname
5179       \csname \zap@space bbl@xeinter@ \bbl@kv@label
5180         @#3@#4@#2 \@empty \endcsname} {} {} {}
5181 \DeclareRobustCommand \enablelocaleinterchar [1] {%
5182   \bbl@ifunset {bbl@ic@#1@ \language name}%
5183   {\bbl@error {unknown-interchar} {#1} {} {}}%
5184   {\bbl@csarg \let {ic@#1@ \language name} \@firstofone}}
5185 \DeclareRobustCommand \disablelocaleinterchar [1] {%
5186   \bbl@ifunset {bbl@ic@#1@ \language name}%
5187   {\bbl@error {unknown-interchar-b} {#1} {} {}}%
5188   {\bbl@csarg \let {ic@#1@ \language name} \@gobble}}
5189 </xetex>

```

10.3. Layout

Note elements like headlines and margins can be modified easily with packages like fancyhdr, typearea or titleps, and geometry.

`\bbl@startskip` and `\bbl@endskip` are available to package authors. Thanks to the $\TeX$ expansion mechanism the following constructs are valid: `\adim\bbl@startskip`, `\advance\bbl@startskip\adim`, `\bbl@startskip\adim`.

Consider `txtbabel` as a shorthand for *tex-xet babel*, which is the bidi model in both pdfTeX and xetex.

```

5190 < *xetex | texxet>

```



```

5191 \providecommand\bbl@provide@intraspace{}
5192 \bbl@trace{Redefinitions for bidi layout}

    Finish here if there in no layout.

5193 \ifx\bbl@opt@layout\@nnil\else % if layout=..
5194 \IfBabelLayout{nopars}
5195 {}
5196 {\edef\bbl@opt@layout{\bbl@opt@layout.pars.}}%
5197 \def\bbl@startskip{\ifcase\bbl@thepardir\leftskip\else\rightskip\fi}
5198 \def\bbl@endskip{\ifcase\bbl@thepardir\rightskip\else\leftskip\fi}
5199 \ifnum\bbl@bidimode>\z@
5200 \IfBabelLayout{pars}
5201 {\def\@hangfrom#1{%
5202   \setbox\@tempboxa\hbox{#1}%
5203   \hangindent\ifcase\bbl@thepardir\wd\@tempboxa\else-\wd\@tempboxa\fi
5204   \noindent\box\@tempboxa}
5205 \def\raggedright{%
5206   \let\\\@centercr
5207   \bbl@startskip\z@skip
5208   \@rightskip\@flushglue
5209   \bbl@endskip\@rightskip
5210   \parindent\z@
5211   \parfillskip\bbl@startskip}
5212 \def\raggedleft{%
5213   \let\\\@centercr
5214   \bbl@startskip\@flushglue
5215   \bbl@endskip\z@skip
5216   \parindent\z@
5217   \parfillskip\bbl@endskip}}
5218 {}
5219 \fi
5220 \IfBabelLayout{lists}
5221 {\bbl@sreplace\list
5222   {\@totalleftmargin\leftmargin}{\@totalleftmargin\bbl@listleftmargin}%
5223   \def\bbl@listleftmargin{%
5224     \ifcase\bbl@thepardir\leftmargin\else\rightmargin\fi}%
5225   \ifcase\bbl@engine
5226     \def\labelenumii{}\theenumii{}\pdfTeX doesn't reverse ()
5227     \def\p@enumiii{\p@enumii}\theenumii{}\p@enumii{}
5228   \fi
5229   \bbl@sreplace\@verbatim
5230     {\leftskip\@totalleftmargin}%
5231     {\bbl@startskip\textwidth
5232       \advance\bbl@startskip-\linewidth}%
5233   \bbl@sreplace\@verbatim
5234     {\rightskip\z@skip}%
5235     {\bbl@endskip\z@skip}}%
5236 {}
5237 \IfBabelLayout{contents}
5238 {\bbl@sreplace\@dottedtocline{\leftskip}{\bbl@startskip}%
5239   \bbl@sreplace\@dottedtocline{\rightskip}{\bbl@endskip}}
5240 {}
5241 \IfBabelLayout{columns}
5242 {\bbl@sreplace\@outputdblcol{\hb@xt@\textwidth}{\bbl@outputbox}%
5243   \def\bbl@outputbox#1{%
5244     \hb@xt@\textwidth{%
5245       \hskip\columnwidth
5246       \hfil
5247       {\normalcolor\vrule \@width\columnseprule}%
5248       \hfil
5249       \hb@xt@\columnwidth{\box\@leftcolumn \hss}%
5250       \hskip-\textwidth
5251       \hb@xt@\columnwidth{\box\@outputbox \hss}%

```

```

5252      \hskip\columnsep
5253      \hskip\columnwidth}}}%
5254  {}

```

Implicitly reverses sectioning labels in bidi=basic, because the full stop is not in contact with L numbers any more. I think there must be a better way.

```

5255 \IfBabelLayout{counters*}%
5256 {\bbl@add\bbl@opt@layout{.counters.}%
5257  \AddToHook{shipout/before}{%
5258   \let\bbl@tempa\babelsublr
5259   \let\babelsublr\@firstofone
5260   \let\bbl@save@thepage\thepage
5261   \protected@edef\thepage{\thepage}%
5262   \let\babelsublr\bbl@tempa}%
5263  \AddToHook{shipout/after}{%
5264   \let\thepage\bbl@save@thepage}}{}
5265 \IfBabelLayout{counters}%
5266 {\let\bbl@latinarabic=\@arabic
5267  \def\@arabic#1{\babelsublr{\bbl@latinarabic#1}}%
5268  \let\bbl@asciroman=\@roman
5269  \def\@roman#1{\babelsublr{\ensureascii{\bbl@asciroman#1}}}%
5270  \let\bbl@asciiRoman=\@Roman
5271  \def\@Roman#1{\babelsublr{\ensureascii{\bbl@asciiRoman#1}}}}{}
5272 \fi % end if layout
5273 </xetex | texxet>

```

10.4. 8-bit TeX

Which start just above, because some code is shared with xetex. Now, 8-bit specific stuff. If just one encoding has been declared, then assume no switching is necessary (1).

```

5274 <*texxet>
5275 \def\bbl@provide@extra#1{%
5276  % == auto-select encoding ==
5277  \ifx\bbl@encoding@select@off\@empty\else
5278   \bbl@ifunset{\bbl@encoding@#1}%
5279   {\def\@elt##1{,##1,}%
5280    \edef\bbl@tempe{\expandafter\@gobbletwo\@fontenc@load@list}%
5281    \count@\z@
5282    \bbl@foreach\bbl@tempe{%
5283     \def\bbl@tempd{##1}% Save last declared
5284     \advance\count@\@ne}%
5285    \ifnum\count@>\@ne % (1)
5286     \getlocaleproperty*\bbl@tempa{#1}{identification/encodings}%
5287     \ifx\bbl@tempa\relax \let\bbl@tempa\@empty \fi
5288     \bbl@replace\bbl@tempa{ }{,}%
5289     \global\bbl@csarg\let{encoding@#1}\@empty
5290     \bbl@xin@{,\bbl@tempd,}{,\bbl@tempa,}%
5291     \ifin@else % if main encoding included in ini, do nothing
5292     \let\bbl@tempb\relax
5293     \bbl@foreach\bbl@tempa{%
5294      \ifx\bbl@tempb\relax
5295       \bbl@xin@{,##1,}{,\bbl@tempe,}%
5296       \ifin@\def\bbl@tempb{##1}\fi
5297      \fi}%
5298     \ifx\bbl@tempb\relax\else
5299     \bbl@exp{%
5300      \global\<\bbl@add\<\bbl@preextras@#1>\<\bbl@encoding@#1>}%
5301      \gdef\<\bbl@encoding@#1>{%
5302       \\babel@save\\f@encoding
5303       \\bbl@add\\originalTeX{\\selectfont}%
5304       \\fontencoding{\bbl@tempb}%
5305       \\selectfont}}}%
5306  \fi

```

```

5307         \fi
5308     \fi}%
5309 {}%
5310 \fi}
5311 </texet>

```

10.5. LuaTeX

The loader for luatex is based solely on `language.dat`, which is read on the fly. The code shouldn't be executed when the format is build, so we check if `\AddBabelHook` is defined. Then comes a modified version of the loader in `hyphen.cfg` (without the `hyphenmins` stuff, which is under the direct control of `babel`).

The names `\l@<language>` are defined and take some value from the beginning because all `ldf` files assume this for the corresponding language to be considered valid, but patterns are not loaded (except the first one). This is done later, when the language is first selected (which usually means when the `ldf` finishes). If a language has been loaded, `\bbl@hyphendata@<num>` exists (with the names of the files read).

The default setup preloads the first language into the format. This is intended mainly for 'english', so that it's available without further intervention from the user. To avoid duplicating it, the following rule applies: if the "0th" language and the first language in `language.dat` have the same name then just ignore the latter. If there are new synonymous, they are added, but note if the language patterns have not been preloaded they won't at run time.

Other preloaded languages could be read twice, if they have been preloaded into the format. This is not optimal, but it shouldn't happen very often – with luatex patterns are best loaded when the document is typeset, and the "0th" language is preloaded just for backwards compatibility.

As of 1.1b, lua(e)tex is taken into account. Formerly, loading of patterns on the fly didn't work in this format, but with the new loader it does. Unfortunately, the format is not based on `babel`, and data could be duplicated, because languages are reassigned above those in the format (nothing serious, anyway). Note even with this format `language.dat` is used (under the principle of a single source), instead of `language.def`.

Of course, there is room for improvements, like tools to read and reassign languages, which would require modifying the language list, and better error handling.

We need catcode tables, but no format (targeted by `babel`) provide a command to allocate them (although there are packages like `ctablestack`). FIX - This isn't true anymore. For the moment, a dangerous approach is used - just allocate a high random number and cross the fingers. To complicate things, `etex.sty` changes the way languages are allocated.

This files is read at three places: (1) when `plain.def`, `babel.sty` starts, to read the list of available languages from `language.dat` (for the base option); (2) at `hyphen.cfg`, to modify some macros; (3) in the middle of `plain.def` and `babel.sty`, by `babel.def`, with the commands and other definitions for luatex (e.g., `\babelpatterns`).

```

5312 <!*luatex>
5313 \directlua{ Babel = Babel or {} } % DL2
5314 \ifx\AddBabelHook\undefined % When plain.def, babel.sty starts
5315 \bbl@trace{Read language.dat}
5316 \ifx\bbl@readstream\undefined
5317   \csname newread\endcsname\bbl@readstream
5318 \fi
5319 \begingroup
5320   \toks@{}
5321   \count@ \z@ % 0=start, 1=0th, 2=normal
5322   \def\bbl@process@line#1#2 #3 #4 {%
5323     \ifx=#1%
5324       \bbl@process@synonym{#2}%
5325     \else
5326       \bbl@process@language{#1#2}{#3}{#4}%
5327     \fi
5328     \ignorespaces}
5329   \def\bbl@manylang{%
5330     \ifnum\bbl@last>\@ne
5331       \bbl@info{Non-standard hyphenation setup}%
5332     \fi
5333     \let\bbl@manylang\relax}
5334   \def\bbl@process@language#1#2#3{%

```

```

5335 \ifcase\count@
5336 \ifundefined{zth@#1}{\count@tw@}{\count@ne}%
5337 \or
5338 \count@tw@
5339 \fi
5340 \ifnum\count@=tw@
5341 \expandafter\addlanguage\csname l@#1\endcsname
5342 \language\allocationnumber
5343 \chardef\bbl@last\allocationnumber
5344 \bbl@manylang
5345 \let\bbl@elt\relax
5346 \xdef\bbl@languages{%
5347 \bbl@languages\bbl@elt{#1}{\the\language}{#2}{#3}}%
5348 \fi
5349 \the\toks@
5350 \toks@{}}
5351 \def\bbl@process@synonym@aux#1#2{%
5352 \global\expandafter\chardef\csname l@#1\endcsname#2\relax
5353 \let\bbl@elt\relax
5354 \xdef\bbl@languages{%
5355 \bbl@languages\bbl@elt{#1}{#2}{}}}%
5356 \def\bbl@process@synonym#1{%
5357 \ifcase\count@
5358 \toks@\expandafter{\the\toks@\relax\bbl@process@synonym{#1}}%
5359 \or
5360 \ifundefined{zth@#1}{\bbl@process@synonym@aux{#1}{0}}}%
5361 \else
5362 \bbl@process@synonym@aux{#1}{\the\bbl@last}%
5363 \fi}
5364 \ifx\bbl@languages\undefined % Just a (sensible?) guess
5365 \chardef\l@english\z@
5366 \chardef\l@USenglish\z@
5367 \chardef\bbl@last\z@
5368 \global\@namedef{bbl@hyphendata@0}{{hyphen.tex}}
5369 \gdef\bbl@languages{%
5370 \bbl@elt{english}{0}{hyphen.tex}}%
5371 \bbl@elt{USenglish}{0}{}}
5372 \else
5373 \global\let\bbl@languages@format\bbl@languages
5374 \def\bbl@elt#1#2#3#4{% Remove all except language 0
5375 \ifnum#2>\z@else
5376 \noexpand\bbl@elt{#1}{#2}{#3}{#4}%
5377 \fi}%
5378 \xdef\bbl@languages{\bbl@languages}%
5379 \fi
5380 \def\bbl@elt#1#2#3#4{\@namedef{zth@#1}{}} % Define flags
5381 \bbl@languages
5382 \openin\bbl@readstream=language.dat
5383 \ifeof\bbl@readstream
5384 \bbl@warning{I couldn't find language.dat. No additional\\%
5385 patterns loaded. Reported}%
5386 \else
5387 \loop
5388 \endlinechar\m@ne
5389 \read\bbl@readstream to \bbl@line
5390 \endlinechar`^^M
5391 \if T\ifeof\bbl@readstream F\fi T\relax
5392 \ifx\bbl@line\empty\else
5393 \edef\bbl@line{\bbl@line\space\space\space}%
5394 \expandafter\bbl@process@line\bbl@line\relax
5395 \fi
5396 \repeat
5397 \fi

```

```

5398 \closein\bbl@readstream
5399 \endgroup
5400 \bbl@trace{Macros for reading patterns files}
5401 \def\bbl@get@enc#1:#2:#3\@@{\def\bbl@hyph@enc{#2}}
5402 \ifx\babelcatcodetablenum\undefined
5403 \ifx\newcatcodetable\undefined
5404 \def\babelcatcodetablenum{5211}
5405 \def\bbl@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5406 \else
5407 \newcatcodetable\babelcatcodetablenum
5408 \newcatcodetable\bbl@pattcodes
5409 \fi
5410 \else
5411 \def\bbl@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5412 \fi
5413 \def\bbl@luapatterns#1#2{%
5414 \bbl@get@enc#1:.\@@@
5415 \setbox\z@\hbox\bgroup
5416 \beginingroup
5417 \savecatcodetable\babelcatcodetablenum\relax
5418 \initcatcodetable\bbl@pattcodes\relax
5419 \catcodetable\bbl@pattcodes\relax
5420 \catcode`\#=6 \catcode`\$=3 \catcode`\&=4 \catcode`\^=7
5421 \catcode`\_ =8 \catcode`\{=1 \catcode`\}=2 \catcode`\~=13
5422 \catcode`\@=11 \catcode`\^^I=10 \catcode`\^^J=12
5423 \catcode`\<=12 \catcode`\>=12 \catcode`\*=12 \catcode`\.=12
5424 \catcode`\-=12 \catcode`\/=12 \catcode`\[=12 \catcode`\]=12
5425 \catcode`\`=12 \catcode`\'=12 \catcode`\`=12
5426 \input #1\relax
5427 \catcodetable\babelcatcodetablenum\relax
5428 \endgroup
5429 \def\bbl@tempa{#2}%
5430 \ifx\bbl@tempa\empty\else
5431 \input #2\relax
5432 \fi
5433 \egroup}%
5434 \def\bbl@patterns@lua#1{%
5435 \language=\expandafter\ifx\csname l@#1:\f@encoding\endcsname\relax
5436 \csname l@#1\endcsname
5437 \edef\bbl@tempa{#1}%
5438 \else
5439 \csname l@#1:\f@encoding\endcsname
5440 \edef\bbl@tempa{#1:\f@encoding}%
5441 \fi\relax
5442 \@namedef{lu@texhyphen@loaded@the\language}{}% Temp
5443 \@ifundefined{bbl@hyphendata@the\language}%
5444 {\def\bbl@elt##1##2##3##4{%
5445 \ifnum##2=\csname l@bbl@tempa\endcsname % #2=spanish, dutch:OT1...
5446 \def\bbl@tempb{##3}%
5447 \ifx\bbl@tempb\empty\else % if not a synonymous
5448 \def\bbl@tempc{##3}{##4}%
5449 \fi
5450 \bbl@csarg\xdef{hyphendata@##2}{\bbl@tempc}%
5451 \fi}%
5452 \bbl@languages
5453 \@ifundefined{bbl@hyphendata@the\language}%
5454 {\bbl@info{No hyphenation patterns were set for\%
5455 language '\bbl@tempa'. Reported}}%
5456 {\expandafter\expandafter\expandafter\bbl@luapatterns
5457 \csname bbl@hyphendata@the\language\endcsname}}}%
5458 \endinput\fi

```

Here ends \ifx\AddBabelHook\undefined. A few lines are only read by HYPHEN.CFG.

```

5459 \ifx\DisableBabelHook\@undefined
5460 \AddBabelHook{luatex}{everylanguage}{%
5461   \def\process@language##1##2##3{%
5462     \def\process@line####1####2 ####3 ####4 {}}
5463 \AddBabelHook{luatex}{loadpatterns}{%
5464   \input #1\relax
5465   \expandafter\gdef\csname bbl@hyphendata@the\language\endcsname
5466     {{#1}}}}
5467 \AddBabelHook{luatex}{loadexceptions}{%
5468   \input #1\relax
5469   \def\bbl@tempb##1##2{{##1}{##2}}
5470   \expandafter\xdef\csname bbl@hyphendata@the\language\endcsname
5471     {\expandafter\expandafter\expandafter\bbl@tempb
5472       \csname bbl@hyphendata@the\language\endcsname}}
5473 \endinput\fi

```

Here stops reading code for HYPHEN.CFG. The following is read the 2nd time it's loaded. First, global declarations for lua.

```

5474 \begingroup
5475 \catcode`\%=12
5476 \catcode`\'=12
5477 \catcode`\ "=12
5478 \catcode`\:=12
5479 \directlua{
5480   Babel.locale_props = Babel.locale_props or {}
5481   function Babel.lua_error(e, a)
5482     tex.print([[noexpand\csname bbl@error\endcsname{]] ..
5483       e .. '}' .. (a or '') .. '}'})
5484   end
5485
5486   function Babel.bytes(line)
5487     return line:gsub(".",
5488       function (chr) return unicode.utf8.char(string.byte(chr)) end)
5489   end
5490
5491   function Babel.priority_in_callback(name,description)
5492     for i,v in ipairs(luatexbase.callback_descriptions(name)) do
5493       if v == description then return i end
5494     end
5495     return false
5496   end
5497
5498   function Babel.begin_process_input()
5499     if luatexbase and luatexbase.add_to_callback then
5500       luatexbase.add_to_callback('process_input_buffer',
5501         Babel.bytes, 'Babel.bytes')
5502     else
5503       Babel.callback = callback.find('process_input_buffer')
5504       callback.register('process_input_buffer', Babel.bytes)
5505     end
5506   end
5507   function Babel.end_process_input ()
5508     if luatexbase and luatexbase.remove_from_callback then
5509       luatexbase.remove_from_callback('process_input_buffer', 'Babel.bytes')
5510     else
5511       callback.register('process_input_buffer', Babel.callback)
5512     end
5513   end
5514
5515   function Babel.str_to_nodes(fn, matches, base)
5516     local n, head, last
5517     if fn == nil then return nil end
5518     for s in string.utfvalues(fn(matches)) do

```

```

5519     if base.id == 7 then
5520         base = base.replace
5521     end
5522     n = node.copy(base)
5523     n.char = s
5524     if not head then
5525         head = n
5526     else
5527         last.next = n
5528     end
5529     last = n
5530 end
5531 return head
5532 end
5533
5534 Babel.linebreaking = Babel.linebreaking or {}
5535 Babel.linebreaking.before = {}
5536 Babel.linebreaking.after = {}
5537 Babel.locale = {}
5538 function Babel.linebreaking.add_before(func, pos)
5539     tex.print([[noexpand\csname bbl@luahyphenate\endcsname]])
5540     if pos == nil then
5541         table.insert(Babel.linebreaking.before, func)
5542     else
5543         table.insert(Babel.linebreaking.before, pos, func)
5544     end
5545 end
5546 function Babel.linebreaking.add_after(func)
5547     tex.print([[noexpand\csname bbl@luahyphenate\endcsname]])
5548     table.insert(Babel.linebreaking.after, func)
5549 end
5550
5551 function Babel.addpatterns(pp, lg)
5552     local lg = lang.new(lg)
5553     local pats = lang.patterns(lg) or ''
5554     lang.clear_patterns(lg)
5555     for p in pp:gmatch('[^%s]+') do
5556         ss = ''
5557         for i in string.utfcharacters(p:gsub('%d', '')) do
5558             ss = ss .. '%d?' .. i
5559         end
5560         ss = ss:gsub('^%d%?%.', '%%.') .. '%d?'
5561         ss = ss:gsub('%.%d%?$', '%%.')
5562         pats, n = pats:gsub('%s' .. ss .. '%s', ' ' .. p .. ' ')
5563         if n == 0 then
5564             tex.sprint(
5565                 [[\string\csname\space bbl@info\endcsname{New pattern: }]]
5566                 .. p .. [[{}]])
5567             pats = pats .. ' ' .. p
5568         else
5569             tex.sprint(
5570                 [[\string\csname\space bbl@info\endcsname{Renew pattern: }]]
5571                 .. p .. [[{}]])
5572         end
5573     end
5574     lang.patterns(lg, pats)
5575 end
5576
5577 Babel.characters = Babel.characters or {}
5578 Babel.ranges = Babel.ranges or {}
5579 function Babel.hlist_has_bidi(head)
5580     local has_bidi = false
5581     local ranges = Babel.ranges

```

```

5582     for item in node.traverse(head) do
5583         if item.id == node.id'glyph' then
5584             local itemchar = item.char
5585             local chardata = Babel.characters[itemchar]
5586             local dir = chardata and chardata.d or nil
5587             if not dir then
5588                 for nn, et in ipairs(ranges) do
5589                     if itemchar < et[1] then
5590                         break
5591                     elseif itemchar <= et[2] then
5592                         dir = et[3]
5593                         break
5594                     end
5595                 end
5596             end
5597             if dir and (dir == 'al' or dir == 'r') then
5598                 has_bidi = true
5599             end
5600         end
5601     end
5602     return has_bidi
5603 end
5604 function Babel.set_chranges_b (script, chrng)
5605     if chrng == '' then return end
5606     texio.write('Replacing ' .. script .. ' script ranges')
5607     Babel.script_blocks[script] = {}
5608     for s, e in string.gmatch(chrng..' ', '(.-%.%.(-)%s') do
5609         table.insert(
5610             Babel.script_blocks[script], {tonumber(s,16), tonumber(e,16)})
5611     end
5612 end
5613
5614 function Babel.discard_sublr(str)
5615     if str:find( [[\string\indexentry]] ) and
5616        str:find( [[\string\babelsublr]] ) then
5617         str = str:gsub( [[\string\babelsublr%s*{%b{}}]],
5618                        function(m) return m:sub(2,-2) end )
5619     end
5620     return str
5621 end
5622 }
5623 \endgroup
5624 \ifx\newattribute\undefined\else % Test for plain
5625 \newattribute\bbl@attr@locale % DL4
5626 \directlua{ Babel.attr_locale = luatexbase.registernumber'bbl@attr@locale' }
5627 \AddBabelHook{luatex}{beforeextras}{%
5628     \setattribute\bbl@attr@locale\localeid}
5629 \fi
5630 %
5631 \def\BabelStringsDefault{unicode}
5632 \let\luabbl@stop\relax
5633 \AddBabelHook{luatex}{encodedcommands}{%
5634     \def\bbl@tempa{utf8}\def\bbl@tempb{#1}%
5635     \ifx\bbl@tempa\bbl@tempb\else
5636         \directlua{Babel.begin_process_input()}%
5637         \def\luabbl@stop{%
5638             \directlua{Babel.end_process_input()}}%
5639     \fi}%
5640 \AddBabelHook{luatex}{stopcommands}{%
5641     \luabbl@stop
5642     \let\luabbl@stop\relax}
5643 %
5644 \AddBabelHook{luatex}{patterns}{%

```



```

5645 \ifundefined{bbl@hyphendata@the\language}%
5646 {\def\bbl@elt##1##2##3##4{%
5647   \ifnum##2=\csname l@#2\endcsname % #2=spanish, dutch:OT1...
5648   \def\bbl@tempb{##3}%
5649   \ifx\bbl@tempb@empty\else % if not a synonymous
5650     \def\bbl@tempc{##3}{##4}%
5651   \fi
5652   \bbl@csarg\xdef{hyphendata@##2}{\bbl@tempc}%
5653   \fi}%
5654 \bbl@languages
5655 \ifundefined{bbl@hyphendata@the\language}%
5656 {\bbl@info{No hyphenation patterns were set for\%
5657   language '#2'. Reported}}%
5658 {\expandafter\expandafter\expandafter\bbl@luapatterns
5659   \csname bbl@hyphendata@the\language\endcsname}}}%
5660 \ifundefined{bbl@patterns@}{}%
5661 \begingroup
5662   \bbl@xin{,\number\language,}{,\bbl@pttnlist}%
5663   \ifin@else
5664     \ifx\bbl@patterns@empty\else
5665       \directlua{ Babel.addpatterns(
5666         [[\bbl@patterns@]], \number\language) }%
5667     \fi
5668     \ifundefined{bbl@patterns@#1}%
5669       \@empty
5670       {\directlua{ Babel.addpatterns(
5671         [[\space\csname bbl@patterns@#1\endcsname]],
5672         \number\language) }}%
5673     \xdef\bbl@pttnlist{\bbl@pttnlist\number\language,}%
5674   \fi
5675 \endgroup}%
5676 \bbl@exp{%
5677   \bbl@ifunset{bbl@prehc@language\name}}}%
5678   {\bbl@ifblank{\bbl@cs{prehc@language\name}}}%
5679   {\prehyphenchar=\bbl@cl{prehc}\relax}}}%

```

\babelpatterns This macro adds patterns. Two macros are used to store them: \bbl@patterns@ for the global ones and \bbl@patterns@*(language)* for language ones. We make sure there is a space between words when multiple commands are used.

```

5680 \onlypreamble\babelpatterns
5681 \AtEndOfPackage{%
5682   \newcommand\babelpatterns[2][\@empty]{%
5683     \ifx\bbl@patterns@relax
5684       \let\bbl@patterns@empty
5685     \fi
5686     \ifx\bbl@pttnlist@empty\else
5687       \bbl@warning{%
5688         You must not intermingle \string\selectlanguage\space and\%
5689         \string\babelpatterns\space or some patterns will not\%
5690         be taken into account. Reported}%
5691       \fi
5692       \ifx\@empty#1%
5693         \protected@edef\bbl@patterns@{\bbl@patterns@\space#2}%
5694       \else
5695         \edef\bbl@tempb{\zap@space#1 \@empty}%
5696         \bbl@for\bbl@tempa\bbl@tempb{%
5697           \bbl@fixname\bbl@tempa
5698           \bbl@iflanguage\bbl@tempa{%
5699             \bbl@csarg\protected@edef{patterns@\bbl@tempa}{%
5700               \ifundefined{bbl@patterns@\bbl@tempa}%
5701                 \@empty
5702                 {\csname bbl@patterns@\bbl@tempa\endcsname\space}%
5703               #2}}}%

```

5704 \fi}}

10.6. Southeast Asian scripts

First, some general code for line breaking, used by \babelposthyphenation.

Replace regular (i.e., implicit) discretionaries by spaceskips, based on the previous glyph (which I think makes sense, because the hyphen and the previous char go always together). Other discretionaries are not touched. See Unicode UAX 14.

```
5705 \def\bbl@intraspace#1 #2 #3\@{%
5706   \directlua{
5707     Babel.intraspaces = Babel.intraspaces or {}
5708     Babel.intraspaces['\csname bbl@sbc@language\endcsname'] = %
5709       {b = #1, p = #2, m = #3}
5710     Babel.locale_props[\the\localeid].intraspace = %
5711       {b = #1, p = #2, m = #3}
5712   }}
5713 \def\bbl@intrapenalty#1\@{%
5714   \directlua{
5715     Babel.intrapenalties = Babel.intrapenalties or {}
5716     Babel.intrapenalties['\csname bbl@sbc@language\endcsname'] = #1
5717     Babel.locale_props[\the\localeid].intrapenalty = #1
5718   }}
5719 \begingroup
5720 \catcode`\%=12
5721 \catcode`\&=14
5722 \catcode`\'=12
5723 \catcode`\~=12
5724 \gdef\bbl@seaintraspace&
5725   \let\bbl@seaintraspace\relax
5726   \directlua{
5727     Babel.sea_enabled = true
5728     Babel.sea_ranges = Babel.sea_ranges or {}
5729     function Babel.set_chranges (script, chrng)
5730       local c = 0
5731       for s, e in string.gmatch(chrng..' ', '(.-)%.(.-)%s') do
5732         Babel.sea_ranges[script..c]={tonumber(s,16), tonumber(e,16)}
5733         c = c + 1
5734       end
5735     end
5736     function Babel.sea_disc_to_space (head)
5737       local sea_ranges = Babel.sea_ranges
5738       local last_char = nil
5739       local quad = 655360      &% 10 pt = 655360 = 10 * 65536
5740       for item in node.traverse(head) do
5741         local i = item.id
5742         if i == node.id'glyph' then
5743           last_char = item
5744         elseif i == 7 and item.subtype == 3 and last_char
5745           and last_char.char > 0x0C99 then
5746           quad = font.getfont(last_char.font).size
5747           for lg, rg in pairs(sea_ranges) do
5748             if last_char.char > rg[1] and last_char.char < rg[2] then
5749               lg = lg:sub(1, 4) &% Remove trailing number of, e.g., Cyril
5750               local intraspace = Babel.intraspaces[lg]
5751               local intrapenalty = Babel.intrapenalties[lg]
5752               local n
5753               if intrapenalty ~= 0 then
5754                 n = node.new(14, 0)      &% penalty
5755                 n.penalty = intrapenalty
5756                 node.insert_before(head, item, n)
5757               end
5758               n = node.new(12, 13)      &% (glue, spaceskip)
5759               node.setglue(n, intraspace.b * quad,
```

```

5760             intraspace.p * quad,
5761             intraspace.m * quad)
5762         node.insert_before(head, item, n)
5763         node.remove(head, item)
5764     end
5765 end
5766 end
5767 end
5768 end
5769 }&
5770 \bbl@luahyphenate}

```

10.7. CJK line breaking

Minimal line breaking for CJK scripts, mainly intended for simple documents and short texts as a secondary language. Only line breaking, with a little stretching for justification, without any attempt to adjust the spacing. It is based on (but does not strictly follow) the Unicode algorithm.

We first need a little table with the corresponding line breaking properties. A few characters have an additional key for the width (fullwidth vs. halfwidth), not yet used. There is a separate file, defined below.

```

5771 \catcode`\%=14
5772 \gdef\bbl@cjk intraspace{%
5773   \let\bbl@cjk intraspace\relax
5774   \directlua{
5775     require('babel-data-cjk.lua')
5776     Babel.cjk_enabled = true
5777     function Babel.cjk_linebreak(head)
5778       local GLYPH = node.id'glyph'
5779       local last_char = nil
5780       local quad = 655360      % 10 pt = 655360 = 10 * 65536
5781       local last_class = nil
5782       local last_lang = nil
5783       for item in node.traverse(head) do
5784         if item.id == GLYPH then
5785           local lang = item.lang
5786           local LOCALE = node.get_attribute(item,
5787             Babel.attr_locale)
5788           local props = Babel.locale_props[LOCALE] or {}
5789           local class = Babel.cjk_class[item.char].c
5790           if props.cjk_quotes and props.cjk_quotes[item.char] then
5791             class = props.cjk_quotes[item.char]
5792           end
5793           if class == 'cp' then class = 'cl' % ]] as CL
5794           elseif class == 'id' then class = 'I'
5795           elseif class == 'cj' then class = 'I' % loose
5796           end
5797           local br = 0
5798           if class and last_class and Babel.cjk_breaks[last_class][class] then
5799             br = Babel.cjk_breaks[last_class][class]
5800           end
5801           if br == 1 and props.linebreak == 'c' and
5802             lang ~= \the\l@nohyphenation\space and
5803             last_lang ~= \the\l@nohyphenation then
5804             local intrapenalty = props.intrapenalty
5805             if intrapenalty ~= 0 then
5806               local n = node.new(14, 0)      % penalty
5807               n.penalty = intrapenalty
5808               node.insert_before(head, item, n)
5809             end
5810             local intraspace = props.intraspace
5811             local n = node.new(12, 13)      % (glue, spaceskip)
5812             node.setglue(n, intraspace.b * quad,
5813               intraspace.p * quad,

```

```

5814             intraspace.m * quad)
5815         node.insert_before(head, item, n)
5816     end
5817     if font.getfont(item.font) then
5818         quad = font.getfont(item.font).size
5819     end
5820     last_class = class
5821     last_lang = lang
5822     else % if penalty, glue or anything else
5823         last_class = nil
5824     end
5825 end
5826 lang.hyphenate(head)
5827 end
5828 }%
5829 \bbl@luahyphenate}
5830 \gdef\bbl@luahyphenate{%
5831 \let\bbl@luahyphenate\relax
5832 \directlua{
5833     luatexbase.add_to_callback('hyphenate',
5834     function (head, tail)
5835         if Babel.linebreaking.before then
5836             for k, func in ipairs(Babel.linebreaking.before) do
5837                 func(head)
5838             end
5839         end
5840         lang.hyphenate(head)
5841         if Babel.cjk_enabled then
5842             Babel.cjk_linebreak(head)
5843         end
5844         if Babel.linebreaking.after then
5845             for k, func in ipairs(Babel.linebreaking.after) do
5846                 func(head)
5847             end
5848         end
5849         if Babel.set_hboxed then
5850             Babel.set_hboxed(head)
5851         end
5852         if Babel.sea_enabled then
5853             Babel.sea_disc_to_space(head)
5854         end
5855     end,
5856     'Babel.hyphenate')
5857 }}
5858 \endgroup
5859 %
5860 \def\bbl@provide@intraspace{%
5861 \bbl@ifunset\bbl@intsp@{language\language}\endcsname\empty\else
5862 {\expandafter\ifx\csname\bbl@intsp@{language}\endcsname\empty\else
5863 \bbl@xin@{/c}{\bbl@cl{lnbrk}}}%
5864 \ifin@ % cjk
5865 \bbl@cjk@intraspace
5866 \directlua{
5867     Babel.locale_props = Babel.locale_props or {}
5868     Babel.locale_props[\the\localeid].linebreak = 'c'
5869 }%
5870 \bbl@exp{\bbl@intraspace\bbl@cl{intsp}}\@}%
5871 \ifx\bbl@KVP@intrapenalty\@nnil
5872 \bbl@intrapenalty0\@
5873 \fi
5874 \else % sea
5875 \bbl@sea@intraspace
5876 \bbl@exp{\bbl@intraspace\bbl@cl{intsp}}\@}%

```

```

5877 \directlua{
5878     Babel.sea_ranges = Babel.sea_ranges or {}
5879     Babel.set_chranges('\bbl@cl{sbcpr}',
5880                       '\bbl@cl{chrng}')
5881 }%
5882 \ifx\bbl@KVP@intrapenalty\@nnil
5883     \bbl@intrapenalty0\@@
5884 \fi
5885 \fi
5886 \fi
5887 \ifx\bbl@KVP@intrapenalty\@nnil\else
5888     \expandafter\bbl@intrapenalty\bbl@KVP@intrapenalty\@@
5889 \fi}}

```

10.8. Arabic justification

WIP. \bbl@arabicjust is executed with both elongated and kashida. This must be fine tuned. The attribute kashida is set by transforms with kashida.

```

5890 \ifnum\bbl@bidimode>100 \ifnum\bbl@bidimode<200
5891 \def\bblar@chars{%
5892     0628,0629,062A,062B,062C,062D,062E,062F,0630,0631,0632,0633,%
5893     0634,0635,0636,0637,0638,0639,063A,063B,063C,063D,063E,063F,%
5894     0640,0641,0642,0643,0644,0645,0646,0647,0649}
5895 \def\bblar@elongated{%
5896     0626,0628,062A,062B,0633,0634,0635,0636,063B,%
5897     063C,063D,063E,063F,0641,0642,0643,0644,0646,%
5898     0649,064A}
5899 \begin{group}
5900 \catcode\_ =11 \catcode\`:=11
5901 \gdef\bblar@nofswarn{\gdef\msg_warning:nx##1##2##3{}}
5902 \end{group}
5903 \gdef\bbl@arabicjust{%
5904     \let\bbl@arabicjust\relax
5905     \newattribute\bblar@kashida
5906     \directlua{ Babel.attr_kashida = luatexbase.registernumber'bblar@kashida' }%
5907     \bblar@kashida=z@
5908     \bbl@patchfont{\bbl@parsejalt}}%
5909 \directlua{
5910     Babel.arabic.elong_map = Babel.arabic.elong_map or {}
5911     Babel.arabic.elong_map[\the\localeid] = {}
5912     luatexbase.add_to_callback('post_linebreak_filter',
5913     Babel.arabic.justify, 'Babel.arabic.justify')
5914     luatexbase.add_to_callback('hpack_filter',
5915     Babel.arabic.justify_hbox, 'Babel.arabic.justify_hbox')
5916 }}%

```

Save both node lists to make replacement.

```

5917 \def\bblar@fetchjalt#1#2#3#4{%
5918     \bbl@exp{\bbl@foreach{#1}}{%
5919         \bbl@ifunset{\bblar@JE@##1}%
5920         {\setbox\z@\hbox{\textdir TRT ^^^200d\char"##1#2}}%
5921         {\setbox\z@\hbox{\textdir TRT ^^^200d\char"\@nameuse{\bblar@JE@##1#2}}}%
5922     \directlua{%
5923         local last = nil
5924         for item in node.traverse(tex.box[0].head) do
5925             if item.id == node.id'glyph' and item.char > 0x600 and
5926                not (item.char == 0x200D) then
5927                 last = item
5928             end
5929         end
5930         Babel.arabic.#3['##1#4'] = last.char
5931     }}%

```

Elongated forms. Brute force. No rules at all, yet. The ideal: look at jalt table. And perhaps other tables (falt?, cswt?). What about kaf? And diacritic positioning?

```

5932 \gdef\bbl@parsejalt{%
5933   \ifx\addfontfeature\undefined\else
5934     \bbl@xin@{/e}{/\bbl@cl{\lnbrk}}%
5935     \ifin@
5936       \directlua{%
5937         if Babel.arabic.elong_map[\the\localeid][\fontid\font] == nil then
5938           Babel.arabic.elong_map[\the\localeid][\fontid\font] = {}
5939           tex.print([[string\cswt\space bbl@parsejalti\endcswt]])
5940         end
5941       }%
5942     \fi
5943   \fi}
5944 \gdef\bbl@parsejalti{%
5945   \begingroup
5946     \let\bbl@parsejalt\relax % To avoid infinite loop
5947     \edef\bbl@tempb{\fontid\font}%
5948     \bblar@nofswarn
5949     \@nameuse{tracinglostchars}\z@
5950     \bblar@fetchjalt\bblar@elongated{}{from}{}%
5951     \bblar@fetchjalt\bblar@chars{^^^064a}{from}{a}% Alef maksura
5952     \bblar@fetchjalt\bblar@chars{^^^0649}{from}{y}% Yeh
5953     \addfontfeature{RawFeature+=jalt}%
5954     % \@namedef{bblar@JE@0643}{06AA}% todo: catch medial kaf
5955     \bblar@fetchjalt\bblar@elongated{}{dest}{}%
5956     \bblar@fetchjalt\bblar@chars{^^^064a}{dest}{a}%
5957     \bblar@fetchjalt\bblar@chars{^^^0649}{dest}{y}%
5958     \directlua{%
5959       for k, v in pairs(Babel.arabic.from) do
5960         if Babel.arabic.dest[k] and
5961           not (Babel.arabic.from[k] == Babel.arabic.dest[k]) then
5962           Babel.arabic.elong_map[\the\localeid][\bbl@tempb]
5963             [Babel.arabic.from[k]] = Babel.arabic.dest[k]
5964         end
5965       end
5966     }%
5967   \endgroup}

```

The actual justification (inspired by CHICKENIZE).

```

5968 \begingroup
5969 \catcode`#=11
5970 \catcode`~=11
5971 \directlua{
5972
5973 Babel.arabic = Babel.arabic or {}
5974 Babel.arabic.from = {}
5975 Babel.arabic.dest = {}
5976 Babel.arabic.justify_factor = 0.95
5977 Babel.arabic.justify_enabled = true
5978 Babel.arabic.kashida_limit = -1
5979
5980 function Babel.arabic.justify(head)
5981   if not Babel.arabic.justify_enabled then return head end
5982   for line in node.traverse_id(node.id'hlist', head) do
5983     Babel.arabic.justify_hlist(head, line)
5984   end
5985   % In case the very first item is a line (eg, in \vbox):
5986   while head.prev do head = head.prev end
5987   return head
5988 end
5989
5990 function Babel.arabic.justify_hbox(head, gc, size, pack)

```

```

5991 local has_inf = false
5992 if Babel.arabic.justify_enabled and pack == 'exactly' then
5993     for n in node.traverse_id(12, head) do
5994         if n.stretch_order > 0 then has_inf = true end
5995     end
5996     if not has_inf then
5997         Babel.arabic.justify_hlist(head, nil, gc, size, pack)
5998     end
5999 end
6000 return head
6001 end
6002
6003 function Babel.arabic.justify_hlist(head, line, gc, size, pack)
6004     local d, new
6005     local k_list, k_item, pos_inline
6006     local width, width_new, full, k_curr, wt_pos, goal, shift
6007     local subst_done = false
6008     local elong_map = Babel.arabic.elong_map
6009     local cnt
6010     local last_line
6011     local GLYPH = node.id'glyph'
6012     local KASHIDA = Babel.attr_kashida
6013     local LOCALE = Babel.attr_locale
6014
6015     if line == nil then
6016         line = {}
6017         line.glue_sign = 1
6018         line.glue_order = 0
6019         line.head = head
6020         line.shift = 0
6021         line.width = size
6022     end
6023
6024     % Exclude last line.
6025     if ((line.next ~= nil or line.glue_sign == 1) and line.glue_order == 0) then
6026         elongs = {}      % Stores elongated candidates of each line
6027         k_list = {}      % And all letters with kashida
6028         pos_inline = 0   % Not yet used
6029
6030         for n in node.traverse_id(GLYPH, line.head) do
6031             pos_inline = pos_inline + 1 % To find where it is. Not used.
6032
6033             % Elongated glyphs
6034             if elong_map then
6035                 local locale = node.get_attribute(n, LOCALE)
6036                 if elong_map[locale] and elong_map[locale][n.font] and
6037                     elong_map[locale][n.font][n.char] then
6038                     table.insert(elongs, {node = n, locale = locale})
6039                     node.set_attribute(n.prev, KASHIDA, 0)
6040                 end
6041             end
6042
6043             % Tatwil. First create a list of nodes marked with kashida. The
6044             % rest of nodes can be ignored. The list of used weights is build
6045             % when transforms with the key kashida= are declared.
6046             if Babel.kashida_wts then
6047                 local k_wt = node.get_attribute(n, KASHIDA)
6048                 if k_wt > 0 then % todo. parameter for multi inserts
6049                     table.insert(k_list, {node = n, weight = k_wt, pos = pos_inline})
6050                 end
6051             end
6052
6053             end % of node.traverse_id

```

```

6054
6055 if #elongs == 0 and #k_list == 0 then goto next_line end
6056 full = line.width
6057 shift = line.shift
6058 goal = full * Babel.arabic.justify_factor % A bit crude
6059 width = node.dimensions(line.head) % The 'natural' width
6060
6061 % == Elongated ==
6062 % Original idea taken from 'chickenize'
6063 while (#elongs > 0 and width < goal) do
6064     subst_done = true
6065     local x = #elongs
6066     local curr = elongs[x].node
6067     local oldchar = curr.char
6068     curr.char = elong_map[elongs[x].locale][curr.font][curr.char]
6069     width = node.dimensions(line.head) % Check if the line is too wide
6070     % Substitute back if the line would be too wide and break:
6071     if width > goal then
6072         curr.char = oldchar
6073         break
6074     end
6075     % If continue, pop the just substituted node from the list:
6076     table.remove(elongs, x)
6077 end
6078
6079 % == Tatwil ==
6080 % Traverse the kashida node list so many times as required, until
6081 % the line is filled. The first pass adds a tatweel after each
6082 % node with kashida in the line, the second pass adds another one,
6083 % and so on. In each pass, add first the kashida with the highest
6084 % weight, then with lower weight and so on.
6085 if #k_list == 0 then goto next_line end
6086
6087 width = node.dimensions(line.head) % The 'natural' width
6088 k_curr = #k_list % Traverse backwards, from the end
6089 wt_pos = 1
6090
6091 while width < goal do
6092     subst_done = true
6093     k_item = k_list[k_curr].node
6094     if k_list[k_curr].weight == Babel.kashida_wts[wt_pos] then
6095         d = node.copy(k_item)
6096         d.char = 0x0640
6097         d.yoffset = 0 % TODO. From the prev char. But 0 seems safe.
6098         d.xoffset = 0
6099         line.head, new = node.insert_after(line.head, k_item, d)
6100         width_new = node.dimensions(line.head)
6101         if width > goal or width == width_new then
6102             node.remove(line.head, new) % Better compute before
6103             break
6104         end
6105         if Babel.fix_diacr then
6106             Babel.fix_diacr(k_item.next)
6107         end
6108         width = width_new
6109     end
6110     if k_curr == 1 then
6111         k_curr = #k_list
6112         wt_pos = (wt_pos >= table.getn(Babel.kashida_wts)) and 1 or wt_pos+1
6113     else
6114         k_curr = k_curr - 1
6115     end
6116 end

```



```

6117
6118 % Limit the number of tatweel by removing them. Not very efficient,
6119 % but it does the job in a quite predictable way.
6120 if Babel.arabic.kashida_limit > -1 then
6121   cnt = 0
6122   for n in node.traverse_id(GLYPH, line.head) do
6123     if n.char == 0x0640 then
6124       cnt = cnt + 1
6125       if cnt > Babel.arabic.kashida_limit then
6126         node.remove(line.head, n)
6127       end
6128     else
6129       cnt = 0
6130     end
6131   end
6132 end
6133
6134 ::next_line::
6135
6136 % Must take into account marks and ins, see luatex manual.
6137 % Have to be executed only if there are changes. Investigate
6138 % what's going on exactly.
6139 if subst_done and not gc then
6140   d = node.hpack(line.head, full, 'exactly')
6141   d.shift = shift
6142   node.insert_before(head, line, d)
6143   node.remove(head, line)
6144 end
6145 end % if process line
6146 end
6147 }
6148 \endgroup
6149 \fi\fi % ends Arabic just block: \ifnum\bbl@bidimode>100...

```

10.9. Common stuff

First, a couple of auxiliary macros to set the renderer according to the script. This is done by patching temporarily the low-level fontspec macro containing the current features set with `\defaultfontfeatures`. Admittedly this is somewhat dangerous, but that way the latter command still works as expected, because the renderer is set just before other settings. In xetex they are set to `\relax`.

```

6150 \def\bbl@scr@node@list{%
6151   ,Armenian,Coptic,Cyrillic,Georgian,,Glagolitic,Gothic,%
6152   ,Greek,Latin,Old Church Slavonic Cyrillic,}
6153 \ifnum\bbl@bidimode=102 % bidi-r
6154   \bbl@add\bbl@scr@node@list{Arabic,Hebrew,Syriac}
6155 \fi
6156 \def\bbl@set@renderer{%
6157   \bbl@xin@{\bbl@cl{sname}}{\bbl@scr@node@list}%
6158   \ifin@
6159     \let\bbl@unset@renderer\relax
6160   \else
6161     \bbl@exp{%
6162       \def\\bbl@unset@renderer{%
6163         \def<g__fontspec_default_fontopts_clist>{%
6164           \[g__fontspec_default_fontopts_clist]}%
6165         \def<g__fontspec_default_fontopts_clist>{%
6166           Renderer=Harfbuzz,\[g__fontspec_default_fontopts_clist]}%
6167       \fi}
6168 <@Font selection@>

```

10.10 Automatic fonts and ids switching

After defining the blocks for a number of scripts (must be extended and very likely fine tuned), we define a the function `Babel.locale_map`, which just traverse the node list to carry out the replacements. The table `loc_to_scr` stores the script range for each locale (whose id is the key), copied from this table (so that it can be modified on a locale basis); there is an intermediate table named `chr_to_loc` built on the fly for optimization, which maps a char to the locale. This locale is then used to get the `\language` as stored in `locale_props`, as well as the font (as requested). In the latter table a key starting with `/` maps the font from the global one (the key) to the local one (the value). Maths are skipped and discretionaries are handled in a special way.

There are two situations where the replacement is not carried out: either the `letters` option has been set and the character is not a letter (in the $\TeX$ sense), or the current script is the same as the new one.

```
6169 \directlua{% DL6
6170 Babel.script_blocks = {
6171   ['dflt'] = {},
6172   ['Arab'] = {{0x0600, 0x06FF}, {0x08A0, 0x08FF}, {0x0750, 0x077F},
6173               {0xFE70, 0xFEFF}, {0xFB50, 0xFDFF}, {0x1EE00, 0x1EEFF}},
6174   ['Armn'] = {{0x0530, 0x058F}},
6175   ['Beng'] = {{0x0980, 0x09FF}},
6176   ['Cher'] = {{0x13A0, 0x13FF}, {0xAB70, 0xABBF}},
6177   ['Copt'] = {{0x03E2, 0x03EF}, {0x2C80, 0x2CFF}, {0x102E0, 0x102FF}},
6178   ['Cyr'] = {{0x0400, 0x04FF}, {0x0500, 0x052F}, {0x1C80, 0x1C8F},
6179              {0x2DE0, 0x2DFF}, {0xA640, 0xA69F}},
6180   ['Deva'] = {{0x0900, 0x097F}, {0xA8E0, 0xA8FF}},
6181   ['Ethi'] = {{0x1200, 0x137F}, {0x1380, 0x139F}, {0x2D80, 0x2DDF},
6182              {0xAB00, 0xAB2F}},
6183   ['Geor'] = {{0x10A0, 0x10FF}, {0x2D00, 0x2D2F}},
6184   % Don't follow strictly Unicode, which places some Coptic letters in
6185   % the 'Greek and Coptic' block
6186   ['Grek'] = {{0x0370, 0x03E1}, {0x03F0, 0x03FF}, {0x1F00, 0x1FFF}},
6187   ['Hans'] = {{0x2E80, 0x2EFF}, {0x3000, 0x303F}, {0x31C0, 0x31EF},
6188              {0x3300, 0x33FF}, {0x3400, 0x4DBF}, {0x4E00, 0x9FFF},
6189              {0xF900, 0xFAFF}, {0xFE30, 0xFE4F}, {0xFF00, 0xFFEF},
6190              {0x20000, 0x2A6DF}, {0x2A700, 0x2B73F},
6191              {0x2B740, 0x2B81F}, {0x2B820, 0x2CEAF},
6192              {0x2CEB0, 0x2EBEF}, {0x2F800, 0x2FA1F}},
6193   ['Hebr'] = {{0x0590, 0x05FF},
6194              {0xFB1F, 0xFB4E}}, % <- Includes some <reserved>
6195   ['Jpan'] = {{0x3000, 0x303F}, {0x3040, 0x309F}, {0x30A0, 0x30FF},
6196              {0x4E00, 0x9FAF}, {0xFF00, 0xFFEF}},
6197   ['Khmr'] = {{0x1780, 0x17FF}, {0x19E0, 0x19FF}},
6198   ['Knda'] = {{0x0C80, 0x0CFF}},
6199   ['Kore'] = {{0x1100, 0x11FF}, {0x3000, 0x303F}, {0x3130, 0x318F},
6200              {0x4E00, 0x9FAF}, {0xA960, 0xA97F}, {0xAC00, 0xD7AF},
6201              {0xD7B0, 0xD7FF}, {0xFF00, 0xFFEF}},
6202   ['Lao'] = {{0x0E80, 0x0EFF}},
6203   ['Latn'] = {{0x0000, 0x007F}, {0x0080, 0x00FF}, {0x0100, 0x017F},
6204              {0x0180, 0x024F}, {0x1E00, 0x1EFF}, {0x2C60, 0x2C7F},
6205              {0xA720, 0xA7FF}, {0xAB30, 0xAB6F}},
6206   ['Mahj'] = {{0x11150, 0x1117F}},
6207   ['Mlym'] = {{0x0D00, 0x0D7F}},
6208   ['Mymr'] = {{0x1000, 0x109F}, {0xAA60, 0xAA7F}, {0xA9E0, 0xA9FF}},
6209   ['Orya'] = {{0x0B00, 0x0B7F}},
6210   ['Sinh'] = {{0x0D80, 0x0DFF}, {0x111E0, 0x111FF}},
6211   ['Sycr'] = {{0x0700, 0x074F}, {0x0860, 0x086F}},
6212   ['Taml'] = {{0x0B80, 0x0BFF}},
6213   ['Telu'] = {{0x0C00, 0x0C7F}},
6214   ['Tfng'] = {{0x2D30, 0x2D7F}},
6215   ['Thai'] = {{0x0E00, 0x0E7F}},
6216   ['Tibt'] = {{0x0F00, 0x0FFF}},
6217   ['Vaii'] = {{0xA500, 0xA63F}},
6218   ['Yiii'] = {{0xA000, 0xA48F}, {0xA490, 0xA4CF}}
```

```

6219 }
6220
6221 Babel.script_blocks.Cyrs = Babel.script_blocks.Cyrl
6222 Babel.script_blocks.Hant = Babel.script_blocks.Hans
6223 Babel.script_blocks.Kana = Babel.script_blocks.Jpan
6224
6225 function Babel.locale_map(head)
6226   if not Babel.locale_mapped then return head end
6227
6228   local LOCALE = Babel.attr_locale
6229   local GLYPH = node.id('glyph')
6230   local inmath = false
6231   local toloc_save
6232   for item in node.traverse(head) do
6233     local toloc
6234     if not inmath and item.id == GLYPH then
6235       % Optimization: build a table with the chars found
6236       if Babel.chr_to_loc[item.char] then
6237         toloc = Babel.chr_to_loc[item.char]
6238       else
6239         for lc, maps in pairs(Babel.loc_to_scr) do
6240           for _, rg in pairs(maps) do
6241             if item.char >= rg[1] and item.char <= rg[2] then
6242               Babel.chr_to_loc[item.char] = lc
6243               toloc = lc
6244               break
6245             end
6246           end
6247         end
6248         % Treat composite chars in a different fashion, because they
6249         % 'inherit' the previous locale.
6250         if (item.char >= 0x0300 and item.char <= 0x036F) or
6251            (item.char >= 0x1AB0 and item.char <= 0x1AFF) or
6252            (item.char >= 0x1DC0 and item.char <= 0x1DFF) then
6253           Babel.chr_to_loc[item.char] = -2000
6254           toloc = -2000
6255         end
6256         if not toloc then
6257           Babel.chr_to_loc[item.char] = -1000
6258         end
6259       end
6260       if toloc == -2000 then
6261         toloc = toloc_save
6262       elseif toloc == -1000 then
6263         toloc = nil
6264       end
6265       if toloc and Babel.locale_props[toloc] and
6266          Babel.locale_props[toloc].letters and
6267          tex.getcatcode(item.char) \string~= 11 then
6268         toloc = nil
6269       end
6270       if toloc and Babel.locale_props[toloc].script
6271          and Babel.locale_props[node.get_attribute(item, LOCALE)].script
6272          and Babel.locale_props[toloc].script ==
6273          Babel.locale_props[node.get_attribute(item, LOCALE)].script then
6274         toloc = nil
6275       end
6276       if toloc then
6277         if Babel.locale_props[toloc].lg then
6278           item.lang = Babel.locale_props[toloc].lg
6279           node.set_attribute(item, LOCALE, toloc)
6280         end
6281         if Babel.locale_props[toloc]['/'..item.font] then

```

```

6282         item.font = Babel.locale_props[toloc]['/'..item.font]
6283     end
6284 end
6285 toloc_save = toloc
6286 elseif not inmath and item.id == 7 then % Apply recursively
6287     item.replace = item.replace and Babel.locale_map(item.replace)
6288     item.pre      = item.pre and Babel.locale_map(item.pre)
6289     item.post     = item.post and Babel.locale_map(item.post)
6290 elseif item.id == node.id'math' then
6291     inmath = (item.subtype == 0)
6292 end
6293 end
6294 return head
6295 end
6296 }

```

The code for `\babelcharproperty` is straightforward. Just note the modified lua table can be different.

```

6297 \newcommand\babelcharproperty[1]{%
6298   \count@=#1\relax
6299   \ifvmode
6300     \expandafter\bbl@chprop
6301   \else
6302     \bbl@error{charproperty-only-vertical}{}{}{}%
6303   \fi}
6304 \newcommand\bbl@chprop[3][\the\count@]{%
6305   \@tempcnta=#1\relax
6306   \bbl@ifunset{bbl@chprop@#2}% {unknown-char-property}
6307   {\bbl@error{unknown-char-property}{}{#2}{}%
6308     {}%
6309   \loop
6310     \bbl@cs{chprop@#2}{#3}%
6311     \ifnum\count@<\@tempcnta
6312       \advance\count@\@ne
6313     \repeat}
6314 %
6315 \def\bbl@chprop@direction#1{%
6316   \directlua{
6317     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6318     Babel.characters[\the\count@]['d'] = '#1'
6319   }}
6320 \let\bbl@chprop@bc\bbl@chprop@direction
6321 %
6322 \def\bbl@chprop@mirror#1{%
6323   \directlua{
6324     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6325     Babel.characters[\the\count@]['m'] = '\number#1'
6326   }}
6327 \let\bbl@chprop@bmg\bbl@chprop@mirror
6328 %
6329 \def\bbl@chprop@linebreak#1{%
6330   \directlua{
6331     Babel.cjk_characters[\the\count@] = Babel.cjk_characters[\the\count@] or {}
6332     Babel.cjk_characters[\the\count@]['c'] = '#1'
6333   }}
6334 \let\bbl@chprop@lb\bbl@chprop@linebreak
6335 %
6336 \def\bbl@chprop@locale#1{%
6337   \directlua{
6338     Babel.chr_to_loc = Babel.chr_to_loc or {}
6339     Babel.chr_to_loc[\the\count@] =
6340       \bbl@ifblank{#1}{-1000}{\the\bbl@cs{id@#1}}\space
6341   }}

```

Post-handling hyphenation patterns for non-standard rules, like ff to ff-f. There are still some issues with speed (not very slow, but still slow). The Lua code is below.

```
6342 \directlua{% DL7
```

```
6343   Babel.nohyphenation = \the\l@nohyphenation
```

```
6344 }
```

Now the TeX high level interface, which requires the function defined above for converting strings to functions returning a string. These functions handle the $\{n\}$ syntax. For example, $\text{pre}=\{1\}\{1\}$ becomes `function(m) return m[1]..m[1]..'-' end`, where m are the matches returned after applying the pattern. With a mapped capture the functions are similar to `function(m) return Babel.capt_map(m[1],1) end`, where the last argument identifies the mapping to be applied to $m[1]$. The way it is carried out is somewhat tricky, but the effect is not dissimilar to `lua load` – save the code as string in a TeX macro, and expand this macro at the appropriate place. As `\directlua` does not take into account the current catcode of `@`, we just avoid this character in macro names (which explains the internal group, too).

```
6345 \begingroup
```

```
6346 \catcode`\-=12
```

```
6347 \catcode`\%=12
```

```
6348 \catcode`\&=14
```

```
6349 \catcode`\|=12
```

```
6350 \gdef\babelprehyphenation{%&
```

```
6351   \@ifnextchar[{\bbl@settransform{0}}{\bbl@settransform{0}[]}]
```

```
6352 \gdef\babelposthyphenation{%&
```

```
6353   \@ifnextchar[{\bbl@settransform{1}}{\bbl@settransform{1}[]}]
```

```
6354 %
```

```
6355 \gdef\bbl@settransform#1[#2]#3#4#5{%&
```

```
6356   \ifcase#1
```

```
6357     \bbl@activateprehyphen
```

```
6358   \or
```

```
6359     \bbl@activateposthyphen
```

```
6360   \fi
```

```
6361 \begingroup
```

```
6362   \def\babeltempa{\bbl@add@list\babeltempb}%&
```

```
6363   \let\babeltempb\@empty
```

```
6364   \def\bbl@tempa{#5}%&
```

```
6365   \bbl@replace\bbl@tempa{,}{,}%& TODO. Ugly trick to preserve {}
```

```
6366   \expandafter\bbl@foreach\expandafter{\bbl@tempa}{%&
```

```
6367     \bbl@ifsamestring{##1}{remove}%&
```

```
6368     {\bbl@add@list\babeltempb{nil}}}%&
```

```
6369     {\directlua{
```

```
6370       local rep = [=[#1]=]
```

```
6371       local three_args = '%s*=%s*([%-d%.%a{ }|]+)%s+([%-d%.%a{ }|]+)%s+([%-d%.%a{ }|]+)'
6372       &% Numeric passes directly: kern, penalty...
```

```
6373       rep = rep:gsub('^%s*(remove)%s*$', 'remove = true')
```

```
6374       rep = rep:gsub('^%s*(insert)%s*$', 'insert = true,')
```

```
6375       rep = rep:gsub('^%s*(after)%s*$', 'after = true,')
```

```
6376       rep = rep:gsub('(string)%s*=%s*([%-s,]*)', Babel.capture_func)
```

```
6377       rep = rep:gsub('node%s*=%s*([%-a+)%s*([%-a+])', Babel.capture_node)
```

```
6378       rep = rep:gsub('(norule)' .. three_args,
```

```
6379         'norule = {' .. '%2, %3, %4' .. '})')
```

```
6380       if #1 == 0 or #1 == 2 then
```

```
6381         rep = rep:gsub('(space)' .. three_args,
```

```
6382         'space = {' .. '%2, %3, %4' .. '})')
```

```
6383         rep = rep:gsub('(spacefactor)' .. three_args,
```

```
6384         'spacefactor = {' .. '%2, %3, %4' .. '})')
```

```
6385         rep = rep:gsub('(kashida)%s*=%s*([%-s,]*)', Babel.capture_kashida)
```

```
6386       &% Transform values
```

```
6387       rep, n = rep:gsub('({[%-a%-.]})|([%-a%-.]})',
```

```
6388         function(v,d)
```

```
6389           return string.format (
```

```
6390             '{\the\csname bbl@id@@@#3\endcsname,"%s",%s}',
```

```
6391             v,
```

```
6392             load( 'return Babel.locale_props'..
```

```

6393         '\the\csname bbl@id@@#3\endcsname.' .. d() )
6394     end )
6395     rep, n = rep:gsub( '{([%a-%.]*)|([%-d%.]*)}',
6396         '\the\csname bbl@id@@#3\endcsname,"%1",%2}')
6397 end
6398 if #1 == 1 then
6399     rep = rep:gsub( '(no)%s*=%s*([^\s,]*)', Babel.capture_func)
6400     rep = rep:gsub( '(pre)%s*=%s*([^\s,]*)', Babel.capture_func)
6401     rep = rep:gsub( '(post)%s*=%s*([^\s,]*)', Babel.capture_func)
6402 end
6403 tex.print([[string\babeltempa{[]] .. rep .. [[]]])
6404 }]}&%
6405 \bbl@foreach\babeltempb{&%
6406     \bbl@forkv{##1}}{&%
6407         \in@{,###1,},{,nil,step,data,remove,insert,string,no,pre,no,&%
6408             post,penalty,kashida,space,spacefactor,kern,node,after,norule,}&%
6409         \ifin\else
6410             \bbl@error{bad-transform-option}{###1}}{&%
6411         \fi}}&%
6412 \let\bbl@kv@attribute\relax
6413 \let\bbl@kv@label\relax
6414 \let\bbl@kv@fonts\@empty
6415 \let\bbl@kv@prepend\relax
6416 \bbl@forkv{#2}{\bbl@csarg\edef{kv@##1}{##2}}&%
6417 \ifx\bbl@kv@fonts\@empty\else\bbl@settransfont\fi
6418 \ifx\bbl@kv@attribute\relax
6419     \ifx\bbl@kv@label\relax\else
6420         \bbl@exp{\bbl@trim@def\bbl@kv@fonts{\bbl@kv@fonts}}&%
6421         \bbl@replace\bbl@kv@fonts{ }{,}&%
6422         \edef\bbl@kv@attribute{\bbl@ATR@\bbl@kv@label @#3@\bbl@kv@fonts}&%
6423         \count@\z@
6424         \def\bbl@elt##1##2##3{&%
6425             \bbl@ifsamestring{#3,\bbl@kv@label}{##1,##2}&%
6426             {\bbl@ifsamestring{\bbl@kv@fonts}{##3}&%
6427                 {\count@\@ne}&%
6428                 {\bbl@error{font-conflict-transforms}{}}{}}}&%
6429             }&%
6430         \bbl@transfont@list
6431         \ifnum\count@=\z@
6432             \bbl@exp{\global\bbl@add\bbl@transfont@list
6433                 {\bbl@elt{#3}{\bbl@kv@label}{\bbl@kv@fonts}}}&%
6434         \fi
6435         \bbl@ifunset{\bbl@kv@attribute}&%
6436         {\global\bbl@carg\newattribute{\bbl@kv@attribute}}&%
6437         {}&%
6438         \global\bbl@carg\setattribute{\bbl@kv@attribute}\@ne
6439     \fi
6440 \else
6441     \edef\bbl@kv@attribute{\expandafter\bbl@stripslash\bbl@kv@attribute}&%
6442 \fi
6443 \directlua{
6444     local lbkr = Babel.linebreaking.replacements[#1]
6445     local u = unicode.utf8
6446     local id, attr, label
6447     if #1 == 0 then
6448         id = \the\csname bbl@id@@#3\endcsname\space
6449     else
6450         id = \the\csname l@#3\endcsname\space
6451     end
6452     \ifx\bbl@kv@attribute\relax
6453         attr = -1
6454     \else
6455         attr = luatexbase.registernumber'\bbl@kv@attribute'

```

```

6456 \fi
6457 \ifx\bbl@kv@label\relax\else &% Same refs:
6458 label = [==[\bbl@kv@label]==]
6459 \fi
6460 &% Convert pattern:
6461 local patt = string.gsub([==[#4]==], '%s', '')
6462 if #1 == 0 then
6463 patt = string.gsub(patt, '|', ' ')
6464 end
6465 if not u.find(patt, '()', nil, true) then
6466 patt = '()' .. patt .. '()'
6467 end
6468 patt = string.gsub(patt, '%(%)^', '^()')
6469 patt = string.gsub(patt, '%$$(%)', '()$')
6470 patt = u.gsub(patt, '{(.)}',
6471 function (n)
6472 return '%' .. (tonumber(n) and (tonumber(n)+1) or n)
6473 end)
6474 patt = u.gsub(patt, '{(%x%x%x%x+)}',
6475 function (n)
6476 return u.gsub(u.char(tonumber(n, 16)), '(%p)', '%%1')
6477 end)
6478 lbkr[id] = lbkr[id] or {}
6479 table.insert(lbkr[id], \ifx\bbl@kv@prepend\relax\else 1,\fi
6480 { label=label, attr=attr, pattern=patt, replace={\babeltempb} })
6481 }&%
6482 \endgroup}
6483 \endgroup
6484 %
6485 \let\bbl@transfont@list\@empty
6486 \def\bbl@settransfont{%
6487 \global\let\bbl@settransfont\relax % Execute only once
6488 \gdef\bbl@transfont{%
6489 \def\bbl@elt####1####2####3{%
6490 \bbl@ifblank{####3}%
6491 {\count@tw@}% Do nothing if no fonts
6492 {\count@z@
6493 \bbl@vforeach{####3}{%
6494 \def\bbl@tempd{#####1}%
6495 \edef\bbl@tempe{\bbl@transfam/\f@series/\f@shape}%
6496 \ifx\bbl@tempd\bbl@tempe
6497 \count@\@ne
6498 \else\ifx\bbl@tempd\bbl@transfam
6499 \count@\@ne
6500 \fi\fi}%
6501 \ifcase\count@
6502 \bbl@csarg\unsetattribute{ATR@####2@####1@####3}%
6503 \or
6504 \bbl@csarg\setattribute{ATR@####2@####1@####3}\@ne
6505 \fi}%
6506 \bbl@transfont@list}%
6507 \AddToHook{selectfont}{\bbl@transfont}% Hooks are global.
6508 \gdef\bbl@transfam{-unknown-}%
6509 \bbl@foreach\bbl@font@fams{%
6510 \AddToHook{##1family}{\def\bbl@transfam{##1}}%
6511 \bbl@ifsamestring{\@nameuse{##1default}}\familydefault
6512 {\xdef\bbl@transfam{##1}}%
6513 }}}
6514 %
6515 \DeclareRobustCommand\enablelocaletransform[1]{%
6516 \bbl@ifunset{\bbl@ATR@#1@\language @}%
6517 {\bbl@error{transform-not-available}{#1}{}}%
6518 {\bbl@csarg\setattribute{ATR@#1@\language @}\@ne}}

```

```

6519 \DeclareRobustCommand\disablelocaletransform[1]{%
6520   \bbl@ifunset{bbl@ATR@#1@\language @}{%
6521     {\bbl@error{transform-not-available-b}{#1}}{}}%
6522     {\bbl@csarg\unsetattribute{ATR@#1@\language @}}}

```

The following two macros load the Lua code for transforms, but only once. The only difference is in `add_after` and `add_before`.

```

6523 \def\bbl@activateposthyphen{%
6524   \let\bbl@activateposthyphen\relax
6525   \ifx\bbl@attr@hboxed\undefined
6526     \newattribute\bbl@attr@hboxed
6527   \fi
6528   \directlua{
6529     require('babel-transforms.lua')
6530     Babel.linebreaking.add_after(Babel.post_hyphenate_replace)
6531   }}
6532 \def\bbl@activateprehyphen{%
6533   \let\bbl@activateprehyphen\relax
6534   \ifx\bbl@attr@hboxed\undefined
6535     \newattribute\bbl@attr@hboxed
6536   \fi
6537   \directlua{
6538     require('babel-transforms.lua')
6539     Babel.linebreaking.add_before(Babel.pre_hyphenate_replace)
6540   }}
6541 \newcommand\SetTransformValue[3]{%
6542   \directlua{
6543     Babel.locale_props[\the\csname bbl@id@#1\endcsname].vars["#2"] = #3
6544   }}

```

The code in `babel-transforms.lua` prints at some points the current string being transformed. This macro first make sure this file is loaded. Then, activates temporarily this feature and typeset inside a box the text in the argument.

```

6545 \newcommand\ShowBabelTransforms[1]{%
6546   \bbl@activateprehyphen
6547   \bbl@activateposthyphen
6548   \begingroup
6549     \directlua{ Babel.show_transforms = true }%
6550     \setbox\z@\vbox{#1}%
6551     \directlua{ Babel.show_transforms = false }%
6552   \endgroup}

```

The following experimental (and unfinished) macro applies the prehyphenation transforms for the current locale to a string (characters and spaces) and processes it in a fully expandable way (among other limitations, the string can't contain `]`). The way it operates is admittedly rather cumbersome: it converts the string to a node list, processes it, and converts it back to a string. The lua code is in the lua file below.

```

6553 \newcommand\localeprehyphenation[1]{%
6554   \directlua{ Babel.string_prehyphenation([=#1=], \the\localeid) }}

```

10.11.Bidi

As a first step, add a handler for bidi and digits (and potentially other processes) just before `luaotfload` is applied, which is loaded by default by $\TeX$. Just in case, consider the possibility it has not been loaded.

```

6555 \def\bbl@activate@preotf{%
6556   \let\bbl@activate@preotf\relax % only once
6557   \directlua{
6558     function Babel.pre_otfload_v(head)
6559       if Babel.numbers and Babel.digits_mapped then
6560         head = Babel.numbers(head)
6561       end
6562       if Babel.bidi_enabled then

```



```

6563     head = Babel.bidi(head, false, dir)
6564 end
6565 return head
6566 end
6567 %
6568 function Babel.pre_otfload_h(head, gc, sz, pt, dir)
6569     if Babel.numbers and Babel.digits_mapped then
6570         head = Babel.numbers(head)
6571     end
6572     if Babel.bidi_enabled then
6573         head = Babel.bidi(head, false, dir)
6574     end
6575     return head
6576 end
6577 %
6578 luatexbase.add_to_callback('pre_linebreak_filter',
6579     Babel.pre_otfload_v,
6580     'Babel.pre_otfload_v',
6581     Babel.priority_in_callback('pre_linebreak_filter',
6582         'luaotfload.node_processor') or nil)
6583 %
6584 luatexbase.add_to_callback('hpack_filter',
6585     Babel.pre_otfload_h,
6586     'Babel.pre_otfload_h',
6587     Babel.priority_in_callback('hpack_filter',
6588         'luaotfload.node_processor') or nil)
6589 }}

```

The basic setup. The output is modified at a very low level to set the `\bodydir` to the `\pagedir`. Sadly, we have to deal with boxes in math with basic, so the `\bbl@mathboxdir` hack is activated every math with the package option `bidi=`. The hack for the PUA is no longer necessary with basic (24.8), but it's kept in basic-r.

```

6590 \ifnum\bbl@bidimode>\@ne % Any bidi= except default (=1)
6591 \let\bbl@beforeforeign\leavevmode
6592 \AtEndOfPackage{\EnableBabelHook{babel-bidi}}
6593 \RequirePackage{luatexbase}
6594 \bbl@activate@preotf
6595 \directlua{
6596     require('babel-data-bidi.lua')
6597     \ifcase\expandafter\@gobbletwo\the\bbl@bidimode\or
6598         require('babel-bidi-basic.lua')
6599     \or
6600         require('babel-bidi-basic-r.lua')
6601     table.insert(Babel.ranges, {0xE000, 0xF8FF, 'on'})
6602     table.insert(Babel.ranges, {0xF0000, 0xFFFFFD, 'on'})
6603     table.insert(Babel.ranges, {0x100000, 0x10FFFFD, 'on'})
6604     \fi}
6605 \newattribute\bbl@attr@dir
6606 \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
6607 \bbl@exp{\output{\bodydir\pagedir\the\output}}
6608 \fi
6609 %
6610 \chardef\bbl@thetextdir\z@
6611 \chardef\bbl@thepardir\z@
6612 \def\bbl@setluadir#1#2{% 1=\text/pardirection 2= 0:l 1:r 2:a1
6613     \ifcase#2\relax
6614         \ifcase#1\else#1=\z@\fi
6615     \else
6616         \ifcase#1#1=\@ne\fi
6617     \fi}

```

`\bbl@attr@dir` stores the directions with a mask: `..00PPTT`, with masks `0xC` (PP is the par dir) and `0x3` (TT is the text dir). These macro names are shared by the 3 engines, with different definitions.

```

6618 \def\bbl@thedir{0}

```

```

6619 \def\bbl@textdir#1{%
6620   \bbl@setluadir\textdirection{#1}%
6621   \chardef\bbl@thetextdir#1\relax
6622   \edef\bbl@thedir{\the\numexpr\bbl@thepardir*4+#1}%
6623   \setattribute\bbl@attr@dir{\numexpr\bbl@thepardir*4+#1}}
6624 \def\bbl@pardir#1{% Used twice
6625   \bbl@setluadir\pardirection{#1}%
6626   \chardef\bbl@thepardir#1\relax}
6627 \def\bbl@bodydir{\bbl@setluadir\bodydirection}% Used once
6628 \def\bbl@dirparastext{\pardirection=\textdirection\relax}% Used once

```

RTL text inside math needs special attention. It affects not only to actual math stuff, but also to ‘tabular’, which is based on a fake math.

```

6629 \ifnum\bbl@bidimode>\z@ % Any bidi=
6630   \def\bbl@insidemath{0}%
6631   \def\bbl@everymath{\def\bbl@insidemath{1}}
6632   \def\bbl@everydisplay{\def\bbl@insidemath{2}}
6633   \frozen@everymath\expandafter{%
6634     \expandafter\bbl@everymath\the\frozen@everymath}
6635   \frozen@everydisplay\expandafter{%
6636     \expandafter\bbl@everydisplay\the\frozen@everydisplay}
6637   \AtBeginDocument{
6638     \directlua{
6639       function Babel.math_box_dir(head)
6640         if not (token.get_macro('bbl@insidemath') == '0') then
6641           if Babel.hlist_has_bidi(head) then
6642             local d = node.new(node.id'dir')
6643             d.dir = '+TRT'
6644             for item in node.traverse(head) do
6645               if item.id == 11 or item.id == node.id'glyph' then
6646                 head = node.insert_before(head, item, d)
6647                 break
6648             end
6649           end
6650           local inmath = false
6651           for item in node.traverse(head) do
6652             if item.id == 11 then
6653               inmath = (item.subtype == 0)
6654             elseif not inmath then
6655               node.set_attribute(item,
6656                 Babel.attr_dir, token.get_macro('bbl@thedir'))
6657             end
6658           end
6659         end
6660       end
6661       return head
6662     end
6663     luatexbase.add_to_callback("hpack_filter", Babel.math_box_dir,
6664       "Babel.math_box_dir", 0)
6665     if Babel.unset_atdir then
6666       luatexbase.add_to_callback("pre_linebreak_filter", Babel.unset_atdir,
6667         "Babel.unset_atdir")
6668       luatexbase.add_to_callback("hpack_filter", Babel.unset_atdir,
6669         "Babel.unset_atdir")
6670     end
6671   } }%
6672 \fi

```

Experimental. Tentative name.

```

6673 \DeclareRobustCommand\localebox[1]{%
6674   {\def\bbl@insidemath{0}%
6675     \mbox{\foreignlanguage{\language}{#1}}}}

```

10.12 Layout

Unlike xetex, luatex requires only minimal changes for right-to-left layouts, particularly in monolingual documents (the engine itself reverses boxes – including column order or headings –, margins, etc.) with `bidi=basic`, without having to patch almost any macro where text direction is relevant.

Still, there are three areas deserving special attention, namely, tabular, math, and graphics, text and intrinsically left-to-right elements are intermingled. I’ve made some progress in graphics, but they’re essentially hacks; I’ve also made some progress in ‘tabular’, but when I decided to tackle math (both standard math and ‘amsmath’) the nightmare began. I’m still not sure how ‘amsmath’ should be modified, but the main problem is that, boxes are “generic” containers that can hold text, math, and graphics (even at the same time; remember that inline math is included in the list of text nodes marked with ‘math’ (11) nodes too).

`\@hangfrom` is useful in many contexts and it is redefined always with the `layout` option.

There are, however, a number of issues when the text direction is not the same as the box direction (as set by `\bodydir`), and when `\parbox` and `\hangindent` are involved. Fortunately, latest releases of luatex simplify a lot the solution with `\shapemode`.

With the issue #15 I realized commands are best patched, instead of redefined. With a few lines, a modification could be applied to several classes and packages. Now, `tabular` seems to work (at least in simple cases) with `array`, `tabularx`, `hline`, `colortbl`, `longtable`, `booktabs`, etc. However, `dcolumn` still fails.

```
6676 \bbl@trace{Redefinitions for bidi layout}
6677 %
6678 << *More package options >> ≡
6679 \chardef\bbl@eqnpos\z@
6680 \DeclareOption{leqno}{\chardef\bbl@eqnpos\@ne}
6681 \DeclareOption{fleqn}{\chardef\bbl@eqnpos\tw@}
6682 << /More package options >>
6683 %
6684 \ifnum\bbl@bidimode>\z@ % Any bidi=
6685   \matheqdirmode\@ne % Several luatex primitives.
6686   \mathemptydisplaymode\@ne % For fixes.
6687   \breakafterdirmode\@ne %
6688   \fixupboxesmode\@ne %
6689   \let\bbl@eqnodir\relax
6690   \def\bbl@eqdel{()}
6691   \def\bbl@eqnum{%
6692     {\normalfont\normalcolor
6693       \expandafter\@firstoftwo\bbl@eqdel
6694       \theequation
6695       \expandafter\@secondoftwo\bbl@eqdel}}
6696   \def\bbl@puteqno#1{\eqno\hbox{#1}}
6697   \def\bbl@putleqno#1{\leqno\hbox{#1}}
6698   \def\bbl@eqno@flip#1{% So
6699     \ifdim\predisplaysize=-\maxdimen % For consecutive displays
6700       \eqno
6701       \hb@xt@.01pt{%
6702         \hb@xt@\displaywidth{\hss{#1}\glet\bbl@upset\@currentlabel}}\hss}%
6703     \else
6704       \leqno\hbox{#1}\glet\bbl@upset\@currentlabel}%
6705     \fi
6706     \bbl@exp{\def\\@currentlabel{\[bbl@upset]}}
6707   \def\bbl@leqno@flip#1{%
6708     \ifdim\predisplaysize=-\maxdimen % For consecutive displays
6709       \leqno
6710       \hb@xt@.01pt{%
6711         \hss\hb@xt@\displaywidth{\hss{#1}\glet\bbl@upset\@currentlabel}}\hss}%
6712     \else
6713       \eqno\hbox{#1}\glet\bbl@upset\@currentlabel}%
6714     \fi
6715     \bbl@exp{\def\\@currentlabel{\[bbl@upset]}}
6716 %
6717   \AtBeginDocument{%
```

```

6718 \ifx\bb@noamsmath\relax\else
6719 \ifx\maketag@@@ \@undefined % Normal equation, eqnarray
6720 \ifnum\bb@eqnpos=\tw@
6721 \bb@replace\equation{\hb@xt@\linewidth}%
6722 {\hbox bdir\mathdirection to\linewidth}%
6723 \bb@carg\bb@sreplace{[ ]}{\hb@xt@\linewidth}%
6724 {\hbox bdir\mathdirection to\linewidth}%
6725 \fi
6726 \AddToHook{env/equation/begin}{%
6727 \ifnum\bb@thetextdir>z@
6728 \def\bb@mathboxdir{\def\bb@insidemath{1}}%
6729 \let\@eqnnum\bb@eqnum
6730 \edef\bb@eqnodir{\noexpand\bb@textdir{\the\bb@thetextdir}}%
6731 \chardef\bb@thetextdir\z@
6732 \bb@add\normalfont{\bb@eqnodir}%
6733 \ifcase\bb@eqnpos
6734 \let\bb@puteqno\bb@eqno@flip
6735 \or
6736 \let\bb@puteqno\bb@leqno@flip
6737 \fi
6738 \fi}%
6739 \ifnum\bb@eqnpos=\tw@\else
6740 \def\endequation{\bb@puteqno{\@eqnnum}$$\@ignoretrue}%
6741 \fi
6742 \AddToHook{env/eqnarray/begin}{%
6743 \ifnum\bb@thetextdir>z@
6744 \def\bb@mathboxdir{\def\bb@insidemath{1}}%
6745 \edef\bb@eqnodir{\noexpand\bb@textdir{\the\bb@thetextdir}}%
6746 \chardef\bb@thetextdir\z@
6747 \bb@add\normalfont{\bb@eqnodir}%
6748 \ifnum\bb@eqnpos=\@ne
6749 \def\@eqnnum{%
6750 \setbox\z@\hbox{\bb@eqnum}%
6751 \hbox to0.01pt{\hss\hbox to\displaywidth{\box\z@\hss}}}%
6752 \else
6753 \let\@eqnnum\bb@eqnum
6754 \fi
6755 \fi}
6756 \else % amstex
6757 \ifnum\bb@eqnpos=\tw@
6758 \bb@exp{% Hack to hide maybe undefined conditionals:
6759 \\\bb@sreplace\\multline@crcr{\<if@fleqn>\tabskip\z@skip\<fi>\crcr}}%
6760 \fi
6761 \bb@exp{% Hack to hide maybe undefined conditionals:
6762 \chardef\bb@eqnpos=0%
6763 \<iftagsleft@>1\<else>\<if@fleqn>2\<fi>\<fi>\relax}%
6764 \ifnum\bb@eqnpos=\@ne
6765 \let\bb@ams@lap\hbox
6766 \else
6767 \let\bb@ams@lap\llap
6768 \fi
6769 \ExplSyntaxOn % Required by \bb@sreplace with \intertext@
6770 \bb@sreplace\intertext@{\normalbaselines}%
6771 {\normalbaselines
6772 \ifx\bb@eqnodir\relax\else\bb@pardir\@ne\bb@eqnodir\fi}%
6773 \ExplSyntaxOff
6774 \def\bb@ams@tagbox#1#2{#1{\bb@eqnodir#2}}% #1=hbox|@lap|flip
6775 \ifx\bb@ams@lap\hbox % leqno
6776 \def\bb@ams@flip#1{%
6777 \hbox to 0.01pt{\hss\hbox to\displaywidth{\#1}\hss}}%
6778 \else % eqno
6779 \def\bb@ams@flip#1{%
6780 \hbox to 0.01pt{\hbox to\displaywidth{\hss{\#1}\hss}}%

```

```

6781 \fi
6782 \def\bbl@ams@preset#1{%
6783   \def\bbl@mathboxdir{\def\bbl@insidemath{1}}%
6784   \ifnum\bbl@thetextdir>\z@
6785     \edef\bbl@eqnodir{\noexpand\bbl@textdir{\the\bbl@thetextdir}}%
6786     \bbl@sreplace\textdef@{\hbox}{\bbl@ams@tagbox\hbox}%
6787     \bbl@sreplace\maketag@@@{\hbox}{\bbl@ams@tagbox#1}%
6788   \fi}%
6789 \def\bbl@ams@equation{%
6790   \def\bbl@mathboxdir{\def\bbl@insidemath{1}}%
6791   \ifnum\bbl@thetextdir>\z@
6792     \edef\bbl@eqnodir{\noexpand\bbl@textdir{\the\bbl@thetextdir}}%
6793     \chardef\bbl@thetextdir\z@
6794     \bbl@add\normalfont{\bbl@eqnodir}%
6795     \ifcase\bbl@eqnpos
6796       \def\veqno##1##2{\bbl@eqno@flip{##1##2}}%
6797     \or
6798       \def\veqno##1##2{\bbl@leqno@flip{##1##2}}%
6799     \fi
6800   \fi}%
6801 \AddToHook{env/equation/begin}{\bbl@ams@equation}%
6802 \AddToHook{env/equation*/begin}{\bbl@ams@equation}%
6803 \AddToHook{env/cases/begin}{\bbl@ams@preset\bbl@ams@lap}%
6804 \AddToHook{env/multline/begin}{\bbl@ams@preset\hbox}%
6805 \AddToHook{env/gather/begin}{\bbl@ams@preset\bbl@ams@lap}%
6806 \AddToHook{env/gather*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6807 \AddToHook{env/align/begin}{\bbl@ams@preset\bbl@ams@lap}%
6808 \AddToHook{env/align*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6809 \AddToHook{env/alignat/begin}{\bbl@ams@preset\bbl@ams@lap}%
6810 \AddToHook{env/alignat*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6811 \AddToHook{env/eqnalign/begin}{\bbl@ams@preset\hbox}%
6812 % Hackish, for proper alignment. Don't ask me why it works!
6813 \bbl@exp{% Avoid a 'visible' conditional
6814   \\\AddToHook{env/align*/end}{\<iftag@>\<else>\\tag*{}\<fi>}%
6815   \\\AddToHook{env/alignat*/end}{\<iftag@>\<else>\\tag*{}\<fi>}}%
6816 \AddToHook{env/flalign/begin}{\bbl@ams@preset\hbox}%
6817 \AddToHook{env/split/before}{%
6818   \def\bbl@mathboxdir{\def\bbl@insidemath{1}}%
6819   \ifnum\bbl@thetextdir>\z@
6820     \bbl@ifsamestring\@currentvir{equation}%
6821     {\ifx\bbl@ams@lap\hbox % leqno
6822       \def\bbl@ams@flip#1{%
6823         \hbox to 0.01pt{\hbox to\displaywidth{{#1}\hss}\hss}}%
6824       \else
6825         \def\bbl@ams@flip#1{%
6826           \hbox to 0.01pt{\hss\hbox to\displaywidth{\hss{#1}}}}%
6827       \fi}%
6828     {}%
6829   \fi}%
6830 \fi\fi}
6831 \fi

```

Declarations specific to lua, called by \babelprovide.

```

6832 \def\bbl@provide@extra#1{%
6833   % == onchar ==
6834   \ifx\bbl@KVP@onchar\@nnil\else
6835     \bbl@luahyphenate
6836     \bbl@exp{%
6837       \\\AddToHook{env/document/before}{%
6838         {\let\\bbl@ifrestoring\\@firstoftwo
6839           \\\select@language{#1}{}}}%
6840     \directlua{
6841       if Babel.locale_mapped == nil then

```

```

6842     Babel.locale_mapped = true
6843     Babel.linebreaking.add_before(Babel.locale_map, 1)
6844     Babel.loc_to_scr = {}
6845     Babel.chr_to_loc = Babel.chr_to_loc or {}
6846 end
6847 Babel.locale_props[\the\localeid].letters = false
6848 }%
6849 \bbl@xin@{ letters }{ \bbl@KVP@onchar\space}%
6850 \ifin@
6851   \directlua{
6852     Babel.locale_props[\the\localeid].letters = true
6853   }%
6854 \fi
6855 \bbl@xin@{ ids }{ \bbl@KVP@onchar\space}%
6856 \ifin@
6857   \ifx\bbl@starthyphens\@undefined % Needed if no explicit selection
6858     \AddBabelHook{babel-onchar}{beforestart}{\bbl@starthyphens}}%
6859   \fi
6860   \bbl@exp{\bbl@add{\bbl@starthyphens
6861     {\bbl@patterns@lua{\language\language}}}%
6862   \directlua{
6863     if Babel.script_blocks['\bbl@cl{sbcp}'] then
6864       Babel.loc_to_scr[\the\localeid] = Babel.script_blocks['\bbl@cl{sbcp}']
6865       Babel.locale_props[\the\localeid].lg = \the\@nameuse{l@\language}\space
6866     end
6867   }%
6868 \fi
6869 \bbl@xin@{ fonts }{ \bbl@KVP@onchar\space}%
6870 \ifin@
6871   \bbl@ifunset{bbl@lsys@\language}{\bbl@provide@lsys{\language}}}%
6872   \bbl@ifunset{bbl@wdir@\language}{\bbl@provide@dirs{\language}}}%
6873   \directlua{
6874     if Babel.script_blocks['\bbl@cl{sbcp}'] then
6875       Babel.loc_to_scr[\the\localeid] =
6876         Babel.script_blocks['\bbl@cl{sbcp}']
6877     end}%
6878   \ifx\bbl@mapselect\@undefined
6879     \AtBeginDocument{%
6880       \bbl@patchfont{\bbl@mapselect}}%
6881     {\selectfont}}%
6882   \def\bbl@mapselect{%
6883     \let\bbl@mapselect\relax
6884     \edef\bbl@prefontid{\fontid\font}}%
6885   \def\bbl@mapdir##1{%
6886     \begingroup
6887       \setbox\z@\hbox{% Force text mode
6888         \def\language{##1}%
6889         \let\bbl@ifrestoring\@firstoftwo % To avoid font warning
6890         \bbl@switchfont
6891         \ifnum\fontid\font>\z@ % A hack, for the pgf nullfont hack
6892           \directlua{
6893             Babel.locale_props[\the\csname bbl@id@##1\endcsname]%
6894               ['/\bbl@prefontid'] = \fontid\font\space}%
6895           \fi}%
6896       \endgroup}%
6897   \fi
6898   \bbl@exp{\bbl@add{\bbl@mapselect{\bbl@mapdir{\language}}}}%
6899 \fi
6900 \fi
6901 % == mapfont ==
6902 % For bidi texts, to switch the font based on direction. Deprecated
6903 \ifx\bbl@KVP@mapfont\@nnil\else
6904   \bbl@ifsamestring{\bbl@KVP@mapfont}{direction}}}%

```

```

6905     {\bbl@error{unknown-mapfont}}{}{}{}%
6906 \bbl@ifunset{\bbl@lsys@{language}}{\bbl@provide@lsys{language}}{}{}%
6907 \bbl@ifunset{\bbl@wdir@{language}}{\bbl@provide@dirs{language}}{}{}%
6908 \ifx\bbl@mapselect\undefined
6909     \AtBeginDocument{%
6910         \bbl@patchfont{{\bbl@mapselect}}%
6911         {\selectfont}}%
6912     \def\bbl@mapselect{%
6913         \let\bbl@mapselect\relax
6914         \edef\bbl@prefontid{\fontid\font}}%
6915     \def\bbl@mapdir##1{%
6916         {\def\language{##1}%
6917         \let\bbl@ifrestoring\@firstoftwo % avoid font warning
6918         \bbl@switchfont
6919         \directlua{Babel.fontmap
6920         [\the\csname bbl@wdir@##1\endcsname]%
6921         [\bbl@prefontid]=\fontid\font}}}%
6922 \fi
6923 \bbl@exp{\bbl@add{\bbl@mapselect{\bbl@mapdir{language}}}}%
6924 \fi
6925 % == Line breaking: CJK quotes ==
6926 \ifcase\bbl@engine\or
6927     \bbl@xin{/c}{/\bbl@ccl{lnbrk}}%
6928     \ifin@
6929         \bbl@ifunset{\bbl@quote@{language}}{}%
6930         {\directlua{
6931             Babel.locale_props[\the\localeid].cjk_quotes = {}
6932             local cs = 'op'
6933             for c in string.utfvalues(
6934                 [[\csname bbl@quote@{language}\endcsname]]) do
6935                 if Babel.cjk_characters[c].c == 'qu' then
6936                     Babel.locale_props[\the\localeid].cjk_quotes[c] = cs
6937                 end
6938                 cs = ( cs == 'op') and 'cl' or 'op'
6939             end
6940         }}%
6941     \fi
6942 \fi
6943 % == Counters: mapdigits ==
6944 % Native digits
6945 \ifx\bbl@KVP@mapdigits\@nnil\else
6946     \bbl@ifunset{\bbl@dgnat@{language}}{}%
6947     {\bbl@activate@preotf
6948     \directlua{
6949         Babel.digits_mapped = true
6950         Babel.digits = Babel.digits or {}
6951         Babel.digits[\the\localeid] =
6952             table.pack(string.utfvalue('\bbl@ccl{dgnat}'))
6953         if not Babel.numbers then
6954             function Babel.numbers(head)
6955                 local LOCALE = Babel.attr_locale
6956                 local GLYPH = node.id'glyph'
6957                 local inmath = false
6958                 for item in node.traverse(head) do
6959                     if not inmath and item.id == GLYPH then
6960                         local temp = node.get_attribute(item, LOCALE)
6961                         if Babel.digits[temp] then
6962                             local chr = item.char
6963                             if chr > 47 and chr < 58 then
6964                                 item.char = Babel.digits[temp][chr-47]
6965                             end
6966                         end
6967                     elseif item.id == node.id'math' then

```

```

6968             inmath = (item.subtype == 0)
6969         end
6970     end
6971     return head
6972 end
6973 end
6974 }}%
6975 \fi
6976 % == transforms ==
6977 \ifx\bbk@KVP@transforms\@nnil\else
6978   \def\bbk@elt##1##2##3{%
6979     \in@{$transforms.}{$##1}%
6980     \ifin@
6981       \def\bbk@tempa{##1}%
6982       \bbk@replace\bbk@tempa{transforms.}{}%
6983       \bbk@carg\bbk@transforms{babel\bbk@tempa}{##2}{##3}%
6984     \fi}%
6985 \bbk@exp{%
6986   \\\bbk@ifblank{\bbk@cl{dgnat}}}%
6987   {\let\\bbk@tempa\relax}%
6988   {\def\\bbk@tempa{%
6989     \\\bbk@elt{transforms.prehyphenation}%
6990     {digits.native.1.0}{([0-9])}%
6991     \\\bbk@elt{transforms.prehyphenation}%
6992     {digits.native.1.1}{string={1|string|0123456789|string|\bbk@cl{dgnat}}}}}%
6993 \ifx\bbk@tempa\relax\else
6994   \toks@{\expandafter\expandafter\expandafter{%
6995     \csname \bbk@inidata@\language\endcsname}%
6996     \bbk@carg\edef{inidata@\language}{%
6997       \unexpanded\expandafter{\bbk@tempa}%
6998       \the\toks@}%
6999   \fi
7000   \csname \bbk@inidata@\language\endcsname
7001   \bbk@release@transforms\relax % \relax closes the last item.
7002 \fi}

```

Start tabular here:

```

7003 \def\localerestoredirs{%
7004   \ifcase\bbk@thetextdir
7005     \ifnum\textdirection=\z@\else\textdirection=\z@\fi
7006   \else
7007     \ifnum\textdirection=\@ne\else\textdirection=\@ne\fi
7008   \fi
7009   \ifcase\bbk@thepardir
7010     \ifnum\pardirection=\z@\else\pardirection=\z@\bodydirection=\z@\fi
7011   \else
7012     \ifnum\pardirection=\@ne\else\pardirection=\@ne\bodydirection=\@ne\fi
7013   \fi}
7014 %
7015 \IfBabelLayout{tabular}%
7016   {\chardef\bbk@tabular@mode\tw@}% All RTL
7017   {\IfBabelLayout{notabular}%
7018     {\chardef\bbk@tabular@mode\z@}%
7019     {\chardef\bbk@tabular@mode\@ne}}% Mixed, with LTR cols
7020 %
7021 \ifnum\bbk@bidimode>\@ne % Any lua bidi= except default=1
7022 % Redefine: vrules mess up dirs (why?).
7023 \AtBeginDocument{\def\@arstrut{\relax\copy\@arstrutbox}}%
7024 \ifcase\bbk@tabular@mode\or % 1 = Mixed - default
7025   \let\bbk@parabefore\relax
7026   \AddToHook{para/before}{\bbk@parabefore}
7027   \AtBeginDocument{%
7028     \bbk@replace\@tabular{$}{$%

```



```

7029     \def\bbl@insidemath{0}%
7030     \def\bbl@parabefore{\localerestoredirs}}%
7031     \ifnum\bbl@tabular@mode=\@ne
7032     \bbl@ifunset{@tabclassz}{}%
7033     \bbl@exp{% Hide conditionals
7034         \\bbl@sreplace\\@tabclassz
7035         {\<ifcase>\\@chnum}%
7036         {\\localerestoredirs\<ifcase>\\@chnum}}}%
7037     \@ifpackageloaded{colortbl}%
7038     {\bbl@sreplace\@classz
7039     {\hbox\bgroup\bgroup}{\hbox\bgroup\bgroup\localerestoredirs}}%
7040     {\@ifpackageloaded{array}%
7041     {\bbl@exp{% Hide conditionals
7042         \\bbl@sreplace\\@classz
7043         {\<ifcase>\\@chnum}%
7044         {\bgroup\\localerestoredirs\<ifcase>\\@chnum}%
7045         \\bbl@sreplace\\@classz
7046         {\\do@row@strut\<fi>}{\\do@row@strut\<fi>\egroup}}}%
7047     {}}%
7048     \fi}%
7049 \or % 2 = All RTL - tabular
7050 \let\bbl@parabefore\relax
7051 \AddToHook{para/before}{\bbl@parabefore}%
7052 \AtBeginDocument{%
7053     \@ifpackageloaded{colortbl}%
7054     {\bbl@replace\@tabular{$}{$}%
7055     \def\bbl@insidemath{0}%
7056     \def\bbl@parabefore{\localerestoredirs}}%
7057     \bbl@sreplace\@classz
7058     {\hbox\bgroup\bgroup}{\hbox\bgroup\bgroup\localerestoredirs}}%
7059     {}}%
7060 \fi

```

Very likely the `\output` routine must be patched in a quite general way to make sure the `\bodydir` is set to `\pagedir`. Note outside `\output` they can be different (and often are). For the moment, two *ad hoc* changes.

```

7061 \AtBeginDocument{%
7062     \@ifpackageloaded{multicol}%
7063     {\toks@ \expandafter{\multi@column@out}%
7064     \edef\multi@column@out{\bodydir\pagedir\the\toks@}}%
7065     {}}%
7066     \@ifpackageloaded{paracol}%
7067     {\edef\pcol@output{%
7068         \bodydir\pagedir\unexpanded\expandafter{\pcol@output}}}%
7069     {}}%
7070 \fi

```

Finish here if there in no layout.

```

7071 \ifx\bbl@opt@layout\@nnil\endinput\fi

```

OMEGA provided a companion to `\mathdir` (`\nextfakemath`) for those cases where we did not want it to be applied, so that the writing direction of the main text was left unchanged. `\bbl@nextfake` is an attempt to emulate it, because `luatex` has removed it without an alternative. Used in `tabular`, `\underline` and `\LaTeX`. Also, `\hangindent` does not honour direction changes by default, so we need to redefine `\@hangfrom`.

```

7072 \chardef\bbl@trace@vboxdir\z@
7073 \ifnum\bbl@bidimode>\z@ % Any bidi=
7074 \def\bbl@nextfake#1{% non-local changes, use always inside a group!
7075     \bbl@exp{%
7076         \mathdir\the\bodydir
7077         #1%         Once entered in math, set boxes to restore values
7078         \def\\bbl@insidemath{0}%
7079         \<ifmmode>%
7080         \everyvbox{%

```

```

7081         \the\everyvbox
7082         \bodydir\the\bodydir
7083         \mathdir\the\mathdir
7084         \everyhbox{\the\everyhbox}%
7085         \everyvbox{\the\everyvbox}}%
7086     \everyhbox{%
7087         \the\everyhbox
7088         \bodydir\the\bodydir
7089         \mathdir\the\mathdir
7090         \everyhbox{\the\everyhbox}%
7091         \everyvbox{\the\everyvbox}}%
7092     \<fi>}}%
7093 \IfBabelLayout{nopars}{}
7094 {\edef\bbl@opt@layout{\bbl@opt@layout.pars.}}%
7095 \IfBabelLayout{pars}
7096 {\chardef\bbl@trace@vboxdir\@ne
7097  \def\@hangfrom#1{%
7098    \setbox\@tempboxa\hbox{\{#1\}}%
7099    \hangindent\wd\@tempboxa
7100    \ifnum\bbl@vbox@bodydir=\pardirection\else
7101      \shapemode\@ne
7102    \fi
7103    \noindent\box\@tempboxa}}
7104 {}
7105 \fi
7106 %
7107 \IfBabelLayout{tabular}
7108 {\let\bbl@OL@@tabular\@tabular
7109  \bbl@exp{\in@\UseMathForPositioningText}{\@tabular}}%
7110  \ifin@
7111    \bbl@replace\@tabular{\UseMathForPositioningText}%
7112    {\bbl@nextfake{\UseMathForPositioningText}}%
7113  \else
7114    \bbl@replace\@tabular{\$}{\bbl@nextfake$}%
7115  \fi
7116  \let\bbl@NL@@tabular\@tabular
7117  \AtBeginDocument{%
7118    \ifx\bbl@NL@@tabular\@tabular\else
7119      \bbl@exp{\in@\UseMathForPositioningText}{\@tabular}}%
7120    \ifin@\else
7121      \ifin@
7122        \bbl@replace\@tabular{\UseMathForPositioningText}%
7123        {\bbl@nextfake{\UseMathForPositioningText}}%
7124      \else
7125        \bbl@replace\@tabular{\$}{\bbl@nextfake$}%
7126      \fi
7127    \fi
7128    \let\bbl@NL@@tabular\@tabular
7129    \fi}}
7130 {}

```

We need to patch lists in documents with both LTR and RTL paragraphs. See issue #395 in GitHub. There was a partial solution, but a better one has been devised by Udi Fogiel (in 26.4).

```

7131 \IfBabelLayout{lists}
7132 {\chardef\bbl@trace@vboxdir\@ne
7133  \let\bbl@OL@list\list
7134  \bbl@sreplace\list{\parshape}{\bbl@listparshape}%
7135  \let\bbl@NL@list\list
7136  \def\bbl@listparshape#1#2#3{%
7137    \parshape #1 #2 #3 %
7138    \ifnum\bbl@vbox@bodydir=\pardirection\else
7139      \shapemode\tw@
7140    \fi}}

```

```

7141 {}
7142 %
7143 \ifcase\bb@l@trace@vboxdir\else
7144 \AtBeginDocument{\chardef\bb@l@vbox@bodydir\pagedirection}%
7145 \def\bb@l@vbox@lists{\chardef\bb@l@vbox@bodydir\bodydirection}%
7146 \let\bb@l@bidi@vbox\everyvbox
7147 \@nameuse{newtoks}\everyvbox % \outer in Plain
7148 \everyvbox\expandafter{\the\bb@l@bidi@vbox}%
7149 \bb@l@bidi@vbox{\bb@l@vbox@lists\the\everyvbox}%
7150 \fi
7151 %
7152 \IfBabelLayout{graphics}
7153 {\let\bb@l@pictresetdir\relax
7154 \def\bb@l@pictsetdir#1{%
7155 \ifcase\bb@l@thetextdir
7156 \let\bb@l@pictresetdir\relax
7157 \else
7158 \ifcase#1\bodydir TLT % Remember this sets the inner boxes
7159 \or\textdir TLT
7160 \else\bodydir TLT \textdir TLT
7161 \fi
7162 % \(\text|par)dir required in pgf:
7163 \def\bb@l@pictresetdir{\bodydir TRT\pardir TRT\textdir TRT\relax}%
7164 \fi}%
7165 \AddToHook{env/picture/begin}{\bb@l@pictsetdir\tw@}%
7166 \directlua{
7167 Babel.get_picture_dir = true
7168 Babel.picture_has_bidi = 0
7169 %
7170 function Babel.picture_dir (head)
7171 if not Babel.get_picture_dir then return head end
7172 if Babel.hlist_has_bidi(head) then
7173 Babel.picture_has_bidi = 1
7174 end
7175 return head
7176 end
7177 luatexbase.add_to_callback("hpack_filter", Babel.picture_dir,
7178 "Babel.picture_dir")
7179 }%
7180 \AddToHook{package/graphics/after}{%
7181 \bb@l@exp{%
7182 \\\bb@l@sreplace\\rotatebox{\hbox{\string#2}}
7183 {\hbox bdir\z@\hbox bdir<ifcase>\bb@l@thetextdir\z@\<else>\@ne<fi>{\string#2}}}}
7184 \AddToHook{package/graphicx/after}{%
7185 \bb@l@exp{%
7186 \\\bb@l@sreplace\\Grot@box@std{\hbox{\string#2}}
7187 {\hbox bdir\z@\hbox bdir<ifcase>\bb@l@thetextdir\z@\<else>\@ne<fi>{\string#2}}}%
7188 \\\bb@l@sreplace\\Grot@box@kv{\hbox{\string#3}}
7189 {\hbox bdir\z@\hbox bdir<ifcase>\bb@l@thetextdir\z@\<else>\@ne<fi>{\string#3}}}%
7190 \\\bb@l@sreplace\\Grot@box{\hbox}{\hbox bdir\z@}}
7191 \AtBeginDocument{%
7192 \long\def\put(#1,#2)#3{%
7193 \@killglue
7194 % Try:
7195 \ifx\bb@l@pictresetdir\relax
7196 \def\bb@l@tempc{0}%
7197 \else
7198 \directlua{
7199 Babel.get_picture_dir = true
7200 Babel.picture_has_bidi = 0
7201 }%
7202 \setbox\z@\hb@xt@{\z@}%
7203 \@defaultunitsset@tempdimc{#1}\unitlength

```

```

7204         \kern\@tempdimc
7205         #3\hss}%
7206         \edef\bbl@tempc{\directlua{tex.print(Babel.picture_has_bidi)}}%
7207         \fi
7208         % Do:
7209         \@defaultunitsset\@tempdimc{#2}\unitlength
7210         \raise\@tempdimc\hb@xt@\z@{%
7211         \@defaultunitsset\@tempdimc{#1}\unitlength
7212         \kern\@tempdimc
7213         {\ifnum\bbl@tempc>\z@\bbl@pictresetdir\fi#3}\hss}%
7214         \ignorespaces}%
7215         \MakeRobust\put}%
7216     \AtBeginDocument
7217     {\AddToHook{cmd/diagbox@pict/before}{\let\bbl@pictsetdir\@gobble}%
7218     \ifx\pgfpicture\undefined\else
7219     \AddToHook{env/pgfpicture/begin}{\bbl@pictsetdir\@ne}%
7220     \bbl@add\pgfinterruptpicture{\bbl@pictresetdir}%
7221     \bbl@add\pgfsys@beginpicture{\bbl@pictsetdir\z@}%
7222     \fi
7223     \ifx\tikzpicture\undefined\else
7224     \AddToHook{env/tikzpicture/begin}{\bbl@pictsetdir\tw}%
7225     \bbl@add\tikz@atbegin@node{\bbl@pictresetdir}%
7226     \bbl@sreplace\tikz{\begingroup}{\begingroup\bbl@pictsetdir\tw}%
7227     \bbl@sreplace\tikzpicture{\begingroup}{\begingroup\bbl@pictsetdir\tw}%
7228     \fi
7229     }}
7230 {}

```

Implicitly reverses sectioning labels in bidi=basic-r, because the full stop is not in contact with L numbers any more. I think there must be a better way. Assumes bidi=basic, but there are some additional readjustments for bidi=default.

```

7231 \IfBabelLayout{counters*}%
7232 {\bbl@add\bbl@opt@layout{.counters.}%
7233 \directlua{
7234     luatexbase.add_to_callback("process_output_buffer",
7235     Babel.discard_sublr , "Babel.discard_sublr") }%
7236 }{}
7237 \IfBabelLayout{counters}%
7238 {\let\bbl@0L@@textsuperscript\@textsuperscript
7239 \bbl@sreplace\@textsuperscript{\m@th}{\m@th\mathdir\pagedir}%
7240 \let\bbl@latinarabic=\@arabic
7241 \let\bbl@0L@@arabic\@arabic
7242 \def\@arabic#1{\babelsublr{\bbl@latinarabic#1}}%
7243 \@ifpackagewith{babel}{bidi=default}%
7244 {\let\bbl@asciroman=\@roman
7245 \let\bbl@0L@@roman\@roman
7246 \def\@roman#1{\babelsublr{\ensureascii{\bbl@asciroman#1}}}%
7247 \let\bbl@asciRoman=\@Roman
7248 \let\bbl@0L@@Roman\@Roman
7249 \def\@Roman#1{\babelsublr{\ensureascii{\bbl@asciRoman#1}}}%
7250 \let\bbl@0L@labelenumii\labelenumii
7251 \def\labelenumii{}\theenumii}%
7252 \let\bbl@0L@p@enumiii\p@enumiii
7253 \def\p@enumiii{\p@enumii}\theenumii{}\}\}\}{}

```

Some $\TeX$ macros use internally the math mode for text formatting. They have very little in common and are grouped here, as a single option.

```

7254 \IfBabelLayout{extras}%
7255 {\bbl@ncarg\let\bbl@0L@underline{underline }%
7256 \bbl@carg\bbl@sreplace{underline }{\hbox}%
7257 {\def\bbl@insidemath{0}\hbox bdir\ifcase\bbl@thetextdir\z@else\@ne\fi}%
7258 \let\bbl@0L@LaTeXe\LaTeXe
7259 \DeclareRobustCommand{\LaTeXe}{\mbox{\m@th
7260 \if b\expandafter\@car\series\@nil\boldmath\fi

```

```

7261 \babelsublr{%
7262 \LaTeX\kern.15em2\bbl@nextfake$_{\textstyle\varepsilon}$}}
7263 {}
7264 </luatex>

```

10.13 Lua: transforms

After declaring the table containing the patterns with their replacements, we define some auxiliary functions: `str_to_nodes` converts the string returned by a function to a node list, taking the node at base as a model (font, language, etc.); `fetch_word` fetches a series of glyphs and discretionary, which pattern is matched against (if there is a match, it is called again before trying other patterns, and this is very likely the main bottleneck).

`post_hyphenate_replace` is the callback applied after `lang.hyphenate`. This means the automatic hyphenation points are known. As empty captures return a byte position (as explained in the `luatex` manual), we must convert it to a utf8 position. With `first`, the last byte can be the leading byte in a utf8 sequence, so we just remove it and add 1 to the resulting length. With `last` we must take into account the capture position points to the next character. Here `word_head` points to the starting node of the text to be matched.

```

7265 <{*transforms}>
7266 Babel.linebreaking.replacements = {}
7267 Babel.linebreaking.replacements[0] = {} -- pre
7268 Babel.linebreaking.replacements[1] = {} -- post
7269
7270 function Babel.tovalue(v)
7271   if type(v) == 'table' then
7272     return Babel.locale_props[v[1]].vars[v[2]] or v[3]
7273   else
7274     return v
7275   end
7276 end
7277
7278 Babel.attr_hboxed = luatexbase.registernumber'bbl@attr@hboxed'
7279
7280 function Babel.set_hboxed(head, gc)
7281   for item in node.traverse(head) do
7282     node.set_attribute(item, Babel.attr_hboxed, 1)
7283   end
7284   return head
7285 end
7286
7287 Babel.fetch_subtext = {}
7288
7289 Babel.ignore_pre_char = function(node)
7290   return (node.lang == Babel.nohyphenation)
7291 end
7292
7293 Babel.show_transforms = false
7294
7295 -- Merging both functions doesn't seem feasible, because there are too
7296 -- many differences.
7297 Babel.fetch_subtext[0] = function(head)
7298   local word_string = ''
7299   local word_nodes = {}
7300   local lang
7301   local item = head
7302   local inmath = false
7303
7304   while item do
7305     if item.id == 11 then
7306       inmath = (item.subtype == 0)
7307     end
7308     item = item.next
7309   end

```

```

7310     if inmath then
7311         -- pass
7312
7313     elseif item.id == 29 then
7314         local locale = node.get_attribute(item, Babel.attr_locale)
7315
7316         if lang == locale or lang == nil then
7317             lang = lang or locale
7318             if Babel.ignore_pre_char(item) then
7319                 word_string = word_string .. Babel.us_char
7320             else
7321                 if node.has_attribute(item, Babel.attr_hboxed) then
7322                     word_string = word_string .. Babel.us_char
7323                 else
7324                     word_string = word_string .. unicode.utf8.char(item.char)
7325                 end
7326             end
7327             word_nodes[#word_nodes+1] = item
7328         else
7329             break
7330         end
7331
7332     elseif item.id == 12 and item.subtype == 13 then
7333         if node.has_attribute(item, Babel.attr_hboxed) then
7334             word_string = word_string .. Babel.us_char
7335         else
7336             word_string = word_string .. ' '
7337         end
7338         word_nodes[#word_nodes+1] = item
7339
7340         -- Ignore leading unrecognized nodes, too.
7341         elseif word_string ~= '' then
7342             word_string = word_string .. Babel.us_char
7343             word_nodes[#word_nodes+1] = item -- Will be ignored
7344         end
7345
7346         item = item.next
7347     end
7348
7349     -- Here and above we remove some trailing chars but not the
7350     -- corresponding nodes. But they aren't accessed.
7351     if word_string:sub(-1) == ' ' then
7352         word_string = word_string:sub(1,-2)
7353     end
7354     if Babel.show_transforms then texio.write_nl(word_string) end
7355     word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7356     return word_string, word_nodes, item, lang
7357 end
7358
7359 Babel.fetch_subtext[1] = function(head)
7360     local word_string = ''
7361     local word_nodes = {}
7362     local lang
7363     local item = head
7364     local inmath = false
7365
7366     while item do
7367
7368         if item.id == 11 then
7369             inmath = (item.subtype == 0)
7370         end
7371
7372         if inmath then

```

```

7373     -- pass
7374
7375     elseif item.id == 29 then
7376         if item.lang == lang or lang == nil then
7377             lang = lang or item.lang
7378             if node.has_attribute(item, Babel.attr_hboxed) then
7379                 word_string = word_string .. Babel.us_char
7380             elseif (item.char == 124) or (item.char == 61) then -- not =, not |
7381                 word_string = word_string .. Babel.us_char
7382             else
7383                 word_string = word_string .. unicode.utf8.char(item.char)
7384             end
7385             word_nodes[#word_nodes+1] = item
7386         else
7387             break
7388         end
7389
7390     elseif item.id == 7 and item.subtype == 2 then
7391         if node.has_attribute(item, Babel.attr_hboxed) then
7392             word_string = word_string .. Babel.us_char
7393         else
7394             word_string = word_string .. '='
7395         end
7396         word_nodes[#word_nodes+1] = item
7397
7398     elseif item.id == 7 and item.subtype == 3 then
7399         if node.has_attribute(item, Babel.attr_hboxed) then
7400             word_string = word_string .. Babel.us_char
7401         else
7402             word_string = word_string .. '|'
7403         end
7404         word_nodes[#word_nodes+1] = item
7405
7406         -- (1) Go to next word if nothing was found, and (2) implicitly
7407         -- remove leading USs.
7408         elseif word_string == '' then
7409             -- pass
7410
7411         -- This is the responsible for splitting by words.
7412         elseif (item.id == 12 and item.subtype == 13) then
7413             break
7414
7415         else
7416             word_string = word_string .. Babel.us_char
7417             word_nodes[#word_nodes+1] = item -- Will be ignored
7418         end
7419
7420         item = item.next
7421     end
7422     if Babel.show_transforms then texio.write_nl(word_string) end
7423     word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7424     return word_string, word_nodes, item, lang
7425 end
7426
7427 function Babel.pre_hyphenate_replace(head)
7428     Babel.hyphenate_replace(head, 0)
7429 end
7430
7431 function Babel.post_hyphenate_replace(head)
7432     Babel.hyphenate_replace(head, 1)
7433 end
7434
7435 Babel.us_char = string.char(31)

```

```

7436
7437 function Babel.hyphenate_replace(head, mode)
7438     local u = unicode.utf8
7439     local lbkr = Babel.linebreaking.replacements[mode]
7440     local tovalue = Babel.tovalue
7441
7442     local word_head = head
7443
7444     if Babel.show_transforms then
7445         texio.write_nl('\n==== Showing ' .. (mode == 0 and 'pre' or 'post') .. 'hyphenation ====')
7446     end
7447
7448     while true do -- for each subtext block
7449
7450         local w, w_nodes, nw, lang = Babel.fetch_subtext[mode](word_head)
7451
7452         if Babel.debug then
7453             print()
7454             print((mode == 0) and '@@@<' or '@@@>', w)
7455         end
7456
7457         if nw == nil and w == '' then break end
7458
7459         if not lang then goto next end
7460         if not lbkr[lang] then goto next end
7461
7462         -- For each saved (pre|post)hyphenation. TODO. Reconsider how
7463         -- loops are nested.
7464         for k=1, #lbkr[lang] do
7465             local p = lbkr[lang][k].pattern
7466             local r = lbkr[lang][k].replace
7467             local attr = lbkr[lang][k].attr or -1
7468
7469             if Babel.debug then
7470                 print('*****', p, mode)
7471             end
7472
7473             -- This variable is set in some cases below to the first *byte*
7474             -- after the match, either as found by u.match (faster) or the
7475             -- computed position based on sc if w has changed.
7476             local last_match = 0
7477             local step = 0
7478
7479             -- For every match.
7480             while true do
7481                 if Babel.debug then
7482                     print('====')
7483                 end
7484                 local new -- used when inserting and removing nodes
7485                 local dummy_node -- used by after
7486
7487                 local matches = { u.match(w, p, last_match) }
7488
7489                 if #matches < 2 then break end
7490
7491                 -- Get and remove empty captures (with ())'s, which return a
7492                 -- number with the position), and keep actual captures
7493                 -- (from (...)), if any, in matches.
7494                 local first = table.remove(matches, 1)
7495                 local last = table.remove(matches, #matches)
7496                 -- Non re-fetched substrings may contain \31, which separates
7497                 -- subsubstrings.
7498                 if string.find(w:sub(first, last-1), Babel.us_char) then break end

```



```

7499
7500     local save_last = last -- with A()BC()D, points to D
7501
7502     -- Fix offsets, from bytes to unicode. Explained above.
7503     first = u.len(w:sub(1, first-1)) + 1
7504     last = u.len(w:sub(1, last-1)) -- now last points to C
7505
7506     -- This loop stores in a small table the nodes
7507     -- corresponding to the pattern. Used by 'data' to provide a
7508     -- predictable behavior with 'insert' (w_nodes is modified on
7509     -- the fly), and also access to 'remove'd nodes.
7510     local sc = first-1 -- Used below, too
7511     local data_nodes = {}
7512
7513     local enabled = true
7514     for q = 1, last-first+1 do
7515         data_nodes[q] = w_nodes[sc+q]
7516         if enabled
7517             and attr > -1
7518             and not node.has_attribute(data_nodes[q], attr)
7519         then
7520             enabled = false
7521         end
7522     end
7523
7524     -- This loop traverses the matched substring and takes the
7525     -- corresponding action stored in the replacement list.
7526     -- sc = the position in substr nodes / string
7527     -- rc = the replacement table index
7528     local rc = 0
7529
7530     ----- TODO. dummy_node?
7531     while rc < last-first+1 or dummy_node do -- for each replacement
7532         if Babel.debug then
7533             print('.....', rc + 1)
7534         end
7535         sc = sc + 1
7536         rc = rc + 1
7537
7538         if Babel.debug then
7539             Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7540             local ss = ''
7541             for itt in node.traverse(head) do
7542                 if itt.id == 29 then
7543                     ss = ss .. unicode.utf8.char(itt.char)
7544                 else
7545                     ss = ss .. '{' .. itt.id .. '}'
7546                 end
7547             end
7548             print('*****', ss)
7549         end
7550     end
7551
7552     local crep = r[rc]
7553     local item = w_nodes[sc]
7554     local item_base = item
7555     local placeholder = Babel.us_char
7556     local d
7557
7558     if crep and crep.data then
7559         item_base = data_nodes[crep.data]
7560     end
7561

```

```

7562     if crep then
7563         step = crep.step or step
7564     end
7565
7566     if crep and crep.after then
7567         crep.insert = true
7568         if dummy_node then
7569             item = dummy_node
7570         else -- TODO. if there is a node after?
7571             d = node.copy(item_base)
7572             head, item = node.insert_after(head, item, d)
7573             dummy_node = item
7574         end
7575     end
7576
7577     if crep and not crep.after and dummy_node then
7578         node.remove(head, dummy_node)
7579         dummy_node = nil
7580     end
7581
7582     if not enabled then
7583         last_match = save_last
7584         goto next
7585
7586     elseif crep and next(crep) == nil then -- = {}
7587         if step == 0 then
7588             last_match = save_last    -- Optimization
7589         else
7590             last_match = utf8.offset(w, sc+step)
7591         end
7592         goto next
7593
7594     elseif crep == nil or crep.remove then
7595         node.remove(head, item)
7596         table.remove(w_nodes, sc)
7597         w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7598         sc = sc - 1 -- Nothing has been inserted.
7599         last_match = utf8.offset(w, sc+1+step)
7600         goto next
7601
7602     elseif crep and crep.kashida then
7603         node.set_attribute(item,
7604             Babel.attr_kashida,
7605             crep.kashida)
7606         last_match = utf8.offset(w, sc+1+step)
7607         goto next
7608
7609     elseif crep and crep.string then
7610         local str = crep.string(matches)
7611         if str == '' then -- Gather with nil
7612             node.remove(head, item)
7613             table.remove(w_nodes, sc)
7614             w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7615             sc = sc - 1 -- Nothing has been inserted.
7616         else
7617             local loop_first = true
7618             for s in string.utfvalues(str) do
7619                 d = node.copy(item_base)
7620                 d.char = s
7621                 if loop_first then
7622                     loop_first = false
7623                     head, new = node.insert_before(head, item, d)
7624                     if sc == 1 then

```

```

7625         word_head = head
7626     end
7627     w_nodes[sc] = d
7628     w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc+1)
7629 else
7630     sc = sc + 1
7631     head, new = node.insert_before(head, item, d)
7632     table.insert(w_nodes, sc, new)
7633     w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc)
7634 end
7635 if Babel.debug then
7636     print('.....', 'str')
7637     Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7638 end
7639 end -- for
7640 node.remove(head, item)
7641 end -- if ''
7642 last_match = utf8.offset(w, sc+1+step)
7643 goto next
7644
7645 elseif mode == 1 and crep and (crep.pre or crep.no or crep.post) then
7646     d = node.new(7, 3) -- (disc, regular)
7647     d.pre = Babel.str_to_nodes(crep.pre, matches, item_base)
7648     d.post = Babel.str_to_nodes(crep.post, matches, item_base)
7649     d.replace = Babel.str_to_nodes(crep.no, matches, item_base)
7650     d.attr = item_base.attr
7651     if crep.pre == nil then -- TeXbook p96
7652         d.penalty = tovalue(crep.penalty) or tex.hyphenpenalty
7653     else
7654         d.penalty = tovalue(crep.penalty) or tex.exhyphenpenalty
7655     end
7656     placeholder = '|'
7657     head, new = node.insert_before(head, item, d)
7658
7659 elseif mode == 0 and crep and (crep.pre or crep.no or crep.post) then
7660     -- ERROR
7661
7662 elseif crep and crep.penalty then
7663     d = node.new(14, 0) -- (penalty, userpenalty)
7664     d.attr = item_base.attr
7665     d.penalty = tovalue(crep.penalty)
7666     head, new = node.insert_before(head, item, d)
7667
7668 elseif crep and crep.space then
7669     -- 655360 = 10 pt = 10 * 65536 sp
7670     d = node.new(12, 13) -- (glue, spaceskip)
7671     local quad = font.getfont(item_base.font).size or 655360
7672     node.setglue(d, tovalue(crep.space[1]) * quad,
7673                 tovalue(crep.space[2]) * quad,
7674                 tovalue(crep.space[3]) * quad)
7675     if mode == 0 then
7676         placeholder = ' '
7677     end
7678     head, new = node.insert_before(head, item, d)
7679
7680 elseif crep and crep.norule then
7681     -- 655360 = 10 pt = 10 * 65536 sp
7682     d = node.new(2, 3) -- (rule, empty) = \no*rule
7683     local quad = font.getfont(item_base.font).size or 655360
7684     d.width = tovalue(crep.norule[1]) * quad
7685     d.height = tovalue(crep.norule[2]) * quad
7686     d.depth = tovalue(crep.norule[3]) * quad
7687     head, new = node.insert_before(head, item, d)

```

```

7688
7689     elseif crep and crep.spacefactor then
7690         d = node.new(12, 13)      -- (glue, spaceskip)
7691         local base_font = font.getfont(item_base.font)
7692         node.setglue(d,
7693             tovalue(crep.spacefactor[1]) * base_font.parameters['space'],
7694             tovalue(crep.spacefactor[2]) * base_font.parameters['space_stretch'],
7695             tovalue(crep.spacefactor[3]) * base_font.parameters['space_shrink'])
7696         if mode == 0 then
7697             placeholder = ' '
7698         end
7699         head, new = node.insert_before(head, item, d)
7700
7701     elseif mode == 0 and crep and crep.space then
7702         -- ERROR
7703
7704     elseif crep and crep.kern then
7705         d = node.new(13, 1)      -- (kern, user)
7706         local quad = font.getfont(item_base.font).size or 655360
7707         d.attr = item_base.attr
7708         d.kern = tovalue(crep.kern) * quad
7709         head, new = node.insert_before(head, item, d)
7710
7711     elseif crep and crep.node then
7712         d = node.new(crep.node[1], crep.node[2])
7713         d.attr = item_base.attr
7714         head, new = node.insert_before(head, item, d)
7715
7716     end -- i.e., replacement cases
7717
7718     -- Shared by disc, space(factor), kern, node and penalty.
7719     if sc == 1 then
7720         word_head = head
7721     end
7722     if crep.insert then
7723         w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc)
7724         table.insert(w_nodes, sc, new)
7725         last = last + 1
7726     else
7727         w_nodes[sc] = d
7728         node.remove(head, item)
7729         w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc+1)
7730     end
7731
7732     last_match = utf8.offset(w, sc+1+step)
7733
7734     ::next::
7735
7736 end -- for each replacement
7737
7738 if Babel.show_transforms then texio.write_nl('> ' .. w) end
7739 if Babel.debug then
7740     print('.....', '/')
7741     Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7742 end
7743
7744 if dummy_node then
7745     node.remove(head, dummy_node)
7746     dummy_node = nil
7747 end
7748
7749 end -- for match
7750

```

```

7751     end -- for patterns
7752
7753     ::next::
7754     word_head = nw
7755 end -- for substring
7756
7757 if Babel.show_transforms then texio.write_nl(string.rep('-', 32) .. '\n') end
7758 return head
7759 end
7760
7761 -- This table stores capture maps, numbered consecutively
7762 Babel.capture_maps = {}
7763
7764 function Babel.esc_hex_to_char(h)
7765   if tex.getcatcode(tonumber(h, 16)) ~= 11 and
7766     tex.getcatcode(tonumber(h, 16)) ~= 12 then
7767     return string.format([[\\Uchar"%X ]], tonumber(h,16))
7768   else
7769     return unicode.utf8.char(tonumber(h, 16))
7770   end
7771 end
7772
7773 -- The following functions belong to the next macro
7774 function Babel.capture_func(key, cap)
7775   local ret = "[[" .. cap:gsub('{{([0-9])}}', "[ ]..m[%1]..[[[" .. "]]]"
7776   local cnt
7777   local u = unicode.utf8
7778   ret = u.gsub(ret, '{(%x%x%x%x+)}', '\\x01\\x04')
7779   ret, cnt = ret:gsub('{{([0-9])|([^[ ]+)|(.-)}}', Babel.capture_func_map)
7780   ret = u.gsub(ret, '\\x01(%x%x%x%x+)\x04', Babel.esc_hex_to_char)
7781   ret = ret:gsub("%[%]%%.", '')
7782   ret = ret:gsub("%.%[%]%%", '')
7783   return key .. [[=function(m) return ]] .. ret .. [[ end]]
7784 end
7785
7786 function Babel.capt_map(from, mapno)
7787   return Babel.capture_maps[mapno][from] or from
7788 end
7789
7790 -- Handle the {n|abc|ABC} syntax in captures
7791 function Babel.capture_func_map(capno, from, to)
7792   local u = unicode.utf8
7793   from = u.gsub(from, '\\x01(%x%x%x%x+)\x04',
7794     function (n)
7795       return u.char(tonumber(n, 16))
7796     end)
7797   to = u.gsub(to, '\\x01(%x%x%x%x+)\x04',
7798     function (n)
7799       return u.char(tonumber(n, 16))
7800     end)
7801   local froms = {}
7802   for s in string.utfcharacters(from) do
7803     table.insert(froms, s)
7804   end
7805   local cnt = 1
7806   table.insert(Babel.capture_maps, {})
7807   local mlen = table.getn(Babel.capture_maps)
7808   for s in string.utfcharacters(to) do
7809     Babel.capture_maps[mlen][froms[cnt]] = s
7810     cnt = cnt + 1
7811   end
7812   return "]]..Babel.capt_map(m[" .. capno .. "], " ..
7813     (mlen) .. " ).." .. "[["

```

```

7814 end
7815
7816 -- Create/Extend reversed sorted list of kashida weights:
7817 function Babel.capture_kashida(key, wt)
7818   wt = tonumber(wt)
7819   if Babel.kashida_wts then
7820     for p, q in ipairs(Babel.kashida_wts) do
7821       if wt == q then
7822         break
7823       elseif wt > q then
7824         table.insert(Babel.kashida_wts, p, wt)
7825         break
7826       elseif table.getn(Babel.kashida_wts) == p then
7827         table.insert(Babel.kashida_wts, wt)
7828       end
7829     end
7830   else
7831     Babel.kashida_wts = { wt }
7832   end
7833   return 'kashida = ' .. wt
7834 end
7835
7836 function Babel.capture_node(id, subtype)
7837   local sbt = 0
7838   for k, v in pairs(node.subtypes(id)) do
7839     if v == subtype then sbt = k end
7840   end
7841   return 'node = {' .. node.id(id) .. ', ' .. sbt .. '}'
7842 end
7843
7844 -- Experimental: applies prehyphenation transforms to a string (letters
7845 -- and spaces).
7846 function Babel.string_prehyphenation(str, locale)
7847   local n, head, last, res
7848   head = node.new(8, 0) -- dummy (hack just to start)
7849   last = head
7850   for s in string.utfvalues(str) do
7851     if s == 20 then
7852       n = node.new(12, 0)
7853     else
7854       n = node.new(29, 0)
7855       n.char = s
7856     end
7857     node.set_attribute(n, Babel.attr_locale, locale)
7858     last.next = n
7859     last = n
7860   end
7861   head = Babel.hyphenate_replace(head, 0)
7862   res = ''
7863   for n in node.traverse(head) do
7864     if n.id == 12 then
7865       res = res .. ' '
7866     elseif n.id == 29 then
7867       res = res .. unicode.utf8.char(n.char)
7868     end
7869   end
7870   tex.print(res)
7871 end
7872 </transforms>

```

10.14 Lua: Auto bidi with basic and basic-r

The file `babel-data-bidi.lua` currently only contains data. It is a large and boring file and it is not shown here (see the generated file), but here is a sample:

```
% [0x25]={d='et'},
% [0x26]={d='on'},
% [0x27]={d='on'},
% [0x28]={d='on', m=0x29},
% [0x29]={d='on', m=0x28},
% [0x2A]={d='on'},
% [0x2B]={d='es'},
% [0x2C]={d='cs'},
%
```

For the meaning of these codes, see the Unicode standard.

Now the `basic-r` bidi mode. One of the aims is to implement a fast and simple bidi algorithm, with a single loop. I managed to do it for R texts, with a second smaller loop for a special case. The code is still somewhat chaotic, but its behavior is essentially correct. I cannot resist copying the following text from Emacs `bidi.c` (which also attempts to implement the bidi algorithm with a single loop):

Arrrrgh!! The UAX#9 algorithm is too deeply entrenched in the assumption of batch-style processing [...]. May the fleas of a thousand camels infest the armpits of those who design supposedly general-purpose algorithms by looking at their own implementations, and fail to consider other possible implementations!

Well, it took me some time to guess what the batch rules in UAX#9 actually mean (in other word, *what* they do and *why*, and not only *how*), but I think (or I hope) I've managed to understand them.

In some sense, there are two bidi modes, one for numbers, and the other for text. Furthermore, setting just the direction in R text is not enough, because there are actually *two* R modes (set explicitly in Unicode with RLM and ALM). In babel the dir is set by a higher protocol based on the language/script, which in turn sets the correct dir (<l>, <r> or <al>).

From UAX#9: “Where available, markup should be used instead of the explicit formatting characters”. So, this simple version just ignores formatting characters. Actually, most of that annex is devoted to how to handle them.

BD14-BD16 are not implemented. Unicode (and the W3C) are making a great effort to deal with some special problematic cases in “streamed” plain text. I don’t think this is the way to go – particular issues should be fixed by a high level interface taking into account the needs of the document. And here is where `luatex` excels, because everything related to bidi writing is under our control.

```
7873 < *basic-r>
7874 Babel.bidi_enabled = true
7875
7876 require('babel-data-bidi.lua')
7877
7878 local characters = Babel.characters
7879 local ranges = Babel.ranges
7880
7881 local DIR = node.id("dir")
7882
7883 local function dir_mark(head, from, to, outer)
7884   dir = (outer == 'r') and 'TLT' or 'TRT' -- i.e., reverse
7885   local d = node.new(DIR)
7886   d.dir = '+' .. dir
7887   node.insert_before(head, from, d)
7888   d = node.new(DIR)
7889   d.dir = '-' .. dir
7890   node.insert_after(head, to, d)
7891 end
7892
7893 function Babel.bidi(head, ispar)
7894   local first_n, last_n          -- first and last char with nums
7895   local last_es                  -- an auxiliary 'last' used with nums
7896   local first_d, last_d          -- first and last char in L/R block
7897   local dir, dir_real
```

Next also depends on script/lang (<al>/<r>). To be set by babel. tex.pardir is dangerous, could be (re)set but it should be changed only in vmode. There are two strong's – strong = l/al/r and strong_lr = l/r (there must be a better way):

```

7898 local strong = ('TRT' == tex.pardir) and 'r' or 'l'
7899 local strong_lr = (strong == 'l') and 'l' or 'r'
7900 local outer = strong
7901
7902 local new_dir = false
7903 local first_dir = false
7904 local inmath = false
7905
7906 local last_lr
7907
7908 local type_n = ''
7909
7910 for item in node.traverse(head) do
7911   -- three cases: glyph, dir, otherwise
7912   if item.id == node.id'glyph'
7913     or (item.id == 7 and item.subtype == 2) then
7914     local itemchar
7915     if item.id == 7 and item.subtype == 2 then
7916       itemchar = item.replace.char
7917     else
7918       itemchar = item.char
7919     end
7920     local chardata = characters[itemchar]
7921     dir = chardata and chardata.d or nil
7922     if not dir then
7923       for nn, et in ipairs(ranges) do
7924         if itemchar < et[1] then
7925           break
7926         elseif itemchar <= et[2] then
7927           dir = et[3]
7928           break
7929         end
7930       end
7931     end
7932     dir = dir or 'l'
7933     if inmath then dir = ('TRT' == tex.mathdir) and 'r' or 'l' end

```

Next is based on the assumption babel sets the language *and* switches the script with its dir. We treat a language block as a separate Unicode sequence. The following piece of code is executed at the first glyph after a 'dir' node. We don't know the current language until then. This is not exactly true, as the math mode may insert explicit dirs in the node list, so, for the moment there is a hack by brute force (just above).

```

7936   if new_dir then
7937     attr_dir = 0
7938     for at in node.traverse(item.attr) do
7939       if at.number == Babel.attr_dir then
7940         attr_dir = at.value & 0x3
7941       end
7942     end
7943     if attr_dir == 1 then
7944       strong = 'r'
7945     elseif attr_dir == 2 then
7946       strong = 'al'
7947     else
7948       strong = 'l'
7949     end
7950     strong_lr = (strong == 'l') and 'l' or 'r'
7951     outer = strong_lr

```



```

7952     new_dir = false
7953 end
7954
7955 if dir == 'nsm' then dir = strong end          -- W1

```

Numbers. The dual $\langle al \rangle / \langle r \rangle$ system for R is somewhat cumbersome.

```

7956     dir_real = dir          -- We need dir_real to set strong below
7957 if dir == 'al' then dir = 'r' end -- W3

```

By W2, there are no $\langle en \rangle$ $\langle et \rangle$ $\langle es \rangle$ if strong == $\langle al \rangle$, only $\langle an \rangle$. Therefore, there are not $\langle et \rangle$ nor $\langle en \rangle$, W5 can be ignored, and W6 applied:

```

7958 if strong == 'al' then
7959     if dir == 'en' then dir = 'an' end          -- W2
7960     if dir == 'et' or dir == 'es' then dir = 'on' end -- W6
7961     strong_lr = 'r'                             -- W3
7962 end

```

Once finished the basic setup for glyphs, consider the two other cases: dir node and the rest.

```

7963 elseif item.id == node.id'dir' and not inmath then
7964     new_dir = true
7965     dir = nil
7966 elseif item.id == node.id'math' then
7967     inmath = (item.subtype == 0)
7968 else
7969     dir = nil          -- Not a char
7970 end

```

Numbers in R mode. A sequence of $\langle en \rangle$, $\langle et \rangle$, $\langle an \rangle$, $\langle es \rangle$ and $\langle cs \rangle$ is typeset (with some rules) in L mode. We store the starting and ending points, and only when anything different is found (including nil, i.e., a non-char), the textdir is set. This means you cannot insert, say, a whatsit, but this is what I would expect (with luacolor you may colorize some digits). Anyway, this behavior could be changed with a switch in the future. Note in the first branch only $\langle an \rangle$ is relevant if $\langle al \rangle$.

```

7971 if dir == 'en' or dir == 'an' or dir == 'et' then
7972     if dir ~= 'et' then
7973         type_n = dir
7974     end
7975     first_n = first_n or item
7976     last_n = last_es or item
7977     last_es = nil
7978 elseif dir == 'es' and last_n then -- W3+W6
7979     last_es = item
7980 elseif dir == 'cs' then          -- it's right - do nothing
7981 elseif first_n then -- & if dir = any but en, et, an, es, cs, inc nil
7982     if strong_lr == 'r' and type_n ~= '' then
7983         dir_mark(head, first_n, last_n, 'r')
7984     elseif strong_lr == 'l' and first_d and type_n == 'an' then
7985         dir_mark(head, first_n, last_n, 'r')
7986         dir_mark(head, first_d, last_d, outer)
7987         first_d, last_d = nil, nil
7988     elseif strong_lr == 'l' and type_n ~= '' then
7989         last_d = last_n
7990     end
7991     type_n = ''
7992     first_n, last_n = nil, nil
7993 end

```

R text in L, or L text in R. Order of dir_ mark's are relevant: d goes outside n, and therefore it's emitted after. See dir_mark to understand why (but is the nesting actually necessary or is a flat dir structure enough?). Only L, R (and AL) chars are taken into account – everything else, including spaces, whatsits, etc., are ignored:

```

7994 if dir == 'l' or dir == 'r' then
7995     if dir ~= outer then
7996         first_d = first_d or item
7997         last_d = item

```

```

7998     elseif first_d and dir ~= strong_lr then
7999         dir_mark(head, first_d, last_d, outer)
8000         first_d, last_d = nil, nil
8001     end
8002 end

```

Mirroring. Each chunk of text in a certain language is considered a “closed” sequence. If <r on r> and <l on l>, it’s clearly <r> and <l>, resp’tly, but with other combinations depends on outer. From all these, we select only those resolving <on> → <r>. At the beginning (when last_lr is nil) of an R text, they are mirrored directly. Numbers in R mode are processed. It should not be done, but it doesn’t hurt.

```

8003     if dir and not last_lr and dir ~= 'l' and outer == 'r' then
8004         item.char = characters[item.char] and
8005             characters[item.char].m or item.char
8006     elseif (dir or new_dir) and last_lr ~= item then
8007         local mir = outer .. strong_lr .. (dir or outer)
8008         if mir == 'rrr' or mir == 'lrr' or mir == 'rrl' or mir == 'rlr' then
8009             for ch in node.traverse(node.next(last_lr)) do
8010                 if ch == item then break end
8011                 if ch.id == node.id'glyph' and characters[ch.char] then
8012                     ch.char = characters[ch.char].m or ch.char
8013                 end
8014             end
8015         end
8016     end

```

Save some values for the next iteration. If the current node is ‘dir’, open a new sequence. Since dir could be changed, strong is set with its real value (dir_real).

```

8017     if dir == 'l' or dir == 'r' then
8018         last_lr = item
8019         strong = dir_real -- Don't search back - best save now
8020         strong_lr = (strong == 'l') and 'l' or 'r'
8021     elseif new_dir then
8022         last_lr = nil
8023     end
8024 end

```

Mirror the last chars if they are no directed. And make sure any open block is closed, too.

```

8025     if last_lr and outer == 'r' then
8026         for ch in node.traverse_id(node.id'glyph', node.next(last_lr)) do
8027             if characters[ch.char] then
8028                 ch.char = characters[ch.char].m or ch.char
8029             end
8030         end
8031     end
8032     if first_n then
8033         dir_mark(head, first_n, last_n, outer)
8034     end
8035     if first_d then
8036         dir_mark(head, first_d, last_d, outer)
8037     end

```

In boxes, the dir node could be added before the original head, so the actual head is the previous node.

```

8038     return node.prev(head) or head
8039 end
8040 </basic-r>

```

And here the Lua code for bidi=basic:

```

8041 <*basic>
8042 -- e.g., Babel.fontmap[1][<prefontid>]=<dirfontid>
8043
8044 Babel.fontmap = Babel.fontmap or {}
8045 Babel.fontmap[0] = {} -- l

```

```

8046 Babel.fontmap[1] = {}      -- r
8047 Babel.fontmap[2] = {}      -- al/an
8048
8049 -- To cancel mirroring. Also OML, OMS, U?
8050 Babel.symbol_fonts = Babel.symbol_fonts or {}
8051 Babel.symbol_fonts[font.id('tenln')] = true
8052 Babel.symbol_fonts[font.id('tenlnw')] = true
8053 Babel.symbol_fonts[font.id('tencirc')] = true
8054 Babel.symbol_fonts[font.id('tencircw')] = true
8055
8056 Babel.bidi_enabled = true
8057 Babel.mirroring_enabled = true
8058
8059 require('babel-data-bidi.lua')
8060
8061 local characters = Babel.characters
8062 local ranges = Babel.ranges
8063
8064 local DIR = node.id('dir')
8065 local GLYPH = node.id('glyph')
8066
8067 local function insert_implicit(head, state, outer)
8068   local new_state = state
8069   if state.sim and state.eim and state.sim ~= state.eim then
8070     dir = ((outer == 'r') and 'TLT' or 'TRT') -- i.e., reverse
8071     local d = node.new(DIR)
8072     d.dir = '+' .. dir
8073     node.insert_before(head, state.sim, d)
8074     local d = node.new(DIR)
8075     d.dir = '-' .. dir
8076     node.insert_after(head, state.eim, d)
8077   end
8078   new_state.sim, new_state.eim = nil, nil
8079   return head, new_state
8080 end
8081
8082 local function insert_numeric(head, state)
8083   local new
8084   local new_state = state
8085   if state.san and state.ean and state.san ~= state.ean then
8086     local d = node.new(DIR)
8087     d.dir = '+TLT'
8088     _, new = node.insert_before(head, state.san, d)
8089     if state.san == state.sim then state.sim = new end
8090     local d = node.new(DIR)
8091     d.dir = '-TLT'
8092     _, new = node.insert_after(head, state.ean, d)
8093     if state.ean == state.eim then state.eim = new end
8094   end
8095   new_state.san, new_state.ean = nil, nil
8096   return head, new_state
8097 end
8098
8099 local function glyph_not_symbol_font(node)
8100   if node.id == GLYPH then
8101     return not Babel.symbol_fonts[node.font]
8102   else
8103     return false
8104   end
8105 end
8106
8107 -- TODO - \hbox with an explicit dir can lead to wrong results
8108 -- <R \hbox dir TLT{<R>}> and <L \hbox dir TRT{<L>}>. A small attempt

```

```

8109 -- was made to improve the situation, but the problem is the 3-dir
8110 -- model in babel/Unicode and the 2-dir model in LuaTeX don't fit
8111 -- well.
8112
8113 function Babel.bidi(head, ispar, hdir)
8114   local d    -- d is used mainly for computations in a loop
8115   local prev_d = ''
8116   local new_d = false
8117
8118   local nodes = {}
8119   local outer_first = nil
8120   local inmath = false
8121
8122   local glue_d = nil
8123   local glue_i = nil
8124
8125   local has_en = false
8126   local first_et = nil
8127
8128   local has_hyperlink = false
8129
8130   local ATDIR = Babel.attr_dir
8131   local attr_d, temp
8132   local locale_d
8133
8134   local save_outer
8135   local locale_d = node.get_attribute(head, ATDIR)
8136   if locale_d then
8137     locale_d = locale_d & 0x3
8138     save_outer = (locale_d == 0 and 'l') or
8139                  (locale_d == 1 and 'r') or
8140                  (locale_d == 2 and 'al')
8141   elseif ispar then    -- Or error? Shouldn't happen
8142     -- when the callback is called, we are just _after_ the box,
8143     -- and the textdir is that of the surrounding text
8144     save_outer = ('TRT' == tex.pardir) and 'r' or 'l'
8145   else                -- Empty box
8146     save_outer = ('TRT' == hdir) and 'r' or 'l'
8147   end
8148   local outer = save_outer
8149   local last = outer
8150   -- 'al' is only taken into account in the first, current loop
8151   if save_outer == 'al' then save_outer = 'r' end
8152
8153   local fontmap = Babel.fontmap
8154
8155   for item in node.traverse(head) do
8156
8157     -- Mask: DxxxPPTT (Done, Pardir [0-2], Textdir [0-2])
8158     locale_d = node.get_attribute(item, ATDIR)
8159     node.set_attribute(item, ATDIR, 0x80)
8160
8161     -- In what follows, #node is the last (previous) node, because the
8162     -- current one is not added until we start processing the neutrals.
8163     -- three cases: glyph, dir, otherwise
8164     if glyph_not_symbol_font(item)
8165       or (item.id == 7 and item.subtype == 2) then
8166
8167       if locale_d == 0x80 then goto nextnode end
8168       locale_d = locale_d or ((save_outer=='l') and 0 or 1)
8169
8170       local d_font = nil
8171       local item_r

```

```

8172     if item.id == 7 and item.subtype == 2 then
8173         item_r = item.replace    -- automatic discs have just 1 glyph
8174     else
8175         item_r = item
8176     end
8177
8178     local chardata = characters[item_r.char]
8179     d = chardata and chardata.d or nil
8180     if not d or d == 'nsm' then
8181         for nn, et in ipairs(ranges) do
8182             if item_r.char < et[1] then
8183                 break
8184             elseif item_r.char <= et[2] then
8185                 if not d then d = et[3]
8186                 elseif d == 'nsm' then d_font = et[3]
8187                 end
8188                 break
8189             end
8190         end
8191     end
8192     d = d or 'l'
8193
8194     -- A short 'pause' in bidi for mapfont
8195     -- %%% TODO. move if fontmap here
8196     d_font = d_font or d
8197     d_font = (d_font == 'l' and 0) or
8198             (d_font == 'nsm' and 0) or
8199             (d_font == 'r' and 1) or
8200             (d_font == 'al' and 2) or
8201             (d_font == 'an' and 2) or nil
8202     if d_font and fontmap and fontmap[d_font][item_r.font] then
8203         item_r.font = fontmap[d_font][item_r.font]
8204     end
8205
8206     if new_d then
8207         table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8208         if inmath then
8209             attr_d = 0
8210         else
8211             attr_d = locale_d & 0x3
8212         end
8213         if attr_d == 1 then
8214             outer_first = 'r'
8215             last = 'r'
8216         elseif attr_d == 2 then
8217             outer_first = 'r'
8218             last = 'al'
8219         else
8220             outer_first = 'l'
8221             last = 'l'
8222         end
8223         outer = last
8224         has_en = false
8225         first_et = nil
8226         new_d = false
8227     end
8228
8229     if glue_d then
8230         if (d == 'l' and 'l' or 'r') ~= glue_d then
8231             table.insert(nodes, {glue_i, 'on', nil})
8232         end
8233         glue_d = nil
8234         glue_i = nil

```

```

8235     end
8236
8237 elseif item.id == DIR then
8238     d = nil
8239     new_d = true
8240
8241 elseif item.id == node.id'glue' and item.subtype == 13 then
8242     glue_d = d
8243     glue_i = item
8244     d = nil
8245
8246 elseif item.id == node.id'math' then
8247     inmath = (item.subtype == 0)
8248     if (locale_d & 0x3) == tex.mathdirection then
8249         temp = node.new("dir")
8250         if inmath then
8251             temp.dir = '+TLT'
8252             head = node.insert_after(head, item, temp)
8253         else
8254             temp.dir = '-TLT'
8255             head = node.insert_after(head, item, temp)
8256         end
8257     end
8258
8259 elseif item.id == 8 and item.subtype == 19 then
8260     has_hyperlink = true
8261
8262 else
8263     d = nil
8264 end
8265
8266 -- AL <= EN/ET/ES      -- W2 + W3 + W6
8267 if last == 'al' and d == 'en' then
8268     d = 'an'           -- W3
8269 elseif last == 'al' and (d == 'et' or d == 'es') then
8270     d = 'on'           -- W6
8271 end
8272
8273 -- EN + CS/ES + EN      -- W4
8274 if d == 'en' and #nodes >= 2 then
8275     if (nodes[#nodes][2] == 'es' or nodes[#nodes][2] == 'cs')
8276         and nodes[#nodes-1][2] == 'en' then
8277         nodes[#nodes][2] = 'en'
8278     end
8279 end
8280
8281 -- AN + CS + AN         -- W4 too, because uax9 mixes both cases
8282 if d == 'an' and #nodes >= 2 then
8283     if (nodes[#nodes][2] == 'cs')
8284         and nodes[#nodes-1][2] == 'an' then
8285         nodes[#nodes][2] = 'an'
8286     end
8287 end
8288
8289 -- ET/EN                -- W5 + W7->l / W6->on
8290 if d == 'et' then
8291     first_et = first_et or (#nodes + 1)
8292 elseif d == 'en' then
8293     has_en = true
8294     first_et = first_et or (#nodes + 1)
8295 elseif first_et then    -- d may be nil here !
8296     if has_en then
8297         if last == 'l' then

```

```

8298         temp = 'l'      -- W7
8299     else
8300         temp = 'en'      -- W5
8301     end
8302 else
8303     temp = 'on'          -- W6
8304 end
8305 for e = first_et, #nodes do
8306     if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8307 end
8308 first_et = nil
8309 has_en = false
8310 end
8311
8312 -- Force mathdir in math if ON (currently works as expected only
8313 -- with 'l')
8314
8315 if inmath and d == 'on' then
8316     d = ('TRT' == tex.mathdir) and 'r' or 'l'
8317 end
8318
8319 if d then
8320     if d == 'al' then
8321         d = 'r'
8322         last = 'al'
8323     elseif d == 'l' or d == 'r' then
8324         last = d
8325     end
8326     prev_d = d
8327     table.insert(nodes, {item, d, outer_first})
8328 end
8329
8330 outer_first = nil
8331
8332 ::nextnode::
8333
8334 end -- for each node
8335
8336 -- TODO -- repeated here in case EN/ET is the last node. Find a
8337 -- better way of doing things:
8338 if first_et then      -- dir may be nil here !
8339     if has_en then
8340         if last == 'l' then
8341             temp = 'l'      -- W7
8342         else
8343             temp = 'en'      -- W5
8344         end
8345     else
8346         temp = 'on'          -- W6
8347     end
8348     for e = first_et, #nodes do
8349         if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8350     end
8351 end
8352
8353 -- dummy node, to close things
8354 table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8355
8356 ----- NEUTRAL -----
8357
8358 outer = save_outer
8359 last = outer
8360

```

```

8361 local first_on = nil
8362
8363 for q = 1, #nodes do
8364     local item
8365
8366     local outer_first = nodes[q][3]
8367     outer = outer_first or outer
8368     last = outer_first or last
8369
8370     local d = nodes[q][2]
8371     if d == 'an' or d == 'en' then d = 'r' end
8372     if d == 'cs' or d == 'et' or d == 'es' then d = 'on' end --- W6
8373
8374     if d == 'on' then
8375         first_on = first_on or q
8376     elseif first_on then
8377         if last == d then
8378             temp = d
8379         else
8380             temp = outer
8381         end
8382         for r = first_on, q - 1 do
8383             nodes[r][2] = temp
8384             item = nodes[r][1] -- MIRRORING
8385             if Babel.mirroring_enabled and glyph_not_symbol_font(item)
8386                 and temp == 'r' and characters[item.char] then
8387                 local font_mode = ''
8388                 if item.font > 0 and font.fonts[item.font].properties then
8389                     font_mode = font.fonts[item.font].properties.mode
8390                 end
8391                 if font_mode ~= 'harf' and font_mode ~= 'plug' then
8392                     item.char = characters[item.char].m or item.char
8393                 end
8394             end
8395         end
8396         first_on = nil
8397     end
8398
8399     if d == 'r' or d == 'l' then last = d end
8400 end
8401
8402 ----- IMPLICIT, REORDER -----
8403
8404 outer = save_outer
8405 last = outer
8406
8407 local state = {}
8408 state.has_r = false
8409
8410 for q = 1, #nodes do
8411     local item = nodes[q][1]
8412
8413     outer = nodes[q][3] or outer
8414
8415     local d = nodes[q][2]
8416
8417     if d == 'nsm' then d = last end -- W1
8418     if d == 'en' then d = 'an' end
8419     local isdir = (d == 'r' or d == 'l')
8420
8421     if outer == 'l' and d == 'an' then
8422         state.san = state.san or item
8423     end

```



```

8424     state.ean = item
8425 elseif state.san then
8426     head, state = insert_numeric(head, state)
8427 end
8428
8429 if outer == 'l' then
8430     if d == 'an' or d == 'r' then      -- im -> implicit
8431         if d == 'r' then state.has_r = true end
8432         state.sim = state.sim or item
8433         state.eim = item
8434     elseif d == 'l' and state.sim and state.has_r then
8435         head, state = insert_implicit(head, state, outer)
8436     elseif d == 'l' then
8437         state.sim, state.eim, state.has_r = nil, nil, false
8438     end
8439 else
8440     if d == 'an' or d == 'l' then
8441         if nodes[q][3] then -- nil except after an explicit dir
8442             state.sim = item -- so we move sim 'inside' the group
8443         else
8444             state.sim = state.sim or item
8445         end
8446         state.eim = item
8447     elseif d == 'r' and state.sim then
8448         head, state = insert_implicit(head, state, outer)
8449     elseif d == 'r' then
8450         state.sim, state.eim = nil, nil
8451     end
8452 end
8453
8454 if isdir then
8455     last = d      -- Don't search back - best save now
8456 elseif d == 'on' and state.san then
8457     state.san = state.san or item
8458     state.ean = item
8459 end
8460
8461 end
8462
8463 head = node.prev(head) or head
8464 % \end{macrocode}
8465 %
8466 % Now direction nodes has been distributed with relation to characters
8467 % and spaces, we need to take into account \TeX-specific elements in
8468 % the node list, to move them at an appropriate place. Firstly, with
8469 % hyperlinks. Secondly, we avoid them between penalties and spaces, so
8470 % that the latter are still discardable.
8471 %
8472 % \begin{macrocode}
8473 --- FIXES ---
8474 if has_hyperlink then
8475     local flag, linking = 0, 0
8476     for item in node.traverse(head) do
8477         if item.id == DIR then
8478             if item.dir == '+TRT' or item.dir == '+TLT' then
8479                 flag = flag + 1
8480             elseif item.dir == '-TRT' or item.dir == '-TLT' then
8481                 flag = flag - 1
8482             end
8483         elseif item.id == 8 and item.subtype == 19 then
8484             linking = flag
8485         elseif item.id == 8 and item.subtype == 20 then
8486             if linking > 0 then

```

```

8487         if item.prev.id == DIR and
8488             (item.prev.dir == '-TRT' or item.prev.dir == '-TLT') then
8489             d = node.new(DIR)
8490             d.dir = item.prev.dir
8491             node.remove(head, item.prev)
8492             node.insert_after(head, item, d)
8493         end
8494     end
8495     linking = 0
8496 end
8497 end
8498 end
8499
8500 for item in node.traverse_id(10, head) do
8501     local p = item
8502     local flag = false
8503     while p.prev and p.prev.id == 14 do
8504         flag = true
8505         p = p.prev
8506     end
8507     if flag then
8508         node.insert_before(head, p, node.copy(item))
8509         node.remove(head, item)
8510     end
8511 end
8512
8513 return head
8514 end
8515
8515 function Babel.unset_atdir(head)
8516     local ATDIR = Babel.attr_dir
8517     for item in node.traverse(head) do
8518         node.set_attribute(item, ATDIR, 0x80)
8519     end
8520     return head
8521 end
8522 </basic>

```

11. Data for CJK

It is a boring file and it is not shown here (see the generated file), but here is a sample:

```

% [0x0021]={c='ex'},
% [0x0024]={c='pr'},
% [0x0025]={c='po'},
% [0x0028]={c='op'},
% [0x0029]={c='cp'},
% [0x002B]={c='pr'},
%

```

For the meaning of these codes, see the Unicode standard.

12. The ‘nil’ language

This ‘language’ does nothing, except setting the hyphenation patterns to nohyphenation. For this language currently no special definitions are needed or available.

The macro `\LdfInit` takes care of preventing that this file is loaded more than once, checking the category code of the `@sign`, etc.

```

8523 < *nil>
8524 \ProvidesLanguage{nil}[<@date@> v<@version@> Nil language]
8525 \LdfInit{nil}{datenil}

```

When this file is read as an option, i.e., by the `\usepackage` command, `nil` could be an ‘unknown’ language in which case we have to make it known.

```
8526 \ifx\l@nil\@undefined
8527   \newlanguage\l@nil
8528   \@namedef{bbl@hyphendata@the\l@nil}{\{}}% Remove warning
8529   \let\bbl@elt\relax
8530   \edef\bbl@languages{% Add it to the list of languages
8531     \bbl@languages\bbl@elt{nil}{\the\l@nil}{\{}}
8532 \fi
```

This macro is used to store the values of the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`.

```
8533 \providehyphenmins{\CurrentOption}{\m@ne\m@ne}
```

The next step consists of defining commands to switch to (and from) the ‘nil’ language.

\captionnil

\datenil

```
8534 \let\captionnil\@empty
8535 \let\datenil\@empty
```

There is no locale file for this pseudo-language, so the corresponding fields are defined here.

```
8536 \def\bbl@inidata@nil{%
8537   \bbl@elt{identification}{tag.ini}{und}%
8538   \bbl@elt{identification}{load.level}{0}%
8539   \bbl@elt{identification}{charset}{utf8}%
8540   \bbl@elt{identification}{version}{1.0}%
8541   \bbl@elt{identification}{date}{2022-05-16}%
8542   \bbl@elt{identification}{name.local}{nil}%
8543   \bbl@elt{identification}{name.english}{nil}%
8544   \bbl@elt{identification}{name.babel}{nil}%
8545   \bbl@elt{identification}{tag.bcp47}{und}%
8546   \bbl@elt{identification}{language.tag.bcp47}{und}%
8547   \bbl@elt{identification}{tag.opentype}{dflt}%
8548   \bbl@elt{identification}{script.name}{Latin}%
8549   \bbl@elt{identification}{script.tag.bcp47}{Latn}%
8550   \bbl@elt{identification}{script.tag.opentype}{DFLT}%
8551   \bbl@elt{identification}{level}{1}%
8552   \bbl@elt{identification}{encodings}{}%
8553   \bbl@elt{identification}{derivate}{no}}
8554 \@namedef{bbl@tbcpl@nil}{und}
8555 \@namedef{bbl@lbcpl@nil}{und}
8556 \@namedef{bbl@casing@nil}{und}
8557 \@namedef{bbl@lotf@nil}{dflt}
8558 \@namedef{bbl@elname@nil}{nil}
8559 \@namedef{bbl@lname@nil}{nil}
8560 \@namedef{bbl@esname@nil}{Latin}
8561 \@namedef{bbl@sname@nil}{Latin}
8562 \@namedef{bbl@sbcpl@nil}{Latn}
8563 \@namedef{bbl@sotf@nil}{latn}
```

The macro `\ldf@finish` takes care of looking for a configuration file, setting the main language to be switched on at `\begin{document}` and resetting the category code of `@` to its original value.

```
8564 \ldf@finish{nil}
8565 </nil>
```

13. Calendars

The code for specific calendars are placed in the specific files, loaded when requested by an ini file in the identification section with `require.calendars`.

Start with function to compute the Julian day. It’s based on the little library `calendar.js`, by John Walker, in the public domain.

```

8566 <<*Compute Julian day>> ≡
8567 \def\bbl@fpmo#1#2{(#1-#2*floor(#1/#2))}
8568 \def\bbl@cs@gregleap#1{%
8569   (\bbl@fpmo{#1}{4} == 0) &&
8570   (!((\bbl@fpmo{#1}{100} == 0) && (\bbl@fpmo{#1}{400} != 0)))}
8571 \def\bbl@cs@jd#1#2#3{ % year, month, day
8572   \fpeval{ 1721424.5 + (365 * (#1 - 1)) +
8573     floor((#1 - 1) / 4) + (-floor((#1 - 1) / 100)) +
8574     floor((#1 - 1) / 400) + floor(((367 * #2) - 362) / 12) +
8575     ((#2 <= 2) ? 0 : (\bbl@cs@gregleap{#1} ? -1 : -2)) + #3 } }
8576 <</Compute Julian day>>

```

13.1. Islamic

The code for the Civil calendar is based on it, too.

```

8577 <*ca-islamic>
8578 <@Compute Julian day@>
8579 % == islamic (default)
8580 % Not yet implemented
8581 \def\bbl@ca@islamic#1-#2-#3\@#4#5#6{

```

The Civil calendar.

```

8582 \def\bbl@cs@isltojd#1#2#3{ % year, month, day
8583   ((#3 + ceil(29.5 * (#2 - 1)) +
8584     (#1 - 1) * 354 + floor((3 + (11 * #1)) / 30) +
8585     1948439.5) - 1) }
8586 \namedef\bbl@ca@islamic-civil++{\bbl@ca@islamicvl@x{+2}}
8587 \namedef\bbl@ca@islamic-civil+{\bbl@ca@islamicvl@x{+1}}
8588 \namedef\bbl@ca@islamic-civil{\bbl@ca@islamicvl@x{}}
8589 \namedef\bbl@ca@islamic-civil-{\bbl@ca@islamicvl@x{-1}}
8590 \namedef\bbl@ca@islamic-civil--{\bbl@ca@islamicvl@x{-2}}
8591 \def\bbl@ca@islamicvl@x#1#2-#3-#4\@#5#6#7{%
8592   \edef\bbl@tempa{%
8593     \fpeval{ floor(\bbl@cs@jd{#2}{#3}{#4})+0.5 #1}}%
8594   \edef#5{%
8595     \fpeval{ floor(((30*(\bbl@tempa-1948439.5)) + 10646)/10631) }}%
8596   \edef#6{\fpeval{
8597     min(12, ceil((\bbl@tempa - (29+\bbl@cs@isltojd{#5}{1}{1}))/29.5)+1) }}%
8598   \edef#7{\fpeval{ \bbl@tempa - \bbl@cs@isltojd{#5}{#6}{1} + 1} }}

```

The Umm al-Qura calendar, used mainly in Saudi Arabia, is based on moment-hijri, by Abdullah Alsigar (license MIT).

Since the main aim is to provide a suitable \today, and maybe some close dates, data just covers Hijri ~1435/~1460 (Gregorian ~2014/~2038).

```

8599 \def\bbl@cs@umalqura@data{56660, 56690,56719,56749,56778,56808,%
8600 56837,56867,56897,56926,56956,56985,57015,57044,57074,57103,%
8601 57133,57162,57192,57221,57251,57280,57310,57340,57369,57399,%
8602 57429,57458,57487,57517,57546,57576,57605,57634,57664,57694,%
8603 57723,57753,57783,57813,57842,57871,57901,57930,57959,57989,%
8604 58018,58048,58077,58107,58137,58167,58196,58226,58255,58285,%
8605 58314,58343,58373,58402,58432,58461,58491,58521,58551,58580,%
8606 58610,58639,58669,58698,58727,58757,58786,58816,58845,58875,%
8607 58905,58934,58964,58994,59023,59053,59082,59111,59141,59170,%
8608 59200,59229,59259,59288,59318,59348,59377,59407,59436,59466,%
8609 59495,59525,59554,59584,59613,59643,59672,59702,59731,59761,%
8610 59791,59820,59850,59879,59909,59939,59968,59997,60027,60056,%
8611 60086,60115,60145,60174,60204,60234,60264,60293,60323,60352,%
8612 60381,60411,60440,60469,60499,60528,60558,60588,60618,60648,%
8613 60677,60707,60736,60765,60795,60824,60853,60883,60912,60942,%
8614 60972,61002,61031,61061,61090,61120,61149,61179,61208,61237,%
8615 61267,61296,61326,61356,61385,61415,61445,61474,61504,61533,%
8616 61563,61592,61621,61651,61680,61710,61739,61769,61799,61828,%
8617 61858,61888,61917,61947,61976,62006,62035,62064,62094,62123,%

```

```

8618 62153,62182,62212,62242,62271,62301,62331,62360,62390,62419,%
8619 62448,62478,62507,62537,62566,62596,62625,62655,62685,62715,%
8620 62744,62774,62803,62832,62862,62891,62921,62950,62980,63009,%
8621 63039,63069,63099,63128,63157,63187,63216,63246,63275,63305,%
8622 63334,63363,63393,63423,63453,63482,63512,63541,63571,63600,%
8623 63630,63659,63689,63718,63747,63777,63807,63836,63866,63895,%
8624 63925,63955,63984,64014,64043,64073,64102,64131,64161,64190,%
8625 64220,64249,64279,64309,64339,64368,64398,64427,64457,64486,%
8626 64515,64545,64574,64603,64633,64663,64692,64722,64752,64782,%
8627 64811,64841,64870,64899,64929,64958,64987,65017,65047,65076,%
8628 65106,65136,65166,65195,65225,65254,65283,65313,65342,65371,%
8629 65401,65431,65460,65490,65520}
8630 \@namedef{bbl@ca@islamic-umalqura+}{\bbl@ca@islamcuqr@x{+1}}
8631 \@namedef{bbl@ca@islamic-umalqura}{\bbl@ca@islamcuqr@x{}}
8632 \@namedef{bbl@ca@islamic-umalqura-}{\bbl@ca@islamcuqr@x{-1}}
8633 \def\bbl@ca@islamcuqr@x#1#2-#3-#4\@#5#6#7{%
8634 \ifnum#2>2014 \ifnum#2<2038
8635 \bbl@afterfi\expandafter\@gobble
8636 \fi\fi
8637 {\bbl@error{year-out-range}{2014-2038}}{}}%
8638 \edef\bbl@tempd{\fpeval{ % (Julian) day
8639 \bbl@cs@jd{#2}{#3}{#4} + 0.5 - 2400000 #1}}%
8640 \count@\@ne
8641 \bbl@foreach\bbl@cs@umalqura@data{%
8642 \advance\count@\@ne
8643 \ifnum##1>\bbl@tempd\else
8644 \edef\bbl@tempe{\the\count@}%
8645 \edef\bbl@tempb{##1}%
8646 \fi}%
8647 \edef\bbl@templ{\fpeval{ \bbl@tempe + 16260 + 949 }}% month~lunar
8648 \edef\bbl@tempa{\fpeval{ floor((\bbl@templ - 1 ) / 12) }}% annus
8649 \edef#5{\fpeval{ \bbl@tempa + 1 }}%
8650 \edef#6{\fpeval{ \bbl@templ - (12 * \bbl@tempa) }}%
8651 \edef#7{\fpeval{ \bbl@tempd - \bbl@tempb + 1 }}%
8652 \bbl@add\bbl@precalendar{%
8653 \bbl@replace\bbl@ld@calendar{-civil}}}%
8654 \bbl@replace\bbl@ld@calendar{-umalqura}}}%
8655 \bbl@replace\bbl@ld@calendar{+}}}%
8656 \bbl@replace\bbl@ld@calendar{-}}}%
8657 </ca-islamic>

```

13.2. Hebrew

This is basically the set of macros written by Michail Rozman in 1991, with corrections and adaptations by Rama Porrat, Misha, Dan Haran and Boris Lavva. This must be eventually replaced by computations with l3fp. An explanation of what's going on can be found in hebcal.sty

```

8658 <*ca-hebrew>
8659 \newcount\bbl@cntcommon
8660 \def\bbl@remainder#1#2#3{%
8661 #3=#1\relax
8662 \divide #3 by #2\relax
8663 \multiply #3 by -#2\relax
8664 \advance #3 by #1\relax}%
8665 \newif\ifbbl@divisible
8666 \def\bbl@checkifdivisible#1#2{%
8667 {\countdef\tmp=0
8668 \bbl@remainder{#1}{#2}{\tmp}%
8669 \ifnum \tmp=0
8670 \global\bbl@divisibletrue
8671 \else
8672 \global\bbl@divisiblefalse
8673 \fi}}
8674 \newif\ifbbl@gregleap

```

```

8675 \def\bbl@ifggregleap#1{%
8676   \bbl@checkifdivisible{#1}{4}%
8677   \ifbbl@divisible
8678     \bbl@checkifdivisible{#1}{100}%
8679     \ifbbl@divisible
8680       \bbl@checkifdivisible{#1}{400}%
8681       \ifbbl@divisible
8682         \bbl@gregleaptrue
8683       \else
8684         \bbl@gregleapfalse
8685     \fi
8686   \else
8687     \bbl@gregleaptrue
8688   \fi
8689 \else
8690   \bbl@gregleapfalse
8691 \fi
8692 \ifbbl@gregleap}
8693 \def\bbl@gregdayspriormonths#1#2#3{%
8694   {#3=\ifcase #1 0 \or 0 \or 31 \or 59 \or 90 \or 120 \or 151 \or
8695     181 \or 212 \or 243 \or 273 \or 304 \or 334 \fi
8696   \bbl@ifggregleap{#2}%
8697   \ifnum #1 > 2
8698     \advance #3 by 1
8699   \fi
8700 \fi
8701 \global\bbl@cntcommon=#3}%
8702 #3=\bbl@cntcommon}
8703 \def\bbl@gregdaysprioryears#1#2{%
8704   {\countdef\tmpc=4
8705   \countdef\tmpb=2
8706   \tmpb=#1\relax
8707   \advance \tmpb by -1
8708   \tmpc=\tmpb
8709   \multiply \tmpc by 365
8710   #2=\tmpc
8711   \tmpc=\tmpb
8712   \divide \tmpc by 4
8713   \advance #2 by \tmpc
8714   \tmpc=\tmpb
8715   \divide \tmpc by 100
8716   \advance #2 by -\tmpc
8717   \tmpc=\tmpb
8718   \divide \tmpc by 400
8719   \advance #2 by \tmpc
8720   \global\bbl@cntcommon=#2\relax}%
8721   #2=\bbl@cntcommon}
8722 \def\bbl@absfromgreg#1#2#3#4{%
8723   {\countdef\tmpd=0
8724   #4=#1\relax
8725   \bbl@gregdayspriormonths{#2}{#3}{\tmpd}%
8726   \advance #4 by \tmpd
8727   \bbl@gregdaysprioryears{#3}{\tmpd}%
8728   \advance #4 by \tmpd
8729   \global\bbl@cntcommon=#4\relax}%
8730   #4=\bbl@cntcommon}
8731 \newif\ifbbl@hebrleap
8732 \def\bbl@checkleaphebryear#1{%
8733   {\countdef\tmpa=0
8734   \countdef\tmpb=1
8735   \tmpa=#1\relax
8736   \multiply \tmpa by 7
8737   \advance \tmpa by 1

```

```

8738 \bbl@remainder{\tmpa}{19}{\tmpb}%
8739 \ifnum \tmpb < 7
8740 \global\bbl@hebrleaptrue
8741 \else
8742 \global\bbl@hebrleapfalse
8743 \fi}}
8744 \def\bbl@hebrrelapsedmonths#1#2{%
8745 {\countdef\tmpa=0
8746 \countdef\tmpb=1
8747 \countdef\tmpc=2
8748 \tmpa=#1\relax
8749 \advance \tmpa by -1
8750 #2=\tmpa
8751 \divide #2 by 19
8752 \multiply #2 by 235
8753 \bbl@remainder{\tmpa}{19}{\tmpb}% \tmpa=years%19-years this cycle
8754 \tmpc=\tmpb
8755 \multiply \tmpb by 12
8756 \advance #2 by \tmpb
8757 \multiply \tmpc by 7
8758 \advance \tmpc by 1
8759 \divide \tmpc by 19
8760 \advance #2 by \tmpc
8761 \global\bbl@cntcommon=#2}%
8762 #2=\bbl@cntcommon}
8763 \def\bbl@hebrrelapseddays#1#2{%
8764 {\countdef\tmpa=0
8765 \countdef\tmpb=1
8766 \countdef\tmpc=2
8767 \bbl@hebrrelapsedmonths{#1}{#2}%
8768 \tmpa=#2\relax
8769 \multiply \tmpa by 13753
8770 \advance \tmpa by 5604
8771 \bbl@remainder{\tmpa}{25920}{\tmpc}% \tmpc == ConjunctionParts
8772 \divide \tmpa by 25920
8773 \multiply #2 by 29
8774 \advance #2 by 1
8775 \advance #2 by \tmpa
8776 \bbl@remainder{#2}{7}{\tmpa}%
8777 \ifnum \tmpc < 19440
8778 \ifnum \tmpc < 9924
8779 \else
8780 \ifnum \tmpa=2
8781 \bbl@checkleaphebrewyear{#1}% of a common year
8782 \ifbbl@hebrleap
8783 \else
8784 \advance #2 by 1
8785 \fi
8786 \fi
8787 \fi
8788 \ifnum \tmpc < 16789
8789 \else
8790 \ifnum \tmpa=1
8791 \advance #1 by -1
8792 \bbl@checkleaphebrewyear{#1}% at the end of leap year
8793 \ifbbl@hebrleap
8794 \advance #2 by 1
8795 \fi
8796 \fi
8797 \fi
8798 \else
8799 \advance #2 by 1
8800 \fi

```

```

8801 \bbl@remainder{#2}{7}{\tmpa}%
8802 \ifnum \tmpa=0
8803 \advance #2 by 1
8804 \else
8805 \ifnum \tmpa=3
8806 \advance #2 by 1
8807 \else
8808 \ifnum \tmpa=5
8809 \advance #2 by 1
8810 \fi
8811 \fi
8812 \fi
8813 \global\bbl@cntcommon=#2\relax}%
8814 #2=\bbl@cntcommon}
8815 \def\bbl@daysinhebrewyear#1#2{%
8816 {\countdef\tmpe=12
8817 \bbl@hebreleapseddays{#1}{\tmpe}%
8818 \advance #1 by 1
8819 \bbl@hebreleapseddays{#1}{#2}%
8820 \advance #2 by -\tmpe
8821 \global\bbl@cntcommon=#2}%
8822 #2=\bbl@cntcommon}
8823 \def\bbl@hebrdayspriormonths#1#2#3{%
8824 {\countdef\tmpf= 14
8825 #3=\ifcase #1
8826 0 \or
8827 0 \or
8828 30 \or
8829 59 \or
8830 89 \or
8831 118 \or
8832 148 \or
8833 148 \or
8834 177 \or
8835 207 \or
8836 236 \or
8837 266 \or
8838 295 \or
8839 325 \or
8840 400
8841 \fi
8842 \bbl@checkleaphebrewyear{#2}%
8843 \ifbbl@hebrleap
8844 \ifnum #1 > 6
8845 \advance #3 by 30
8846 \fi
8847 \fi
8848 \bbl@daysinhebrewyear{#2}{\tmpf}%
8849 \ifnum #1 > 3
8850 \ifnum \tmpf=353
8851 \advance #3 by -1
8852 \fi
8853 \ifnum \tmpf=383
8854 \advance #3 by -1
8855 \fi
8856 \fi
8857 \ifnum #1 > 2
8858 \ifnum \tmpf=355
8859 \advance #3 by 1
8860 \fi
8861 \ifnum \tmpf=385
8862 \advance #3 by 1
8863 \fi

```



```

8864 \fi
8865 \global\bbl@cntcommon=#3\relax}%
8866 #3=\bbl@cntcommon}
8867 \def\bbl@absfromhebr#1#2#3#4{%
8868 {#4=#1\relax
8869 \bbl@hebrdayspriormonths{#2}{#3}{#1}%
8870 \advance #4 by #1\relax
8871 \bbl@hebreleaseddays{#3}{#1}%
8872 \advance #4 by #1\relax
8873 \advance #4 by -1373429
8874 \global\bbl@cntcommon=#4\relax}%
8875 #4=\bbl@cntcommon}
8876 \def\bbl@hebrfromgreg#1#2#3#4#5#6{%
8877 {\countdef\tmpx= 17
8878 \countdef\tmpy= 18
8879 \countdef\tmpz= 19
8880 #6=#3\relax
8881 \global\advance #6 by 3761
8882 \bbl@absfromgreg{#1}{#2}{#3}{#4}%
8883 \tmpz=1 \tmpy=1
8884 \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
8885 \ifnum \tmpx > #4\relax
8886 \global\advance #6 by -1
8887 \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
8888 \fi
8889 \advance #4 by -\tmpx
8890 \advance #4 by 1
8891 #5=#4\relax
8892 \divide #5 by 30
8893 \loop
8894 \bbl@hebrdayspriormonths{#5}{#6}{\tmpx}%
8895 \ifnum \tmpx < #4\relax
8896 \advance #5 by 1
8897 \tmpy=\tmpx
8898 \repeat
8899 \global\advance #5 by -1
8900 \global\advance #4 by -\tmpy}}
8901 \newcount\bbl@hebrday \newcount\bbl@hebrmonth \newcount\bbl@hebyear
8902 \newcount\bbl@gregday \newcount\bbl@gregmonth \newcount\bbl@gregyear
8903 \def\bbl@ca@hebrew#1-#2-#3\@#4#5#6{%
8904 \bbl@gregday=#3\relax \bbl@gregmonth=#2\relax \bbl@gregyear=#1\relax
8905 \bbl@hebrfromgreg
8906 {\bbl@gregday}{\bbl@gregmonth}{\bbl@gregyear}%
8907 {\bbl@hebrday}{\bbl@hebrmonth}{\bbl@hebyear}%
8908 \edef#4{\the\bbl@hebyear}%
8909 \edef#5{\the\bbl@hebrmonth}%
8910 \edef#6{\the\bbl@hebrday}}
8911 </ca-hebrew>

```

13.3. Persian

There is an algorithm written in TeX by Jabri, Abolhassani, Pournader and Esfahbod, created for the first versions of the FarsiTeX system (no longer available), but the original license is GPL, so its use with LPPL is problematic. The code here follows loosely that by John Walker, which is free and accurate, but sadly very complex, so the relevant data for the years 2013-2050 have been pre-calculated and stored. Actually, all we need is the first day (either March 20 or March 21).

```

8912 < *ca-persian>
8913 <@Compute Julian day@>
8914 \def\bbl@cs@firstjal@xx{2012,2016,2020,2024,2028,2029,% March 20
8915 2032,2033,2036,2037,2040,2041,2044,2045,2048,2049}
8916 \def\bbl@ca@persian#1-#2-#3\@#4#5#6{%
8917 \edef\bbl@tempa{#1}% 20XX-03-\bbl@tempe = 1 farvardin:
8918 \ifnum\bbl@tempa>2012 \ifnum\bbl@tempa<2051

```

```

8919 \bbl@afterfi\expandafter\@gobble
8920 \fi\fi
8921 {\bbl@error{year-out-range}{2013-2050}{}}}%
8922 \bbl@xin@{\bbl@tempa}{\bbl@cs@firstjal@xx}%
8923 \ifin@def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
8924 \edef\bbl@tempc{\fpeval{\bbl@cs@jd{\bbl@tempa}{#2}{#3}+.5}}% current
8925 \edef\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}% begin
8926 \ifnum\bbl@tempc<\bbl@tempb
8927 \edef\bbl@tempa{\fpeval{\bbl@tempa-1}}% go back 1 year and redo
8928 \bbl@xin@{\bbl@tempa}{\bbl@cs@firstjal@xx}%
8929 \ifin@def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
8930 \edef\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}%
8931 \fi
8932 \edef#4{\fpeval{\bbl@tempa-621}}% set Jalali year
8933 \edef#6{\fpeval{\bbl@tempc-\bbl@tempb+1}}% days from 1 farvardin
8934 \edef#5{\fpeval{% set Jalali month
8935 (#6 <= 186) ? ceil(#6 / 31) : ceil((#6 - 6) / 30)}}
8936 \edef#6{\fpeval{% set Jalali day
8937 (#6 - ((#5 <= 7) ? ((#5 - 1) * 31) : ((#5 - 1) * 30) + 6))}}%
8938 </ca-persian>

```

13.4. Coptic and Ethiopic

Adapted from `jquery.calendars.package-1.1.4`, written by Keith Wood, 2010. Dual license: GPL and MIT. The only difference is the epoch.

```

8939 <*ca-coptic>
8940 <@Compute Julian day@>
8941 \def\bbl@ca@coptic#1-#2-#3\@#4#5#6{%
8942 \edef\bbl@tempd{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + 0.5}}%
8943 \edef\bbl@tempc{\fpeval{\bbl@tempd - 1825029.5}}%
8944 \edef#4{\fpeval{%
8945 floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
8946 \edef\bbl@tempc{\fpeval{%
8947 \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1825029.5}}%
8948 \edef#5{\fpeval{floor(\bbl@tempc / 30) + 1}}%
8949 \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}%
8950 </ca-coptic>
8951 <*ca-ethiopic>
8952 <@Compute Julian day@>
8953 \def\bbl@ca@ethiopic#1-#2-#3\@#4#5#6{%
8954 \edef\bbl@tempd{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + 0.5}}%
8955 \edef\bbl@tempc{\fpeval{\bbl@tempd - 1724220.5}}%
8956 \edef#4{\fpeval{%
8957 floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
8958 \edef\bbl@tempc{\fpeval{%
8959 \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1724220.5}}%
8960 \edef#5{\fpeval{floor(\bbl@tempc / 30) + 1}}%
8961 \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}%
8962 </ca-ethiopic>

```

13.5. Julian

Based on [ReinDersh].

```

8963 <*ca-julian>
8964 <@Compute Julian day@>
8965 \def\bbl@ca@julian#1-#2-#3\@#4#5#6{%
8966 \edef\bbl@tempj{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + .5}}%
8967 \edef\bbl@tempa{\fpeval{\bbl@tempj + 32082.5}}%
8968 \edef\bbl@tempb{\fpeval{floor((4 * \bbl@tempa + 3) / 1461)}}%
8969 \edef\bbl@tempc{\fpeval{\bbl@tempa - floor(1461*\bbl@tempb/4)}}%
8970 \edef\bbl@tempd{\fpeval{floor((5 * \bbl@tempc + 2) / 153)}}%
8971 \edef#6{\fpeval{\bbl@tempc - floor((153*\bbl@tempd+2) / 5) + 1}}%

```

```

8972 \edef#5{\fpeval{\bbl@tempd + 3 - 12 * floor(\bbl@tempd / 10)}}%
8973 \edef#4{\fpeval{\bbl@tempb - 4800 + floor(\bbl@tempd / 10)}}%
8974 </ca-julian>

```

13.6. Buddhist

That's very simple.

```

8975 <*ca-buddhist>
8976 \def\bbl@ca@buddhist#1-#2-#3\@@#4#5#6{%
8977 \edef#4{\number\numexpr#1+543\relax}%
8978 \edef#5{#2}%
8979 \edef#6{#3}}
8980 </ca-buddhist>
8981 %
8982 % \subsection{Chinese}
8983 %
8984 % Brute force, with the Julian day of first day of each month. The
8985 % table has been computed with the help of \textsf{python-lunardate} by
8986 % Ricky Yeung, GPLv2 (but the code itself has not been used). The range
8987 % is 2015-2044.
8988 %
8989 % \begin{macrocode}
8990 <*ca-chinese>
8991 \ExplSyntaxOn
8992 <@Compute Julian day@>
8993 \def\bbl@ca@chinese#1-#2-#3\@@#4#5#6{%
8994 \edef\bbl@tempd{\fpeval{%
8995 \bbl@cs@jd{#1}{#2}{#3} - 2457072.5 }}%
8996 \count@z@
8997 \@tempcnta=2015
8998 \bbl@foreach\bbl@cs@chinese@data{%
8999 \ifnum##1>\bbl@tempd\else
9000 \advance\count@\@ne
9001 \ifnum\count@>12
9002 \count@\@ne
9003 \advance\@tempcnta\@ne\fi
9004 \bbl@xin@{,##1,}{, \bbl@cs@chinese@leap,}%
9005 \ifin@
9006 \advance\count@\m@ne
9007 \edef\bbl@tempe{\the\numexpr\count@+12\relax}%
9008 \else
9009 \edef\bbl@tempe{\the\count@}%
9010 \fi
9011 \edef\bbl@tempb{##1}%
9012 \fi}%
9013 \edef#4{\the\@tempcnta}%
9014 \edef#5{\bbl@tempe}%
9015 \edef#6{\the\numexpr\bbl@tempd-\bbl@tempb+1\relax}}
9016 \def\bbl@cs@chinese@leap{%
9017 885,1920,2953,3809,4873,5906,6881,7825,8889,9893,10778}
9018 \def\bbl@cs@chinese@data{0,29,59,88,117,147,176,206,236,266,295,325,
9019 354,384,413,443,472,501,531,560,590,620,649,679,709,738,%
9020 768,797,827,856,885,915,944,974,1003,1033,1063,1093,1122,%
9021 1152,1181,1211,1240,1269,1299,1328,1358,1387,1417,1447,1477,%
9022 1506,1536,1565,1595,1624,1653,1683,1712,1741,1771,1801,1830,%
9023 1860,1890,1920,1949,1979,2008,2037,2067,2096,2126,2155,2185,%
9024 2214,2244,2274,2303,2333,2362,2392,2421,2451,2480,2510,2539,%
9025 2569,2598,2628,2657,2687,2717,2746,2776,2805,2835,2864,2894,%
9026 2923,2953,2982,3011,3041,3071,3100,3130,3160,3189,3219,3248,%
9027 3278,3307,3337,3366,3395,3425,3454,3484,3514,3543,3573,3603,%
9028 3632,3662,3691,3721,3750,3779,3809,3838,3868,3897,3927,3957,%
9029 3987,4016,4046,4075,4105,4134,4163,4193,4222,4251,4281,4311,%
9030 4341,4370,4400,4430,4459,4489,4518,4547,4577,4606,4635,4665,%

```

```

9031 4695,4724,4754,4784,4814,4843,4873,4902,4931,4961,4990,5019,%
9032 5049,5079,5108,5138,5168,5197,5227,5256,5286,5315,5345,5374,%
9033 5403,5433,5463,5492,5522,5551,5581,5611,5640,5670,5699,5729,%
9034 5758,5788,5817,5846,5876,5906,5935,5965,5994,6024,6054,6083,%
9035 6113,6142,6172,6201,6231,6260,6289,6319,6348,6378,6408,6437,%
9036 6467,6497,6526,6556,6585,6615,6644,6673,6703,6732,6762,6791,%
9037 6821,6851,6881,6910,6940,6969,6999,7028,7057,7087,7116,7146,%
9038 7175,7205,7235,7264,7294,7324,7353,7383,7412,7441,7471,7500,%
9039 7529,7559,7589,7618,7648,7678,7708,7737,7767,7796,7825,7855,%
9040 7884,7913,7943,7972,8002,8032,8062,8092,8121,8151,8180,8209,%
9041 8239,8268,8297,8327,8356,8386,8416,8446,8475,8505,8534,8564,%
9042 8593,8623,8652,8681,8711,8740,8770,8800,8829,8859,8889,8918,%
9043 8948,8977,9007,9036,9066,9095,9124,9154,9183,9213,9243,9272,%
9044 9302,9331,9361,9391,9420,9450,9479,9508,9538,9567,9597,9626,%
9045 9656,9686,9715,9745,9775,9804,9834,9863,9893,9922,9951,9981,%
9046 10010,10040,10069,10099,10129,10158,10188,10218,10247,10277,%
9047 10306,10335,10365,10394,10423,10453,10483,10512,10542,10572,%
9048 10602,10631,10661,10690,10719,10749,10778,10807,10837,10866,%
9049 10896,10926,10956,10986,11015,11045,11074,11103}
9050 \ExplSyntaxOff
9051 </ca-chinese>

```

14. Support for Plain T_EX (plain.def)

14.1. Not renaming hyphen.tex

As Don Knuth has declared that the filename `hyphen.tex` may only be used to designate *his* version of the american English hyphenation patterns, a new solution has to be found in order to be able to load hyphenation patterns for other languages in a plain-based T_EX-format. When asked he responded:

That file name is “sacred”, and if anybody changes it they will cause severe upward/downward compatibility headaches.

People can have a file `localhyphen.tex` or whatever they like, but they mustn’t diddle with `hyphen.tex` (or `plain.tex` except to preload additional fonts).

The files `bplain.tex` and `blplain.tex` can be used as replacement wrappers around `plain.tex` and `lplain.tex` to achieve the desired effect, based on the `babel` package. If you load each of them with `iniTEX`, you will get a file called either `bplain.fmt` or `blplain.fmt`, which you can use as replacements for `plain.fmt` and `lplain.fmt`.

As these files are going to be read as the first thing `iniTEX` sees, we need to set some category codes just to be able to change the definition of `\input`.

```

9052 <{*bplain | blplain}
9053 \catcode`\{=1 % left brace is begin-group character
9054 \catcode`\}=2 % right brace is end-group character
9055 \catcode`\#=6 % hash mark is macro parameter character

```

If a file called `hyphen.cfg` can be found, we make sure that *it* will be read instead of the file `hyphen.tex`. We do this by first saving the original meaning of `\input` (and I use a one letter control sequence for that so as not to waste multi-letter control sequence on this in the format).

```

9056 \openin 0 hyphen.cfg
9057 \ifeof0
9058 \else
9059 \let\input

```

Then `\input` is defined to forget about its argument and load `hyphen.cfg` instead. Once that’s done the original meaning of `\input` can be restored and the definition of `\a` can be forgotten.

```

9060 \def\input #1 {%
9061 \let\input\input
9062 \a hyphen.cfg
9063 \let\input\input
9064 }
9065 \fi
9066 </bplain | blplain>

```

Now that we have made sure that `hyphen.cfg` will be loaded at the right moment it is time to load `plain.tex`.

```
9067 <bplain>\a plain.tex
9068 <bplain>\a lplain.tex
```

Finally we change the contents of `\fmtname` to indicate that this is *not* the plain format, but a format based on plain with the `babel` package preloaded.

```
9069 <bplain>\def\fmtname{babel-plain}
9070 <bplain>\def\fmtname{babel-lplain}
```

When you are using a different format, based on `plain.tex` you can make a copy of `blplain.tex`, rename it and replace `plain.tex` with the name of your format file.

14.2. Emulating some $\text{\LaTeX}$ features

The file `babel.def` expects some definitions made in the $\text{\LaTeX}_{2\epsilon}$ style file. So, in Plain we must provide at least some predefined values as well some tools to set them (even if not all options are available). There are no package options, and therefore an alternative mechanism is provided. For the moment, only `\babeloptionstrings` and `\babeloptionmath` are provided, which can be defined before loading `babel`. `\BabelModifiers` can be set too (but not sure it works).

```
9071 <<*Emulate LaTeX>> ≡
9072 \def\@empty{}
9073 \def\loadlocalcfg#1{%
9074   \openin0#1.cfg
9075   \ifeof0
9076     \closein0
9077   \else
9078     \closein0
9079     {\immediate\write16{*****}%
9080      \immediate\write16{* Local config file #1.cfg used}%
9081      \immediate\write16{*}%
9082     }
9083     \input #1.cfg\relax
9084   \fi
9085   \@endoflfd}
```

14.3. General tools

A number of $\text{\LaTeX}$ macro's that are needed later on.

```
9086 \long\def\@firstofone#1{#1}
9087 \long\def\@firstoftwo#1#2{#1}
9088 \long\def\@secondoftwo#1#2{#2}
9089 \def\@nnil{\@nil}
9090 \def\@gobbletwo#1#2{}
9091 \def\@ifstar#1{\@ifnextchar *{\@firstoftwo{#1}}}
9092 \def\@star@or@long#1{%
9093   \@ifstar
9094   {\let\l@ngrel@x\relax#1}%
9095   {\let\l@ngrel@x\long#1}}
9096 \let\l@ngrel@x\relax
9097 \def\@car#1#2\@nil{#1}
9098 \def\@cdr#1#2\@nil{#2}
9099 \let\@typeset@protect\relax
9100 \let\protected@edef\edef
9101 \long\def\@gobble#1{}
9102 \edef\@backslashchar{\expandafter\@gobble\string\}
9103 \def\strip@prefix#1>{}
9104 \def\g@addto@macro#1#2{{%
9105   \toks@\expandafter{#1#2}%
9106   \xdef#1{\the\toks@}}}
9107 \def\@namedef#1{\expandafter\def\csname #1\endcsname}
9108 \def\@nameuse#1{\csname #1\endcsname}
```

```

9109 \def\@ifundefined#1{%
9110   \expandafter\ifx\csname#1\endcsname\relax
9111     \expandafter\@firstoftwo
9112   \else
9113     \expandafter\@secondoftwo
9114   \fi}
9115 \def\@expandtwoargs#1#2#3{%
9116   \edef\reserved@a{\noexpand#1{#2}{#3}}\reserved@a}
9117 \def\zap@space#1 #2{%
9118   #1%
9119   \ifx#2\@empty\else\expandafter\zap@space\fi
9120   #2}
9121 \let\bbl@trace\@gobble
9122 \def\bbl@error#1{% Implicit #2#3#4
9123   \begingroup
9124     \catcode`\=0 \catcode`\==12 \catcode`\=12
9125     \catcode`\^^M=5 \catcode`\%=14
9126     \input errbabel.def
9127   \endgroup
9128   \bbl@error{#1}}
9129 \def\bbl@warning#1{%
9130   \begingroup
9131     \newlinechar=`^^J
9132     \def\{^^J(babel) }%
9133     \message{\{#1}%
9134   \endgroup}
9135 \let\bbl@infowarn\bbl@warning
9136 \def\bbl@info#1{%
9137   \begingroup
9138     \newlinechar=`^^J
9139     \def\{^^J}%
9140     \wlog{#1}%
9141   \endgroup}

```

$\text{\LaTeX 2}_\epsilon$ has the command `\@onlypreamble` which adds commands to a list of commands that are no longer needed after `\begin{document}`.

```

9142 \ifx\@preamblecmds\undefined
9143   \def\@preamblecmds{}
9144 \fi
9145 \def\@onlypreamble#1{%
9146   \expandafter\gdef\expandafter\@preamblecmds\expandafter{%
9147     \@preamblecmds\do#1}}
9148 \@onlypreamble\@onlypreamble

```

Mimic $\text{\LaTeX}$'s `\AtBeginDocument`; for this to work the user needs to add `\begindocument` to his file.

```

9149 \def\begindocument{%
9150   \@begindocumenthook
9151   \global\let\@begindocumenthook\@undefined
9152   \def\do##1{\global\let##1\@undefined}%
9153   \@preamblecmds
9154   \global\let\do\noexpand}

9155 \ifx\@begindocumenthook\@undefined
9156   \def\@begindocumenthook{}
9157 \fi
9158 \@onlypreamble\@begindocumenthook
9159 \def\AtBeginDocument{\g@addto@macro\@begindocumenthook}

```

We also have to mimic $\text{\LaTeX}$'s `\AtEndOfPackage`. Our replacement macro is much simpler; it stores its argument in `\@endoflfd`.

```

9160 \def\AtEndOfPackage#1{\g@addto@macro\@endoflfd{#1}}
9161 \@onlypreamble\AtEndOfPackage
9162 \def\@endoflfd{}
9163 \@onlypreamble\@endoflfd

```

```

9164 \let\bbl@afterlang\@empty
9165 \chardef\bbl@opt@hyphenmap\z@

```

$\LaTeX$ needs to be able to switch off writing to its auxiliary files; plain doesn't have them by default. There is a trick to hide some conditional commands from the outer `\ifx`. The same trick is applied below.

```

9166 \catcode`\&=\z@
9167 \ifx&\if@filesw\@undefined
9168   \expandafter\let\csname if@filesw\expandafter\endcsname
9169     \csname iffalse\endcsname
9170 \fi
9171 \catcode`\&=4

```

Mimic $\LaTeX$'s commands to define control sequences.

```

9172 \def\newcommand{\@star@or@long\new@command}
9173 \def\new@command#1{%
9174   \@testopt{\@newcommand#1}0}
9175 \def\@newcommand#1[#2]{%
9176   \ifnextchar [{\@xargdef#1[#2]}%
9177     {\@argdef#1[#2]}}
9178 \long\def\@argdef#1[#2]#3{%
9179   \@yargdef#1\@ne{#2}{#3}}
9180 \long\def\@xargdef#1[#2][#3]#4{%
9181   \expandafter\def\expandafter#1\expandafter{%
9182     \expandafter\@protected@testopt\expandafter #1%
9183     \csname\string#1\expandafter\endcsname{#3}}}%
9184   \expandafter\@yargdef \csname\string#1\endcsname
9185   \tw@{#2}{#4}}
9186 \long\def\@yargdef#1#2#3{%
9187   \@tempcnta#3\relax
9188   \advance \@tempcnta \@ne
9189   \let\@hash@\relax
9190   \edef\reserved@a{\ifx#2\tw@ [\@hash@1]\fi}%
9191   \@tempcntb #2%
9192   \@whilenum \@tempcntb < \@tempcnta
9193   \do{%
9194     \edef\reserved@a{\reserved@a\@hash@\the\@tempcntb}%
9195     \advance\@tempcntb \@ne}%
9196   \let\@hash@##%
9197   \l@ngrelx\expandafter\def\expandafter#1\reserved@a}
9198 \def\providecommand{\@star@or@long\provide@command}
9199 \def\provide@command#1{%
9200   \begingroup
9201     \escapechar\m@ne\xdef\@gtempa{\{string#1\}}%
9202   \endgroup
9203   \expandafter\ifundefined\@gtempa
9204     {\def\reserved@a{\new@command#1}}%
9205     {\let\reserved@a\relax
9206      \def\reserved@a{\new@command\reserved@a}}%
9207   \reserved@a}%

9208 \def\DeclareRobustCommand{\@star@or@long\declare@robustcommand}
9209 \def\declare@robustcommand#1{%
9210   \edef\reserved@a{string#1}%
9211   \def\reserved@b{#1}%
9212   \edef\reserved@b{\expandafter\strip@prefix\meaning\reserved@b}%
9213   \edef#1{%
9214     \ifx\reserved@a\reserved@b
9215       \noexpand\x@protect
9216       \noexpand#1%
9217     \fi
9218     \noexpand\protect
9219     \expandafter\noexpand\csname
9220       \expandafter\@gobble\string#1 \endcsname

```

```

9221 }%
9222 \expandafter\new@command\csname
9223 \expandafter@gobble\string#1 \endcsname
9224 }
9225 \def\x@protect#1{%
9226 \ifx\protect\@typeset@protect\else
9227 \x@protect#1%
9228 \fi
9229 }
9230 \catcode`\&=\z@ % Trick to hide conditionals
9231 \def\x@protect#1&fi#2#3{&fi\protect#1}

```

The following little macro `\in@` is taken from `latex.ltx`; it checks whether its first argument is part of its second argument. It uses the boolean `\in@`; allocating a new boolean inside conditionally executed code is not possible, hence the construct with the temporary definition of `\bbl@tempa`.

```

9232 \def\bbl@tempa{\csname newif\endcsname&ifin@}
9233 \catcode`\&=4
9234 \ifx\in@\@undefined
9235 \def\in@#1#2{%
9236 \def\in@##1##2##3\in@@{%
9237 \ifx\in@##2\in@false\else\in@true\fi}%
9238 \in@##1\in@\in@@}
9239 \else
9240 \let\bbl@tempa\@empty
9241 \fi
9242 \bbl@tempa

```

$\TeX$ has a macro to check whether a certain package was loaded with specific options. The command has two extra arguments which are code to be executed in either the true or false case. This is used to detect whether the document needs one of the accents to be activated (`activegrave` and `activeacute`). For plain $\TeX$ we assume that the user wants them to be active by default. Therefore the only thing we do is execute the third argument (the code for the true case).

```

9243 \def\@ifpackagewith#1#2#3#4{#3}

```

The $\TeX$ macro `\@ifl@aded` checks whether a file was loaded. This functionality is not needed for plain $\TeX$ but we need the macro to be defined as a no-op.

```

9244 \def\@ifl@aded#1#2#3#4{}

```

For the following code we need to make sure that the commands `\newcommand` and `\providecommand` exist with some sensible definition. They are not fully equivalent to their $\TeX 2_{\epsilon}$ versions; just enough to make things work in plain $\TeX$ environments.

```

9245 \ifx\@tempcnta\@undefined
9246 \csname newcount\endcsname\@tempcnta\relax
9247 \fi
9248 \ifx\@tempcntb\@undefined
9249 \csname newcount\endcsname\@tempcntb\relax
9250 \fi

```

To prevent wasting two counters in $\TeX$ (because counters with the same name are allocated later by it) we reset the counter that holds the next free counter (`\count10`).

```

9251 \ifx\bye\@undefined
9252 \advance\count10 by -2\relax
9253 \fi
9254 \ifx\@ifnextchar\@undefined
9255 \def\@ifnextchar#1#2#3{%
9256 \let\reserved@d=#1%
9257 \def\reserved@a{#2}\def\reserved@b{#3}%
9258 \futurelet\@let@token\@ifnch}
9259 \def\@ifnch{%
9260 \ifx\@let@token\@sptoken
9261 \let\reserved@c\@xifnch
9262 \else
9263 \ifx\@let@token\reserved@d
9264 \let\reserved@c\reserved@a

```



```

9265     \else
9266     \let\reserved@c\reserved@b
9267     \fi
9268     \fi
9269     \reserved@c}
9270 \def:{\let\@sptoken= } \: % this makes \@sptoken a space token
9271 \def:{\@xifnch} \expandafter\def:{\futurelet\@let@token\@ifnch}
9272 \fi
9273 \def\@testopt#1#2{%
9274   \@ifnextchar[#{1}{#1[#2]}}
9275 \def\@protected@testopt#1{%
9276   \ifx\protect\@typeset@protect
9277     \expandafter\@testopt
9278   \else
9279     \@x@protect#1%
9280   \fi}
9281 \long\def\@whilenum#1\do #2{\ifnum #1\relax #2\relax\@iwhilenum{#1\relax
9282   #2\relax}\fi}
9283 \long\def\@iwhilenum#1{\ifnum #1\expandafter\@iwhilenum
9284   \else\expandafter\@gobble\fi{#1}}

```

14.4. Encoding related macros

Code from `ltoutenc.dtx`, adapted for use in the plain $\TeX$ environment.

```

9285 \def\DeclareTextCommand{%
9286   \@dec@text@cmd\providecommand
9287 }
9288 \def\ProvideTextCommand{%
9289   \@dec@text@cmd\providecommand
9290 }
9291 \def\DeclareTextSymbol#1#2#3{%
9292   \@dec@text@cmd\chardef#1{#2}#3\relax
9293 }
9294 \def\@dec@text@cmd#1#2#3{%
9295   \expandafter\def\expandafter#2%
9296     \expandafter{%
9297       \csname#3-cmd\expandafter\endcsname
9298       \expandafter#2%
9299       \csname#3\string#2\endcsname
9300     }%
9301 %   \let\@ifdefinable\@rc@ifdefinable
9302   \expandafter#1\csname#3\string#2\endcsname
9303 }
9304 \def\@current@cmd#1{%
9305   \ifx\protect\@typeset@protect\else
9306     \noexpand#1\expandafter\@gobble
9307   \fi
9308 }
9309 \def\@changed@cmd#1#2{%
9310   \ifx\protect\@typeset@protect
9311     \expandafter\ifx\csname\cf@encoding\string#1\endcsname\relax
9312       \expandafter\ifx\csname ?\string#1\endcsname\relax
9313         \expandafter\def\csname ?\string#1\endcsname{%
9314           \@changed@x@err{#1}%
9315         }%
9316       \fi
9317       \global\expandafter\let
9318         \csname\cf@encoding \string#1\expandafter\endcsname
9319         \csname ?\string#1\endcsname
9320     \fi
9321     \csname\cf@encoding\string#1%
9322       \expandafter\endcsname
9323   \else

```

```

9324     \noexpand#1%
9325     \fi
9326 }
9327 \def\@changed@x@err#1{%
9328     \errhelp{Your command will be ignored, type <return> to proceed}%
9329     \errmessage{Command \protect#1 undefined in encoding \cf@encoding}}
9330 \def\DeclareTextCommandDefault#1{%
9331     \DeclareTextCommand#1?%
9332 }
9333 \def\ProvideTextCommandDefault#1{%
9334     \ProvideTextCommand#1?%
9335 }
9336 \expandafter\let\csname OT1-cmd\endcsname\@current@cmd
9337 \expandafter\let\csname?-cmd\endcsname\@changed@cmd
9338 \def\DeclareTextAccent#1#2#3{%
9339     \DeclareTextCommand#1{#2}[1]{\accent#3 ##1}
9340 }
9341 \def\DeclareTextCompositeCommand#1#2#3#4{%
9342     \expandafter\let\expandafter\reserved@a\csname#2\string#1\endcsname
9343     \edef\reserved@b{\string##1}%
9344     \edef\reserved@c{%
9345         \expandafter\@strip@args\meaning\reserved@a:-\@strip@args}%
9346     \ifx\reserved@b\reserved@c
9347         \expandafter\expandafter\expandafter\ifx
9348             \expandafter\@car\reserved@a\relax\relax\@nil
9349             \@text@composite
9350         \else
9351             \edef\reserved@b##1{%
9352                 \def\expandafter\noexpand
9353                     \csname#2\string#1\endcsname####1{%
9354                         \noexpand\@text@composite
9355                         \expandafter\noexpand\csname#2\string#1\endcsname
9356                         ####1\noexpand\@empty\noexpand\@text@composite
9357                         {##1}%
9358                     }%
9359                 }%
9360             \expandafter\reserved@b\expandafter{\reserved@a{##1}}%
9361         \fi
9362         \expandafter\def\csname\expandafter\string\csname
9363             #2\endcsname\string#1-\string#3\endcsname{#4}
9364     \else
9365         \errhelp{Your command will be ignored, type <return> to proceed}%
9366         \errmessage{\string\DeclareTextCompositeCommand\space used on
9367             inappropriate command \protect#1}
9368     \fi
9369 }
9370 \def\@text@composite#1#2#3\@text@composite{%
9371     \expandafter\@text@composite@x
9372     \csname\string#1-\string#2\endcsname
9373 }
9374 \def\@text@composite@x#1#2{%
9375     \ifx#1\relax
9376         #2%
9377     \else
9378         #1%
9379     \fi
9380 }
9381 %
9382 \def\@strip@args#1:#2-#3\@strip@args{#2}
9383 \def\DeclareTextComposite#1#2#3#4{%
9384     \def\reserved@a{\DeclareTextCompositeCommand#1{#2}{#3}}%
9385     \bgroup
9386     \lccode`\@=#4%

```

```

9387 \lowercase{%
9388 \egroup
9389 \reserved@a @%
9390 }%
9391 }
9392 %
9393 \def\UseTextSymbol#1#2{#2}
9394 \def\UseTextAccent#1#2#3{}
9395 \def\@use@text@encoding#1{}
9396 \def\DeclareTextSymbolDefault#1#2{%
9397 \DeclareTextCommandDefault#1{\UseTextSymbol{#2}#1}%
9398 }
9399 \def\DeclareTextAccentDefault#1#2{%
9400 \DeclareTextCommandDefault#1{\UseTextAccent{#2}#1}%
9401 }
9402 \def\cf@encoding{OT1}

```

Currently we only use the $\text{\LaTeX}$ method for accents for those that are known to be made active in *some* language definition file.

```

9403 \DeclareTextAccent{"}{OT1}{127}
9404 \DeclareTextAccent{'}{OT1}{19}
9405 \DeclareTextAccent{^}{OT1}{94}
9406 \DeclareTextAccent`}{OT1}{18}
9407 \DeclareTextAccent{~}{OT1}{126}

```

The following control sequences are used in `babel.def` but are not defined for `PLAIN  $\text{\TeX}$` .

```

9408 \DeclareTextSymbol{\textquotedblleft}{OT1}{92}
9409 \DeclareTextSymbol{\textquotedblright}{OT1}{`\"}
9410 \DeclareTextSymbol{\textquoteleft}{OT1}{``}
9411 \DeclareTextSymbol{\textquoteright}{OT1}{``'}
9412 \DeclareTextSymbol{\i}{OT1}{16}
9413 \DeclareTextSymbol{\ss}{OT1}{25}

```

For a couple of languages we need the $\text{\LaTeX}$ -control sequence `\scriptsize` to be available. Because plain $\text{\TeX}$ doesn't have such a sophisticated font mechanism as $\text{\LaTeX}$ has, we just `\let` it to `\sevenrm`.

```

9414 \ifx\scriptsize@undefined
9415 \let\scriptsize\sevenrm
9416 \fi

```

And a few more “dummy” definitions.

```

9417 \def\language{english}%
9418 \let\bbl@opt@shorthands@nnil
9419 \def\bbl@ifshorthand#1#2#3{#2}%
9420 \let\bbl@language@opts@empty
9421 \let\bbl@provide@locale\relax
9422 \ifx\babeloptionstrings@undefined
9423 \let\bbl@opt@strings@nnil
9424 \else
9425 \let\bbl@opt@strings\babeloptionstrings
9426 \fi
9427 \def\BabelStringsDefault{generic}
9428 \def\bbl@tempa{normal}
9429 \ifx\babeloptionmath\bbl@tempa
9430 \def\bbl@mathnormal{\noexpand\textormath}
9431 \fi
9432 \def\AfterBabelLanguage#1#2{}
9433 \ifx\BabelModifiers@undefined\let\BabelModifiers\relax\fi
9434 \let\bbl@afterlang\relax
9435 \def\bbl@opt@safe{BR}
9436 \ifx\@uclclist@undefined\let\@uclclist@empty\fi
9437 \ifx\bbl@trace@undefined\def\bbl@trace#1{}\fi
9438 \expandafter\newif\csname ifbbl@single\endcsname
9439 \chardef\bbl@bidimode\z@
9440 <</Emulate LaTeX>>

```

A proxy file:

```
9441 <*\plain>
9442 \input babel.def
9443 </\plain>
```

15. Acknowledgements

In the initial stages of the development of babel, Bernd Raichle provided many helpful suggestions and Michel Goossens supplied contributions for many languages. Ideas from Nico Poppelier, Piet van Oostrum and many others have been used. Paul Wackers and Werenfried Spit helped find and repair bugs.

More recently, there are significant contributions by Salim Bou, Ulrike Fischer, Loren Davis and Udi Fogiel.

Barbara Beeton has helped in improving the manual.

There are also many contributors for specific languages, which are mentioned in the respective files. Without them, babel just wouldn't exist.

References

- [1] Huda Smitschuijzen Abifares, *Arabic Typography*, Saqi, 2001.
- [2] Johannes Braams, Victor Eijkhout and Nico Poppelier, *The development of national $\TeX$ styles*, *TUGboat* 10 (1989) #3, pp. 401–406.
- [3] Yannis Haralambous, *Fonts & Encodings*, O'Reilly, 2007.
- [4] Donald E. Knuth, *The $\TeX$ book*, Addison-Wesley, 1986.
- [5] Jukka K. Korpela, *Unicode Explained*, O'Reilly, 2006.
- [6] Leslie Lamport, *$\TeX$, A document preparation System*, Addison-Wesley, 1986.
- [7] Leslie Lamport, in: $\TeX$ hax Digest, Volume 89, #13, 17 February 1989.
- [8] Ken Lunde, *CJKV Information Processing*, O'Reilly, 2nd ed., 2009.
- [9] Edward M. Reingold and Nachum Dershowitz, *Calendrical Calculations: The Ultimate Edition*, Cambridge University Press, 2018.
- [10] Hubert Partl, *German $\TeX$* , *TUGboat* 9 (1988) #1, pp. 70–72.
- [11] Joachim Schrod, *International $\TeX$ is ready to use*, *TUGboat* 11 (1990) #1, pp. 87–90.
- [12] Apostolos Syropoulos, Antonis Tsolomitis and Nick Sofroniu, *Digital typography using $\TeX$* , Springer, 2002, pp. 301–373.
- [13] K.F. Treebus, *Tekstwijzer, een gids voor het grafisch verwerken van tekst*, SDU Uitgeverij ('s-Gravenhage, 1988).